

# **TestMAX™ ATPG and TestMAX Diagnosis DRC Rules**

---

Version W-2024.09, September 2024



# Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

## Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

## Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

## Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

## Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

## Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

[www.synopsys.com](http://www.synopsys.com)

# Contents

---

|                        |    |
|------------------------|----|
| Conventions .....      | 21 |
| Customer Support ..... | 22 |

---

|                                              |           |
|----------------------------------------------|-----------|
| <b>1. Category A - Assertion Rules .....</b> | <b>23</b> |
| DRC Rule A1 .....                            | 23        |
| DRC Rule A2 .....                            | 24        |
| DRC Rule A3 .....                            | 24        |
| DRC Rule A5 .....                            | 25        |

---

|                                          |           |
|------------------------------------------|-----------|
| <b>2. Category B - Build Rules .....</b> | <b>27</b> |
| DRC Rule B1 .....                        | 28        |
| DRC Rule B2 .....                        | 29        |
| DRC Rule B3 .....                        | 29        |
| DRC Rule B4 .....                        | 30        |
| DRC Rule B5 .....                        | 30        |
| DRC Rule B6 .....                        | 31        |
| DRC Rule B7 .....                        | 31        |
| DRC Rule B8 .....                        | 32        |
| DRC Rule B9 .....                        | 32        |
| DRC Rule B10 .....                       | 33        |
| DRC Rule B11 .....                       | 34        |
| DRC Rule B12 .....                       | 34        |
| DRC Rule B13 .....                       | 35        |
| DRC Rule B14 .....                       | 35        |
| DRC Rule B15 .....                       | 36        |
| DRC Rule B16 .....                       | 37        |
| DRC Rule B17 .....                       | 37        |
| DRC Rule B18 .....                       | 38        |
| DRC Rule B19 .....                       | 39        |

## Contents

|                                          |           |
|------------------------------------------|-----------|
| DRC Rule B20 .....                       | 39        |
| DRC Rule B21 .....                       | 40        |
| DRC Rule B22 .....                       | 40        |
| DRC Rule B23 .....                       | 41        |
| DRC Rule B24 .....                       | 43        |
| DRC Rule B25 .....                       | 43        |
| DRC Rule B26 .....                       | 44        |
| DRC Rule B27 .....                       | 44        |
| DRC Rule B28 .....                       | 45        |
| DRC Rule B29 .....                       | 45        |
| DRC Rule B30 .....                       | 46        |
| DRC Rule B31 .....                       | 47        |
| DRC Rule B32 .....                       | 47        |
| DRC Rule B33 .....                       | 48        |
| DRC Rule B34 .....                       | 49        |
| <hr/>                                    |           |
| <b>3. Category C - Clock Rules .....</b> | <b>52</b> |
| DRC Rule C1 .....                        | 53        |
| DRC Rule C2 .....                        | 56        |
| DRC Rule C3 .....                        | 57        |
| DRC Rule C4 .....                        | 58        |
| DRC Rule C5 .....                        | 60        |
| DRC Rule C6 .....                        | 62        |
| DRC Rule C7 .....                        | 63        |
| DRC Rule C8 .....                        | 64        |
| DRC Rule C9 .....                        | 65        |
| DRC Rule C10 .....                       | 66        |
| DRC Rule C11 .....                       | 67        |
| DRC Rule C12 .....                       | 68        |
| DRC Rule C13 .....                       | 70        |
| DRC Rule C14 .....                       | 71        |
| DRC Rule C15 .....                       | 73        |

## Contents

|                    |    |
|--------------------|----|
| DRC Rule C16 ..... | 74 |
| DRC Rule C17 ..... | 75 |
| DRC Rule C18 ..... | 76 |
| DRC Rule C19 ..... | 76 |
| DRC Rule C20 ..... | 77 |
| DRC Rule C21 ..... | 79 |
| DRC Rule C22 ..... | 79 |
| DRC Rule C23 ..... | 81 |
| DRC Rule C24 ..... | 82 |
| DRC Rule C25 ..... | 83 |
| DRC Rule C26 ..... | 84 |
| DRC Rule C27 ..... | 85 |
| DRC Rule C28 ..... | 85 |
| DRC Rule C29 ..... | 86 |
| DRC Rule C30 ..... | 87 |
| DRC Rule C31 ..... | 87 |
| DRC Rule C34 ..... | 88 |
| DRC Rule C35 ..... | 89 |
| DRC Rule C36 ..... | 90 |
| DRC Rule C37 ..... | 90 |
| DRC Rule C38 ..... | 91 |
| DRC Rule C39 ..... | 91 |
| DRC Rule C40 ..... | 94 |
| DRC Rule C41 ..... | 95 |
| DRC Rule C43 ..... | 96 |
| DRC Rule C44 ..... | 96 |

---

|                                        |           |
|----------------------------------------|-----------|
| <b>4. Category D - DFT Rules .....</b> | <b>98</b> |
| DRC Rule D1 .....                      | 99        |
| DRC Rule D2 .....                      | 100       |
| DRC Rule D3 .....                      | 101       |
| DRC Rule D4 .....                      | 102       |

## Contents

|                    |     |
|--------------------|-----|
| DRC Rule D5 .....  | 103 |
| DRC Rule D6 .....  | 103 |
| DRC Rule D7 .....  | 104 |
| DRC Rule D8 .....  | 105 |
| DRC Rule D9 .....  | 106 |
| DRC Rule D10 ..... | 108 |
| DRC Rule D11 ..... | 109 |
| DRC Rule D12 ..... | 109 |
| DRC Rule D13 ..... | 110 |
| DRC Rule D14 ..... | 111 |
| DRC Rule D15 ..... | 112 |
| DRC Rule D16 ..... | 113 |
| Example .....      | 114 |
| DRC Rule D17 ..... | 115 |
| DRC Rule D18 ..... | 116 |
| DRC Rule D20 ..... | 116 |
| DRC Rule D21 ..... | 117 |
| DRC Rule D22 ..... | 117 |
| DRC Rule D23 ..... | 118 |
| DRC Rule D39 ..... | 118 |

---

|                                          |            |
|------------------------------------------|------------|
| <b>5. Category K - LBIST Rules .....</b> | <b>120</b> |
| DRC Rule K1 .....                        | 124        |
| DRC Rule K2 .....                        | 126        |
| DRC Rule K3 .....                        | 127        |
| DRC Rule K4 .....                        | 128        |
| DRC Rule K5 .....                        | 130        |
| DRC Rule K6 .....                        | 131        |
| DRC Rule K7 .....                        | 133        |
| DRC Rule K8 .....                        | 135        |
| DRC Rule K9 .....                        | 136        |
| DRC Rule K10 .....                       | 138        |

## Contents

|                    |     |
|--------------------|-----|
| DRC Rule K11 ..... | 140 |
| DRC Rule K12 ..... | 141 |
| DRC Rule K13 ..... | 143 |
| DRC Rule K14 ..... | 144 |
| DRC Rule K15 ..... | 145 |
| DRC Rule K16 ..... | 146 |
| DRC Rule K17 ..... | 147 |
| DRC Rule K18 ..... | 149 |
| DRC Rule K19 ..... | 149 |
| DRC Rule K20 ..... | 150 |
| DRC Rule K21 ..... | 152 |
| DRC Rule K22 ..... | 153 |
| DRC Rule K23 ..... | 154 |
| DRC Rule K24 ..... | 155 |
| DRC Rule K25 ..... | 156 |
| DRC Rule K26 ..... | 157 |
| DRC Rule K27 ..... | 159 |
| DRC Rule K28 ..... | 160 |
| DRC Rule K29 ..... | 161 |
| DRC Rule K30 ..... | 162 |
| DRC Rule K31 ..... | 163 |
| DRC Rule K32 ..... | 164 |
| DRC Rule K33 ..... | 165 |
| DRC Rule K34 ..... | 167 |
| DRC Rule K35 ..... | 168 |
| DRC Rule K36 ..... | 169 |
| DRC Rule K37 ..... | 170 |
| DRC Rule K38 ..... | 171 |
| DRC Rule K39 ..... | 172 |
| DRC Rule K40 ..... | 173 |
| DRC Rule K41 ..... | 174 |
| DRC Rule K42 ..... | 175 |

## Contents

|                    |     |
|--------------------|-----|
| DRC Rule K43 ..... | 176 |
| DRC Rule K44 ..... | 177 |
| DRC Rule K45 ..... | 178 |
| DRC Rule K46 ..... | 179 |
| DRC Rule K47 ..... | 180 |
| DRC Rule K48 ..... | 181 |
| DRC Rule K49 ..... | 182 |
| DRC Rule K50 ..... | 183 |
| DRC Rule K51 ..... | 184 |
| DRC Rule K52 ..... | 185 |
| DRC Rule K53 ..... | 186 |
| DRC Rule K54 ..... | 187 |
| DRC Rule K55 ..... | 188 |
| DRC Rule K56 ..... | 189 |
| DRC Rule K57 ..... | 190 |
| DRC Rule K58 ..... | 191 |
| DRC Rule K59 ..... | 192 |
| DRC Rule K60 ..... | 193 |
| DRC Rule K61 ..... | 194 |
| DRC Rule K62 ..... | 195 |
| DRC Rule K63 ..... | 196 |
| DRC Rule K64 ..... | 197 |
| DRC Rule K65 ..... | 199 |
| DRC Rule K66 ..... | 200 |
| DRC Rule K67 ..... | 201 |
| DRC Rule K68 ..... | 202 |
| DRC Rule K69 ..... | 203 |
| DRC Rule K70 ..... | 204 |
| DRC Rule K71 ..... | 205 |
| DRC Rule K72 ..... | 207 |
| DRC Rule K73 ..... | 208 |
| DRC Rule K74 ..... | 209 |



## Contents

|                     |     |
|---------------------|-----|
| DRC Rule K75 .....  | 210 |
| DRC Rule K76 .....  | 211 |
| DRC Rule K77 .....  | 213 |
| DRC Rule K78 .....  | 214 |
| DRC Rule K79 .....  | 216 |
| DRC Rule K80 .....  | 217 |
| DRC Rule K81 .....  | 219 |
| DRC Rule K82 .....  | 220 |
| DRC Rule K83 .....  | 221 |
| DRC Rule K84 .....  | 222 |
| DRC Rule K85 .....  | 223 |
| DRC Rule K86 .....  | 223 |
| DRC Rule K87 .....  | 225 |
| DRC Rule K88 .....  | 226 |
| DRC Rule K89 .....  | 227 |
| DRC Rule K90 .....  | 228 |
| DRC Rule K91 .....  | 229 |
| DRC Rule K92 .....  | 229 |
| DRC Rule K93 .....  | 230 |
| DRC Rule K94 .....  | 231 |
| DRC Rule K95 .....  | 232 |
| DRC Rule K96 .....  | 233 |
| DRC Rule K97 .....  | 234 |
| DRC Rule K98 .....  | 236 |
| DRC Rule K99 .....  | 237 |
| DRC Rule K100 ..... | 238 |
| DRC Rule K101 ..... | 239 |
| DRC Rule K102 ..... | 240 |
| DRC Rule K103 ..... | 240 |
| DRC Rule K104 ..... | 241 |
| DRC Rule K105 ..... | 241 |
| DRC Rule K106 ..... | 242 |

## Contents

|                     |     |
|---------------------|-----|
| DRC Rule K107 ..... | 242 |
| DRC Rule K108 ..... | 243 |
| DRC Rule K109 ..... | 244 |
| DRC Rule K110 ..... | 245 |
| DRC Rule K111 ..... | 246 |
| DRC Rule K112 ..... | 247 |
| DRC Rule K113 ..... | 250 |
| DRC Rule K114 ..... | 251 |
| DRC Rule K115 ..... | 252 |
| DRC Rule K116 ..... | 253 |
| DRC Rule K117 ..... | 254 |
| DRC Rule K118 ..... | 255 |
| DRC Rule K119 ..... | 256 |
| DRC Rule K120 ..... | 257 |

---

|                                            |            |
|--------------------------------------------|------------|
| <b>6. Category N - Netlist Rules .....</b> | <b>259</b> |
| DRC Rule N1 .....                          | 260        |
| DRC Rule N2 .....                          | 261        |
| DRC Rule N3 .....                          | 261        |
| DRC Rule N4 .....                          | 261        |
| DRC Rule N5 .....                          | 262        |
| DRC Rule N6 .....                          | 262        |
| DRC Rule N7 .....                          | 263        |
| DRC Rule N8 .....                          | 263        |
| DRC Rule N9 .....                          | 264        |
| DRC Rule N10 .....                         | 264        |
| DRC Rule N11 .....                         | 265        |
| DRC Rule N12 .....                         | 266        |
| DRC Rule N13 .....                         | 266        |
| DRC Rule N14 .....                         | 267        |
| DRC Rule N15 .....                         | 267        |
| DRC Rule N16 .....                         | 268        |

## Contents

|                                               |            |
|-----------------------------------------------|------------|
| DRC Rule N17 .....                            | 270        |
| DRC Rule N18 .....                            | 270        |
| DRC Rule N19 .....                            | 271        |
| DRC Rule N20 .....                            | 271        |
| DRC Rule N21 .....                            | 273        |
| DRC Rule N22 .....                            | 275        |
| DRC Rule N23 .....                            | 275        |
| DRC Rule N24 .....                            | 277        |
| DRC Rule N25 .....                            | 277        |
| DRC Rule N26 .....                            | 278        |
| DRC Rule N27 .....                            | 279        |
| DRC Rule N28 .....                            | 279        |
| DRC Rule N29 .....                            | 280        |
| DRC Rule N30 .....                            | 280        |
| DRC Rule N31 .....                            | 282        |
| DRC Rule N32 .....                            | 282        |
| DRC Rule N33 .....                            | 283        |
| <hr/>                                         |            |
| <b>7. Category P - Path Delay Rules .....</b> | <b>284</b> |
| DRC Rule P1 .....                             | 285        |
| Example .....                                 | 285        |
| DRC Rule P2 .....                             | 286        |
| DRC Rule P3 .....                             | 286        |
| Example .....                                 | 287        |
| DRC Rule P4 .....                             | 287        |
| Example .....                                 | 288        |
| DRC Rule P5 .....                             | 288        |
| Example .....                                 | 289        |
| DRC Rule P6 .....                             | 289        |
| Example .....                                 | 289        |
| DRC Rule P7 .....                             | 290        |
| Example .....                                 | 291        |
| DRC Rule P8 .....                             | 291        |

## Contents

|                    |     |
|--------------------|-----|
| Example .....      | 292 |
| DRC Rule P9 .....  | 293 |
| Example .....      | 294 |
| DRC Rule P10 ..... | 294 |
| Example .....      | 295 |
| DRC Rule P11 ..... | 295 |
| Example .....      | 296 |
| DRC Rule P12 ..... | 296 |
| Example .....      | 297 |
| DRC Rule P13 ..... | 297 |
| Example .....      | 298 |
| DRC Rule P14 ..... | 298 |
| Example .....      | 299 |
| DRC Rule P15 ..... | 299 |
| Example .....      | 300 |
| DRC Rule P16 ..... | 300 |
| Example .....      | 301 |
| DRC Rule P17 ..... | 301 |
| Example .....      | 302 |
| DRC Rule P18 ..... | 302 |
| Example .....      | 303 |
| DRC Rule P19 ..... | 303 |
| Example .....      | 304 |
| DRC Rule P20 ..... | 304 |
| Example .....      | 305 |
| DRC Rule P21 ..... | 305 |
| Example .....      | 306 |
| DRC Rule P22 ..... | 306 |
| Example .....      | 307 |
| DRC Rule P23 ..... | 307 |
| Example .....      | 308 |
| DRC Rule P24 ..... | 308 |
| Example .....      | 309 |
| DRC Rule P25 ..... | 309 |

|                                               |            |
|-----------------------------------------------|------------|
| DRC Rule P26 .....                            | 310        |
| <hr/>                                         |            |
| <b>8. Category R - Compressor Rules .....</b> | <b>312</b> |
| DRC Rule R1 .....                             | 314        |
| DRC Rule R2 .....                             | 315        |
| DRC Rule R3 .....                             | 315        |
| DRC Rule R4 .....                             | 316        |
| DRC Rule R5 .....                             | 316        |
| DRC Rule R6 .....                             | 317        |
| DRC Rule R7 .....                             | 317        |
| DRC Rule R8 .....                             | 319        |
| DRC Rule R9 .....                             | 320        |
| DRC Rule R10 .....                            | 321        |
| DRC Rule R11 .....                            | 323        |
| DRC Rule R12 .....                            | 325        |
| DRC Rule R13 .....                            | 325        |
| DRC Rule R14 .....                            | 326        |
| DRC Rule R15 .....                            | 327        |
| DRC Rule R16 .....                            | 328        |
| DRC Rule R17 .....                            | 329        |
| DRC Rule R18 .....                            | 330        |
| DRC Rule R19 .....                            | 330        |
| DRC Rule R20 .....                            | 331        |
| DRC Rule R21 .....                            | 331        |
| DRC Rule R22 .....                            | 332        |
| DRC Rule R23 .....                            | 333        |
| DRC Rule R24 .....                            | 334        |
| DRC Rule R25 .....                            | 336        |
| DRC Rule R26 .....                            | 336        |
| DRC Rule R27 .....                            | 337        |
| DRC Rule R28 .....                            | 338        |
| What's Next .....                             | 339        |

## Contents

|                    |     |
|--------------------|-----|
| DRC Rule R29 ..... | 339 |
| DRC Rule R30 ..... | 340 |
| DRC Rule R31 ..... | 340 |
| DRC Rule R32 ..... | 341 |
| DRC Rule R33 ..... | 342 |
| DRC Rule R34 ..... | 344 |
| DRC Rule R35 ..... | 345 |
| DRC Rule R36 ..... | 346 |
| DRC Rule R37 ..... | 347 |
| DRC Rule R38 ..... | 348 |
| DRC Rule R39 ..... | 349 |
| DRC Rule R40 ..... | 350 |
| DRC Rule R41 ..... | 351 |
| DRC Rule R42 ..... | 351 |
| DRC Rule R43 ..... | 352 |
| DRC Rule R44 ..... | 353 |
| DRC Rule R45 ..... | 354 |
| DRC Rule R46 ..... | 354 |
| DRC Rule R47 ..... | 355 |
| DRC Rule R48 ..... | 356 |
| DRC Rule R49 ..... | 357 |
| DRC Rule R50 ..... | 357 |
| DRC Rule R51 ..... | 358 |
| DRC Rule R52 ..... | 359 |
| DRC Rule R53 ..... | 360 |
| DRC Rule R54 ..... | 361 |
| DRC Rule R55 ..... | 362 |
| DRC Rule R56 ..... | 362 |
| DRC Rule R57 ..... | 363 |
| DRC Rule R58 ..... | 363 |
| DRC Rule R59 ..... | 364 |
| DRC Rule R60 ..... | 365 |

|                                               |            |
|-----------------------------------------------|------------|
| DRC Rule R61 .....                            | 366        |
| DRC Rule R62 .....                            | 366        |
| DRC Rule R63 .....                            | 367        |
| DRC Rule R64 .....                            | 368        |
| DRC Rule R65 .....                            | 368        |
| <hr/>                                         |            |
| <b>9. Category S - Scan Chain Rules .....</b> | <b>369</b> |
| DRC Rule S1 .....                             | 370        |
| DRC Rule S2 .....                             | 372        |
| DRC Rule S3 .....                             | 373        |
| DRC Rule S4 .....                             | 374        |
| DRC Rule S5 .....                             | 374        |
| Example .....                                 | 375        |
| DRC Rule S6 .....                             | 375        |
| DRC Rule S7 .....                             | 376        |
| DRC Rule S8 .....                             | 376        |
| DRC Rule S9 .....                             | 377        |
| DRC Rule S10 .....                            | 378        |
| DRC Rule S11 .....                            | 378        |
| DRC Rule S12 .....                            | 379        |
| DRC Rule S13 .....                            | 379        |
| DRC Rule S14 .....                            | 380        |
| DRC Rule S15 .....                            | 381        |
| DRC Rule S16 .....                            | 382        |
| DRC Rule S17 .....                            | 382        |
| DRC Rule S18 .....                            | 383        |
| DRC Rule S19 .....                            | 384        |
| Example .....                                 | 384        |
| DRC Rule S20 .....                            | 386        |
| DRC Rule S21 .....                            | 386        |
| DRC Rule S22 .....                            | 387        |
| DRC Rule S23 .....                            | 389        |

## Contents

|                                            |            |
|--------------------------------------------|------------|
| DRC Rule S24 .....                         | 389        |
| Example .....                              | 390        |
| DRC Rule S25 .....                         | 390        |
| Example .....                              | 391        |
| DRC Rule S26 .....                         | 391        |
| Example .....                              | 392        |
| DRC Rule S27 .....                         | 393        |
| Example .....                              | 393        |
| DRC Rule S28 .....                         | 394        |
| Example .....                              | 394        |
| DRC Rule S29 .....                         | 394        |
| DRC Rule S30 .....                         | 396        |
| DRC Rule S32 .....                         | 397        |
| DRC Rule S34 .....                         | 398        |
| DRC Rule S35 .....                         | 399        |
| DRC Rule S36 .....                         | 399        |
| DRC Rule S37 .....                         | 400        |
| DRC Rule S38 .....                         | 400        |
| <hr/>                                      |            |
| <b>10. Category V - Vector Rules .....</b> | <b>402</b> |
| DRC Rule V1 .....                          | 403        |
| DRC Rule V2 .....                          | 404        |
| DRC Rule V3 .....                          | 404        |
| DRC Rule V4 .....                          | 405        |
| DRC Rule V5 .....                          | 406        |
| DRC Rule V6 .....                          | 406        |
| DRC Rule V7 .....                          | 407        |
| DRC Rule V8 .....                          | 408        |
| DRC Rule V9 .....                          | 410        |
| DRC Rule V10 .....                         | 410        |
| DRC Rule V11 .....                         | 410        |
| DRC Rule V12 .....                         | 411        |
| DRC Rule V13 .....                         | 412        |



## Contents

|                    |     |
|--------------------|-----|
| DRC Rule V14 ..... | 413 |
| DRC Rule V15 ..... | 414 |
| DRC Rule V16 ..... | 415 |
| DRC Rule V17 ..... | 415 |
| See Also .....     | 417 |
| DRC Rule V18 ..... | 417 |
| DRC Rule V19 ..... | 419 |
| DRC Rule V20 ..... | 419 |
| DRC Rule V21 ..... | 420 |
| DRC Rule V22 ..... | 421 |
| DRC Rule V23 ..... | 422 |
| DRC Rule V24 ..... | 423 |
| DRC Rule V25 ..... | 424 |
| DRC Rule V26 ..... | 425 |
| DRC Rule V27 ..... | 425 |
| DRC Rule V28 ..... | 426 |
| DRC Rule V29 ..... | 426 |
| DRC Rule V31 ..... | 427 |
| DRC Rule V32 ..... | 429 |
| DRC Rule V33 ..... | 430 |
| DRC Rule V34 ..... | 430 |
| DRC Rule V35 ..... | 432 |
| DRC Rule V36 ..... | 433 |
| DRC Rule V37 ..... | 434 |
| DRC Rule V39 ..... | 435 |

---

|                                             |            |
|---------------------------------------------|------------|
| <b>11. Category X - X-State Rules .....</b> | <b>436</b> |
| DRC Rule X1 .....                           | 436        |
| Example .....                               | 436        |
| DRC Rule X2 .....                           | 438        |

---

|                                          |            |
|------------------------------------------|------------|
| <b>12. Category Y - PHDS Rules .....</b> | <b>440</b> |
|------------------------------------------|------------|

## Contents

|                                              |            |
|----------------------------------------------|------------|
| DRC Rule Y1 .....                            | 441        |
| DRC Rule Y2 .....                            | 441        |
| DRC Rule Y3 .....                            | 442        |
| DRC Rule Y4 .....                            | 442        |
| DRC Rule Y5 .....                            | 443        |
| DRC Rule Y6 .....                            | 443        |
| DRC Rule Y7 .....                            | 444        |
| DRC Rule Y8 .....                            | 444        |
| DRC Rule Y9 .....                            | 444        |
| DRC Rule Y10 .....                           | 445        |
| DRC Rule Y11 .....                           | 445        |
| DRC Rule Y12 .....                           | 446        |
| DRC Rule Y13 .....                           | 446        |
| DRC Rule Y14 .....                           | 446        |
| DRC Rule Y15 .....                           | 447        |
| DRC Rule Y16 .....                           | 447        |
| DRC Rule Y17 .....                           | 448        |
| DRC Rule Y18 .....                           | 448        |
| DRC Rule Y19 .....                           | 448        |
| DRC Rule Y20 .....                           | 449        |
| DRC Rule Y21 .....                           | 450        |
| DRC Rule Y22 .....                           | 450        |
| DRC Rule Y23 .....                           | 451        |
| DRC Rule Y24 .....                           | 451        |
| DRC Rule Y25 .....                           | 452        |
| DRC Rule Y26 .....                           | 452        |
| DRC Rule Y27 .....                           | 453        |
| DRC Rule Y28 .....                           | 453        |
| <hr/>                                        |            |
| <b>13. Category Z - Tristate Rules .....</b> | <b>454</b> |
| DRC Rule Z1 .....                            | 454        |
| DRC Rule Z2 .....                            | 456        |

## Contents

|                                                    |            |
|----------------------------------------------------|------------|
| Example .....                                      | 456        |
| DRC Rule Z3 .....                                  | 457        |
| Example .....                                      | 458        |
| DRC Rule Z4 .....                                  | 458        |
| DRC Rule Z5 .....                                  | 459        |
| Example .....                                      | 460        |
| DRC Rule Z6 .....                                  | 461        |
| DRC Rule Z7 .....                                  | 461        |
| DRC Rule Z8 .....                                  | 463        |
| DRC Rule Z9 .....                                  | 464        |
| DRC Rule Z10 .....                                 | 465        |
| DRC Rule Z11 .....                                 | 466        |
| Example .....                                      | 466        |
| DRC Rule Z12 .....                                 | 467        |
| Example .....                                      | 468        |
| <hr/>                                              |            |
| <b>14. Category Others - All Other Rules .....</b> | <b>469</b> |
| SEL-003 .....                                      | 469        |
| SEL-004 .....                                      | 469        |
| SEL-005 .....                                      | 470        |

# About This Book

The *TestMAX ATPG and TestMAX Diagnosis DRC Rules* book describes the usage and methodology for TestMAX ATPG and TestMAX Diagnosis DRC rules.

This preface includes the following sections:

- [Conventions](#)
- [Customer Support](#)

## Conventions

The following conventions are used in Synopsys documentation.

| Convention            | Description                                                                                                                                                                                                                              |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Courier               | Indicates syntax, such as <code>write_file</code> .                                                                                                                                                                                      |
| <i>Courier italic</i> | Indicates a user-defined value in syntax, such as <code>write_file design_list</code>                                                                                                                                                    |
| <b>Courier bold</b>   | Indicates user input—text you type verbatim—in examples, such as<br><code>prompt&gt; write_file top</code>                                                                                                                               |
| Purple                | <ul style="list-style-type: none"> <li>• Within an example, indicates information of special interest.</li> <li>• Within a command-syntax section, indicates a default, such as <code>include_enclosing = true   false</code></li> </ul> |
| [ ]                   | Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>                                                                                                                                                      |
| ...                   | Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .                                                                                                                              |
|                       | Indicates a choice among alternatives, such as <code>low   medium   high</code>                                                                                                                                                          |
| \                     | Indicates a continuation of a command line.                                                                                                                                                                                              |
| /                     | Indicates levels of directory structure.                                                                                                                                                                                                 |
| <b>Bold</b>           | Indicates a graphical user interface (GUI) element that has an action associated with it.                                                                                                                                                |
| <b>Edit &gt; Copy</b> | Indicates a path to a menu command, such as opening the <b>Edit</b> menu and choosing <b>Copy</b> .                                                                                                                                      |
| Ctrl+C                | Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.                                                                                                                                                      |

## Customer Support

Customer support is available through SolvNetPlus.

### Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

### Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

# 1

## Category A - Assertion Rules

---

### See Also

- [A1 - assert during shift failure](#)
- [A2 - assert during capture failure](#)
- [A3 - assert stable failure](#)

---

## DRC Rule A1

### Message Text

```
A1: Assert [0|1|Stable|Not-X] during shift on Gate [ instance_name ]([  
gate_id ]) failed (A1- n ).. That is,
```

### Severity

Error

### Description

This message appears when a user-defined shift assertion for a cell instance fails during DRC. It displays the type of assertion (0|1|Stable|Not-X), cell instance name (*instance\_name*), location (*gate\_id*), and message code (A1-*n*).

An assertion failure indicates that the identified cell instance causes a pattern failure during VCS simulation. You can find descriptions of all TestMAX ATPG shift assertions in the Assertions topic.

### What Next

A design change is required to fix the connection failure. You can downgrade this message to pass DRC. However, the risk is that TestMAX ATPG generates patterns that fails during VCS simulation.

---

## DRC Rule A2

### Message Text

```
A2: Assert [0|1|Stable|Not-X] during capture on Gate [ instance_name ] ([ gate_id ]) failed (A2- n ).
```

### Severity

Error

### Description

This message appears when a user-defined capture assertion for a cell instance fails during DRC. It displays the type of assertion (0|1|Stable|Not-X), cell instance name (instance\_name), location (gate\_id), and message code (A2-n).

An assertion failure indicates that the identified cell instance causes a pattern failure during VCS simulation. You can find descriptions of all TestMAX ATPG shift assertions in the Assertions topic.

`set_rules A2/A3 -verbose` prints gates with capture mask and ports with PO mask due to Assertion violations

### What Next

A design change is required to fix the connection failure. You can downgrade this message to pass DRC. However, the risk is that TestMAX ATPG generates patterns that fails during VCS simulation.

`set_rules A2/A3 warn` downgrades assertion errors to warning. Default behavior of masking gates and PO's due to assertion violations.

`set_rules A2/A3 warn -nomask` switch allows ATPG to not mask assertion failure lib cells all instances output to X. This disables default behavior of masking gates and PO's due to Assertion violations.

---

## DRC Rule A3

### Message Text

```
A3: Assert Stable on Gate [ instance_name ] ( gate_id ) failed during [shift|capture|shift and capture] (A3- n ).
```

### Severity

Error



### Description

This message appears when a user-defined stable assertion for a cell instance fails during the DRC shift, capture, or both shift and capture processes. It displays the cell instance name (`instance_name`), location (`gate_id`), operation (`shift|capture|shift and capture`) and message code (`A3-n`).

An assertion failure indicates that the identified cell instance causes a pattern failure during VCS simulation. You can find descriptions of all TestMAX ATPG shift assertions in the Assertions topic.

`set_rules A2/A3 -verbose` prints gates with capture mask and ports with PO mask due to Assertion violations

### What Next

A design change is likely required to fix the connection failure. You can downgrade this message to pass DRC. However, the risk is that TestMAX ATPG generates patterns that fails during VCS simulation.

`set_rules A2/A3 warn` downgrades assertion errors to warning. Default behavior of masking gates and PO's due to assertion violations.

`set_rules A2/A3 warn -nomask` switch allows ATPG to not mask assertion failure lib cells all instances output to X. This disables default behavior of masking gates and PO's due to Assertion violations.

---

## DRC Rule A5

### Message Text

```
A5: Assert pin constraint [0|1|Stable|Not-X] on [ instance_name ] ( PI/  
TDR ) for [capture|test_setup] (A5- n)
```

### Severity

Warning

### Description

This message is displayed when there is a modification in the core pin value constraint within the STIL file.

The `PatternPortingAssertions` section is added to the STIL file using `write_patterns` when enabling `set_drc -hierarchical_atpg`. To validate the accuracy of the assertion, it is necessary to read back the STIL file for DRC.

An alert from V5 assertion warns about an unintended alteration in a pin constraint value, potentially leading to a pattern failure in VCS simulation. Detailed information on all TestMAX ATPG assertions is available in the Assertions section.

### **What Next**

Review the script responsible for generating or modifying the pattern to identify the underlying reason for unintended pin constraint value changes. Correcting this prevents the TestMAX ATPG from producing patterns that result in failures during VCS simulation.

# 2

## Category B - Build Rules

---

### See Also

- [B1 - Invalid top module](#)
- [B2 - Multiple choices for top module](#)
- [B3 - Invalid number of net entries for submodule](#)
- [B4 - Inout used unidirectionally](#)
- [B5 - Undefined module referenced](#)
- [B6 - Undriven module inout pin](#)
- [B7 - Undriven module output pin](#)
- [B8 - Unconnected module input pin](#)
- [B9 - Undriven module internal net](#)
- [B10 - Unconnected module internal net](#)
- [B11 - Unsupported library primitive](#)
- [B12 - Undriven instance input pin](#)
- [B13 - Undriven instance inout pin](#)
- [B14 - Unconnected instance output pin](#)
- [B15 - Invalid instance pin connection](#)
- [B16 - Invalid name](#)
- [B17 - Driven tied signal net](#)
- [B18 - Tristate and non-tristate drivers combined](#)
- [B19 - Tristate driver on wired gate](#)
- [B20 - Non-tristate driver on buskeeper](#)
- [B21 - PI drives tristate and non-tristate gates](#)

- [B22 - Dropped design view](#)
- [B23 - Feedback path](#)
- [B24 - Memory never written](#)
- [B25 - No memory init file](#)
- [B26 - Invalid memory init file](#)
- [B27 - Incorrect memory init file size](#)
- [B28 - Invalid memory](#)
- [B29 - Invalid net connection](#)
- [B30 - Lost connection strength](#)
- [B31 - Module references itself](#)
- [B32 - Illegal buskeeper net](#)
- [B33 - Cascaded BUSES](#)
- [B34 - Illegal wired net](#)

---

## DRC Rule B1

### Message Text

Build Rule B1: Invalid top module name (M).

### Severity

Fatal Error

### Description

The module name that you specified either does not exist or does not match the design because of case-sensitivity problems.

### What Next

Make sure that you have read in a netlist. An empty netlist produces this error. Use the `report_module-summary` command to see a summary of modules defined.

Check the spelling of the top module name that you used. Compare it to the module name in netlists that TestMAX ATPG has read in. A `report_module-all` command lists the names of all modules known.

---

## DRC Rule B2

### Message Text

Build Rule B2 There are multiple choices for top module (M selected).

### Severity

Warning

### Description

There are multiple unreferenced modules in a netlist that can be chosen as the default top module. If the DRC severity is not set to error, the last unreferenced netlist module is chosen as the top module.

*M* is the module name of the selected top module.

### What Next

If the reported top module is not correct, specify the correct top module name in the `run_build_model` command.

---

## DRC Rule B3

### Message Text

Build Rule B3: Invalid number of pins (N1/N2) for module primitive (M/I).

### Severity

Fatal error

### Description

The number of net entries given for an instantiation of a module that uses an ordered list of pins must be consistent with the number of external pins for the module.

*N1* is the number of net entries of the instance, *N2* is the number of external pins for the instantiated module, *M* is the name of the module that contains the instance, and *I* is the name of the instance.

### What Next

The netlist of the module in error must be corrected and read in again before a successful model build can be done.

---

## DRC Rule B4

### Message Text

Build Rule B4 I/O used unidirectionally (M) .

### Severity

Warning

### Description

A module port pin declared as bidirectional is connected to internal gate pins that are only inputs or only outputs.

The presence of this violation has no effect on ATPG pattern generation or test coverage.

### What Next

Consider changing the module's port declaration from bidirectional to input or output.

The DRC severity can be changed to ignore using the `set_rules b` command.

---

## DRC Rule B5

### Message Text

Build Rule B5: Module (M) referenced undefined module (U) .

### Severity

Error

### Description

When using the `run_build_model` command, all modules referenced by the top module must be defined or specified as a black box or empty box. In this case, a module referenced by the top module was not defined.

`M` is the name of the module that referenced the undefined module and `U` is the name of the undefined module.

### What Next

You can review a list of missing modules using the `-undefined` option of the `report_modules` command. You must provide definitions for this list of missing modules or mark them as black box or empty box models.

Some Verilog primitives are not supported. When these primitives are referenced by the design or by library cells, they appear as undefined modules. The primitives are: `rtran`,

tranif, rtranif0, rtranif1, tranif0, tranif1. In this case, you need to create a custom ATPG model as a replacement. For more information, see ATPG Modeling Primitive Summary.

You can use the `-black_box` and `-empty_box` options of the `set_build` command to explicitly name modules that are missing and for which you would like black box or empty box behavior. To determine the list of missing modules, adjust the severity of the B5 rule to a warning and repeat the `run_build_model` command. Next, report all missing modules using either the `report_violationsb5` command or the `report_modules-undefined` command. Use this list to construct an explicit series of `set_build` commands with the missing modules identified as black box or empty box. Then repeat the build and continue.

---

## DRC Rule B6

### Message Text

Build Rule B6: Undriven module inout pin (M/D).

### Severity

Warning

### Description

An inout pin is not driven and is treated as inout (default), input, or output.

### What Next

The selection of how to treat an undriven pin can be made using the `set_build-undriven_bidi` command.

---

## DRC Rule B7

### Message Text

Build Rule B7: Undriven module output pin (M/P).

### Severity

Warning

### Description

An output pin of a module must be driven by circuitry inside the module. A module was encountered with a port declared as an output, which did not have a connection to an internal gate output. The output port is then undriven.

`M` is the name of the module and `P` is the name of the pin.

This DRC is performed at the *module* level, and does not consider the effect of any `add_net_connections` commands which can affect the final in-memory image after run build completes.

### What Next

Review the module definition and make changes to the module.

If needed, you can lower the DRC severity to warning or ignore, and the output port is set to TIEZ. All connected pins are also set to TIEZ. ATPG pattern generation can then proceed but overall coverage is affected.

---

## DRC Rule B8

### Message Text

Build Rule B8: Unconnected module input pin (M/P).

### Severity

Warning

### Description

An input pin of a module should drive circuitry inside the module. When a module port list is declared with an input pin and this input pin has no connection to internal gates, then this DRC violation occurs.

*M* is the name of the module and *P* is the name of the pin.

Note: This DRC is performed at the module level, and does not consider the effect of any `add_net_connections` commands which can affect the final in-memory image after run build completes.

### What Next

After investigating the source module, if no change to the module is made, change the severity of the DRC to warning or ignore. This should have no affect on ATPG pattern generation.

---

## DRC Rule B9

### Message Text

Build Rule B9: Undriven module internal net (M/N).



### Severity

Warning

### Description

An internal net of a module must be driven by circuitry inside the module.

**M** is the name of the module and **N** is the name of the net.

This message is generated when an internal net is defined or implied by instance connections and the net is not connected to an input port, a bidirectional port, or an internal instance output pin. When this happens, there is no signal driver for this net.

Note: This DRC is performed at the module level, and does not consider the effect of any `add_net_connections` commands which can affect the final in-memory image after run build completes.

### What Next

Frequently this message occurs when a net/wire is defined within the module but not actually used.

Review the module definition to determine if this problem is harmless or serious.

If needed, you can use the `set_rules` command to set the DRC severity to warning or ignore and any input pins attached to the undriven net is connected to TIEX for ATPG pattern generation purposes.

---

## DRC Rule B10

### Message Text

Build Rule B10: Unconnected module internal net (M/N).

### Severity

Warning

### Description

An internal net of a module must drive circuitry inside the module. The net might be connected to a driving gate output, but has no connection to another gate input or to a module output or inout port.

**M** is the name of the module and **N** is the name of the net.

This type of violation is common when designs change and sections of circuitry are deleted but a few nets become unused and are not removed. This violation poses no danger to pattern generation and will have no effect on test coverage. It can be ignored.

Note: This DRC is performed at the *module* level, and does not consider the effect of any `add_net_connections` commands which can affect the final in-memory image after run build completes.

### What Next

If the DRC severity is set to warning or ignore, the net is left dangling and will automatically be removed if default `set_build-delete_unused_gates` options are in effect. This will have no affect on test coverage or ATPG pattern generation.

If there are also B9 violations in the same module you might want to investigate as there might be a net name mismatch error or typo which is causing an output pin that goes nowhere (B10) and an associated input without a driver (B9).

---

## DRC Rule B11

### Message Text

Build Rule B11: Unmodeled netlist simulation primitive (P) modeled as TIEX.

### Severity

Warning

### Description

A netlist simulation primitive must be able to be modeled as an ATPG primitive.

P is the name of the netlist primitive.

### What Next

If the DRC severity is set to warning or ignore, the netlist instance are modeled as a TIEX gate. The outputs is constant X values. If this is not desirable, provide an alternate ATPG model.

---

## DRC Rule B12

### Message Text

Build Rule B12: Undriven instance input pin (P).

### Severity

Error

### Description

An instantiation of a module has a floating input pin.

### What Next

Investigate the floating input. A floating input is very often a netlist error and should be corrected. If you determine that a floating input is not an error, then use the `set_rules` command to adjust the severity of this rule to warning or ignore and then continue with another `run_build_model` command. All floating inputs is connected to TIEX sources during the build operation.

---

## DRC Rule B13

### Message Text

Build Rule B13: Undriven instance inout pin (P).

### Severity

Warning

### Description

An instantiation of a module has a floating bidirectional pin.

P is the pathname for the pin.

### What Next

You might want to investigate the floating bidirectional pin. If you had intended the pin be used as only an output there will not be a problem. However, if you had intended that the pin be used as an input there is no connection to another driving gate in the design and this might be a netlist error.

---

## DRC Rule B14

### Message Text

Build Rule B14: Unconnected instance output pin (P).

### Severity

Ignored

### Description

An instance output pin should be connected to some other pins or primary outputs or faults cannot be detected upon the output.

P is the pathname for the pin.

### What Next

If the DRC severity is set to warning or ignore, the unused output pin is left dangling and results in unused circuitry. ATPG pattern generation can still be performed. The unused outputs will result in UU faults.

Unused pins can be removed during unused gate optimization using the `set_build-delete_unused_gates` command.

---

## DRC Rule B15

### Message Text

Build Rule B15: Invalid instance pin usage (P).

### Severity

fatal error

### Description

The design usage of an instance pin must be consistent with the declared direction from the defining module.

P is the pin pathname within the design where the violation occurs.

This violation will occur if a module's pin direction is in conflict with how the pin has been connected in the design. For example, a pin defined as an output is connected to a TIE1, TIE0, or TIEX source within the netlist.

### What Next

Check the list of ports used when the module is instantiated in the design against the list of ports in the module definition. Some mismatch can have occurred, especially if the port connections are made "by order", rather than "by name".

This error can occur when module A is defined which has vectored ports in its port list, then another module B which references A has been defined, and then module A is read again or a different version of module A is read which redefines the port list or has the potential of redefining the port list. You should check the list of N5 violations to see if any modules have been redefined. If so, and one of the redefined modules corresponds to the module involved with this violation message, then try to eliminate the duplicate module definitions. You might also try changing the order in which modules are read, as this will affect which module definition is used when there are duplicates.

If there has been no duplication of module definitions, and the module in question is correct, then the instantiation of this module is in error and the design netlist must be corrected and the design re-read before a successful model build can be completed.

---

## DRC Rule B16

### Message Text

Build Rule B16: Invalid instance pin name (P).

### Severity

Fatal error

### Description

The name of a pin of an instantiated module with a named pin list must be consistent with its associated module pin. When the module was instantiated in the design netlist one pin name was used, but the definition of the module itself called out a different pin name.

P is the instance pathname of the pin.

Failure to satisfy this rule is a fatal error and the flattening process is halted. Without a flattened model you cannot proceed in the ATPG pattern generation flow.

### What Next

Review the names of the pins as defined in the module versus the names of the pins called out when the module is instantiated in the design. The two usages must match or the error will continue to occur. After the mismatch has been corrected, re-read the design and libraries before attempting another `run_build_model` command.

Sometimes the error can occur because the design has been flattened and the original instance path names retained using an escape mechanism such as:

```
\\I$1/ABC/I$31/U3
```

You can try adjusting the default hierarchy character (which is "/") to overcome this problem. The `-hierarchy_delimiter` option of the `set_build` command is used for this. Try using a period "."

---

## DRC Rule B17

### Message Text

Build Rule B17: Tied signal net N (nsource=#, nioconns=#) is driven by  
<list of drivers>.

### Severity

Fatal Error

### Description

A net which is connected to a `supply0` or `supply1` statement in Verilog or is tied to a zero or one by instantiating a module and then placing a `1'b0` or `1'b1` or other constant in its port list is considered a tied net. A tied net connected to a module port which is declared to be either output or bidirectional will produce this violation.

The tied net is identified by `N` along with the number of driver sources as well as bidirectional sources attached to this net in addition to the tied value. The additional drivers are listed by their pin pathnames.

Failure to satisfy this rule is a fatal error and the flattening process is halted. Without a flattened model you cannot proceed in the ATPG pattern generation flow.

### What Next

A common cause of this error is the definition inside of a module of a port direction of `inout` or `output` which should be `input`.

The netlist of the module in error must be corrected and read in again before repeating a `run_build_model` command.

---

## DRC Rule B18

### Message Text

Build Rule B18: Three-state and nonthree-state drivers on T gate I (G).

### Severity

Warning

### Description

A net is driven by at least one three-state and at least one nonthree-stateable driver, which results in the net not being able to be three-stated.

`T` is the type of the driven gate (typically a BUS gate), `I` is the instance name and `G` is the primitive ID number.

### What Next

You should investigate the design. If left uncorrected, some test coverage loss occurs.

There is an option to `run_build_model`, called `-weakgates`, which treats all non tristate drivers on the net with a tristate driver as having a weak drive strength. If your

design matches this condition, you might want to rebuild the in memory image using the `-weakgates` option.

---

## DRC Rule B19

### Message Text

Build Rule B19: Three-state driver on wired T gate I (G).

### Severity

Warning

### Description

A wired-OR or wired-AND gate cannot be driven by a three-state driver.

T is the type of the driven gate, I is the instance name and G is the primitive ID number.

### What Next

Investigate the design. If left uncorrected, some test coverage loss occurs.

---

## DRC Rule B20

### Message Text

Build Rule B20: Nonthree-state drivers on T gate I (G).

### Severity

Warning

### Description

A buskeeper primitive is driven by a non tristate driver, which renders the buskeeper useless.

T is the type of the driven gate, I is the instance name and G is the primitive ID number.

### What Next

Investigate the design. If left uncorrected, some test coverage loss occurs because the buskeeper gate never has any influence on the propagated value.

---

## DRC Rule B21

### Message Text

Build Rule B21: T gate I (ID) drives three-state and nonthree-state gates.

### Severity

Ignored

### Description

A primary input fans out to both three-state and non three-state gates.

T is the type of the gate (PI), I is the instance name and ID is the primitive ID number.

### What Next

No action is required and no loss of test coverage occurs. For some design styles, this is an informative message that might suggest design flaws.

---

## DRC Rule B22

### Message Text

Build Rule B22: Message variations:

#1 - Pin P of internal BUS could not be placed anywhere, dropped design view of I with N pins.

#2 - Pin P of removed equivalent DLAT could not be placed anywhere, dropped design view with N pins.

#3 - Dropped design view of I with N pins, merged with I2.

#4 - Dropped design view of I with N pins.

#5 - Pullup/downs removed from PIOs: N pins lost.

### Severity

Warning

### Description

For the following discussion, P is a pin name, I is an instance path, and N is a count.

For message type #1, when multiple BUS or WIRE primitives are directly connected in series they are merged into a single primitive. This is done for efficiency of the ATPG



algorithm, for bus analysis, and so forth. When multiple BUS or WIRE gates are merged there are pins in the middle which disappear. An attempt is made to move these named pins (only named pins can hold faults) to equivalent positions. If this is not possible the B22 violation is issued. After a pin of an instance is lost, the instance is not viewable at the design-level, only at the primitive level.

Message type #2 is similar to #1 except that you have two DLAT or DFF primitives in parallel which are being merged into a single gate.

For message type #3, the situation is again similar to #1 except that pins are dropping the from an entire instance, I, as it is merged with another instance, I2. A design cell might be a candidate for message type #1 and #3 and which message is generated is somewhat random.

For message type #4, the situation is again similar to #1 and #3. This is a catch-all message for the cases where pins could be moved and instances are not merged yet you cannot keep the design-level representation. An example would be a BUS primitive that was created by merging several other BUS primitives for which one of the original BUS primitives had a pin with no name. The final BUS is missing at least one pin name and cannot be shown at the design level, only the primitive level.

For #5, pullup/downs were removed from all bidis as a result of specifying  
`run_build_model-remove_pio_pull.`

Failure to satisfy the B22 rule can result in a slight drop in fault population. There is no danger that patterns are created which do not simulate.

### What Next

For most users, no action is required.

For the test purist you can count up the number of pins dropped due to B22 violations and use this information to manually adjust the final test coverage or fault coverage. For most users this number is negligible.

---

## DRC Rule B23

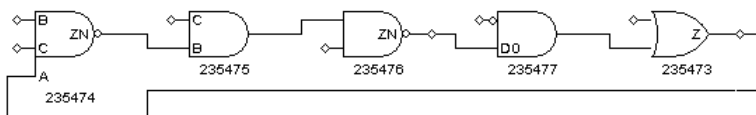
### Message Text

Build Rule B23: Feedback path#ID contains N1 gates and N2 sources.

### Severity

Warning

One example, many variations are possible:



## Description

A combinational feedback path was identified. As each combinational feedback path is identified, it is recorded as a B23 violation. `ID` is the identification number assigned to this feedback path, `N1` is the number of gates in the path, and `N2` is the number of sources in the path. Feedback path sources are a subset of the gates in the feedback path that, if removed, would eliminate the feedback. The sources subset might not be unique. For example, in the picture above, any one gate can be considered a source. The more sources a feedback path has, the more "branches and loops" it has.

## What Next

A few feedback loops are not uncommon and should not greatly affect performance or test coverage. However, an excessive number of feedback loops (> 50) or a loop with a high number of gates could increase CPU effort to create patterns and lead to lower test coverage. In addition, if the number of feedback loops is unexpected, this could be a sign of a library problem or the use of a library modeled at the transistor or pass-gate level.

You can also use the `report_feedback_paths` command to review the number of gates and sources in the feedback loop. You can also use it to get a detailed list of instance path names of gates in the loop. The "source" gates of the feedback loop are always listed first when the verbose option is used.

You can analyze a B23 violation using the graphical schematic viewer by selecting the ANALYZE button on the left, and then choosing the specific occurrence of the B23 violation from the menu. You can also perform an analysis by manually entering the `analyze_violation` command with the `-display` option. This sets the pin data reporting to constrain value data and graphically displays all the gates in the feedback path.

If you are experiencing ATPG performance problems (runtime is long) and you suspect it might be due to the presence of feedback loops, you might want to consider manually breaking the loop for ATPG efforts by disconnecting selected gates in the loop. The `add_net_connections` command along with the `TIEX` and `-disconnect` options might be used to disconnect all inputs from selected gates in the feedback path. However, the use of net connections can lead to performance issues during circuit flattening, so you might want to modify the netlist (used only in the ATPG environment) to break the loop rather than use the `add_net_connections` command.

---

## DRC Rule B24

### Message Text

Build Rule B24: Memory (M) has no write ports and no init file.

### Severity

Error

### Description

A RAM memory must have at least one write port and a ROM memory must have an initialization file. If neither is true, the memory is always at X.

M is the memory instance name.

### What Next

Review the netlist or the model to make sure that the RAM has at least one write port.

If the device is a ROM, ensure that it has an initialization file and that the pathname in the model definition for the initialization file can be found.

You can also downgrade the severity of this rule to warning or ignore using the `set_rules` command. The memory with the violation is treated as a TIEX source for any outputs which can lower test coverage.

---

## DRC Rule B25

### Message Text

Build Rule B25: Cannot open memory init file (F).

### Severity

Error

### Description

The memory initialization file must be readable.

F is the file pathname that was attempted.

### What Next

Check the spelling and file pathname specified for the memory initialization file.

Check the permissions and ownership of the file to make sure you have read access to it.

The `read_memory_file` command can be used to attempt to reread the memory initialization file associated with a particular memory instance.

TestMAX ATPG follows Verilog rules for locating specified initialization files from paths inside Verilog models by resolving file references in the same manner as a Verilog simulator. Since a design/module file contains hard-coded relative paths, make sure it has not moved to a different location; you can address this either by changing the path inside the Verilog model, or by replicating the path to the external file in the new location.

---

## DRC Rule B26

### Message Text

Build Rule B26: Line `L` (`filename`), invalid memory init file entry (`D`).

### Severity

Error

### Description

The memory initialization file must be syntactically correct.

`L` is the line in the file, and `D` is the text segment or address where the invalid syntax was encountered.

### What Next

Correct the syntax problem in the file and try again. If you are in DRC mode you can use the `read_memory_file` command to interactively try re-reading the memory initialization file.

---

## DRC Rule B27

### Message Text

Build Rule B27: Line `L` (`filename`), `D` data values in memory init file.

### Severity

Ignored

### Description

The number of data values in the memory initialization file must be equal to the number of data bits in the memory.

### What Next

Adjust the number of bits in the file or redefine the RAM model to match the data width of the file. Unspecified memory bits are considered to be X.

---

## DRC Rule B28

### Message Text

Build Rule B28: Line L (*filename*), invalid memory (<additional info>).

### Severity

fatal error

### Description

The memory described must be valid.

### What Next

The netlist of the module in error must be corrected and read in again before a successful model build can be done.

---

## DRC Rule B29

### Message Text

Build Rule B29: Invalid net connection (<additional info>).

### Severity

Error

### Description

The defined net connection could not be found in the design during the design build process.

A net connection cannot be checked when it is first defined, because the complete design might not yet have been read in and the top-level module has not yet been selected. Only during the flattening of the design (which comes with `run_build_model` command) can the net connection be properly checked for a match. When no match is found, the B29 violation is issued.

### What Next

Check your spelling and case sensitivity. Your defined net connection must match exactly the net pathname used in the design. If Verilog-escaped names are used, you might want

to review faults located within the same design instance for an idea of the pin pathnames to the corresponding area. This can assist in forming a correct net pathname.

If the violation message indicates "Cannot add net connection on source- and sink-less net", then your target net is a floating net. This can occur when using the `-alloption` and some modules declare nets such as `GND`, but these nets are not used. This type of violation can be ignored as it causes no harm to the ATPG environment. However, you might want to edit the design and delete the unused net. This net will also show up as a B10 violation.

After investigating the B29 violations, if you find that they can be ignored, you might want to use the `set_rules` command to downgrade the rule to a severity of ignore, or just continue to ignore the warnings generated.

---

## DRC Rule B30

### Message Text

Build Rule B30: Transfer path from weak input of BUS I (G) to strong input of BUS I (G).

### Severity

Warning

### Description

The tool cannot properly model a purely transfer path that changes strength from weak to strong.

The message indicates the instance path of the two devices as "I" and the primitive ID as "G".

This violation occurs when there exists a transfer path (that is, a path through only BUS primitives and the data path of SW primitives) that starts with a weak input to a BUS and ends with a strong input to another BUS. If the value of the first BUS is given by its weak input (all other strong inputs being Z, and all other weak inputs being either Z or driving the same value) then this value will appear through at least one SW primitive as a *\*strong\** input to the second BUS. This can cause mismatches against a simulator that accurately considers all strength levels.

Mismatches between the 2-strength ATPG simulator and a full strength-accurate simulator can result in several cases of multiple strengths designs, but this is the only case where a mismatch can result in a design with only two strengths (normal and weak).

### What Next

You can use the ANALYZE button to see the source gates graphically. However, there is not much you can do about the issue. There is a potential that ATPG patterns cannot match simulation results.

---

## DRC Rule B31

### Message Text

Build Rule B31: Module (M) references itself.

### Severity

fatal error

### Description

A module must not directly or indirectly reference itself.

### What Next

The netlist of the module in error must be corrected and read in again before a successful model build can be done.

---

## DRC Rule B32

### Message Text

Build Rule B32: Illegal Bus Keeper Net.

### Severity

fatal error

### Description

A bus keeper has been defined which cannot be initialized to a value because attachment to another driving gate is absent.

### What Next

The netlist of the module in error must be corrected and read in again before a successful model build can be done.

One commonly occurring cause of this type of error is a bus keeper circuit attached to a net which is declared as a module output with no other internal gates which can drive the net. Changing the port definition to an inout is required to overcome the error.

## DRC Rule B33

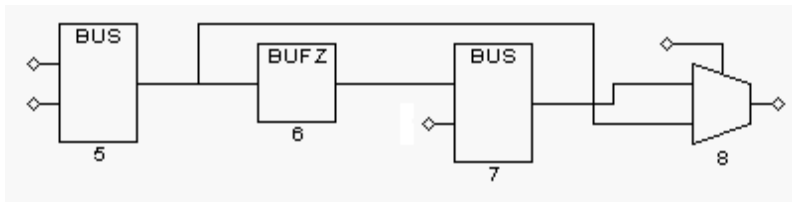
### Message Text

Build Rule B33: Cascaded Buses.

### Severity

Warning

One example, many variations are possible:



### Description

The flattened model contained two cascaded BUS primitives, or two cascaded BUS primitives separated by a BUFZ primitive. BUS primitives are inserted during flattening to resolve the output of tristateable drivers and bus holds.

This condition could cause TestMAX ATPG to see contention and reject patterns due to contention that would otherwise be kept. This is because the contention detection algorithm requires that all gates driving a BUS primitive have an enable (TSD's), or be weak drives (BUSK's). When a BUFZ occurs between two BUS primitives it is regarded as a gate that can't be disabled and the BUS prim is marked as having contention.

### What Next

The occurrence in the design of logic that would justify cascaded BUS devices is very rare and is most often an error in the netlist. Carefully check the source of the gates involved, as well as the defined directions of the ports in the parent module.

If the netlist has no error, you might want to disable contention checking to avoid discarding patterns due to this false contention condition. Unfortunately, the contention checking is globally disabled and this cannot be acceptable.

Sometimes insertion into the original design of a unidirectional gate can overcome this issue.



## DRC Rule B34

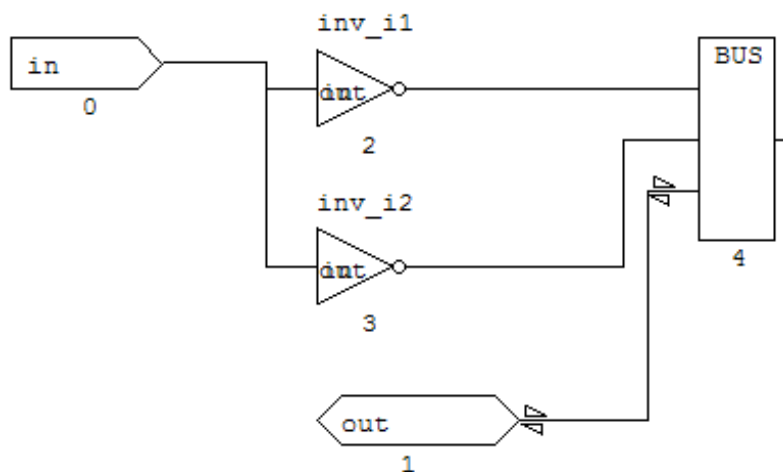
### Message Text

Build Rule B34: Illegal wired net

### Severity

Error

The following figure shows one example of the B34 build rule violation; many variations are possible:

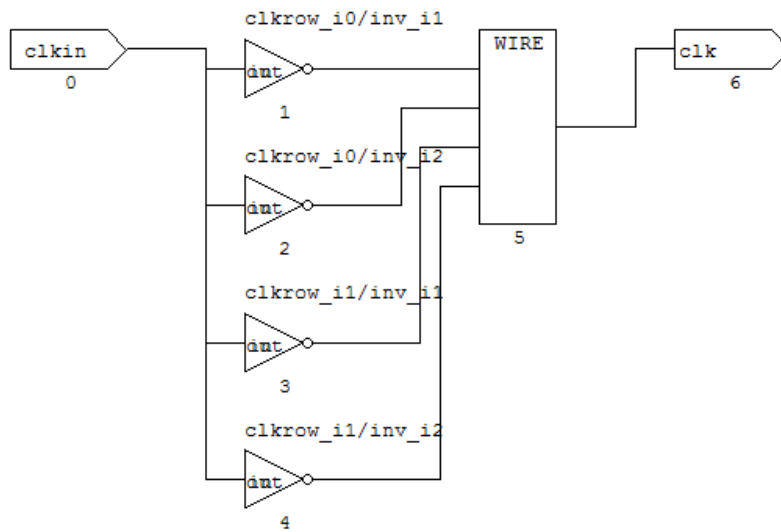


### Description

TestMAX ATPG cannot properly model or represent wired\_and or wired\_or nets with bidirectional connections.

The flattened model above contains two primitives in parallel. The BUS primitive is inserted during flattening to resolve the output of tristateable drivers and bus holds.

If instantiating two of the previous modules and wired\_or the out this cannot be represented properly in TestMAX ATPG. When you downgrade the B34 build rule, TestMAX ATPG builds the model ignoring the wired statement.



```

module top (clkin, clk);
input clkin;
output clk;
wor clk;

clkrow clkrow_i0 ( .out ( clk ) , .in ( clkin ));
clkrow clkrow_i1 ( .out ( clk ) , .in ( clkin ));
endmodule

module clkrow ( out , in );
input in ;
inout out ;

    invert_it inv_i1 (.out ( out ) , .in ( in ));
    invert_it inv_i2 (.out ( out ) , .in ( in ));
endmodule

module invert_it (out, in);
input in;
output out;

    not inv1 (out,in);
endmodule

```

## What Next

When you downgrade the B34 build rule from its default rule violation, TestMAX ATPG builds the model ignoring the wired statement. The final netlist might not be the required representation.

A design with a logic that justifies the wired\_and or wired\_or devices with bidirectional output ports is very rare. It is most probably an error in the netlist. Carefully check the

source of the gates involved as well as the defined directions of the ports in the parent module.

To overcome this issue, insert a unidirectional gate or port in the original design.

# 3

## Category C - Clock Rules

---

### See Also

- [C1 - Unstable scan cells when clocks off](#)
- [C2 - Unstable nonscan DFF when clocks off](#)
- [C3 - No latch transparency when clocks off](#)
- [C4 - Clock unable to capture](#)
- [C5 - LS port captured data affected by new capture](#)
- [C6 - TE port captured data affected by new capture](#)
- [C7 - LE port captured data affected by new capture](#)
- [C8 - LS port clock path affected by new capture](#)
- [C9 - TE port clock path affected by new capture](#)
- [C10 - LE port clock path affected by new capture](#)
- [C11 - LS port captured data affected by clock](#)
- [C12 - LE port captured data affected by clock](#)
- [C13 - TE port captured data affected by clock](#)
- [C14 - Clock connected to multiple ports of same cell](#)
- [C15 - Scan cell port unable to capture](#)
- [C16 - Nonscan cell port unable to capture](#)
- [C17 - Clock connected to PO](#)
- [C18 - Clock connected to PO affected by new capture](#)
- [C19 - Clock connected to non-contention-free BUS](#)
- [C20 - Unstable RAM when clocks off](#)
- [C21 - RAM port unable to capture](#)

- C22 - Clock as data for unstable cells
- C23 - State element connected to unstable cell clock input
- C24 - Nonclock PI connected to unstable cell clock input
- C25 - Unstable cell clock input connected from multiple sources
- C26 - Clock as data different from capture clock for stable cell
- C27 - Missing PLL conditioning data
- C28 - Invalid PLL source for internal clock
- C29 - Undefined PLL source for internal clock
- C30 - Scan PLL conditioning affected by nonscancells
- C31 - Scan PLL conditioning not stable during capture
- C34 - Unsensitized path between PLL source and internal clock
- C35 - Multiple sensitizations between PLL source and internal clock
- C36 - Mistimed sensitizations between PLL source and internal clock
- C37 - Cannot satisfy all internal clocks off for all cycles
- C38 - Bad off-conditioning between PLL source and internal clock
- C39 - Nonlogical clock C connects to scancell T
- C40 - Internal clock is restricted
- C41 - Equivalent clocks are not groupable
- C43 - Scan enable does not facilitate capture after N delay cycles.
- C44 - Scan enable facilitates capture at N1 delay cycles but expected after N2 delay cycles.

---

## DRC Rule C1

### Message Text

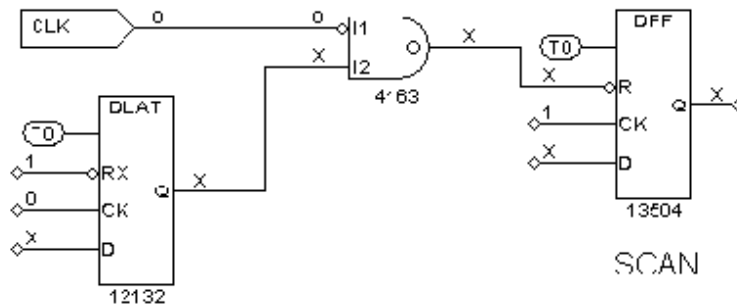
Clock Rule C1: Clock PIs off failed to force off clock input N of scan S I (G) .

### Severity

Error

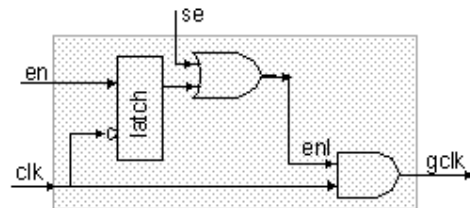
## Examples

### Example 1: Scan chain with a master-slave configuration



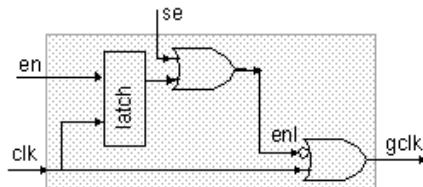
Example 2, below, displays an integrated cell using a latch-based gating style, rising-edge-triggered. The integrated cell contains test logic scan enable.

### Example 2: Rising-Edge Latch-Based Integrated Cell With Post-Control (latch\_posedge\_postcontrol)



Example 3, below, displays an integrated cell using a latch-based gating style, falling-edge-triggered. The integrated cell contains test logic scan enable.

### Example 3: Falling-Edge Latch-Based Integrated Cell With Post-Control (latch\_negedge\_postcontrol)



When all clocks ports are at their off state, all clock/set/reset inputs of scan state elements must be at their off state. The only exception to this would be if these inputs were causing a capture of data from within the scan cell in which they reside such as when a slave latch captures data from its master latch.

$N$  is the input port number of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...),  $S$  is the name of the scan chain,  $I$  is the instance pathname, and  $G$  is the gate ID number of the device.

Failure to satisfy this rule can result in scan cells failing to hold their value before capture, and there is a high risk that patterns generated would fail in simulation. Scan based simulation continues to assume that the values captured during a capture clock cycle are the result of state element values that existed at the end of the load operation. This assumption might be false if scan cells fail to hold their value and lead to patterns generated that fail simulation.

This check is performed by simulating the logic values that result when:

clocks are at their off state

constrained ports are set to their constrained values

constant value nonscan cells are set to their constant state

nonscan cells which have constant values are determined automatically during circuit learning.

This rule violation occurs if any clock/set/reset input of any scan cell memory element is not at its off state.

### What Next

A common cause of this violation is a missing clock definition or a clock defined with the wrong polarity.

Another common cause is a clock which passes through a MUX, and the select line of the MUX is not constant. If the MUX is controlled by some sort of test mode port, that port should be constrained with an `add_pi_constraint` command.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to clock-off data and displays the failing cell with all the gates in the backtrace cone of the failing input. This shows the failing clock/set/reset input and tracing back from this input can assist you in determining how to correct the problem.





## What Next

A common cause of this violation is a missing clock definition or a clock defined with the wrong polarity.

Another common cause is a clock which passes through a MUX, and the select line of the MUX is not constant. If the MUX is controlled by some sort of test mode port, that port should be constrained with an `add_pi_constraint` command.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to clock-off data and displays the failing cell with all the gates in the back trace cone of the failing input. This shows the failing clock/set/reset input and tracing back from this input can assist you in determining how to correct the problem.

Use of the `-mask` option of the `set_rules` command can also be used to mask the C2 rule DRC violation. This allows use of `set_simulation -capture_mask_always_x` to mask nonscan cells with an active set/reset that also have a C2 violation when using threaded ATPG. The option `set_simulation -capture_mask_always_x` is not supported with non-threaded ATPG.

---

## DRC Rule C3

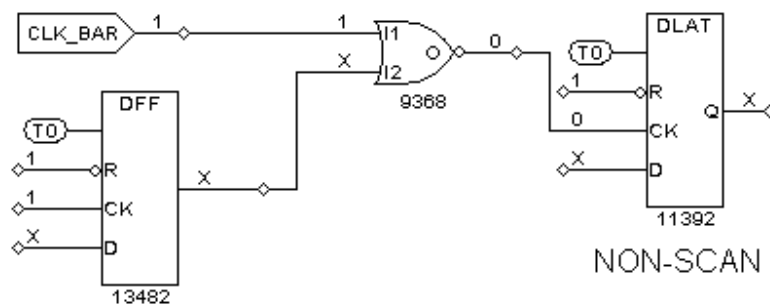
### Message Text

Clock Rule C3: Clock PIs off failed to allow transparency of nonscan DLAT I (G) .

### Severity

Warning

One example, many variations are possible:



### Description

When all defined clocks are at their off state, at least one clock input of a nonscan DLAT (latch primitive) must be capable of being on.

*I* is the instance pathname, and *G* is the gate ID number of the device.

Failure to satisfy this rule means the DLAT primitive is not usable as a transparent latch. The Basic-Scan ATPG algorithm treats the DLAT as if it were a TIEX device and the coverage that uses this algorithm might be degraded. The Fast-Sequential or Full-Sequential algorithm might be required to increase coverage.

This check is performed by simulating the logic values that result when:

clocks are at their off state

constrained ports are set to their constrained values

constant value nonscan cells are set to their constant state

nonscan cells which have constant values are determined automatically during circuit learning.

The rule violation occurs if all clock/set/reset inputs of a nonscan latch are at their off state. DLATs which fail this rule are treated as if their outputs are at X.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to clock-off data and displays the failing cell with all the gates in the backtrace cone of the failing input. This shows the failing clock/set/reset input and tracing back from this input can assist you in determining why the latch cannot be placed in a transparent mode.

Enabling Fast-Sequential or Full-Sequential ATPG pattern generation might help to reduce test coverage loss from latches that cannot be made transparent.

---

## DRC Rule C4

### Message Text

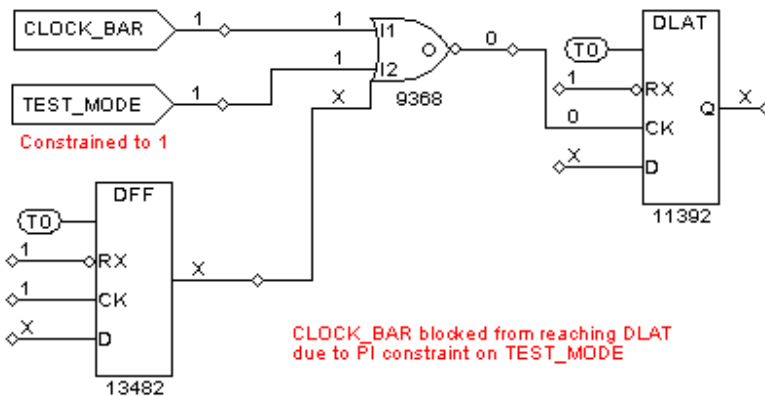
Clock Rule C4: Clock *port\_name* cannot capture data with other clocks off.

Clock Rule C4: Multiple clocks or a single clock through multiple paths drives scan-cell *instance\_name* during shift.

### Severity

Error

One example, many variations are possible:



## Description

There are two different variations of the C4 rule violation message.

The first variation of this rule violation message occurs if all clock/set/reset/write inputs of all sequential elements (including RAMs) are off during this condition. In this case, each defined clock port must turn on a clock, set, reset, or write\_control input of at least one sequential device or memory element when all other clocks are off. Failure to satisfy this rule indicates a defined clock cannot perform a capture operation which can result in some loss of test coverage.

This check is performed by simulating the logic values that result when:

the specified clock (and any equivalent ports) are set to X

all other defined clocks are set to their off states

constrained ports are set to their constrained values

constant value nonscan cells are set to their constant state

The second variation of the C4 rule violation message occurs when the `set_drc -check_multiple_shift_clocks` command is specified. In this case, a C4 violation is reported for each scan cell whose shift clock comes from multiple clocks that are active during shift, or from a single clock which fans out through multiple paths that reconverge.

## What Next

The explanation for the first variation of C4 rule violation message (Clock *port\_name* cannot capture data with other clocks off.) is as follows:

The most common cause of the first variation of this violation is defining a clock that does not behave as a clock. Removing the invalid clock definition corrects this problem.

Another common cause of the first variation of this violation is circuitry blockage that is in the way of a clock pulse reaching the internal sequential device or memory element. This can occur when a clock passes through an AND gate and the control side is 0, or an OR gate with a control side of 1.

Another possibility when the instance is a nonscan device is that there is another pin on the device such as an asynchronous set or reset that is controlled from a primary input that has not been defined as a clock. For example, a CLK pin has been defined as a clock but a RESETB pin has not and so pulsing the CLK pin cannot capture data while RESETB=X. This can be corrected by defining the missing clock.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to constrain data and displays the failing clock with gates along a path to a potential capture cell.

You might also want to switch the display to Clock Off data for a different perspective of the problem.

After reviewing the cause of the violation you might want to continue by setting the severity of the C4 rule to a warning using the `set_rules` command, and then re-issuing the `run_drc` command.

There are certain signals identified as clocks that also generate C4 violations. These types of clock signals are not connected to clock inputs of sequential elements; they are only connected to data inputs. To review these clocks, use the `report_clocks-verbose` command, and determine if a clock uses any sequential elements. You should not define signals without clock usage as clocks; this affects the usage of these signals. If the design context requires these signals to be identified as clocks even though they have no clock usage, you can use the `set_rulesC4 -autofix` command to remove violations for these types of signals. This option does not remove the blocked clocks from generating C4 violations.

The explanation for the second variation of the C4 rule violation message (`Multiple clocks or a single clock through multiple paths drives scan-cellinstance_name during shift.`) is as follows:

This variation of the violation is reported by the `set_drc -check_multiple_shift_clocks` command and is normally ignored. But it can cause problems in static timing analysis or simulations with full delays. Fixing it requires redesign or reconfiguration of the clock generation circuitry of the affected scan cells.

---

## DRC Rule C5

### Message Text

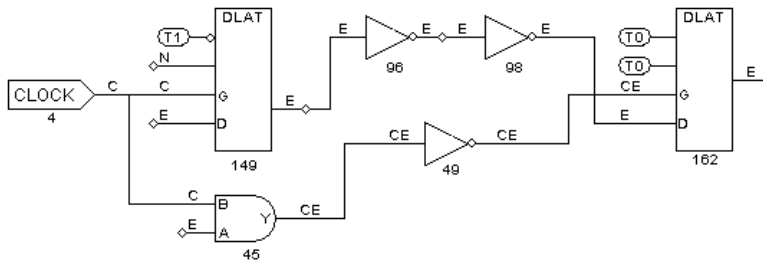
Clock Rule C5: Clock C can capture new data on LS input N of <type> I1 (G1) .

Source of violation: input N of <type> I2 (G2)

## Severity

Warning

One example, many variations are possible:



## Description

A clock must not capture data into a level sensitive (LS) port (latch or RAM) if that data might be affected by new captured data.

C is the clock port; the <type> is either DLAT or DFF; I1 and I2 are instance pathnames; G1 and G2 are gate IDs. Input N refers to the ATPG gate's inputs following the scheme 0=set, 1=reset, 2=clk1, 3=data1, 4=clk2, 5=data2...

There is a risk that test generation will not correctly control the new value that is captured, which will result in a loss of test coverage. In this case, use the Fast-Sequential pattern operator and the command `set_atpg -lete_fastseq`.

This check is performed using clock cone data. A potential rule violation occurs on a clock if a clock input of a LS port is in the clock cone and its data input (including RAM address) is in the effect cone. This port is called the sink port of a violation.

A list of all potential source ports for the violation is identified by tracing back from the sink data line through gates in the effect cone. To be considered a violation, the source port must not be a trailing edge (TE) port, the source port clock must be capable of being at an on state when the sink port clock is at an on state, and the path from source to sink must be propagatable under this condition.

## What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

Use of the `-mask` option of the `set_rules` command can also be used to compensate for the DRC violation. This causes the ATPG algorithm to mask off the observe value in an

attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

## DRC Rule C6

### Message Text

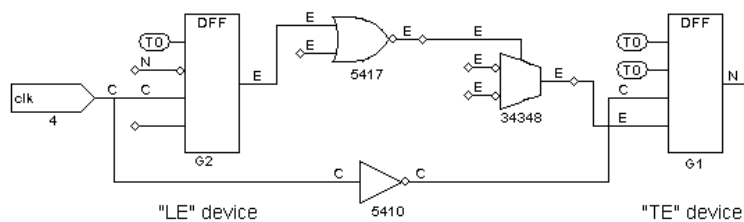
Clock Rule C6: Clock C can capture new data on TE input N of <type> I1 (G1) .

Source of violation: input N of <type> I2 (G2) .

### Severity

Warning

One example, many variations are possible:



### Description

A clock must not capture data into a trailing edge (TE) port if that data might be affected by new captured data. In the violation message,

C is the clock port; the <type> is either DLAT or DFF; I1 and I2 are instance pathnames; G1 and G2 are gate IDs.

There is a risk that test generation does not correctly control the new value that is captured. This results in some loss of test coverage and in some cases simulation mismatches. In such a scenario, proper masking can be applied using `set_rules C6 -mask` to avoid any mismatches.

This check is performed using clock cone data and a potential rule violation occurs on a clock if a clock input of a TE port is in the clock cone and its data input (including RAM address) is in the effect cone. This port is called the sink port of a violation. To be considered a violation, the source port must not be a TE port, the source port clock must be capable of being at 1 (0 if LE) when the sink port clock is at 1, and the path from source to sink must be propagatable under this condition.

Note that C6 violations are not issued on DSLAVE cells. This is because ATPG uses these cells only for controllability and does not observe data from them during scan chain unload.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

A list of all potential source ports for the violation is identified by tracing back from the sink clock line through gates in both the clock and its effect cone.

---

## DRC Rule C7

### Message Text

Clock Rule C7: Clock C can capture new data on LE input I1 of T1 N1 (G1).  
Source of violation: input I2 of T2 N2 (G2).

### Severity

Ignored

### Description

A clock must not capture data into a leading edge (LE) port if that data might be affected by the newly captured data.

Failure to satisfy this rule is not likely to result in a race condition. However, if a race condition occurs this can lead to inaccurate simulation results and the creation of ATPG patterns which fail simulation. Scan based simulation assumes that the values captured during a capture clock cycle are the result of state element values that existed at the end of the load operation.

This check is performed using clock cone data and a potential rule violation occurs on a clock if a clock input of a LE port is in the clock cone and its data input (including RAM address) is in the effect cone. This port is called the sink port of a violation.

A list of all potential source ports for the violation is identified by tracing back from the sink data line through gates in the effect cone. To be considered a violation, the source port must be an LS port, the source port clock must be capable of being at 1 when the sink port clock is at 1, and the path from source to sink must be propagatable under this condition.

## What Next

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

Use the `-mask` option of the `set_rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG algorithm to mask off the control or observe functions, or both, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

---

## DRC Rule C8

### Message Text

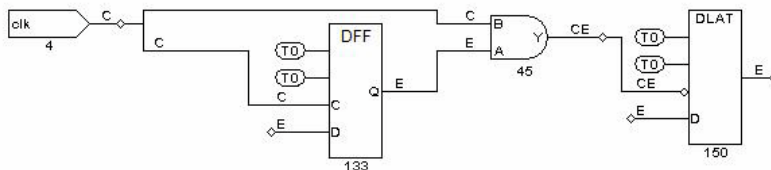
Clock Rule C8: C clock path affected by new capture on LS input N of <type> I1 (G1).

Source of violation: input N of <type> I2 (G2).

### Severity

Warning

One example, many variations are possible:



### Description

The path from a clock to a level sensitive (LS) port must not be affected by its new captured data.

C is the clock port; the <type> is either DLAT or DFF; I1 and I2 are instance pathnames; G1 and G2 are gate ID's.

Failure to satisfy this rule can result in a race condition which can create inaccurate ATPG simulation results and lead to patterns that fail simulation. Scan based simulation continues to assume that the values captured during a capture clock cycle are the result of state element values that existed at the end of the load operation. This assumption could be false in the presence of a race condition and produce patterns that fail simulation.



This check is performed using clock cone data and a potential rule violation occurs on a clock if a clock input of a level sensitive port is in both the clock cone and effect cone. This port is called the sink port of a violation. To be considered a violation, the source port must not be a TE port, the source port clock must be capable of being at 1 when the sink port clock is at 1, and the path from source to sink must be propagatable under this condition.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

A list of all potential source ports for the violation is identified by tracing back from the sink clock line through gates in both the clock and its effect cone.

Use the `-mask` option of the `set_rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG algorithm to mask off the observe value, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is a reduction of test coverage.

---

## DRC Rule C9

### Message Text

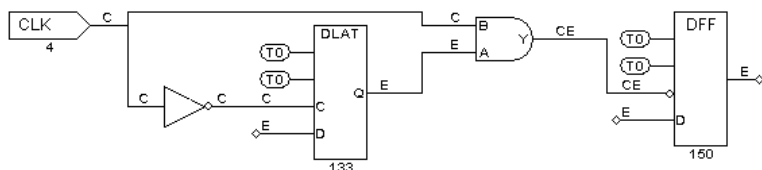
Clock Rule C9: C clock path affected by new capture on TE input N1 of T1 I1 (G1).

Source of violation: input N2 of T2 I2 (G2).

### Severity

Warning

One example, many variations are possible:



### Description

The path from a clock to a trailing edge port must not be affected by its new captured data.

C is the clock port name; N1/N2 are the input numbers of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...); T1/T2 are the primitive types of either DLAT or DFF; I1/I2 are the instance pathnames containing the primitive; G1/G2 are the gate ID's of the primitives;

Failure to satisfy this rule results in an additional simulation pass to allow the trailing edge (TE) port to capture new data from leading edge (LE) or level sensitive (LS) ports. There is also risk that test generation will not correctly control the new value that is captured, which will result in some loss of test coverage.

This check is performed using clock cone data and a rule violation occurs on a clock if a clock input of a TE port is in both the clock cone and effect cone. This port is called the sink port of a violation. To be considered a violation, the source port must not be a TE port, the source port clock must be capable of being at 1 (0 if LE) when the sink port clock is at 1, and the path from source to sink must be propagatable under this condition.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to the clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

A list of all potential source ports for the violation is identified by tracing back from the sink clock line through gates in both the clock and its effect cone.

---

## DRC Rule C10

### Message Text

Clock Rule C10: C clock path affected by new capture on LE input N1 of T1 I1 (G1).

Source of violation: input N2 of T2 I2 (G2).

### Severity

Ignored

### Description

The path from a clock to a leading edge (LE) port must not be affected by its new captured data.

C is the clock port name; N1/N2 are the input numbers of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...); T1/T2 are the primitive types of either DLAT or DFF; I1/I2 are the instance pathnames containing the primitive; G1/G2 are the gate ID's of the primitives;

This check is performed using clock cone data, and a potential rule violation occurs on a clock if a clock input of a LE port is in both the clock cone and effect cone. This port is called the sink port of a violation. To be considered a violation, the source port must be a level sensitive (LS) port, the source port clock must be capable of being at 1 when the sink port clock is at 1, and the path from source to sink must be propagatable under this condition.

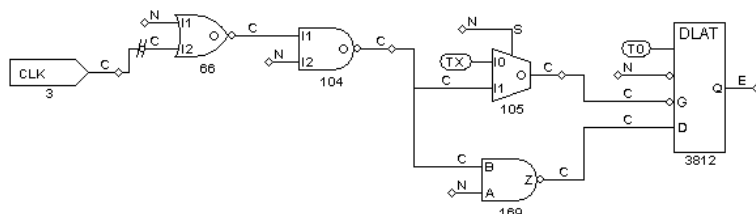
A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to the clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

Use of the `-mask` option of the `set_rules` command to compensate for the DRC violation. By setting the rule severity to `warning` and enabling the `mask` option, this causes the ATPG algorithm to mask off the control or observe functions, or both, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

### Message Text

## Severity

One example, many variations are possible:



### Description

A clock has a combinational path to both the data and clock ports of a level sensitive (LS) device.

`C` is the clock port name; `N1/N2` are the input numbers of the ATPG primitive (`set=0`, `rst=1`, `clk1=2`, `data1=3...`); `I` is the instance pathname containing the primitive; `G` is the gate ID of the primitive;

A clock must not capture data for a level sensitive (LS) port whose data line is affected by the clock.

Failure to satisfy this rule can result in a race condition, which can create inaccurate simulation results. Simulation is performed assuming a captured data value for the clock at an on state. If this assumption is not correct, patterns can fail simulation.

This check is performed by using clock cone data, and a rule violation occurs on a clock when a clock input of a level sensitive clock line and its data line are in the same clock cone.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to the clock cone data and displays all gates in the paths from the clock to both the clock input and data input of the failing port.

If the sequential element involved in the DRC violation is part of a scan chain, you might be able to define a Cell Constraint of OX (observe X) for this cell and reduce the possibility of generating ATPG patterns that fail simulation. This however, will result in a lowering of test coverage.

Use the `-mask` option of the `set_rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG algorithm to mask off the control or observe functions, or both, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

---

## DRC Rule C12

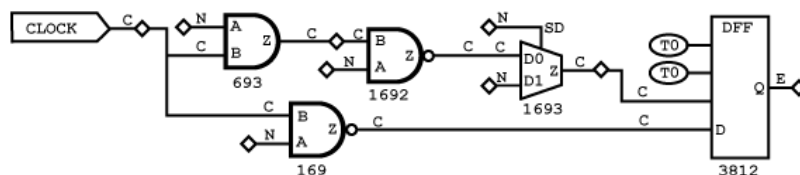
### Message Text

Clock Rule C12: Clock C connects to LE clock/data inputs N1/N2 of I (G).

### Severity

Warning

One example, many variations are possible:



## Description

This check is performed using clock cone data. A violation occurs if a common path (a combinational gate) from a defined clock affects both the data and clock ports of a leading edge (LE) triggered sequential device.

The **C** in the diagram is the clock port name; **N1/N2** are the input numbers of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...); **I** is the instance pathname containing the primitive; **G** is the gate ID number of the primitive.

This violation identifies a potential race condition between the changing data input and the clock capturing that data. TestMAX ATPG cannot determine the actual timing between the two paths, so it is not possible to determine which path is faster. By default, TestMAX ATPG assumes the clock path is faster for an LE device. When this assumption is incorrect, it is likely that any patterns generated are incorrect and will fail in simulation.

## What Next

You can analyze this violation by clicking the ANALYZE button in the schematic viewer and selecting the violation ID. This changes the pin data reporting to clock cone data and displays all gates in the paths from the clock to both the clock input and data input of the failing sequential device.

After reviewing the design gates and paths, you can fix the violation by doing one or more of the following:

C12 violations are masked by default, and ATPG and simulation do not mask every capture cycle. Determine if the cycle data remains stable and mask only unstable cycles. Also, if the data change occurs when another register is clocked, the effects of that change are considered to take place after the capture. In this case, do not mask any cycles.

A race can be caused by clock skew. In this case, use a Synopsys Design Constraints (SDC) hold-time exception to mask the capture. For more information on SDC, see Specifying Timing Exceptions from an SDC File.

Also note the following:

Only scan registers with C12 violations are masked. If you need to mask a nonscan register, use the `add_capture_masks` command.

You can use the `set_rulesc12 -nomask` command if coverage is too low. In this case, the flip-flop captures the old value in both the Basic and Fast-Sequential engines.

You can block the path from the clock to the data input as it passes through combinational logic by applying a constraint to a net connecting to the input of the gate. For example, holding the input of an AND gate to zero will block the path of the clock through the data input gate.

You can change your design so the race path is not in ATPG test mode.

## DRC Rule C13

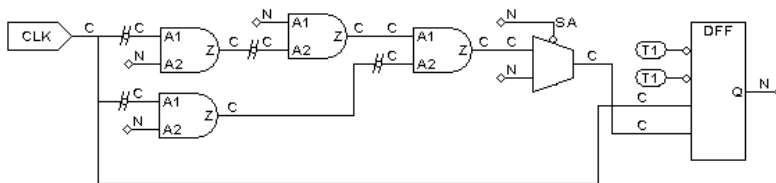
### Message Text

Clock Rule C13: Clock C connects to TE clock/data inputs N1/N2 of I (G).

### Severity

Warning

One example, many variations are possible:



### Description

A common path (combinational gates) exists from a defined clock to both the data and clock ports of a trailing edge (TE) triggered sequential device.

C is the name of the clock, N1/N2 are the input port numbers of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...), I is the instance pathname to the device, and G is the gate ID number of the device.

Failure to pass this rule check indicates there is a potential race condition between the data input changing and the clock capturing that data. The ATPG tool does not have knowledge of the actual timing between the two paths so it is not possible to know which will win the race. By default, the ATPG tool assumes the clock path is faster for a TE device. In other words, the value captured is the value which results at the data input while the clock is asserted. When this assumption is wrong, it is highly likely that any patterns generated is wrong and will fail in simulation.

This check is performed by using data.

This rule violation occurs if a clock input and its data input are in the same clock cone of a trailing edge clock.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to clock cone data and displays all gates in the paths from the clock to both the clock input and data input of the failing sequential device.

After review of the design gates and paths involved a solution can involve one or more of the following:

If the clock path is indeed the faster path, you can ignore the violation by changing the rule to a warning using the `set_rules` command.

It might be possible to block the path from the clock to the data input as it passes through combinational logic by applying a constraint to a net connecting to the other input of one of these combinational gates. For example, holding the other input of an AND gate to zero will block the path of the clock through that gate.

If ATPG patterns generated fail simulation you might be able to sidestep the problem using the `add_cell_constraint` command to mask any data captured into the scan cell.

A design change might be required so that at least in ATPG test mode, this race path is not present.

Use the `-mask` option of the `set_rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG algorithm to mask off the control or observe functions, or both, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

---

## DRC Rule C14

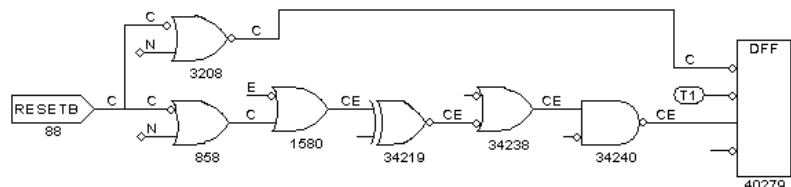
### Message Text

Clock Rule C14: Clock C connects to clock/clock inputs N1/N2 of I (G).

### Severity

Warning

One example, many variations are



possible:

### Description

A common path (combinational gates) exists from a defined clock port to more than one clock input of a sequential device. A clock input includes any asynchronous set or reset control inputs as well as data clocks.

C is the name of the clock, N1/N2 are the input port numbers of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...), I is the instance pathname to the device, and G is the gate ID number of the device.

Failure to pass this rule check indicates there is a potential race condition because a clock port has a combinational gate path to two different clock inputs of the same ATPG sequential primitive. The ATPG tool does not have knowledge of the actual timing between the two paths so it is not possible to know which input will control the stored sequential value. The data captured in the sequential device will come from the data input with the clock at an on state. When this chosen value is incorrect, then simulation failures can result from patterns generated.

This check is performed using clock cone data.

A rule violation occurs on a clock when multiple clock inputs of a state element are in the same clock cone and test generation has identified that it is possible that both clock inputs might be on at the same time.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to clock cone data and displays all gates in the paths from the clock to both clock inputs of the failing sequential device.

After review of the design gates and paths involved a solution can involve one or more of the following:

If the clock path is indeed the faster path, you can ignore the violation by changing the rule to a warning using the `set_rules` command.

If ATPG patterns generated fail simulation you might be able to sidestep the problem using the `add_cell_constraint` command to mask any data captured into the scan cell.



Use the `-mask` option of the `set rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG algorithm to mask off the control or observe functions, or both, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

A violation of this rule introduces no danger of generating bad patterns but is a warning that a potential reduction of test coverage can result because the clock input cannot be used.

#### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to "constrain" data and displays the failing cell with all of the gates in the back trace cone of the failing clock input.

---

## DRC Rule C16

### Message Text

Clock Rule C16: Clock input N of nonscan T I (G) cannot capture data.

### Severity

Warning

### Description

Each clock input of a nonscan cell must be capable of capturing data when a single clock is on and all other clocks are off. Clock inputs include asynchronous set/reset inputs.

**N** is the input number of the ATPG primitive (set=0, rst=1, clk1=2, data1=3...); **T** is a gate type of DFF or DLAT; **I** is the instance pathname to the device; **G** is the gate ID number of the device.

This check is performed using the simulated values that result when one clock is set to X, all other clocks are at their off state, the constrained pins are set to their constrained value, and the constant value nonscan cells are set to their constant state. The rule violation occurs when a clock input of a scan cell always remains off.

A violation of this rule introduces no danger of generating bad patterns but is a warning that a potential reduction of test coverage can result because the clock/set/reset input cannot be used when performing Fast-Sequential ATPG.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to "constrain" data and displays the failing cell with all of the gates in the back trace cone of the failing clock input.

## DRC Rule C17

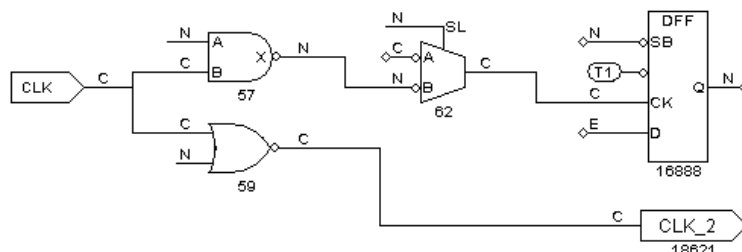
### Message Text

Clock Rule C17: Clock C is connected to primary output P.

### Severity

Warning

One example, many variations are



possible:

### Description

A clock should not have a path to a primary output consisting only of combinational gates. This type of connection is referred to as a clock PO output. Faults along this path which require the clock to be asserted for detection require a special type of pattern called a "Clock ON" pattern which is, by default, not produced by TestMAX ATPG. However, patterns are generated for faults that can be detected with the clock off.

C is the clock port name; P is the primary output port name.

This check is performed by using the clock cone data, and a violation occurs when a primary output (PO) is in a clock cone.

A violation of this rule introduces no danger of generating bad patterns but is a warning that there might be a reduction in test coverage. The ATPG algorithm simulates an additional time frame, which occurs before the capture clock. This is used to determine the correct values for the primary outputs identified.

### What Next

The generation of Clock ON patterns is enabled by using the `-allow_clockon_measures` of the `set_atpg` command and by supplying appropriate procedures in the DRC procedure file. Please Note that not all testers can support this type of pattern.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "clock cone" data and displays all gates in the path from the clock to the failing primary output (PO).

One potential cause of this violation, in addition to the direct combinational path shown above, is a bidirectional port used as a clock. You can ignore C17's occurring on bidirectional clock ports.

See also Supporting Clock ON Patterns in STIL

---

## DRC Rule C18

### Message Text

Clock Rule C18: Path from clock C to PO P affected by new captured data.

### Severity

Ignored

### Description

A path from a clock to a primary output (PO) must not be affected by new data values captured by the clock.

C is the clock port name; P is the primary output port name.

Failure to satisfy this rule has no effect on current functionality.

This check is performed by using the clock cone data, and a violation occurs when a primary output is in both a clock and effect cone.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This sets the pin data reporting to the clock cone data and displays all gates in the path from the clock to the affected primary output (PO).

---

## DRC Rule C19

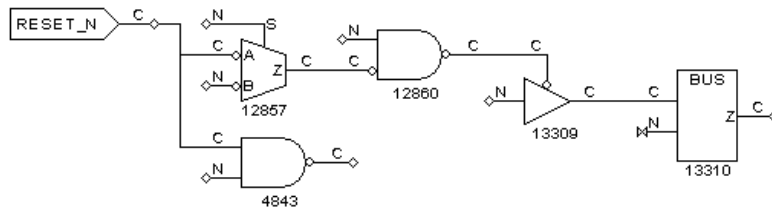
### Message Text

Clock Rule C19: Clock C is connected to at least one non-contention-free BUS (G).

### Severity

Warning

One example, many variations are possible:



## Description

A clock must not have a combinational path to a BUS gate if that BUS gate is not contention free. A BUS gate is inserted into the flattened design for all multi driver nets and acts as a resolution point for the final driven value on this net. **C** is the clock port name; **G** is the gate ID of the BUS gate.

Failure to satisfy this rule creates a risk that bus contention can occur during precapture clock time. This contention condition will result in the pattern being discarded and other patterns is attempted which will increase pattern generation time. Finding a pattern which can avoid contention cannot be possible which will result in a lower test coverage.

This check is performed by using the clock cone data and a violation occurs when a BUS gate is in a clock cone.

## What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This sets the pin data reporting to the clock cone data and displays all gates in the path from the clock to the failing BUS.

As a workaround, you might want to disable contention checking using the `set_contention` command. However, any patterns generated can contain contention and can either fail in simulation or cause stress or device failure when used on the actual silicon, or both.

## DRC Rule C20

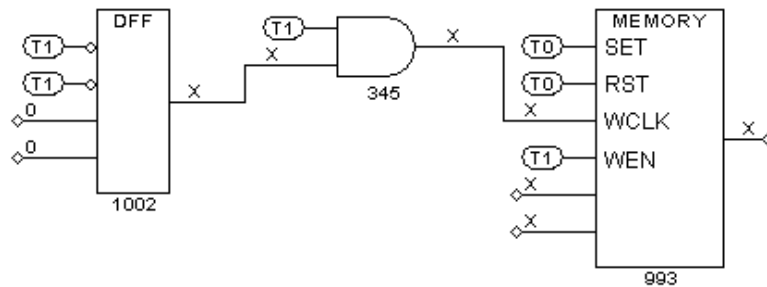
### Message Text

Clock Rule C20: Clock PIs off failed to force off clock input N of RAM I (G) .

### Severity

Warning

One example, many variations are possible:



## Description

When all clocks are at their off state, all clock/set/reset inputs of a MEMORY gate must be in their inactive state.

$N$  is an input starting with 0 at the top and counting down;  $I$  is an instance pathname to the MEMORY;  $G$  is the gate ID of the MEMORY primitive.

Failure to satisfy this rule will result in the MEMORY gate being the equivalent of a TIEX gate for Basic-Scan and Fast-Sequential pattern generation. A reduction in test coverage can occur but can be recovered by using Full-Sequential pattern generation.

This check is performed using the simulated values that result when clocks are at their off state, the constrained pins are set to their constrained value, and the constant value nonscan cells are set to their constant state. The rule violation occurs if any clock/set/reset input of a MEMORY gate is not at its inactive state.

## What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This sets the pin data reporting to the simulation values of the clock-off pattern and displays the failing MEMORY gate with all the gates in the backtrace cone of the failing input. This identifies the failing clock input, and tracing back from this input can help you identify how to correct the problem.

A common cause of this violation is having a primary input which controls MEMORY reads or writes which is not defined as a clock.

If this violation cannot be corrected and the test coverage loss around the memory device is not acceptable, you might wish to explore the use of Full-Sequential pattern generation to attempt to create patterns which make use of the memory, despite this violation.

---

## DRC Rule C21

### Message Text

Clock Rule C21: Write input N of RAM I (G) cannot capture data.

### Severity

Warning

### Description

Each write control of a RAM must be capable of capturing data when a single clock is on and all other clocks are off.

N is an input of the MEMORY primitive starting with 0 at the top and counting down; I is an instance pathname to the MEMORY; G is the gate ID of the MEMORY primitive.

Failure to satisfy this rule does not cause any danger of generating patterns which fail in simulation but it does indicate the potential for a lower test coverage because the write port cannot be used.

This check is performed using the simulated values that result when one clock is set to X, all other clocks are at their off state, the constrained pins are set to their constrained values, and the constant value nonscan cells are set to their constant state. The rule violation occurs when the write input of a RAM is always off rather than X.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This sets the pin data reporting to the "constrain" data and displays the failing MEMORY primitive with all of the gates in the back trace cone of the failing write control input.

You might also wish to switch the viewed data to the `clock off` values in the schematic viewer.

A common cause of this violation is failure to declare the write control input a clock, or not having a top level primary input control for the RAM write control in ATPG test mode.

---

## DRC Rule C22

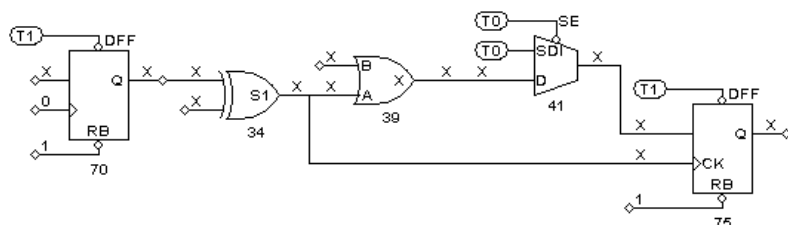
### Message Text

Clock Rule C22: Clock as data connection between G1 and inputs N1/N2 of unstable T (G2).

## Severity

### Warning

One example, many variations are possible:



## Description

The clock input of an unstable nonscan DFF or DLAT was found to have a common connection with its data input and the source of this common connection is a state element or a primary input not declared as a clock.

G1 is the gate ID of the gate that is the common source of the connection; N1 is the input number of the clock input of the failing device; N2 is the input number of the data input of the failing device; T is the gate type (DLAT or DFF) of the failing device; G2 is the gate ID of the failing device.

Failure to satisfy this rule will hinder the ability to detect faults in nonscan areas using the Fast-Sequential or Full-Sequential ATPG pattern capability. This will reduce test coverage.

Failure to satisfy this rule indicates there is a potential race condition between the data input changing and the clock capturing data. Basic-scan and Fast-Sequential ATPG algorithms will treat unstable cells as an X source, so the danger of generating patterns which mismatch in simulation will exist only if Full-Sequential patterns are being generated.

## What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button and changing the pin data displayed to "none". This will draw the trace back from the failing nonscan cell and show the local clock source common to both inputs.

If you are generating Full-Sequential patterns, then it is suggested that you use the `-mask` option of the `set_rules` command to compensate for the DRC violation. By selecting the `-mask` option you cause the ATPG algorithm to mask off the control or observe functions, or both, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.



## DRC Rule C23

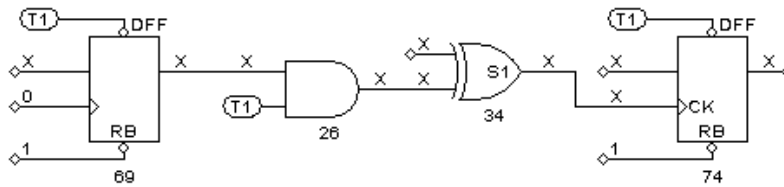
### Message Text

Clock Rule C23: I1 (G1) connected to input N of unstable I2 (G2).

### Severity

Ignored

One example, many variations are possible:



### Description

The clock input of a state element (DFF/DLAT/RAM) has an unblocked path from an unstable cell (DFF/DLAT/RAM).

I1 is an instance pathname to the storage gate that connects to the clock input; G1 is its corresponding gate ID; N is the input of the unstable ATPG primitive starting with 0 at the top and counting down; I2 is an instance pathname to the unstable cell; G2 is its corresponding gate ID;

A violation of this rule indicates that the failing device drives an internally generated clock. Basic-scan and Fast-Sequential ATPG ignore the behavior of unstable cells driven by this internally generated clock, and unless the circuit is modified, Full-Sequential ATPG is required to improve test coverage around unstable cells. This will result in additional ATPG effort and simulation run time, but there is usually no additional risk of simulation mismatches.

This check is performed by identifying storage gate effects for clock inputs of unstable nonscan cells.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This sets the pin data reporting to the "none" and displays all gates in the path between the storage gate and the failing unstable nonscan cell.

## DRC Rule C24

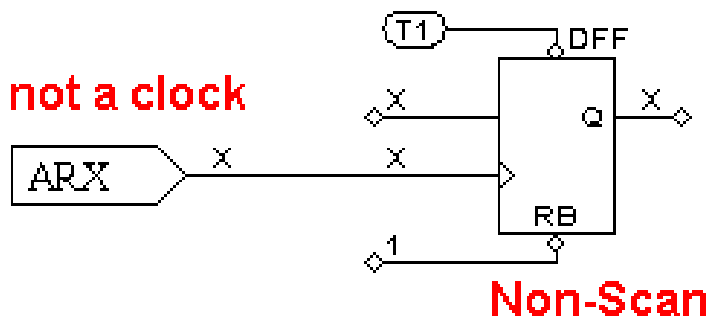
### Message Text

Clock Rule C24: Nonclock PI connected to unstable cell clock input

### Severity

Ignored

One example, many variations are possible:



### Description

The clock input of an unstable state element (DFF/DLAT/RAM) was found to have an unblocked path from primary input which was not declared as a clock. This violation occurs if you neglect to declare a primary input to be a clock when the signal is used to clock an unstable cell.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This sets the pin data reporting to the "clock off" and displays all gates in the path between the primary input and the clock input pin of the unstable state element.

After identifying which Primary Input has a path to the state element, return to DRC mode and define the PI as a clock using the `add_clocks` command.

## DRC Rule C25

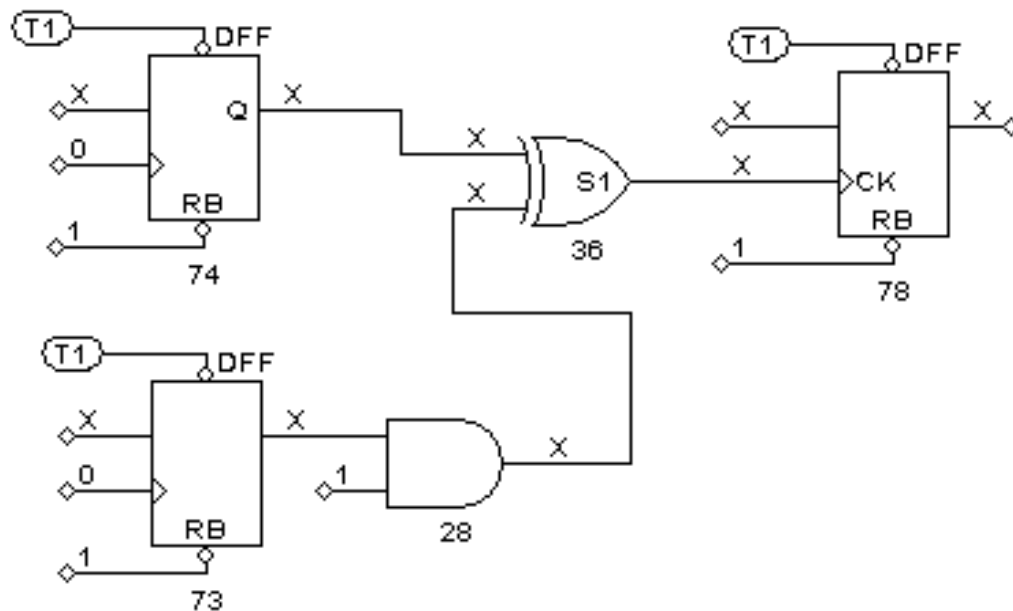
### Message Text

Clock Rule C25: Unstable cell clock input connected from multiple sources.

### Severity

Warning

One example, many variations are possible:



### Description

The clock/set/reset input of an unstable cell (DFF/DLAT/RAM) was found to have an unblocked path from multiple sources which were either state elements or primary inputs not declared as clocks.

For example, the clock input might be the logical OR of two different state elements using the same clock polarity. The inputs to the gating must transition in the same simulation frame, so the combinations LE-TE, PI-LE, PI-TE, and PI-LS are allowed and do not result in C25 violations.

Failure to satisfy this rule indicates there is a potential for a clock glitch to occur. Latches that cause a C25 violation are considered to be X sources immediately after shift only. So

if the latch had a value stored in it during shift, this value would be lost. However, since a latch has the ability to become transparent during ATPG, non-X data can pass through the latch and be stored in it. This will override the initial X following the scan chain load and, in some cases, might allow TestMAX ATPG to test faults through the latch.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This sets the pin data reporting to the "clock off" and displays the unstable cell gate and the gates in the backtrace cone of the appropriate clock input.

The clock generation circuitry should be modified to block one of the paths in test mode, or to ensure in some other way that a glitch is impossible.

Use of the `-mask` option of the `set_rules` command can also be used to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG to mask off the control or observe functions, or both, to ensure that any ATPG pattern generated will pass simulation. The trade-off is that in doing this, there is a reduction of test coverage.

The command `set_rule C25 -mask` is considered only for clock ports and not for set or reset lines. Use the `set_simulation -capture_mask_always_x` command to enable the ability to mask active set/reset values on flops with a C25 and not just the "capture clock" values.

---

## DRC Rule C26

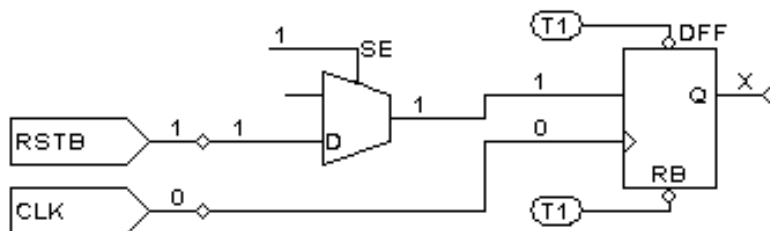
### Message Text

Clock Rule C26: clock as data different from capture clock for stable cell.

### Severity

Warning

One example, many variations are possible:



### Description

The data input of a stable cell (DFF/DLAT/RAM) was found to have an unblocked path to a primary input declared as a clock, and this is a different clock than the one connected to the clock input of the stable cell. Violations of this rule do not create a danger of simulation mismatches, but can indicate a reduction in test coverage.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This sets the pin data reporting to the "constrain\_data" and displays the gate of the stable cell and the backtrace path to primary input declared as a clock.

You should not define the clock going to the data pin as a clock input. Instead, use the `-allow_unstable` option of the `set_drc` command.

---

## DRC Rule C27

### Message Text

Clock Rule C27: Missing PLL conditioning data.

### Severity

Error

### Description

All internal clocks must have the same number of control clauses, corresponding to consecutive cycles, starting with 0.

### What Next

Verify the number of capture cycles, defined by using a `set_drc-num_pll_cycles d` command or by using the ClockStructures block of a Stil Protocol File, and specify control clauses for all capture cycles. See Defining Internal Clocks for PLL Support for the syntax of the ClockStructures block.

---

## DRC Rule C28

### Message Text

Clock Rule C28: Invalid PLL source for internal clock.

### Severity

Fatal Error

### Description

The PLL source declared for an internal clock must be defined as a PLL clock.

### What Next

Define the missing PLL clock by using the `-pllclock` option of the `add_clocks` command or by using the ClockStructures block of a Stil Protocol File. See Defining Internal Clocks for PLL Support for the syntax of a ClockStructures block.

---

## DRC Rule C29

### Message Text

Clock Rule C29: Undefined PLL source for internal clock. Default  
Severity: warning

### Severity

Warning

### Description

For internal clocks, the PLL source must be specified to benefit from PLL clock rule checking; for example, C34 and so on. When defining internal clocks from the command line, specify the corresponding PLL clock source by using the `add_clocks` command.

### Note:

In cases where the missing `pll_source` specifications in the SPF lead to C29 violations, it is recommended that the default setting of `set_simulation` be changed to `set_simulation -nosource_disturb` to prevent the patterns from failing in simulation. The missing accurate `pll_source` specification could lead to optimistic X masking of disturbed clock groups if this switch is not used. Another solution would be to add the missing `pll_source` to resolve the C29 violations and define a unique `pll_source` for each OCC defined in the SPF. This adds proper masking and still provides the benefit of using `source_disturb` simulation.

### What Next

Use the `-pll_source` option of the `add_clocks` command to specify the missing PLL clock. Alternatively, this information can also be provided in the ClockStructures block of the Stil Protocol File. See Defining Internal Clocks for PLL Support for the syntax of a ClockStructures block.

---

## DRC Rule C30

### Message Text

Clock Rule C30: Scan PLL conditioning affected by nonscancells.

### Severity

Error

### Description

For scan-ATPG, internal clocks can only be controlled by scan cells, PIs, and PIOs. PIs and PIOs cannot change after the first "forcePI" in a multi cycle pattern.

### What Next

Provide the required PI constraints or design modification that would allow valid conditioning with a scan cell, PI, or PIO.

This can be debugged in the GSV after downgrading C30 to a warning, so that DRC can finish its analysis. Do not attempt to generate patterns with C30 violations because the patterns will be rejected due to bad clock conditioning. After running `analyze_violations C30-1`, the nonscan cell is shown along with the conditioning node and internal clock node specified in the SPF ClockStructures block or the `add_clocks` command. Verify the behavior of the nonscan cell by right-clicking it and selecting Display Gate Info. Its behavior during scan chain operation can be seen by selecting Shift pin data. If the nonscan cell should be on the scan chain, correct the design or protocol so that it is scanned. Otherwise, correct the conditioning node specification, so that it refers to a scan cell.

If the violation is for a PI or a PIO, the `set_delay -nopi_changes` command is required to prevent the value from changing during the capture cycle. In addition, a PIO must be forced to the input direction so it cannot be overridden by output values.

---

## DRC Rule C31

### Message Text

Clock Rule C31: Scan PLL conditioning not stable during capture.

### Severity

Error

### Description

For scan ATPG, the scan cells that control internal clocks must be stable during capture.

### What Next

Make the required constraint or design modifications to allow stability of the conditioning data through capture.

To make the conditioning data stable, it needs to either come from a stable PI or from a flop in the clock chain that retains its state during the capture clocks. This can be accomplished by wrapping the Q of these flops back to their respective D inputs.

---

## DRC Rule C34

### Message Text

Clock Rule C34: Unsensitized path between PLL source and internal clock.

### Severity

Warning

### Description

This rule checks that the conditions defined for each cycle of an internal clock must sensitize the path between the corresponding PLL source and the internal clock.

If a clock fails the C34 check, the [C35](#), [C36](#), and [C38](#) checks cannot be performed on the same clock. As a result, a C34 check might mask other violations. The other checks are still performed for clocks that do not fail the C34 check.

### Note:

In cases where `pll_source` specifications are repeated in the SPF leading to C34 violations, it is recommended that the default setting of `set_simulation` be changed to `set_simulation -nosource_disturb` to prevent the patterns from failing in simulation. Missing accurate `pll_source` specification could lead to optimistic X masking of disturbed clock groups if this switch is not used. Another solution would be to add unique `pll_source` to resolve the C34 violations for each OCC defined in the SPF. This adds proper masking and still provides the benefit of using `source_disturb` simulation.

### What Next

Use the graphical schematic viewer to trace the path between the PLL source and the internal clock to identify the cause of the sensitization problem.

Make sure the correct PLL clock location was specified for the violated internal clock. If the correspondence is not specified correctly, the C34 violation is the only consequence. The PLL clock location is specified either in the ClockStructures block of the STL procedure file, or in the `add_clocks` command.



---

## DRC Rule C35

### Message Text

Clock Rule C35: Multiple sensitizations between PLL source and internal clock.

### Severity

Error

### Description

The conditions defined for each cycle of an internal clock must sensitize the path between the corresponding PLL source and the internal clock for a single PLL clock pulse.

TestMAX ATPG expects control over the conditioning of internal clocks from one capture cycle to the next (cycle-specific conditioning).

This check cannot be performed on a clock that fails a [C34](#) check.

### What Next

Some OCC controllers use one control bit to enable a double pulse. This type of controller violates a C35 check because it is impossible to control the pulses separately. This clock conditioning is suitable for two-clock transition-delay testing using common launch and capture clocks, but not for any other purpose. If this limitation is acceptable, you can downgrade the C35 rule using the command `set_rules C35 warning`.

You should verify that the PLL clock controller can accommodate cycle-specific conditioning of the internal clocks. A PLL cycle counter is a typical solution, in which the outputs of the counter are used to gate the internal clock conditioning values.

The following examples show how to define internal clocks:

```
add_clocks 0 intclk3 -intclock -pll_source u1/pllclk3 \  
-cycle { 0 clock_chain/cell[5]/Q 1 \  
1 clock_chain/cell[4]/Q 1 }  
  
add_clocks 0 intclk2 -intclock -pll_source u1/pllclk2 \  
-cycle { 0 clock_chain/cell[3]/Q 1 \  
1 clock_chain/cell[2]/Q 1 }  
  
add_clocks 0 intclk1 -intclock -pll_source u1/pllclk1 \  
-cycle { 0 clock_chain/cell[1]/Q 1 \  
1 clock_chain/cell[0]/Q 1 }
```

A counter keeps track of the number of clock pulses from the PLL since the start of the capture cycle. It needs to keep track of as many cycles as the maximum number

defined in any one single `add_clocks` command. Each cycle number is ANDed with the corresponding clock chain bit and the results are then ORed together to enable the clock pulse to pass through the clock MUX.

```
Count (value) 1 is ANDed with bit[1]
Count (value) 2 is ANDed with bit[0]
```

---

## DRC Rule C36

### Message Text

Clock Rule C36: Mistimed sensitizations between PLL source and internal clock.

### Severity

Error

### Description

This rule checks that the conditions defined for each cycle of an internal clock must sensitize the path between the corresponding PLL source and the internal clock in order of the cycles. That is, the conditions for cycle  $n$  must sensitize the path for a PLL pulse preceding the sensitization for cycle  $n+1$ .

This check cannot be performed on a clock that fails a [C34](#) check.

### What Next

Verify that the clock controller provides for cycle-specific conditioning of the internal clocks. A typical solution is to implement a PLL cycle or pulse counter that provides one-hot values used to gate the internal clock conditioning value.

---

## DRC Rule C37

### Message Text

Clock Rule C37: Cannot satisfy all internal clocks off for all cycles.

### Severity

Error

### Description

It must be possible to condition all clocks off for all cycles; otherwise, no chain test can be created and some faults might be untestable.

### What Next

Verify that internal clock conditioning allows for all internal clocks to be turned off by loading the appropriate conditioning values into the clock chain.

---

## DRC Rule C38

### Message Text

Clock Rule C38: Bad off-conditioning between PLL source and internal clock.

### Severity

Error

### Description

This rule checks that the setting of the first condition of the cycle in which the internal clock can be set to its opposite value results in the clock off, independent of other conditions.

This check cannot be performed on a clock that fails a [C34](#) check.

### What Next

You should verify that the conditioning values provide exclusive control over the enabling of the internal clocks.

---

## DRC Rule C39

### Message Text

Clock Rule C39

Nonlogical clock *clock\_name* connects to *number* scan cells (*gate\_id*)

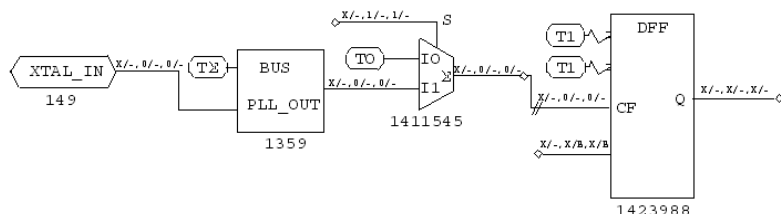
Clock Rule C39

Nonlogical clock *clock\_name* connects to *number* non-state holding pipeline cell(*gate\_id*)

### Severity

Error

One example, many variations are possible:



## Description

This violation occurs when a nonlogical clock propagates to any scan cell input. Nonlogical clocks include reference clocks and controller clocks declared with the `set_drc-controller_clock` command.

TestMAX ATPG does not simulate the effect of pulsing the nonlogical clocks on scan cells. Failure to satisfy this rule can result in patterns which fail in simulation.

The second variation of the C39 violation message occurs only for non-state holding pipeline cells with uncompressed design. In this case, the spf includes this keyword **"PipelineStateHolding "False" "**.

```
PipelineStages {
LoadPipelineStages 2;
UnloadPipelineStages 2;
PipelineStateHolding "False";
}
```

The uncompressed design with non-state holding pipeline cells can throw C39 on non-pipeline cells as well. In that case, you see the first variation of C39 violation message. C39 violation in this design occurs when pipeline clock is connected to free running `ate_clk`.

## What Next

A PLL that is not modeled as a black box is a common source of this violation. In this case, reference clocks might propagate to the scan cell through the PLL logic. You can use the `set_build-black_box` command to explicitly declare a PLL module as a black box.

If the C39 violation appears, the patterns fail simulation because clock pulses that are unsimulated by both ATPG and `run_simulation` can appear before, during, and after the ATPG capture cycles. If a C39 violation cannot be prevented, you can downgrade it to a warning using the `set_rules` command. In addition, you should use the command `add_cell_constraints XX` to both load and unload all unknown values on the affected

registers as well as the `add_capture_mask` command to mask captured value and prevent it from propagating in another cycle to another scan cell.

Also note that C39 violations that were downgraded to warnings are not saved in image files. Make sure that any violated scan cells are masked before saving the image file.

By default, TestMAX ATPG prints only one message for each clock that causes a C39 violation. You can use the `set_rules c39 warning -verbose` command to print out an expanded list of C39 clock rules violation information. In this case, every scan cell affected by a C39 violation is printed.

The following example shows the default output that result after specifying the `set_rules c39 warning` command:

```
set_rules c39 warning
report_violations C39

Warning: Nonlogical clock dft_clock connects to 4 scancells
(u1/ci_reg_3_ (827)). (C39-1)
Warning: Nonlogical clock clkref connects to 19 scancells (u8/LOCKUP
(812)). (C39-2)
```

The next example shows the output after using the `set_rules c39 warning -verbose` option. Note, in this case, that the `set_rules` command must be specified before the `run_drc` command or the violations will not be calculated using a verbose methodology.

```
set_rules c39 warning -verbose
...
run_drc
...
report_violations C39

Warning: Nonlogical clock dft_clock connects to scancell u1/ci_reg_3_
(827). (C39-1)
Warning: Nonlogical clock dft_clock connects to scancell u1/ci_reg_2_
(828). (C39-2)
Warning: Nonlogical clock dft_clock connects to scancell u1/ci_reg_0_
(829). (C39-3)
Warning: Nonlogical clock dft_clock connects to scancell u1/ci_reg_1_
(836). (C39-4)
Warning: Nonlogical clock clkref connects to scancell u8/LOCKUP (812).
(C39-5)
Warning: Nonlogical clock clkref connects to scancell u8/ai_reg_3_
(873). (C39-6)
.
.
.
```

If you see first variation of C39 rule violation in uncompressed design with non-state holding pipeline cells or second variation of C39 rule violation message, you can safely

downgrade the violation using `set_rules C39 warning` to pass DRC and VCS. You must mask the non-state holding pipeline cells by default to pass VCS.

---

## DRC Rule C40

### Message Text

Clock Rule C40

Internal clock *gate\_id* is restricted due to <reason>.

Clock Rule C40-1 Internal clock *name* is restricted due to its <time> latency not aligning with its controller's <time> latency.

### Severity

Warning

### Description

The C40 rule applies only to synchronized internal clocks (declared in a SynchronizedClocks block of the STIL procedure file). TestMAX ATPG manipulates the clock chain data for synchronized clocks to compensate for different periods or start-up latencies. This warning indicates that DRC has encountered a configuration that prevents such manipulation, and the clock is flagged as restricted. A restricted clock cannot be activated in the same pattern as another clock of the same sync group, unless both clocks have the same period and latency. Internal clocks with the same period and latency do not require special clock chain manipulation.

The C40 warning is triggered by ATPG or cell constraints on the clock chain or PLL condition nets. It is also triggered by unsupported clock controller configurations in which the one-to-one relationship between clock chain bits and internal clock pulses is violated. Some examples of unsupported configurations are shared clock chain bits (either within or across clock controllers), or complex logic driving the PLL conditions.

### What Next

You should evaluate the described constraints to see if they can be eliminated or moved away from the clock controller logic.

In most cases, you can ignore this warning and accept the clocking restrictions. This might result in pattern inflation or a reduction in fault coverage.

As a last resort, you can remove the clock from the SynchronizedClocks block of the STIL procedure file. This will eliminate all of the synchronized clock ATPG restrictions, as TestMAX ATPG is ignorant of the timing for this clock. In this case, you should be careful to simulate the patterns outside of TestMAX ATPG to validate the ATPG timing assumptions.

If you get a C40-1 message, you will need to adjust the latency for multiple synchronized clocks on the same controller, and should refer to the details specified in the corresponding M722 message.

---

## DRC Rule C41

### Message Text

Clock Rule C41:Internal clocks *gate\_id* and *gate\_id* are equivalent.

### Severity

Warning

### Description

The two clocks noted in this message have been identified as equivalent clocks, which means they will always both be active or inactive together. Equivalence markers for internal clocks are removed prior to DRC. Equivalence checks for internal clocks are now done before clock checks and It is now impossible to get ungroupable equivalent clocks.

During DRC it will show information on these clocks.

The following clocks were identified as groupable with clock CG/mux\_clk500ma (357).

CG/mux\_clk500mb (358).

### What Next

If the clocks always pulse simultaneously and are skew-balanced in the design, the patterns generated by ATPG will be correct. However, if the clocks are not both simultaneous and skew-balanced, the patterns generated with [M285](#) messages will be incorrect and will fail at test or with full-timing simulation.

Review the grouping opportunities for these clocks using the command `report_clocks-matrix`. Consider design or constraint changes to allow these clocks to be grouped.

Review the conditions or constraints on these clocks that cause them to be identified as equivalent. For instance, "always on" conditions for internal clocks will cause these clocks to be equivalent. Consider reducing these constraints to allow the clocks to be independent.

As a last resort, consider removing the definition of one or the other clock, and running DRC without the combination of clocks present. This will require multiple DRC and ATPG passes to process each clock separately.

---

## DRC Rule C43

### Message Text

Scan enable (name = <scan\_enable\_name>, id = <G\_id>, (got/exp) <E>/<G>) does not facilitate capture after <N> delay cycles.

**Scan\_enable\_name** indicates the instance name of the scan\_enable gate.

**G\_id** indicates gate id of the scan\_enable gate

**N** indicates expected delay-capture cycles after load\_unload before capture occurs(specified as se\_delay in protocol file)

**E** is the expected value on scan\_enable gate after N delay-capture cycles

**G** is the value on the scan\_enable gate after N delay-capture cycles

### Severity

Error

### Description

After the load-unload procedure, the scan\_enable gate (specified in the protocol file) is checked to have the expected value (specified in the protocol file) after se\_delay (specified in the protocol file) number of delay-capture cycles. The C43 violation is displayed if the scan\_enable gate does not have the expected value after specified number of delay-capture cycles.

### What Next

To correct this problem, the netlist or protocol file needs to be changed. You can use the automated violation analysis to debug this problem. This analysis displays the scan\_enable gate when the pindata is set to "set\_pindata -se\_delay" and contains the simulated values of the capture-delay cycles.

This rule can be downgraded but it is not recommended as the generated patterns may be incorrect.

---

## DRC Rule C44

### Message Text

Scan enable (name = <scan\_enable\_name >, id = <G\_id>, (got/exp) <E>/<G>) facilitate capture at <N1> delay cycles but expected after <N2> delay cycles.

**Scan\_enable\_name** indicates the instance name of the scan\_enable gate



**G\_id** indicates gate id of the scan\_enable gate

**N1** indicates current delay-capture cycle at which capture occurs

**N2** indicates expected delay-capture cycle at which capture should occur(specified as se\_delay in protocol file)

**E** is the expected value on scan\_enable gate at N1 delay-capture cycle

**G** is the value on the scan\_enable gate at N1 delay-capture cycle

### Severity

Error

### Description

After the load-unload procedure, scan\_enable gate (specified in the protocol file) is checked to have the expected value (inverted of the value specified in the protocol file) during all the delay-capture cycles except the last delay-capture cycle. C44 violation is thrown if scan\_enable gate has incorrect value during these delay-capture cycles.

### What Next

To correct this problem, netlist or protocol file needs to be changed. The automated violation analysis can be used to debug this problem. This analysis displays the scan\_enable gate and pindata is set to "set\_pindata -se\_delay", which has the simulated values of the capture-delay cycles.

This rule can be downgraded but not recommended as the generated patterns may be incorrect.

# 4

## Category D - DFT Rules

---

### See Also

- D1 - clock input was not controlled
- D2 - set input was not controlled
- D3 - reset input was not controlled
- D4 - clock input was not controlled
- D5 - set input was not controlled
- D6 - reset input was not controlled
- D7 - clock input RAM was not controlled
- D8 - clock cannot capture data
- D9 - clock input not active when set on
- D10 - clock connects to data input
- D11 - clock connects to clock and data inputs
- D12 - clock connects to clock/set/reset inputs
- D13 - clock can capture new data on LS input
- D14 - clock can capture new data on TE input
- D15 - clock affected by new capture on LS input
- D16 - clock affected by new capture on TE input
- D17 - clock input couldn't capture data
- D18 - multiple clock/reset/set inputs (G1/G2) of DFF cell are active
- D20 - bus gate failed contention check
- D21 - bus gate failed state check
- D22 - wire gate failed contention check

- [D23 - feedback path sensitizable through source gate](#)
- [D39 - probability of capture X \(P\) exceeds threshold for scancell](#)

---

## DRC Rule D1

### Message Text

DRC Rule D1: Clock input I of DFF S was not controlled

### Severity

Warning

### Description

When all clock ports are in their off state, all clock inputs of flip-flops must also be in their off state.

Clock input I is the uncontrolled input pin on the cell S. In some cases, pin I is not displayed when the DRC engine cannot point to the exact pin. This occurs when the sequential cell in question is driven by multiple clock inputs.

This check is performed using the simulated values that result when:

Clocks are in their off state

Constrained ports are set to their constrained values

Constant-value nonscan cells are set to their constant state

This rule violation occurs if any clock input of a flip-flop is not in its off state.

If the scan style is a multiplexed flip-flop, violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

If the scan style is other than multiplexed flip-flop, this violation is flagged only when the violation occurs on the functional clock, and will not have any effect even if it is flagged.

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

A common cause of this violation is an incorrect or missing clock definition. Another common cause is a clock signal that is blocked from reaching the input of a scan cell.

Check the signal type specifications, the protocol and the design.

---

## DRC Rule D2

### Message Text

DRC Rule D2: Set input I of DFF S was not controlled

### Severity

Warning

### Description

When all clock ports are in their off state, all set inputs of flip-flops must be in their off state.

I is the uncontrolled input pin to the cell S. The pin I might not be displayed in some cases in which the DRC engine cannot point to the exact pin. This will occur if the sequential cell in question is driven by multiple clock inputs.

This check is performed using the simulated values that result when:

Clocks are in their off state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any set input of any sequential cell is not in its off state.

Violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

A common cause of this violation is an incorrect or missing clock definition. Another common cause is a clock signal that is blocked from reaching the input of a scan cell. Check the signal type specifications, the protocol and the design.

---

## DRC Rule D3

### Message Text

DRC Rule D3: Reset input I of DFF S was not controlled

### Severity

Warning

### Description

When all clock ports are in their off state, all reset inputs of scannable flip-flops must be in their off state.

S is the scan cell-name and I is the input pin that is not controlled. The pin I might not be displayed in some cases where the DRC engine cannot point to the exact pin. This will occur if the sequential cell in question is driven by multiple clock inputs.

This check is performed using the simulated values that result when:

Clocks are in their off state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any set input of any sequential cell is not in its off state.

Violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

A common cause of this violation is an incorrect or missing clock definition. Another common cause is a clock signal that is blocked from reaching the input of a scan cell. Check the signal type specifications, the protocol and the design.

---

## DRC Rule D4

### Message Text

DRC Rule D4: Clock input I of DLAT S was not controlled

### Severity

Warning

### Description

When all clock ports are in their off state, all latch clock inputs must also be at their off state.

S is the scan cell-name and I is the input pin that is not controlled. The pin I might not be displayed in some cases where the DRC engine cannot point to the exact pin. This will occur if the sequential cell in question is driven by multiple clock inputs.

This check is performed by simulating the logic values that result when:

Clocks are in their off state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any clock input of a latch is not in its off state.

Violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

If the scan style is other than multiplexed flip-flop, this violation is flagged only if the violation occurs on the functional clock. There is no effect from the flagging.

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

---

## DRC Rule D5

### Message Text

DRC Rule D5: Set input I of DLAT S was not controlled

### Severity

Warning

### Description

When all clock ports are in their off state, all latch clock inputs must be at their off state also.

S is the cell-name and I is the input pin that is not controlled. The pin I might not be displayed in some cases where the DRC engine cannot point to the exact pin. This will occur if the sequential cell in question is driven by multiple clock inputs.

This check is performed by simulating the logic values that result when:

Clocks are in their off state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any clock input of a latch is not in its off state.

Violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

---

## DRC Rule D6

### Message Text

DRC Rule D6: Reset input I of DLAT S was not controlled

### Severity

Warning

### Description

When all clock ports are in their off state, all latch clock inputs must be in their off state also.

S is the cell-name and I in the input pin that is not controlled. Pin I might not be displayed in cases where the DRC engine cannot point to the exact pin. This occurs if the sequential cell in question is driven by multiple clock inputs.

This check is performed by simulating the logic values that result when:

Clocks are at their off state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any clock input of a latch is not at its off state.

Violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

---

## DRC Rule D7

### Message Text

DRC Rule D7: Clock input I of RAM M was not controlled

### Severity

Warning

### Description

M is the name of the memory element and I is the input pin that is not controlled.



When all clock ports are in their off state, all clock/set/reset inputs of memory elements must be in their off state.

The off state of a clock PI is the value 0 or 1 in which the cell driven by the PI is stable.

This check is performed using the simulated values that result when:

Clocks are in their off state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any clock/set/reset input of a memory element is not in its off state.

Violation of this rule has the following effects:

The cell is violated

Memory cells are considered black boxes unless a simulation model is provided for them. Use the `test_simulation_library` variable to provide a simulation model.

Can lead to C20 violations in post-DFT DRC

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check your design, model, and specification. A common cause of this violation is a primary input that controls memory writes not defined as a clock.

---

## DRC Rule D8

### Message Text

DRC Rule D8: Clock C cannot capture data with other clocks off

### Severity

Warning

### Description

Each defined clock port must be capable of turning on a clock, set, reset, or `write_control` input of in least one sequential device or memory element when all other clocks are off. C is the name of the port.

This check is performed using the simulated values that result when:

The specified port (and any equivalent port) is set to X

All other defined clocks or asynchronous sets and resets are set to their off states

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

The rule violation occurs if all clock/set/reset/write inputs of all sequential elements (including RAMs) are off during this condition. In addition, this violation can be caused by any of the following:

Incorrect or missing clock definition

A clock signal is blocked from reaching the input of the internal sequential cell or memory element. This can occur when the signal passes through an AND gate and the control size is 0, or an OR gate with a control size of 1.

If the instance is a nonscan device and there is another pin on the device such as an asynchronous set or reset that is controlled from a primary input that has not been correctly defined. For example, a CLK pin has been defined as a clock but a RESET pin has not been defined as a reset. Consequently, data cannot be captured when the CLK pin is pulsed and RESET=X. Correct this by defining the missing signal.

Violation of this rule has the following effects:

Failure to satisfy this rule indicates the specified port cannot perform a capture operation

It does not violate any cells but can lead to C4 violations is post-DFT DRC

Note the following:

Pipeline register clocks should have D8 violations if they are definitely dedicated clocks

Reference clocks are not subject to D8 violations

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check the signal-type specifications, the protocol and the design.

---

## DRC Rule D9

### Message Text

DRC Rule D9: Clock input I of <type> S not active when clocks are set on.

## Severity

Warning

## Description

When all clock ports are in their on state, none of the clock inputs of sequential elements must be inactive (X).

*S* is the cell name and *I* is the input pin that is not controlled. The *<type>* is either DLAT (latch) or DFF (flip-flop). The pin *I* might not be displayed in some cases in which the DRC engine cannot point to the exact pin.

This check is performed using the simulated values that result when:

Clocks are in their on state

Constrained ports are set to their constrained values

Constant value nonscan cells are set to their constant state

This rule violation occurs if any clock input of a flip-flop is not active. This rule checks the scan controllability of a sequential cell.

If the scan style is multiplexed flip-flop, violation of this rule has the following effects:

The cell is violated

The cell is not included in scan-chains

DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications

If the scan style is other than multiplexed flip-flop, this violation is flagged only if the violation occurs on the functional clock, and does not have any effect even if it is flagged.

## What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

A common cause of this violation is an incorrect or missing clock definition. Another common cause is a clock signal that is blocked from reaching the input of a scan cell. For example, the clock signal could be blocked by a clock-gating cell that does not have test mode connections that put it into a pass-through mode during test. Check the signal type specifications, the protocol, and the design.

A less common cause of this violation is if the registers are clocked by a clock that has been specified as an ATE clock for on-chip clocking (OCC) controller insertion using the DFT Compiler command `set_dft_signal -type Oscillator`. In this case, the ATE clock is free-running so that the clock for the registers is uncontrollable for scan ATPG.

purposes. This type of dual usage of the clock should be avoided. But if there are no available clocks or ports to use as an ATE clock, there is a workaround.

The workaround is to remove the `set_dft_signal -type Oscillator` specification for the ATE clock. This will result in post-DFT DRC failing with S1 violations, but the resulting netlist and protocol are usable. The ATE clock must be specified in the TestMAX ATPG command file with the command `add_clocks 0 ate_clock -refclock`. The resulting S18 and C39 errors must be downgraded, and command `set_rules S18 warning -mask` must be used to prevent miscompares in Verilog resimulation.

**Warning:** Although this workaround enables stuck-at fault testing for these flip-flops, it causes a significant loss of transition delay fault coverage in ATPG.

---

## DRC Rule D10

### Message Text

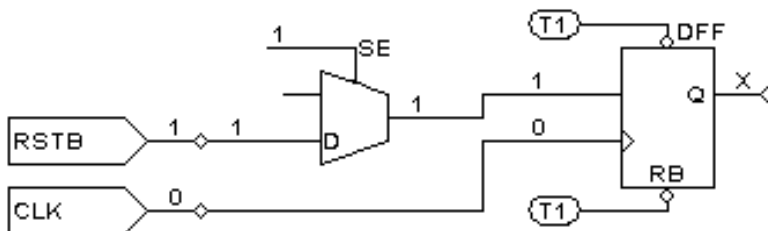
DRC Rule D10: Clock C connects to data input of <type> S

### Severity

Warning

### Example

One example, many variations are possible:



### Description

A clock has a path that can be propagated to a data input of a DFF/DLAT when PI constraints and constant value cells are set. The data input of a sequential cell was found to have an unblocked path to a primary input declared as a clock, and this is a different clock than the one connected to the clock input of the cell. C is the clock port, <type> is either DFF (flip-flop) or DLAT (latch), and S is the cell-name.

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Also make sure the pin data that it uses the clock is on.

Check your design and specifications.

---

## DRC Rule D11

### Message Text

DRC Rule D11: Clock C connects to clock and data inputs I1/I2 of <type> S

### Severity

Warning

### Description

A clock has a combinational path to both the data and clock/set/reset inputs of a sequential cell. C is the clock port name; I1/I2 are the input pins; S is the sequential cell name. A clock must not capture data for a sequential cell whose data line is affected by the same clock.

This check is performed using clock cone data. A rule violation occurs when a clock/set/reset input and its data input are in the same clock cone. A clock cone is the cone of combinational logic fanning out from a single clock input port and extending to one or more sequential device input pins or a primary output port.

Violation of this rule has the following effects:

Can result in a race condition

This does not violate a cell but it can lead to C11/C12/C13 violations in post-DFT DRC

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check the design and specifications. It can be possible to block the path from the clock to the data input as it passes through combinational logic by applying a constraint to a net connecting to the other input of one of these combinational gates. For example, holding the other input of an AND gate to zero will block the path of the clock through that gate.

---

## DRC Rule D12

### Message Text

DRC Rule D12: Clock C connects to clock/set/reset inputs (G1 / G2) of DFF I

### Severity

Warning

### Description

A common path (combinational gates) exists from a defined clock port to more than one clock input of a sequential device. A clock input includes any asynchronous set or reset control inputs as well as data clocks. C is the name of the clock, G1 and G2 are the clock or set or reset pins to the cell I, and I is the instance pathname to the device.

Failure to pass this rule check indicates there is a potential race condition because a clock port has a combinational gate path to two different clock inputs of the same ATPG sequential primitive.

If the scan style is a multiplexed flip-flop, violation of this rule has the following effects:

- The cell is violated.
- The cell is not included in scan-chains.
- DFT insertion unscans the cell if it has already been scan-replaced, depending on user specifications.

If the scan style is other than a multiplexed flip-flop, this violation is flagged only if the violation occurs on the functional clock, and does not have any effect even if it is flagged.

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

You might choose to ignore this rule. If you do, you might need to mask this instance during ATPG to avoid potential mismatches. A timing analysis or simulation will guide you with respect to this masking.

---

## DRC Rule D13

### Message Text

DRC Rule D13: Clock C can capture new data on LS input I1 of <type> S1  
Source of violations: input I2 of <type> S2.

### Severity

Warning

### Description

A clock must not capture data into a level sensitive (LS) input (latch or RAM) if that data can be affected by new captured data. C is the clock port; the <type> is either DLAT (latch) or DFF (flip-flop); I1 and I2 are input pin names; S1 and S2 are cell names.

A potential rule violation occurs on a clock if a clock input of a LS pin is in the clock cone and its data input (including RAM address) is in the effect cone.

A clock cone is the cone of combinational logic fanning out from a single clock input port and extending to one or more sequential device input pins or a primary output port.

An effect cone is the cone of combinational logic fanning out from the output pin of a sequential device (clocked by CLK1 for example) and extending through all combinational logic until it reaches other sequential devices or primary output ports. An effect cone is always relative to a specific clock (CLK1) connected to the clock/set/reset pin of a sequential device.

Violation of this rule has the following effects:

This is a capture violation

It does not violate any cells but can lead to C5 violations in post-DFT DRC

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check the design and specifications.

---

## DRC Rule D14

### Message Text

DRC Rule D14: Clock C can capture new data on TE input I1 of <type> S1

Source of violations: input I2 of <type> S2.

### Severity

Warning

### Description

A clock must not capture data into a trailing edge (TE) input if that data can be affected by new captured data. C is the clock port; the <type> is either DLAT (latch) or DFF (flip-flop); I1 and I2 are input pin names; S1 and S2 are cell names.

A potential rule violation occurs on a clock if a clock input of a TE pin is in the clock cone and its data input (including RAM address) is in the effect cone. This input is called the sink of the violation.

To be considered a violation, all of the following are true:

The source must not be a TE pin

The source clock must be capable of being at 1 (0 if LE) when the sink clock is at 1

It must be possible to propagate the path from source to sink under this condition

A clock cone is the cone of combinational logic fanning out from a single clock input port and extending to one or more sequential device input pins or a primary output port.

An effect cone is the cone of combinational logic fanning out from the output pin of a sequential device (clocked by CLK1 for example) and extending through all combinational logic until it reaches other sequential devices or primary output ports. An effect cone is always relative to a specific clock (CLK1) connected to the clock/set/reset pin of a sequential device.

Failure to satisfy this rule does not violate any cells, but it will lead to C6 violations in post-DFT DRC. This is a capture violation.

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check the design and specifications.

---

## DRC Rule D15

### Message Text

DRC Rule D15: C clock path affected by new capture on LS input I1 of <type> S1

Source of violation: input I2 of <type> S2.

### Severity

Warning

### Description

The path from a clock to a level sensitive (LS) input must not be affected by its new captured data.



C is the clock port; the <type> is either DLAT (latch) or DFF (flip-flop); I1 and I2 are input pin names; S1 and S2 are cell names.

This check is performed using clock cone data and a potential rule violation occurs on a clock if a clock input of a level sensitive pin is in both the clock cone and effect cone. This input is called the sink of a violation. To be considered a violation, the source must not be a TE input, the source clock must be capable of being at 1 when the sink clock is at 1, and it must be possible to propagate the path from source to sink under this condition.

A clock cone is the cone of combinational logic fanning out from a single clock input port and extending to one or more sequential device input pins or a primary output port.

An effect cone is the cone of combinational logic fanning out from the output pin of a sequential device (clocked by CLK1 for example) and extending through all combinational logic until it reaches other sequential devices or primary output ports. An effect cone is always relative to a specific clock (CLK1) connected to the clock/set/reset pin of a sequential device.

Violation of this rule has the following effects:

This is capture violation

Failure to satisfy this rule can result in a race condition that can eventually lead to C8 violations in post-DFT DRC

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check the design and specifications.

---

## DRC Rule D16

### Message Text

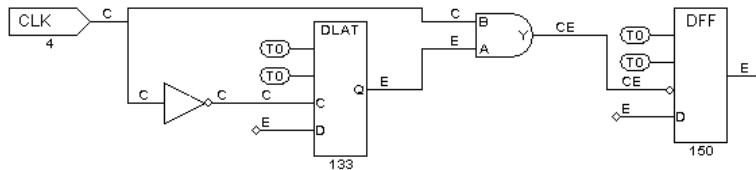
DRC Rule D16: C clock path affected by new capture on TE input I1 of <type> S1.

Source of violation: input I2 of <type> S2.

### Severity

Warning

One example, many variations are possible:



## Example

### Description

The path from a clock to a trailing edge input must not be affected by its new captured data.

C is the clock port; the <type> is either DLAT (latch) or DFF (flip-flop); I1 and I2 are input pin names; S1 and S2 are cell names.

Failure to satisfy this rule results in an additional simulation pass to allow the trailing edge (TE) port to capture new data from leading edge (LE) or level sensitive (LS) ports. There is also risk that test generation will not correctly control the new value that is captured, which will result in some loss of test coverage.

This check is performed using clock cone data and a rule violation occurs on a clock if a clock input of a TE port is in both the clock cone and effect cone. This port is called the sink port of a violation. To be considered a violation, the source port must not be a TE port, the source port clock must be capable of being at 1 (0 if LE) when the sink port clock is at 1, and the path from source to sink must be propagatable under this condition.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the ANALYZE button. This changes the pin data reporting to the clock cone data and displays all gates in the paths from the source port to the sink port, the clock to the source port, and the clock to the sink port.

A list of all potential source ports for the violation is identified by tracing back from the sink clock line through gates in both the clock and its effect cone.

Use the `-mask` option of the `set_rules` command to compensate for the DRC violation. By setting the rule severity to warning and enabling the `-mask` option, this causes the ATPG algorithm to mask off the observe values, in an attempt to ensure that any ATPG pattern generated will pass simulation. The tradeoff is that in doing this, there is a reduction of test coverage.

---

## DRC Rule D17

### Message Text

DRC Rule D17: Clock input I of <type> S couldn't capture data

### Severity

Warning

### Description

Each clock/set/reset input of a state element must be capable of capturing data when a single clock is on and all other clocks are off. I is the clock/set/reset input, <type> is either DLAT (latch) or DFF (flip-flop), and S is the name of the cell. The input I might not be displayed if DRC cannot identify the exact pin.

This check is performed using the simulated values that result when:

One clock is set to X

All other clocks are in their off state

The constrained pins are set to the constrained values

The constant-value nonscan cells are set to their constant values.

The violation occurs when the clock/set/reset input of a sequential cell is off rather than X.

The off state of a clock PI is the value 0 or 1 at which the sequential cell driven by the PI is stable.

If the scan style is multiplexed flip-flop, violation of this rule has the following effects:

The cell is violated

The cell is not included in scan chains

TestMAX DFT unscans the cell if it has already been scan replaced, depending on user specifications

If the scan style is other than multiplexed flip-flop, this violation is flagged only if the violation occurs on the functional clock, and will not have any effect even if it is flagged

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

A common cause of this violation is a clock pin tied to a constant. Check your design and specifications.

---

## DRC Rule D18

### Message Text

DRC Rule D18: Multiple clock/reset/set inputs (G1/G2) of DFF <cell\_name> are active.

### Severity

Warning

### Description

More than one clock input of a sequential device is active at the same time. A clock input can include any asynchronous set or reset control inputs or data clocks. G1 and G2 are the clock or set or reset pins to the cell <cell\_name>, which includes the instance pathname to the device. Failure to pass this rule check indicates there is a potential race condition.

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

---

## DRC Rule D20

### Message Text

DRC Rule D20: Bus gate N failed contention ability check for drivers G1 and G2

### Severity

Warning

### Description

A bus must not be capable of being in a contention state when constraints are set.

N is the instance name of the bus device, and G1 and G2 are two drivers that could contribute to a contention situation. The analysis will stop after identifying two nets in a possible contention state.

This check does not indicate that contention is occurring, only that it is possible. Therefore, monitor the bus closely.

There should be no affect on test coverage.

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check your design and eliminate the contention if possible.

---

## DRC Rule D21

### Message Text

DRC Rule D21: Bus gate N failed Z state ability check

### Severity

Warning

### Description

A bus must not be capable of being in a Z state when constraints are set.

Failure to satisfy this rule will not violate cells or affect test coverage. However, internal Z states become Xs as they pass through other gates and can lead to an increase in tester mask requirements if the Xs make their way into scan chains or outputs.

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check your design and eliminate the Z state if possible.

---

## DRC Rule D22

### Message Text

DRC Rule D22: Wire gate N failed contention ability check for drivers G1 and G2

### Severity

Warning

### Description

A wired-and net must not be capable of being at different state when constraints are set.

Violation of this rule has the following effects:

An X state on the wired-and output, which can lead to a loss of test coverage

The contending condition can also cause harmful device stress if the non-tristate drivers are allowed to be in contention

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check your design and eliminate the contention if possible. Remove the wired-and structures.

---

## DRC Rule D23

### Message Text

DRC Rule D23: Feedback path network X is sensitizable through source gate G

### Severity

Warning

### Description

Feedback paths must not be sensitizable. A combinational feedback path was also found to be sensitizable. In other words, there is a feedback loop that completes the path back on itself and results in a potential oscillation or ring driver.

Failure to satisfy this rule indicates a potential sensitizable feedback path. You must also consider the effect of nonscan cells with constant values or user-specified constraints. Investigate further to determine whether there is a truly sensitizable feedback path.

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check your design. Eliminate the feedback loop if necessary.

---

## DRC Rule D39

### Message Text

DRC Rule D39: Probability of capture X (P) exceeds threshold for scancell  
<gate\_type> gate\_id.

### Severity

Warning

### Description

The probability of capture  $x$  exceeds the threshold for the scan cell. In the message,  $P$  is the percent probability (0 -100). This rule checks for scan cells with data input that has a probability of  $X$  that equals or exceeds the threshold value. If the `set_drc-xchain_threshold_probability` option is set to 0 (the default), or the severity is set to `ignore`, this rule will not be checked. The scan cells checked will all be DFFs which pass rules [D1](#), [D2](#), and [D3](#), or cells which have been selected by the `set_scan_ability` command.

### What Next

You can analyze a rule violation by clicking on the corresponding violation number inside the Violation Browser.

Check the design and specifications.

# 5

## Category K - LBIST Rules

---

### See Also

- [K1](#)
- [K2](#)
- [K3](#)
- [K4](#)
- [K5](#)
- [K6](#)
- [K7](#)
- [K8](#)
- [K9](#)
- [K10](#)
- [K11](#)
- [K12](#)
- [K13](#)
- [K14](#)
- [K15](#)
- [K16](#)
- [K17](#)
- [K18](#)
- [K19](#)
- [K20](#)
- [K21](#)



- [K22](#)
- [K23](#)
- [K24](#)
- [K25](#)
- [K26](#)
- [K27](#)
- [K28](#)
- [K29](#)
- [K30](#)
- [K31](#)
- [K32](#)
- [K33](#)
- [K34](#)
- [K35](#)
- [K36](#)
- [K37](#)
- [K38](#)
- [K39](#)
- [K40](#)
- [K41](#)
- [K42](#)
- [K43](#)
- [K44](#)
- [K45](#)
- [K46](#)
- [K47](#)
- [K48](#)

- [K49](#)
- [K50](#)
- [K51](#)
- [K52](#)
- [K53](#)
- [K54](#)
- [K55](#)
- [K56](#)
- [K57](#)
- [K58](#)
- [K59](#)
- [K60](#)
- [K61](#)
- [K62](#)
- [K63](#)
- [K64](#)
- [K65](#)
- [K66](#)
- [K67](#)
- [K68](#)
- [K69](#)
- [K70](#)
- [K71](#)
- [K72](#)
- [K73](#)
- [K74](#)
- [K75](#)

- [K76](#)
- [K77](#)
- [K78](#)
- [K79](#)
- [K80](#)
- [K81](#)
- [K82](#)
- [K83](#)
- [K84](#)
- [K85](#)
- [K86](#)
- [K87](#)
- [K88](#)
- [K89](#)
- [K90](#)
- [K91](#)
- [K92](#)
- [K93](#)
- [K94](#)
- [K96](#)
- [K97](#)
- [K98](#)
- [K99](#)
- [K100](#)
- [K101](#)
- [K102](#)
- [K103](#)

- [K104](#)
- [K105](#)
- [K106](#)
- [K107](#)
- [K112](#)
- [K119](#)
- [K120](#)

---

## DRC Rule K1

### Message Text

Seq compressor chain length L1 did not equal expected length L2.

Where:

L1 indicates calculated length of `codec_drc_chain` and `occ_chain(optional)`

L2 indicates expected length of `codec_drc_chain` and `occ_chain(options)` from the STIL protocol file

### Severity

Fatal Error

### Description

Seq compressor chain includes `codec_drc_chain` and `occ_chain (optional)`. The `codec_drc_chain` is traced back from its defined chain output port using the simulated values of the `CODEC_DRC_LOAD_UNLOAD` and `CODEC_DRC_SHIFT` procedures. `occ_chain` is also traced back from its defined chain output port using the simulated values of `OCC_LOAD` and `OCC_SHIFT` procedures. If the number of scan cells identified by this tracing does not equal the expected number of cells, then a violation of this rule is issued. The expected number of cells in the particular `seq_compressor_chain(codec_drc_chain)` is the sum of the following which are defined in the STIL protocol file:

Length of the `occ_chain` register (optional - may be zero)

Lengths of the registers on `codec_drc_chain`:

Registers of CODEC:

lengths of the CarePRPG registers

lengths of the XTOLPRPG registers

lengths of the MISR registers

lengths of the PRPGShadow registers

lengths of the CARE\_shadow registers  
lengths of the XTOLShadow registers  
lengths of the XTOL\_enable registers (always 1)  
length of the Power\_enable register (1 if length is not specified)  
length of the SERDESReg  
length of the MaskEnable (always 1)  
length of the XchainsRegister  
Registers of IDU:  
  lengths of SERDESRegs  
  lengths of Address registers  
  lengths of Broadcast registers(always 1)  
  lengths of New\_address\_shadow registers(always 1)  
  lengths of Final\_segment registers(always 1)  
  lengths of Shift\_counter\_shadow registers  
  lengths of Seed\_type registers (always 1)  
  lengths of Capture\_enable\_shadow registers (always 1)  
  lengths of Current\_address registers  
  lengths of First\_address registers  
  lengths of PatternStart registers (always 1)  
  lengths of New address registers  
  lengths of NewAddressCyclesReg registers  
  lengths of IncrAddressCyclesReg registers  
  lengths of OCCCyclesReg registers  
  lengths of OCCLocalCyclesReg registers  
  lengths of MISRCyclesReg registers  
  lengths of PayloadCyclesCounter registers  
  lengths of LocalCyclesCounter registers  
  lengths of CumulativeCodecEnable registers  
  lengths of CodecEnable registers  
  lengths of Shift\_counter registers  
  lengths of Capture\_enable registers  
  lengths of FirstShift registers (always 1)  
  lengths of MaskAllChains registers (always 1)

## What Next

The severity of this violation is a fatal error, so it must be corrected before an ATPG (Automatic Test Pattern Generation) can be performed. To correct the problem, identify which of the register lengths are incorrectly defined in the STIL protocol file and edit them to specify proper values. There is no automated violation analysis that can be used to help debug this problem. The `-codec_drc_chain_cells` option of the `report_compressors` command can be used to display an ordered list of all the cells in the `codec_drc_chain`. The `-cells` option of the `report_compressor` and `report_idus` can be used to display the cells and lengths of various registers of CODEC and IDU respectively. The `-occ_chain_load_segments` option of the `report_compressor` can be used to report the `occ_chain` cells.

---

## DRC Rule K2

### Message Text

DFTMAX LBIST Rule K2: Invalid clock capture edge for compressor cell  
Invalid clock capture edge was used for T seq compressor cell G.

### Severity

Fatal Error

### Description

Note the following:

T indicates register type

G indicates gate ID of failing register cell

The codec\_drc\_chain is traced back from its defined chain output port using the simulated values of the CODEC\_DRC\_LOAD\_UNLOAD and CODEC\_DRC\_SHIFT procedures. The edge on which each register type captures data during the CODEC\_DRC\_SHIFT procedure is checked to ensure that it is correct. The following registers are checked for the given edge:

CarePRPG cells — must capture on leading edge

XTOLPRPG cells — must capture on leading edge

MISR cells — must capture on leading edge

PRPGShadow cells — must capture on leading edge

CARE\_shadow cells — must capture on leading edge

XTOLShadow cells — must capture on leading edge

XTOL\_enable cells — must capture on leading edge

occ\_chain cells — not checked

Power\_enable cells — not checked

### What Next

To correct this problem, you must edit the netlist or the STL procedure file so that the failing cells capture data on the required edge. You can use automated violation analysis to debug this problem. This analysis displays the failing cell with the pin data set to the simulated values of the CODEC\_DRC\_SHIFT procedure.

You can use the `-cells` option of the `report_compressors` command to report the cells for a selected register type.

You can use the `codec_drc_shift` option of the `set_pindata` command to display the simulated values of the `codec_drc` procedure.

This rule violation can be downgraded to a warning. But this is not recommended because this violation is likely to cause the data transfer of the compression circuitry to be out of sync and corrupt any patterns created by TestMAX ATPG.

---

## DRC Rule K3

### Message Text

```
DFTMAX LBIST Rule K3: PRPG output could not be identified  
Hold output(P) of CarePRPG in compressor S could not be identified.  
Hold output of XTOLPRPG in compressor S could not be identified.  
Output N of XTOLPRPG in compressor S could not be identified.
```

### Severity

Fatal Error

### Description

Note the following:

`S` indicates the name of failing DFTMAX LBIST CODEC

`P` is the Power Enable Register Value setting for which hold output is not identified

`N` is the failing output ID

The nodes corresponding to the CarePRPG hold output, the XTOLPRPG hold output, and the XTOLPRPG observe mode selection outputs are inferred based on defined connections to their associated PRPGs. This inference is accomplished by performing parallel (32) pattern simulation using the simulated values of the SHIFT procedure and by placing random values on the PRPG cells. Gates connected to the cone of influence of the PRPGs cells are inspected to identify any gate which is are the expected values or the inverse value. Expected values are calculated by exclusive-OR in the values assigned to the associated connecting PRPG cells. A violation is issued if a gate cannot be found at the expected value.

### Note:

While inferring CAREPRPG hold output, all the `pwr_enable` settings are used to validate the inferred hold output.

For CAREPRPG, the expected values are calculated based on Low Power Hardware versions (detected and informed during DRC) as follows:

For V1:

```
HOLD =  
((Pwr_enable[0] && Pwr_ctrl[0]) ||  
(Pwr_enable[1] && (Pwr_ctrl[1] || Pwr_ctrl[2])) ||  
(Pwr_enable[2] && (Pwr_ctrl[3] || Pwr_ctrl[4] || Pwr_ctrl[5] ||  
Pwr_ctrl[6])))
```

For V2:

```
Pwr_enable=0: HOLD=0  
Pwr_enable=1: HOLD=(Pwr_ctrl[0] && Pwr_ctrl[3]) && !shadow_to_careprpg  
Pwr_enable=2: HOLD=((Pwr_ctrl[0]||Pwr_ctrl[1])&&Pwr_ctrl[3])  
    && !shadow_to_careprpg  
Pwr_enable=3: HOLD=(Pwr_ctrl[0]) && !shadow_to_careprpg  
Pwr_enable=4:  
    HOLD=(Pwr_ctrl[0]||!(Pwr_ctrl[1]||Pwr_ctrl[2]||Pwr_ctrl[3]))  
    && !shadow_to_careprpg  
Pwr_enable=5: HOLD=(Pwr_ctrl[0]||!(Pwr_ctrl[1]||Pwr_ctrl[2]))  
    && !shadow_to_careprpg  
Pwr_enable=6: HOLD=(Pwr_ctrl[1]||Pwr_ctrl[2]) && !shadow_to_careprpg  
Pwr_enable=7: HOLD=(Pwr_ctrl[0]||Pwr_ctrl[1]||Pwr_ctrl[2])  
    && !shadow_to_careprpg
```

## What Next

To correct this problem, edit the netlist or the STL procedure file to identify the failing PRPG output. You can use automated violation analysis to help debug this problem. This analysis displays the PRPG cells that are connected to the failing PRPG output. The displayed pin data is set to the simulated values of the SHIFT procedure with binary values assigned to the connecting PRPG cells. Gates in the fanout cone of the PRPG cells that have binary values are also displayed.

You can use the `-hold_connections` option of the `report_compressors` command to report the connecting PRPG cells.

---

## DRC Rule K4

### Message Text

```
DFTMAX LBIST Rule K4: Chain output could not be identified  
  
Output of chain C in compressor S could not be identified.
```

### Severity

Fatal Error



## Description

Note the following:

c indicates name of failing chain

s indicates name of DFTMAX SEQ CODEC used for failing chain

The nodes corresponding to the outputs of internal chains are inferred based on their defined connections to their associated MISR during single observe mode conditions. This inference is done by performing simulation using the simulated values of the SHIFT procedure, and placing the XTOL\_enable cells to their on state, and XTOLPRPG outputs to select the single observe mode of the given chain. A traceback is performed from the data input of the first connecting MISR cell through gates set to X identifying potential chain output cell candidates. If no candidates or more than one candidate can be found, a violation is issued.

## What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that the failing chain output can be identified. The automated violation analysis can be used to help debug this problem. This analysis will display the first connecting MISR cell for the failing chain. The displayed pin data is set to the simulated values of the SHIFT procedure with XTOL\_enable cells set on and XTOLPRPG outputs set to the single observe mode of the failing chain. Gates in the traceback cone from the first connecting MISR cell which are set to X are also displayed.

After you pull a K4 violation up in the GSV, start tracing back from the MISR bit on the right. That bit should have an x on its D input (but from the scan chain, not from an X (unknown) driving the scan chain...). 0 and 1 are wrong (bad) values. Following back in the circuit from this bit, you will run into another MISR bit. That is the wrong path. Keep going back on the other path (of the XOR) until you get deep into the XOR tree of the compactor. These XORs lead back to AND or OR gates. These are the masking gates. MOST of these AND/OR gates will have a 0/1 on one leg and an X on the other. The 0s and 1s are BLOCKING the x. The x is from a scan chain. Find the AND/OR gate with a 1/0 on one leg and a 0/1/X on the other. X (as I mentioned) is "good" unless it is from a real UNKNOWN... 0 and 1 on the scan data path are likely from static values or reset signals.

Following the XOR tree back goes to a lot of AND and OR gates (the mask gates, as mentioned). You are looking for the non-XOR gates which have NO "x" on the inputs. Usually, that x is being blocked (a good thing) by the other gate inputs (0 on AND, 1 on OR). You will eventually find an XOR fed by a gate that has all 1s and 0s on the inputs. Or a passing value on most inputs, but an X on the last. This is an "unknown" X value.

In summary: `analyze_violation`; trace XOR tree until you hit some OTHER logic gates; see if these gates are set to BLOCK or PASS a value. If passing a value, that value should be x. If it IS an X (all the way to the initial MISR bit pulled up by `analyze_violation`) then why is it X? If a 1 or 0, why is it 1 or 0 and not x?

---

## DRC Rule K5

### Message Text

Invalid MISR capture value for no\_observe mode

N MISR cells (P/G) in compressor S had invalid capture data for no\_observe mode.

N MISR cells (P/G) in seq compressor S had invalid capture data for MASK\_ENABLE no\_observe mode.

### Severity

Error

### Description

Note the following:

N indicates the number of failing MISR cells

P indicates the position of the first failing cell in the MISR

G indicates the gate ID of the first failing cell in the MISR

S indicates the name of DFTMAX codec associated with the failing MISR

When the XTOL\_ENABLE cells are set to their on state and the XTOLPRPG outputs are set to select the no\_observe mode, the data inputs of MISR cells are checked to ensure they are successfully blocked (set to 0).

This check performs parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and places MISR cells to 0, XTOL\_ENABLE cells to their on state, and XTOLPRPG outputs to select the no\_observe mode.

Data inputs of MISR cells which are not at 0 for all of the simulated parallel patterns cause a violation of the K5 rule. Only one violation is issued for a given MISR.

If MASK\_ENABLE cells are used, this check is also performed with MASK\_ENABLE set to the on state.

Data inputs of MISR cells which are not at their full observe mode value without xchain credit for all of the simulated parallel patterns cause a violation of this rule. Only one violation is issued for a given MISR.

If MASK\_ENABLE cells are used, this check is also performed with MASK\_ENABLE set to on. Data inputs of MISR cells which are not at their full observe mode value without xchain credit and blocked selected chains for all of the simulated parallel patterns cause a violation of this rule. Only one violation is issued for a given MISR.

### What Next

To correct this problem, you must change the netlist or the STIL protocol file so that the data input of the failing MISR cell is properly blocked in no\_observe mode.

You can use automated violation analysis to debug this problem. This analysis will display the first failing cell for the failing MISR.

The displayed pin data is set to the simulated values of the SHIFT procedure with MISR cells set to 0, XTOL\_ENABLE cells set to their on state, and XTOLPRPG outputs set to select the no observe mode. Gates in the paths between XTOLPRPG outputs used to select the no observe mode and the failing MISR cell are also displayed.

You can use the `-cells` option of the `report_compressors` command to report the XTOL\_ENABLE or MISR cells.

You can use the `-lfsr_connections` option of the `report_compressors` command to report the XTOLPRPG output values.

You can use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

You can downgrade this rule violation to a warning. But it is not recommended since it will likely cause incorrect BIST pattern intervals.

---

## DRC Rule K6

### Message Text

```
Invalid MISR capture value for single_observe mode unblocked cell N MISR  
unblocked cells (P/G1) in compressor S had invalid capture data for  
single_observe mode of chain C (G2).
```

```
N MISR unblocked cells (P/G1) in seq compressor S had invalid capture  
data for MASK_ENABLE single_observe mode of chain C (G2).
```

### Severity

Error

### Description

Note the following:

`N` indicates the number of failing MISR cells

`P` indicates the position of the first failing cell in the MISR

`G1` indicates the gate ID of the first failing cell in the MISR

`s` indicates the name of the DFTMAX codec associated with the failing MISR

`c` indicates the `chain_id` of the chain observed with single observe mode

`G2` indicates the gate ID of the chain output

When the `XTOL_ENABLE` cells are set to their `on` state and the `XTOLPRPG` outputs are set to select the single observe mode for a given chain, the data inputs of unblocked MISR cells are checked to ensure they successfully capture data from the chain output.

This check performs parallel (32) pattern simulation using the simulated values of the `SHIFT` procedure, and places the chain outputs to random values, MISR cells to 0, `XTOL_ENABLE` cells to their `on` state, and `XTOLPRPG` outputs to select the single observe mode for the chain.

The values of `'mask_all_chain_cells'` are set to 0, `'mask_enable'` to OFF and `'xchains_registers'` to X, which are displayed in the GUI.

The data inputs of any unblocked MISR cells which are not at the same value as the chain output (or inverted) for all of the simulated parallel patterns causes a violation of this rule. Only one violation is issued for a given MISR.

### What Next

To correct this problem, you must change the netlist or the STIL procedures file so the data input of the failing MISR cell successfully captures data from the chain output in the single observe mode of the failing chain.

You can use automated violation analysis to debug this problem. This analysis displays the first failing cell for the failing MISR.

The displayed pin data is set to the simulated values of the `SHIFT` procedure with MISR cells set to 0, `XTOL_ENABLE` cells set to their `on` state, and `XTOLPRPG` outputs set to select the no observe mode. Gates in the paths between `XTOLPRPG` outputs used to select the no observe mode and the failing MISR cell are also displayed.

You can use the `-cells` option of the `report_compressors` command to report the `XTOL_ENABLE` or MISR cells.

You can use the `-lfsr_connections` option of the `report_compressors` command to report the `XTOLPRPG` output values.

You can use the `-mode_data` option of the `report_compressors` command to report the `XTOLPRPG` output values for all observe modes

### Understanding K6 check With Passing Testcase

If you have passing testcase, you can perform the following steps with passing case to debug and fix the problems.

1. Before 'run\_drc', use this command to save the simulation values of failing chain.

```
set_drc -store_k6_data <chain_id_of_failing_chain>
```

**Note:**

If you want to know the `chain_id` of any chain, you can use the following command

```
report_compressors -chain_data -all
```

2. To display the stored value of failed chain, use the following command.

```
set_pindata -debug_sim_data
```

Now, you have the simulation value of passing case, you can easily compare it with failing case by using following command to debug/fix the issue.

```
analyze_violation k6-<N>
```

`N` can be the error number in the failing message.

For example - `analyze_violation k6-1`

You can downgrade this rule violation to a warning. But this is not recommended since it likely causes incorrect BIST pattern intervals

You can use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

---

## DRC Rule K7

### Message Text

```
DFTMAX LBIST Rule K7: Invalid MISR capture value for single_observe mode  
blocked cell
```

```
N MISR blocked cells (P/G1) in compressor S had invalid capture data for  
single_observe mode of chain C (G2).
```

### Severity

Error

### Description

Note the following:

`N` indicates number of failing MISR cells

`P` indicates position of first failing cell in the MISR

G1 indicates gate ID of first failing cell in the MISR

S indicates name of DFTMAX CODEC associated with failing MISR

C indicates name of chain observed with single observe mode

G2 indicates gate ID of chain output

When the XTOL\_enable cells are set to their on state and the XTOLPRPG outputs are set to select the single observe mode for a given chain, the data inputs of blocked MISR cells are checked to ensure that they are successfully blocked (set to 0). This checking is done by performing parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and placing the chain outputs to random values, MISR cells to 0, XTOL\_enable cells to their on state, and XTOLPRPG outputs to select the single observe mode for the desired chain. Data inputs of blocked MISR cells which are not at 0 for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given MISR.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that the data input of the failing MISR cell is properly blocked in the single observe mode of the failing chain. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing MISR. The displayed pin data is set to the simulated values of the SHIFT procedure with chain outputs set to a random binary value, MISR cells set to 0, XTOL\_enable cells set to their on state, and XTOLPRPG outputs set to select the single observe mode for the chain. Gates in the paths between XTOLPRPG outputs used to select the single observe mode and the failing MISR cell are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable or MISR cells.

The `-lfsr_connections` option of the `report_compressors` command can be used to report the XTOLPRPG outputs.

The `-mode_data` option of the `report_compressors` command can be used to report the XTOLPRPG output values for all observe modes.

The `-chain_modes` option of the `report_compressors` command can be used to report the XTOLPRPG output values for the single observe mode for chains.

The `-lfsr_connections` option of the `report_compressors` command can be used to report the connections between chains and the MISR.

---

## DRC Rule K8

### Message Text

DFTMAX LBIST Rule K8: Different chain output to MISR inversion parity  
Cells P1/G1 and P2/G2 of MISR in compressor S had different inversion parity for single\_observe mode of chain C (G3)

### Severity

Error

### Description

Note the following:

P1 indicates position of first cell of failing pair in the MISR

G1 indicates gate ID of first cell of failing pair in the MISR

P2 indicates position of second cell of failing pair in the MISR

G2 indicates gate ID of second cell of failing pair in the MISR

S indicates name of DFTMAX CODEC associated with failing MISR

C indicates name of chain observed with single observe mode

G3 indicates gate ID of chain output

When the XTOL\_enable cells are set to their on state and the XTOLPRPG outputs are set to select the single observe mode for a given chain, the inversion parity between data inputs of unblocked MISR cells and the chain output are checked to ensure they are all the same. This checking is done by performing parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and placing the chain outputs to random values, MISR cells to 0, XTOL\_enable cells to their on state, and XTOLPRPG outputs to select the single observe mode for the desired chain. Any 2 data inputs of unblocked MISR cells which have differing inversion parity will result in a violation of this rule.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that the data inputs of the failing MISR cells have the same inversion parity in the single observe mode of the failing chain. The automated violation analysis can be used to help debug this problem. This analysis will display the 2 failing cells for the failing MISR. The displayed pin data is set to the simulated values of the SHIFT procedure with chain outputs set to a random binary value, MISR cells set to 0, XTOL\_enable cells set to their on state, and XTOLPRPG outputs set to select the single observe mode for the chain. Gates in the paths between the chain output and the failing MISR cells are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable or MISR cells.

The `-lfsr_connections` option of the `report_compressors` command can be used to report the XTOLPRPG outputs.

The `-mode_data` option of the `report_compressors` command can be used to report the XTOLPRPG output values for all observe modes.

The `-chain_modes` option of the `report_compressors` command can be used to report the XTOLPRPG output values for the single observe mode for chains.

The `-lfsr_connections` option of the `report_compressors` command can be used to report the connections between chains and the MISR.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause incorrect patterns for all observe modes.

---

## DRC Rule K9

### Message Text

Invalid MISR capture value for full\_observe mode

N MISR cells (P/G) in compressor S had invalid capture data for full\_observe mode.

N MISR cells (P/G) in seq compressor S had invalid capture data for MASK\_ENABLE full\_observe mode.

N MISR cells (P/G) in seq compressor S had invalid capture data for MASK\_ENABLE xt看\_enable\_off full\_observe mode.

### Severity

Error

### Description

Note the following:

N indicates the number of failing MISR cells

P indicates the position of the first failing cell in the MISR

G indicates the gate ID of the first failing cell in the MISR

S indicates the name of the DFTMAX codec associated with the failing MISR



When the XTOL\_ENABLE cells are set to their on state and the XTOLPRPG outputs are set to select the full observe mode, the data inputs of MISR cells are checked to ensure they successfully capture data from their connecting chains.

This check performs parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and places the chain outputs to random values, MISR cells to 0, XTOL\_ENABLE cells to their on state, and XTOLPRPG outputs to select the full observe mode.

Data inputs of MISR cells which are not at their expected value for all of the simulated parallel patterns cause a violation of this rule. The expected value is calculated by exclusively applying the OR operator to the values of all chain outputs connected to a MISR cell input with consideration given to their precalculated inversion parity. Only one violation is issued for a given MISR.

If MASK\_ENABLE cells are used, this check is also performed with MASK\_ENABLE set to on. Data inputs of MISR cells which are not at their full observe mode value with X-chain credit for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given MISR. In addition, this check is done with MASK\_ENABLE set to on and XTOL\_ENABLE set to off. Data inputs of MISR cells which are not at their full observe mode value without X-chain credit for all simulated parallel patterns cause a violation of this rule. Only one violation is issued for a given MISR.

If the previous check passes and the XTOLPRPG outputs are not all initialized to binary values during the TEST\_SETUP procedure, the full\_observe capture mode is also checked to ensure it works correctly when the XTOL\_ENABLE cells are set to their off state and the XTOLPRPG outputs are set to X. When a failure occurs, the text of the K9 violation message is changed to indicate global full\_observe mode.

If there are no failures due to either of the previous checks, the full-observe capture mode is also checked for each X-chain to ensure it works correctly when the XTOL\_ENABLE cells are set to their on state and the XTOLPRPG outputs are set to select the single observe mode for that X-chain.

When a failure occurs, the text of the K9 violation message is changed to indicate global full\_observe mode and also report the name of a failing X-chain.

### **What Next**

To correct this problem, you must change the netlist or the STIL protocol file so that the data input of the failing MISR cell properly captures data from all of its chain connections in full\_observe mode.

You can use automated violation analysis to debug this problem. This analysis displays the first failing cell for the failing MISR. Gates in the paths between XTOLPRPG outputs used to select the full observe mode and the failing MISR cell are also displayed.

The displayed pin data is set to one of the following:

*Type 1* - The simulated values of the SHIFT procedure with chain outputs set to a random binary value, MISR cells set to 0, XTOL\_ENABLE cells set to their on state, and XTOLPRPG outputs set to select the full observe mode.

*Type 2* - The simulated values of the SHIFT procedure with chain outputs set to a random binary value, MISR cells set to 0, XTOL\_ENABLE cells set to their off state, and XTOLPRPG outputs set to X. A frequent cause of this failure type is the usage of complex synthesized circuitry which exploit X and X-BAR effects to force the full observe mode when XTOL\_ENABLE is set off. This can be corrected by either initializing the XTOLPRPG outputs in the TEST\_SETUP procedure or running synthesis using options that avoid this problem.

*Type 3* - The simulated values of the SHIFT procedure with chain outputs set to a random binary value, MISR cells set to 0, XTOL\_ENABLE cells set to their off state, and XTOLPRPG outputs set to select the single observe mode of the failing X-chain.

You can use the `-cells` option of the `report_compressors` command to report the XTOL\_ENABLE or MISR cells.

You can use the `-lfsr_connections` option of the `report_compressors` command to report the XTOLPRPG output values.

You can use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

You can downgrade this rule violation to a warning. But this is not recommended since it will likely cause incorrect BIST pattern intervals.

---

## DRC Rule K10

### Message Text

```
Invalid MISR capture value for multiple_observe mode
```

```
N MISR cells (P/G) in compressor S had invalid capture data for F
multiple_observe mode.
```

```
N MISR cells (P/G) in compressor S had invalid capture data for
MASK_ENABLE F multiple_observe mode.
```

### Severity

Error

## Description

Note the following:

**N** indicates the number of failing MISR cells

**P** indicates the position of the first failing cell in the MISR

**G** indicates the gate ID of the first failing cell in the MISR

**S** indicates the name of the DFTMAX codec associated with failing MISR

**F** indicates the fraction of scan cells observed by multiple\_observe mode

When the XTOL\_ENABLE cells are set to their on state and the XTOLPRPG outputs are set to select a multiple observe mode, the data inputs of MISR cells are checked to ensure that they successfully capture data from their connecting chains.

This check performs parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and places the chain outputs to random values, MISR cells to 0, XTOL\_ENABLE cells to their on state, and XTOLPRPG outputs to select the multiple\_observe mode.

Data inputs of MISR cells which are not at their expected value for all of the simulated parallel patterns cause a violation of this rule. The expected value is calculated by exclusively applying the OR operator to the values of all chain outputs connected to a MISR cell input with consideration given to their precalculated inversion parity. Only one violation is issued for a given MISR.

If MASK\_ENABLE cells are used, this check is also performed with MASK\_ENABLE set to on. Data inputs of MISR cells which are not at their correct value, assuming all chains are observed except selected chains which are also dynamic X-chains for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given MISR.

## What Next

To correct this problem, you must change the netlist or the STIL protocol file so that the data input of the failing MISR cell properly captures data from all of its chain connections in the multiple observe mode.

You can use automated violation analysis to debug this problem. This analysis displays the first failing cell for the failing MISR. Gates in the paths between XTOLPRPG outputs used to select the full observe mode and the failing MISR cell are also displayed.

You can use the `-cells` option of the `report_compressors` command to report the XTOL\_ENABLE or MISR cells.

You can use the `-lfsr_connections` option of the `report_compressors` command to report the XTOLPRPG output values.

You can use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

You can downgrade this rule violation to a warning. But this is not recommended since it will likely cause incorrect BIST pattern intervals.

---

## DRC Rule K11

### Message Text

Invalid MISR capture value for complement multiple\_observe mode

N MISR cells (P/G) in compressor S had invalid capture data for F multiple\_observe mode.

N MISR cells (P/G) in seq compressor S had invalid capture data for MASK\_ENABLE F complement multiple\_observe mode.

### Severity

Error

### Description

Note the following:

N indicates the number of failing MISR cells

P indicates the position of the first failing cell in the MISR

G indicates the gate ID of the first failing cell in the MISR

S indicates the name of the DFTMAX codec associated with the failing MISR

F indicates the fraction of scan cells observed by multiple\_observe mode

When the XTOL\_ENABLE cells are set to their on state and the XTOLPRPG outputs are set to select a complement multiple observe mode, the data inputs of MISR cells are checked to ensure they successfully capture data from their connecting chains.

This check performs parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and places the chain outputs to random values, MISR cells to 0, XTOL\_ENABLE cells to their on state, and XTOLPRPG outputs to select the multiple observe mode.

Data inputs of MISR cells which are not at their expected value for all of the simulated parallel patterns cause a violation of this rule. The expected value is calculated by exclusively applying the OR operator to the values of all chain outputs connected to a MISR cell input with consideration given to their precalculated inversion parity. Only one violation is issued for a given MISR.

If MASK\_ENABLE cells are used, this check is also performed with MASK\_ENABLE set to on.

Data inputs of MISR cells which are not at their correct value, assuming all chains are observed except selected chains which are also dynamic X-chains for all of the simulated parallel patterns, cause a violation of this rule. Only one violation is issued for a given MISR.

### What Next

To correct this problem, change the netlist or the STL protocol file so that the data input of the failing MISR cell properly captures data from all of its chain connections in the complement multiple\_observe mode.

This check performs parallel (32) pattern simulation using the simulated values of the SHIFT procedure, and places the chain outputs to random values, MISR cells to 0, XTOL\_ENABLE cells to their on state, and XTOLPRPG outputs to select the multiple observe mode.

Data inputs of MISR cells which are not at their expected value for all of the simulated parallel patterns cause a violation of this rule. The expected value is calculated by exclusively ORing the values for all the chain outputs that are connected to a MISR cell input with consideration given to their precalculated inversion parity. Only one violation is issued for a given MISR.

You can use automated violation analysis to debug this problem. This analysis displays the first failing cell for the failing MISR. Gates in the paths between XTOLPRPG outputs used to select the full observe mode and the failing MISR cell are also displayed.

You can use the `-cells` option of the `report_compressors` command to report the XTOL\_ENABLE or MISR cells.

You can use the `-lfsr_connections` option of the `report_compressors` command to report the XTOLPRPG output values.

You can use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

You can downgrade this rule violation to a warning. But this is not recommended since it will likely cause incorrect BIST pattern intervals.

---

## DRC Rule K12

### Message Text

DFTMAX LBIST Rule K12: Invalid MISR capture shift operation

N MISR cells (input I of P/G) in compressor S had invalid shift values during X procedure.

Severity:Error

### Severity

Error

### Description

Note the following:

N indicates number of failing MISR cells

P indicates position of first failing cell in the MISR

G indicates gate ID of first failing cell in the MISR

S indicates name of DFTMAX CODEC associated with failing MISR

F indicates fraction of scancells observed by multiple\_observe mode

X indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the MISR cells are checked to ensure that they are successfully capturing data as expected for full observe mode during the SHIFT and RESEED\_OVERLAP\_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. MISR cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value is calculated by exclusiving-ORing the values for all the chain outputs that are connected to a MISR cell input with consideration given to their precalculated inversion parity and also for MISR tapping. Only one violation is issued for a given MISR.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that MISR cells are properly capturing data in full observe mode as controlled directly by the XTOL\_enable cells during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing MISR. If the failure occurred on a clock/set/reset input, then the pindata is the simulated value for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the paths between connecting chains and the failing MISR cell are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or MISR cells.

The `-lfsr_connections` option of the `report_compressors` command can be used to report the connections between chains and the MISR.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise this MISR capture shift operation to be incorrect.

---

## DRC Rule K13

### Message Text

DFTMAX LBIST Rule K13: Invalid MISR unload operation

N MISR cells (input I of P/G) in compressor S had invalid unload value during X procedure.

### Severity

Error

### Description

Note the following:

N indicates number of failing MISR cells

I indicates input ID of first failing cell in the MISR

P indicates position of first failing cell in the MISR

G indicates gate ID of first failing cell in the MISR

S indicates name of DFTMAX CODEC associated with failing MISR

X indicates name of procedure

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the MISR cells are checked to ensure that they are successfully unloading data during the `RESEED_WITH_MISR_SHIFT` procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. MISR cells which do not successfully unload their value for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given MISR.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that MISR cells are properly unloading data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing MISR. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing MISR cell and the its adjacent cell are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or MISR cells

The `-lfsr_connections` option of the `report_compressors` command can be used to report the connections between chains and the MISR.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise this MISR capture shift operation to be incorrect.

---

## DRC Rule K14

### Message Text

DFTMAX LBIIST Rule K14: Invalid MISR load input

N MISR cells (P/G) in compressor S had invalid load input value during X procedure.

### Severity

Error

### Description

Note the following:

N indicates number of failing MISR cells

P indicates position of first failing cell in the MISR

G indicates gate ID of first failing cell in the MISR

S indicates name of DFTMAX CODEC associated with failing MISR

X indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the MISR cells at the beginning of an unload segment are checked to ensure that they are successfully resetting the MISR to 0 during the RESEED\_WITH\_MISR\_SHIFT procedure. This



checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. MISR cells at the beginning of a segment which do not successfully capture a 0 for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given MISR.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that MISR input ports are properly propagating values to the MISR during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing MISR. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing MISR input port and the its associated MISR cell are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or PRPG\_shadow cells.

The `-chain_segments` option of the `report_compressors` command can be used to report the unload segments of the MISRs.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise this MISR reset operation to be incorrect.

---

## DRC Rule K15

### Message Text

```
DFTMAX LBIST Rule K15: Invalid MISR unload output
```

```
MISR output O value not equal MISR cell (P/G) in compressor S during X  
procedure
```

### Severity

Error

### Description

Note the following:

- indicates name of MISR output port
- P indicates position of first failing cell in the MISR

G indicates gate ID of first failing cell in the MISR

S indicates name of DFTMAX CODEC associated with failing MISR

X indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the MISR output ports are checked to ensure that they are successfully unloading data during the RESEED\_WITH\_MISR\_SHIFT procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL\_enable cells and Power\_enable to their off state, and all other register types to random values. MISR output ports which do not successfully have the same value as their associated MISR cell for all of the simulated parallel patterns will result in a violation of this rule.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that MISR output ports are properly receiving values from the MISR during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell for the failing MISR. The displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing MISR output port and its associated MISR cell are also displayed.

The -cells option of the report\_compressors command can be used to report the XTOL\_enable, Power\_enable, or PRPG\_shadow cells

The -chain\_segments option of the report\_compressors command can be used to report the output ports of the MISR.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns which exercise this MISR unload operation to be incorrect.

---

## DRC Rule K16

### Message Text

DFTMAX LBIST Rule K16: MISR cell disturb

N MISR cells (P/G) in compressor S were disturbed during X procedure.

### Severity

Error

### Description

Note the following:

**N** indicates number of failing MISR cells

**P** indicates position of first failing cell in the MISR

**G** indicates gate ID of first failing cell in the MISR

**S** indicates name of DFTMAX CODEC associated with failing MISR

**X** indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the MISR cells are checked to ensure that they hold state during the RESEED\_SHIFT, SHADOW\_TO\_CAREPRPG, SHADOW\_TO\_XTOLPRPG, and OCC\_SHIFT procedures and also during the capture cycles. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. MISR cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given MISR.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that MISR cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing MISR with all the gates in the traceback cone of the input which was believed to have caused the problem. If the failing procedure was the capture procedure, then the displayed pin data is set to one of the parallel pattern values that failed. Otherwise, the pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or PRPG\_shadow cells.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns which exercise this MISR unload operation to be incorrect.

---

## DRC Rule K17

### Message Text

DFTMAX LBIST Rule K17: Invalid PRPG\_shadow load operation

N PRPG\_shadow cells (input I of P/G) in compressor S had invalid value during X procedure.

## Severity

Error

## Description

Note the following:

**N** indicates number of failing PRPG\_shadow cells

**I** indicates input ID of first failing cell in the PRPG\_shadow register

**P** indicates position of first failing cell in the PRPG\_shadow register

**G** indicates gate ID of first failing cell in the PRPG\_shadow register

**S** indicates name of DFTMAX CODEC associated with the failing PRPG\_shadow register

**X** indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the PRPG\_shadow cells are checked to ensure that they are successfully loading data during the RESEED\_WITH\_MISR\_SHIFT, RESEED\_SHIFT, and RESEED\_OVERLAP\_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the PRPG\_shadow input ports to random values, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. Prpg\_shadow cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for cells which connect to PRPG\_shadow input ports are the values assigned to the input port (or inverted). The expected values for all other cells are the values assigned to the adjacent cells. Only one violation is issued for a given PRPG\_shadow.

## What Next

To correct this problem, the netlist or the STL procedure file must be changed so that PRPG\_shadow cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing PRPG\_shadow register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or PRPG\_shadow cells

The `-chain_segments` option of the `report_compressors` command can be used to report the unload segments of the MISRs.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns which exercise the reseed operation to be incorrect.

---

## DRC Rule K18

### Message Text

DFTMAX LBIST Rule K18: PRPG\_shadow cell disturb

N PRPG\_shadow cells (P/G) in compressor S were disturbed during X procedure.

### Severity

Error

### Description

Note the following:

N indicates number of failing MISR cells

P indicates position of first failing cell in the MISR

G indicates gate ID of first failing cell in the MISR

S indicates name of DFTMAX CODEC associated with failing MISR

X indicates name of procedure

This rule is not currently checked.

### What Next

No action required.

---

## DRC Rule K19

### Message Text

DFTMAX LBIST Rule K19: Invalid CarePRPG shift operation

N CarePRPG cells (input I of P/G) in compressor S had invalid shift values during X procedure.

### Severity

Error

## Description

Note the following:

**N** indicates number of failing CarePRPGcells

**I** indicates input ID of first failing cell in the CarePRPG

**P** indicates position of first failing cell in the CarePRPG

**G** indicates gate ID of first failing cell in the CarePRPG

**S** indicates name of DFTMAX CODEC associated with failing CarePRPG

**X** indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the CarePRPG cells are checked to ensure that they are successfully shifting data during the SHIFT and RESEED\_OVERLAP\_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. CarePRPG cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for cells considers the PRPG definition with tapping. Only one violation is issued for a given CarePRPG.

## What Next

To correct this problem, the netlist or the STL procedure file must be changed so that CarePRPG cells are properly shifting during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing CarePRPG register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the sources of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or CarePRPG cells.

This rule violation can be downgraded to warning but it is not recommended since this will causes patterns which exercise the scan chain load operation from PRPGs to be incorrect.

---

## DRC Rule K20

### Message Text

DFTMAX LBIST Rule K20: Invalid CarePRPG reload operation

Seq compressor chain length L1 did not equal expected length L2.

### Severity

Error

### Description

Note the following:

**N** indicates number of failing CarePRPGcells

**I** indicates input ID of first failing cell in the CarePRPG

**P** indicates position of first failing cell in the CarePRPG

**G** indicates gate ID of first failing cell in the CarePRPG

**S** indicates name of DFTMAX CODEC associated with failing CarePRPG

**X** indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the CarePRPG cells are checked to ensure that they are successfully capturing reseed data from the PRPG\_shadow during the SHADOW\_TO\_CAREPRPG procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. CarePRPG cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for cells are determined by the values from the associated cell in the PRPG\_shadow. Only one violation is issued for a given CarePRPG.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that CarePRPG cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing CarePRPG register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or CarePRPG cells.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns which exercise the scan chain load operation from PRPGs to be incorrect.

---

## DRC Rule K21

### Message Text

DFTMAX LBIIST Rule K21: CarePRPG cell disturb

N CarePRPG cells (P/G) in compressor S were disturbed during X procedure.

### Severity

Error

### Description

Note the following:

N indicates number of failing CarePRPGcells

P indicates position of first failing cell in the CarePRPG

G indicates gate ID of first failing cell in the CarePRPG

S indicates name of DFTMAX CODEC associated with failing CarePRPG

X indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the CarePRPG cells are checked to ensure that they hold state during the RESEED\_WITH\_MISR\_SHIFT, RESEED\_SHIFT, and SHADOW\_TO\_XTOLPRPG procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. CarePRPG cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given CarePRPG.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that CarePRPG cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing CarePRPG with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report\_compressors command can be used to report the XTOL\_enable, Power\_enable, or CarePRPG cells.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns which exercise the scan chain load operation from PRPGs to be incorrect.



---

## DRC Rule K22

### Message Text

DFTMAX LBIST Rule K22: Invalid XTOLPRPG shift operation  
(input I of P/G) in compressor S had invalid shift values during X  
procedure.

### Severity

Error

### Description

Note the following:

**N** indicates number of failing XTOLPRPG

**I** indicates input ID of first failing cell in the XTOLPRPG

**P** indicates position of first failing cell in the XTOLPRPG

**G** indicates gate ID of first failing cell in the XTOLPRPG

**S** indicates name of DFTMAX CODEC associated with failing XTOLPRPG

**X** indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the XTOLPRPG cells are checked to ensure that they are successfully shifting data during the SHIFT and RESEED\_OVERLAP\_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. XTOLPRPG cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for cells considers the PRPG definition with tapping. Only one violation is issued for a given XTOLPRPG.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that XTOLPRPG cells are properly shifting during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing XTOLPRPG register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed, and gates in the path between the failing cell and the sources of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or XTOLPRPG cells.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns which exercise the observe mode control during the MISR capture operation from internal chains to be incorrect.

---

## DRC Rule K23

### Message Text

```
Invalid XTOLPRPG reload operation
```

```
N XTOLPRPG cells (input I of P/G) in seq compressor S had invalid shift  
values during X procedure.
```

### Severity

Error

### Description

Note the following:

`N` indicates the number of failing XTOLPRPG

`I` indicates the input ID of first failing cell in the XTOLPRPG

`P` indicates the position of the first failing cell in the XTOLPRPG

`G` indicates the gate ID of the first failing cell in the XTOLPRPG

`S` indicates the name of the DFTMAX codec associated with the failing XTOLPRPG

`X` indicates the name of the procedure

When the XTOL\_ENABLE and POWER\_ENABLE cells are set to their off state, the XTOLPRPG cells are checked to ensure that they successfully capture reseed data from the PRPG\_SHADOW during the SHADOW\_TO\_XTOLPRPG procedure.

This check performs parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_ENABLE and POWER\_ENABLE cells to their off state, and all other register types to random values.

CarePRPG cells which do not successfully capture their expected value for all of the simulated parallel patterns cause a violation of this rule. The expected values for cells are determined by the values from the associated cell in the PRPG\_SHADOW. Only one violation is issued for a given XTOLPRPG.

If MASK\_ENABLE cells are used, the XTOLPRPG cells are checked to ensure they successfully capture reseed data from the PRPG\_SHADOW during the SHIFT procedure.

### What Next

To correct this problem, you must change the netlist or the STIL protocol file so that XTOLPRPG cells properly capture data during the specified procedure.

You can use automated violation analysis to debug this problem. This analysis displays the first failing cell of the failing XTOLPRPG register.

If the failure occurs on a clock/set/reset input, the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

You can use the `-cells` option of the `report_compressors` command to report the XTOL\_ENABLE, POWER\_ENABLE, or XTOLPRPG cells.

You can downgrade this rule violation to a warning. But it is not recommended since this will likely cause patterns which exercise the observe mode control during the MISR capture operation from internal chains to be incorrect.

---

## DRC Rule K24

### Message Text

DFTMAX LBIST Rule K24: XTOLPRPG cell disturb

N XTOLPRPG cells (P/G) in compressor S were disturbed during X procedure.

### Severity

Error

### Description

Note the following:

N indicates number of failing XTOLPRPG

P indicates position of first failing cell in the XTOLPRPG

G indicates gate ID of first failing cell in the XTOLPRPG

S indicates name of DFTMAX CODEC associated with failing XTOLPRPG

X indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the XTOLPRPG cells are checked to ensure that they hold state during the RESEED\_WITH\_MISR\_SHIFT, RESEED\_SHIFT, and SHADOW\_TO\_CAREPRPG procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. CarePRPG cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given XTOLPRPG.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that XTOLPRPG cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing XTOLPRPG with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or XTOLPRPG cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise the observe mode control during the MISR capture operation from internal chains to be incorrect.

---

## DRC Rule K25

### Message Text

N CARE\_shadow cells (input I of P/G) in seq compressor S had invalid shift values during X procedure. where:

N indicates number of failing CARE\_shadow cells

I indicates input ID of first failing cell in the CARE\_shadow register

P indicates position of failing cell in the CARE\_shadow register

G indicates gate ID of failing cell in the CARE\_shadow register

S indicates name of DFTMAX Seq CODEC associated with failing CARE\_shadow register

X indicates name of procedure

### Severity

Error

## Description

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the `CARE_shadow` cells are checked to ensure that they are successfully shifting data during the `SHIFT` and `RESEED_OVERLAP_SHIFT` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. `CARE_shadow` cells that do not successfully capture their expected value for all of the simulated parallel patterns results in a violation of this rule. The expected values for a `CARE_shadow` cell are the values assigned to the associated cell in the `CarePRPG`. If the `CODEC` has `Low_Power v3`, the expected values for a `CARE_Shadow` cell are ANDing values of the values assigned to the associated `CAREPRPG` and the `silencer_DLAT`. Only one violation is issued for a given `CARE_shadow`.

## What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that `CARE_shadow` cells are properly shifting during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the first failing cell of the failing `CARE_shadow` register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data is also displayed.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or `CARE_shadow` cells.

This rule violation may be downgraded to a warning, but it is not recommended because this may cause patterns which exercise the internal chain load operation from PRPGs to be incorrect.

---

## DRC Rule K26

### Message Text

N `CARE_shadow` cells (P/G) in seq compressor S failed hold during X procedure.

where:

N indicates number of failing `CARE_shadow` cells

P indicates position of failing cell in the `CARE_shadow` register

G indicates gate ID of failing cell in the `CARE_shadow` register

S indicates name of DFTMAX Seq CODEC associated with failing CARE\_shadow register

X indicates name of procedure

### Severity

Error

### Description

When the Power\_enable cell is set to its on state, silencer\_DLATs (if the CODEC has Low\_Power v3 hardware) are set to HI and the hold ability is activated for CarePRPGs, the CARE\_shadow cells are checked to ensure that they are successfully holding state during the SHIFT and RESEED\_OVERLAP\_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing Power\_enable to its on state, activating the hold ability of the CarePRPG, and placing all other register types to random values. CARE\_shadow cells which do not successfully hold their value for all of the simulated parallel patterns results in a violation of this rule. Only one violation is issued for a given CARE\_shadow.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that CARE\_shadow cells are properly holding values during the specified procedure under hold conditioning. The automated violation analysis can be used to help debug this problem. This analysis displays the first failing cell of the failing CARE\_shadow register. If the failure was believed to be due to a problem on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the hold control sources are also displayed. The hold control sources are the Power\_enable cell and the cells of the CarePRPG which are used to create the hold signal.

The -cells option of the report\_compressors command can be used to report the XTOL\_enable, Power\_enable, or CARE\_shadow cells.

The -hold\_connections option of the report\_compressors command can be used to report the CarePRPG cells which are used to create the hold signal.

This rule violation can be downgraded to warning. Since the hold ability is not currently used during test generation, this does not create any problem with current patterns.

---

## DRC Rule K27

### Message Text

N CARE\_shadow cells (input I of P/G) in seq compressor S had invalid value during X procedure.

where:

N indicates number of failing CARE\_shadow cells

I indicates input ID of first failing cell in the CARE\_shadow register

P indicates position of failing cell in the CARE\_shadow register

G indicates gate ID of failing cell in the CARE\_shadow register

S indicates name of DFTMAX Seq CODEC associated with failing CARE\_shadow register

X indicates name of procedure

### Severity

Error

### Description

When the XTOL\_enable and Power\_enable cells are set to their off state, the CARE\_shadow cells are checked to ensure that they are successfully loading data from the CarePRPG during the SHADOW\_TO\_CAREPRPG procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. CARE\_shadow cells which do not successfully capture their expected value for all of the simulated parallel patterns results in a violation of this rule. The expected values for a CARE\_shadow cell are the value assigned to the associated cell in the CarePRPG. If the CODEC has Low\_Power v3, the expected values for a CARE\_Shadow cell are ANDing values of values assigned to the associated CAREPRPG and the silencer\_DLAT. Only one violation is issued for a given CARE\_shadow.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that CARE\_shadow cells are properly capturing data from its CarePRPG during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the first failing cell of the failing CARE\_shadow register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and

gates in the path between the failing cell and the associated cell in the `CarePRPG` are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, `CarePRPG`, or `CARE_shadow` cells.

This rule violation may be downgraded to warning but it is not recommended since this likely causes patterns which exercise the `CarePRPG` reseed operation to be incorrect.

---

## DRC Rule K28

### Message Text

DFTMAX LBIST Rule K2: `CARE_shadow` cell disturb

`N` `CARE_shadow` cells (`P/G`) in compressor `S` were disturbed during `X` procedure..

### Severity

Error

### Description

Note the following:

`N` indicates number of failing `CARE_shadow` cells

`P` indicates position of first failing cell in the `CARE_shadow` register

`G` indicates gate ID of first failing cell in the `CARE_shadow` register

`S` indicates name of DFTMAX CODEC associated with failing `CARE_shadow` register

`X` indicates name of procedure

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the `CARE_shadow` register cells are checked to ensure that they hold state during the `SHADOW_TO_XTOLPRPG` procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. `CARE_shadow` cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given `CARE_shadow`.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `CARE_shadow` cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing



cell for the failing CARE\_shadow register with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or CARE\_shadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise the CarePRPG reseed operation to be incorrect.

---

## DRC Rule K29

### Message Text

```
DFTMAX LBIST Rule K29: Invalid XTOLShadow shift operation
```

```
N XTOLShadow cells (input I of P/G) in compressor S had invalid shift  
values during X procedure.
```

### Severity

Error

### Description

Note the following:

`N` indicates number of failing XTOLShadow cells

`I` indicates input ID of first failing cell in the XTOLShadow register

`P` indicates position of first failing cell in the XTOLShadow register

`G` indicates gate ID of first failing cell in the XTOLShadow register

`S` indicates name of DFTMAX CODEC associated with failing XTOLShadow register

`X` indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the XTOLShadow cells are checked to ensure that they are successfully shifting data during the SHIFT and RESEED\_OVERLAP\_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. XTOLShadow cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for an XTOLShadow cell are the values assigned to the associated cell in the XTOLPRPG. Only one violation is issued for a given XTOLShadow.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that CARE\_shadow cells are properly shifting during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing CARE\_shadow register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or XTOLShadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K30

### Message Text

```
DFTMAX LBIST Rule K30: Invalid XTOLShadow hold operation  
N XTOLShadow cells (P/G) in compressor S failed hold during X procedure.
```

### Severity

Error

### Description

Note the following:

N indicates number of failing XTOLShadow cells

P indicates position of first failing cell in the XTOLShadow register

G indicates gate ID of first failing cell in the XTOLShadow register

S indicates name of DFTMAX CODEC associated with failing XTOLShadow register

X indicates name of procedure

When the XTOL\_enable cells are set to their on state and the hold ability is activated for x-tolerance control, the XTOLShadow cells are checked to ensure that they are successfully holding state during the SHIFT and RESEED\_OVERLAP\_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_enable cells to their on state, activating the hold ability for x-tolerance control, and placing all other register types to

random values. XTOLShadow cells which do not successfully hold their value for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given XTOLShadow.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that XTOLShadow cells are properly holding state during the specified procedure under hold conditioning. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing XTOLShadow register. If the failure was believed to be due to a problem on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the hold control sources are also displayed. The hold control sources are the Power\_enable cell and the cells of the XTOLPRPG which are used to create the hold signal.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or XTOLShadow cells.

The `-hold_connections` option of the `report_compressors` command can be used to report the XTOLPRPG cells which are used to create the hold signal.

The `-mode_data` option of the `report_compressors` command can be used to report the XTOLPRPG output values used for the hold observe mode.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K31

### Message Text

```
DFTMAX LBIST Rule K31: Invalid XTOLShadow reload operation
```

```
N XTOLShadow cells (input I of P/G) in compressor S had invalid value
during X procedure.
```

### Severity

Error

### Description

Note the following:

N indicates number of failing XTOLShadow cells

I indicates input ID of first failing cell in the XTOLShadow register

**P** indicates position of first failing cell in the XTOLShadow register

**G** indicates gate ID of first failing cell in the XTOLShadow register

**S** indicates name of DFTMAX CODEC associated with failing XTOLShadow register

**X** indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the XTOLShadow cells are checked to ensure that they are successfully loading data from the XTOLPRPG during the SHADOW\_TO\_XTOLPRPG procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. XTOLShadow cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for an XTOLShadow cell are the values assigned to the associated cell in the XTOLPRPG. Only one violation is issued for a given XTOLShadow.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that XTOLShadow cells are properly capturing data from its XTOLPRPG during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing XTOLShadow register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the associated cell in the XTOLPRPG are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or XTOLShadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K32

### Message Text

DFTMAX LBIST Rule K32: XTOLShadow cell disturb

N XTOLShadow cells (P/G) in compressor S were disturbed during X procedure.

### Severity

Error

### Description

Note the following:

**N** indicates number of failing XTOLShadow cells

**P** indicates position of first failing cell in the XTOLShadow register

**G** indicates gate ID of first failing cell in the XTOLShadow register

**S** indicates name of DFTMAX CODEC associated with failing XTOLShadow register

**X** indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the XTOLShadow register cells are checked to ensure that they hold state during the SHADOW\_TO\_CAREPRPG procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. XTOLShadow cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given XTOLShadow.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that XTOLShadow cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing XTOLShadow register with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or XTOLShadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K33

### Message Text

DFTMAX LBIST Rule K33: Invalid XTOL\_enable reload operation

XTOL\_enable cell (input I of G) in seq compressor S had invalid capture during X procedure.

### Severity

Error

### Description

Note the following:

I indicates input ID of failing XTOL\_enable cell

G indicates gate ID of failing XTOL\_enable cell

S indicates name of DFTMAX Seq CODEC associated with failing XTOL\_enable cell

X indicates name of procedure

The XTOL\_enable cells are checked to ensure that they successfully load data from the PRPG\_shadow during the SHADOW\_TO\_CAREPRPG, SHADOW\_TO\_XTOLPRPG, and LAST\_SHIFT procedures. This check is performed by running a parallel (32) pattern simulation using the simulated values of the appropriate procedure and placing XTOL\_enable and Power\_enable cells to their off state and all other register types to random values. XTOL\_enable cells that do not successfully capture their expected value for all simulated parallel patterns violate this rule. The expected values for a XTOL\_enable cell are those assigned to the associated cell in the PRPG\_shadow.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that XTOL\_enable cells properly capture data from its PRPG\_shadow during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the failing XTOL\_enable cell. Suppose the failure occurred on a clock/set/reset input. In that case, the pindata is set to the simulated values for the indicated procedure, and all gates in the trace-back cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed, and gates in the path between the failing cell and the associated cell in the PRPG\_shadow are also displayed.

To fix this problem, the netlist or STIL log file needs to be modified to allow XTOL\_enable cells to properly capture data from their PRPG\_shadow during the specified procedure. Automated violation analysis can be used to debug this issue. This analysis shows the faulty XTOL\_enable cell. Suppose the error occurred on a clock/set/reset input; in this case, the PIN data is set to the simulated values for the specified procedure, and all tags in that input's traceback cone are also displayed. Otherwise, the displayed pin data is set to one of the failed parallel pattern values, and gates in the path between the failed cell and the associated cell in the PRPG\_shadow are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable or PRPG\_shadow cells.

This rule violation can be downgraded to a warning but is not recommended as it is likely to result in incorrect patterns.

---

## DRC Rule K34

### Message Text

DFTMAX LBIST Rule K3: XTOL\_enable cell disturb

XTOL\_enable cell (G) in compressor S was disturbed during X procedure.

### Severity

Error

### Description

Note the following:

G indicates gate ID of first failing cell in the XTOL\_enable register

S indicates name of DFTMAX CODEC associated with failing XTOL\_enable register

X indicates name of procedure

The XTOL\_enable cells are checked to ensure that they hold state during the SHIFT and RESEED\_OVERLAP\_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. XTOL\_enable cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that XTOL\_enable cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing XTOL\_enable cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K35

### Message Text

DFTMAX LBIST Rule K35: Invalid internal chain input value

Input node G of chain C in compressor S had invalid simulated value during X procedure..

### Severity

Error

### Description

Note the following:

G indicates gate ID of chain input node

C indicates name of chain whose input had invalid value

S indicates name of DFTMAX CODEC associated with failing chain

X indicates name of procedure

Chain inputs are checked to ensure that they are at the correct value during the SHIFT procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. Chain inputs which are not at their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for a chain input are calculated from the values assigned to the CARE\_shadow cells considering the CarePRPG tapping.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that chain inputs have the correct value during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing chain input node. The displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing chain input and the connecting CARE\_shadow cells are also displayed.

The -cells option of the report\_compressors command can be used to report the XTOL\_enable, Power\_enable, or CARE\_shadow cells.

The -lfsr\_connections option of the report\_compressors command can be used to report the CarePRPG tapping and the chain to CarePRPG connections.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.



---

## DRC Rule K36

### Message Text

DFTMAX LBIST Rule K36: Invalid internal chain shift operation  
N internal chain cells (input I of G) of chain C had invalid value during  
X procedure.

### Severity

Error

### Description

Note the following:

N indicates number of failing internal cells

I indicates input ID of first failing internal cell

G indicates gate ID of first failing cell

C indicates name of chain whose cell failed shift checking

X indicates name of procedure

Internal cells are checked to ensure that they are correctly shifting data during the RESEED\_OVERLAP\_SHIFT procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing internal cells at random values, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. Internal cells which do not capture their expected shift value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for an internal cell are calculated from the values assigned to their adjacent cell considering cell-to-cell inversion. The expected shift values for the cell closest to the scan input are the values assigned to the chain input node. Only one violation is issued for a given chain.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that internal cells successfully capture the correct shift values during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the failing internal cell. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the adjacent cell (or chain input) are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable or Power\_enable cells.

The `report_scan_chains` command can be used to report the chain input nodes or report the internal cells with their inversion.

This rule violation can be downgraded to warning but it is not recommended since this likely causes patterns to be incorrect.

If any compressed scan chain is having higher number of consecutive Lock-Up latches in their design, the `set_drc -max_sim_passes <n>` command can be used to increase the simulation passes to take care of such structure in this DRC rule check. The value of `n` by default is 20.

---

## DRC Rule K37

### Message Text

DFTMAX LBIST Rule K37: Internal chain cell disturb

*N* internal chain cells (*P/G* of chain *C*) in seq compressor *S* were disturbed during *X* procedure, where,

*N* indicates number of failing internal cells int the failing chain.

*P* indicates position of first failing cell in the failing internal chain.

*G* indicates gate ID of first failing cell in the failing internal chain.

*C* indicates name of chain whose cells failed disturb checking.

*S* indicates name of DFTMAX Seq CODEC associated with failing internal chain cell.

*X* indicates name of procedure.

### Severity

Error

### Description

You can check the internal scan cells to ensure that they hold state during the `RESEED_WITH_MISR_SHIFT`, `RESEED_SHIFT`, `SHADOW_TO_CAREPRPG`, `SHADOW_TO_XTOLPRPG`, and `OCC_SHIFT` procedures. You can check this by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the chain outputs to random values, `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. Internal scan cells which do not hold state for all of the simulated parallel patterns result violate this rule.

### What Next

To correct this problem, change the netlist or the STIL protocol file so that the internal scan cells hold state during the specified procedure. Use the automated violation analysis to help debug this problem. This analysis displays the failing internal scan cell with all the gates in the traceback cone of the input that was causing the problem. The pindata is set to the simulated values of the failing procedure.

Use the `-cells` option of the `report_compressors` command to report the `XTOL_enable` or `Power_enable` cells.

Use the `report_scan_chains` command to report the chain input nodes.

Use the `report_scan_cells` command to report the internal cells with their inversion.

### For IDU2:

Tracing the failing internal chain cell provides some information about which DFF is causing the problem. Sometimes the culprit DFF receives incorrect value from the previous procedure simulation.

The following command displays the procedure which is simulated before the current procedure.

```
Set_drc -analyze_drc_violation {K37 gate_id}
```

This rule violation may be downgraded to warning but it is not recommended since this likely causes patterns to be incorrect.

---

## DRC Rule K38

### Message Text

```
DFTMAX LBIST Rule K38: Invalid Power_enable value from test_setup  
Power_enable cell (G) had incorrect value (V) at end of test_setup  
procedure.
```

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing `Power_enable` cell

v indicates value of `Power_enable` cell at end of `test_setup` procedure

The Power\_enable cell is checked to ensure that it is successfully initialized to its off-state during the test\_setup procedure. This checking is done by comparing the simulated value of the Power\_enable cell at the end of the test\_setup procedure and its off-state value.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that the Power\_enable cell is properly initialized to its off-state in the test\_setup procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing Power\_enable cell. The pindata is set to the simulated values of the test\_setup procedure and all gates in the traceback cone of this gate are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the Power\_enable cell with its off state.

The `test_setup` option of the `set_pindata` command can be used to report the stored simulated values of the test\_setup procedure.

The `-store_setup` option of the `set_drc` command can be used to store the simulated values for all patterns of the test\_setup procedure. The default is to only store the final pattern value.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K39

### Message Text

```
DFTMAX LBIST Rule K39: Invalid Power_enable value from low_power_setup  
Power_enable cell (G) had incorrect value (V) at end of low_power_setup  
procedure.
```

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing Power\_enable cell

v indicates value of Power\_enable cell at end of low\_power\_setup procedure

The Power\_enable cell is checked to ensure that it is successfully initialized to its on-state during the low\_power\_setup procedure. This checking is done by comparing the simulated

value of the Power\_enable cell at the end of the low\_power\_setup procedure and its on-state value.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that the Power\_enable cell is properly initialized to its on-state in the low\_power\_setup procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing Power\_enable cell. The pindata is set to the simulated values of the low\_power\_setup procedure and all gates in the traceback cone of this gate are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the Power\_enable cell with its off state.

The `low_power_setup` option of the `set_pindata` command can be used to report the stored simulated values of the test\_setup procedure.

This rule violation can be downgraded to warning. Since there is currently no support for using the low\_power\_setup procedure, it will have no effect until support is available for this procedure.

---

## DRC Rule K40

### Message Text

DFTMAX LBIST Rule K40: Power\_enable cell disturb

Power\_enable cell (G) was disturbed during X procedure.

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing Power\_enable cell

x indicates the name of the procedure

The Power\_enable cell is checked to ensure that it holds state during the SHIFT, RESEED\_WITH\_MISR\_SHIFT, RESEED\_SHIFT, RESEED\_OVERLAP\_SHIFT, SHADOW\_TO\_CAREPRPG, SHADOW\_TO\_XTOLPRPG, OCC\_SHIFT, and capture procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. If the

Power\_enable cell does not hold state for all of the simulated parallel patterns, it will result in a violation of this rule.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that Power\_enable cell holds state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing Power\_enable cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the Power\_enable cell with its off state.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K41

### Message Text

```
DFTMAX LBIST Rule K41: Invalid occ_chain_load operation
```

```
N OCC_chain_cells (input I of P/G) had invalid value during X procedure..
```

### Severity

Error

### Description

Note the following:

`N` indicates number of failing occ\_chain cells

`I` indicates input ID of first failing cell in the occ\_chain

`P` indicates position of first failing cell in the occ\_chain

`G` indicates gate ID of first failing cell in the occ\_chain

`X` indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the occ\_chain cells are checked to ensure that they are successfully loading data during the OCC\_SHIFT procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing the occ\_chain input ports to random values, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. occ\_chain cells which do not successfully capture their

expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for cells which connect to `occ_chain` input ports are the values assigned to the input port (or inverted). The expected values for all other cells are the values assigned to the adjacent cells. Only one violation is issued for a given `occ_chain`.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `occ_chain` cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing `occ_chain`. If the failure occurred on a clock/set/reset input, then the pin data is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-chain_segments` option of the `report_compressors` command can be used to report the input ports of the `occ_chain`.

---

## DRC Rule K42

### Message Text

```
DFTMAX LBIIST Rule K4: occ_chain cell disturb  
N occ_chain cells (P/G) were disturbed during X procedure.
```

### Severity

Error

### Description

Note the following:

`N` indicates number of failing `occ_chain` cells

`P` indicates position of first failing cell in the `occ_chain`

`G` indicates gate ID of first failing cell in the `occ_chain`

`X` indicates name of procedure

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the `occ_chain` cells are checked to ensure that they hold state during the `SHIFT`, `RESEED_WITH_MISR_SHIFT`, `RESEED_SHIFT`, `RESEED_OVERLAP_SHIFT`, `SHADOW_TO_CAREPRPG`, and `SHADOW_TO_XTOLPRPG` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `XTOL_enable` and `Power_enable` cells to their off state, and all

other register types to random values. `occ_chain` cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given `occ_chain`.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `occ_chain` cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing `occ_chain` with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or `occ_chain` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise the `occ_chain` load operation to be incorrect.

---

## DRC Rule K43

### Message Text

```
DFTMAX LBIST Rule K43: misr_mode cell disturb
```

```
Misr_mode cell (G) was disturbed during X procedure.
```

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing `misr_mode` cell

X indicates name of procedure

The MISR cells are checked to ensure that they hold state during the `SHIFT`, `RESEED_WITH_MISR_SHIFT`, `RESEED_SHIFT`, `RESEED_OVERLAP_SHIFT`, `SHADOW_TO_CAREPRPG`, `SHADOW_TO_XTOLPRPG`, `OCC_SHIFT`, and capture procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. If any `misr_mode` cell does not hold state for all of the simulated parallel patterns, it will result in a violation of this rule.



### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that the failing `misr_mode` cell holds state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing `misr_mode` cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `misr_mode` cells with their accumulator mode enable values.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K44

### Message Text

```
DFTMAX LBIIST Rule K44: Invalid misr_mode value from test_setup  
Misr_mode cell (G) had incorrect value V at end of test_setup procedure.
```

### Severity

Error

### Description

Note the following:

`G` indicates gate ID of failing `misr_mode` cell

`v` indicates the incorrect value that was set by the `test_setup` procedure

At the end of the `test_setup` procedure (and extended `test_setup` if used), the `misr_mode` cells are checked to ensure that they are at the correct value as determined by the selected accumulator mode usage. If the accumulator mode is enabled, then the `misr_mode` cells must be set to their enable values. Otherwise, the `misr_mode` cells must be set to the complement of their enable values. `Misr_mode` cells which are not at their correct value at this time will result in a violation of this rule.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `misr_mode` cells are at their intended value at the end of `test_setup`. The active accumulator mode selection may have to be changed to reflect the intended usage. The automated violation analysis can be used to help debug this problem. This analysis will

display the failing `misr_mode` cell with all the gates in its traceback cone. The `pindata` is set to the simulated values at the end of the `test_setup` procedure(s).

The `-cells` option of the `report_compressors` command can be used to report the `misr_mode` cells with their accumulator mode enable values.

The `drc` option of the `report_settings` command can be used to report the current selection of the accumulator mode usage.

The `test_setup` option of the `set_pindata` command can be used to report the stored simulated values of the `test_setup` procedure.

The `-store_setup` option of the `set_drc` command can be used to store the simulated values for all patterns of the `test_setup` procedure.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K45

### Message Text

```
DFTMAX LBIST Rule K45: Invalid seed_type shift operation
```

```
Seed_type cell (input I of G) had invalid shift value during X procedure.
```

### Severity

Error

### Description

Note the following:

`I` indicates input ID of failing `seed_type` cell

`G` indicates gate ID of failing `seed_type` cell

`X` indicates name of procedure

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the `seed_type` cells are checked to ensure that they are successfully shifting data during the `RESEED_PARTIAL_OVERLAP_SHIFT` procedures. This checking is done by first simulating the JTAG load procedure to set values on the JTAG controller. This is followed by performing parallel (32) pattern simulation using the simulated values of the `RESEED_PARTIAL_OVERLAP_SHIFT` procedure, and placing `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. `Seed_type` cells which do not successfully capture their expected value for all of the

simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG ULTRA is used.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that seed\_type cells correctly shift during the RESEED\_PARTIAL\_OVERLAP\_SHIFT procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing seed\_type cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report\_compressors command can be used to report the XTOL\_enable, Power\_enable, or seed\_type cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K46

### Message Text

DFTMAX LBIST Rule K4: seed\_type cell disturb

Seed\_type cell (G) was disturbed during X procedure..

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing seed\_type cell

X indicates name of procedure

To correct this problem, the netlist or the STL procedure file must be changed so that seed\_type cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing seed\_type cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that seed\_type cells hold state during the specified procedure. The automated violation

analysis can be used to help debug this problem. This analysis will display the failing seed\_type cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report\_compressors command can be used to report the XTOL\_enable, Power\_enable, or seed\_type cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K47

### Message Text

```
DFTMAX LBIST Rule K47: Invalid capture_enable_shadow shift operation
```

```
Capture_enable_shadow cell (input I of G) had invalid shift value during  
X procedure.
```

### Severity

Error

### Description

Note the following:

I indicates input ID of failing capture\_enable\_shadow cell

G indicates gate ID of failing capture\_enable\_shadow cell

X indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the capture\_enable\_shadow cells are checked to ensure that they are successfully shifting data during the RESEED\_PARTIAL\_OVERLAP\_SHIFT procedures. This checking is done by first simulating the JTAG\_RESEED\_SETUP procedure to set values on the JTAG controller. This is followed by performing parallel (32) pattern simulation using the simulated values of the RESEED\_PARTIAL\_OVERLAP\_SHIFT procedure, and placing XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. Capture\_enable\_shadow cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that capture\_enable\_shadow cells correctly shift during the

RESEED\_PARTIAL\_OVERLAP\_SHIFT procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing capture\_enable\_shadow cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or capture\_enable\_shadow cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K48

### Message Text

```
DFTMAX LBIST Rule K48: Capture_enable_shadow cell disturb
```

```
Capture_enable_shadow cell (G) was disturbed during X procedure.
```

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing capture\_enable\_shadow cell

X indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the capture\_enable\_shadow cells are checked to ensure that they hold state during selected procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. Capture\_enable\_shadow cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. This rule is not currently checked for any procedure.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that capture\_enable\_shadow cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing capture\_enable\_shadow cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or `capture_enable_shadow` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K49

### Message Text

DFTMAX LBIST Rule K49: Invalid `capture_enable` reload operation

`Capture_enable` cell (input `I` of `G`) had invalid reload value during `X` procedure.

### Severity

Error

### Description

Note the following:

`I` indicates input ID of failing `capture_enable` cell

`G` indicates gate ID of failing `capture_enable` cell

`X` indicates name of procedure

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the `capture_enable` cells are checked to ensure that they successfully load data from their associated `capture_enable_shadow` cells during the `SHADOW_TO_CAREPRPG` and `SHADOW_TO_XTOLPRPG` procedures. This checking is done by first simulating the `JTAG_RESEED_SETUP` procedure to set values on the JTAG controller. This is followed by performing parallel (32) pattern simulation using the simulated values of the failing procedure, and placing `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. `Capture_enable` cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `capture_enable` cells capture their expected values from their associated `capture_enable_shadow` cells during the failing procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing `capture_enable` cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, `capture_enable`, or `capture_enable_shadow` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K50

### Message Text

```
DFTMAX LBIST Rule K50: capture_enable cell disturb
```

```
Capture_enable cell (G) was disturbed during X procedure..
```

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing `capture_enable` cell

X indicates name of procedure

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the `capture_enable` cells are checked to ensure that they hold state during the `RESEED_PARTIAL_OVERLAP_SHIFT`, `JTAG_RESEED_SETUP`, and `JTAG_MISR_SHIFT` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. `Capture_enable` cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `capture_enable` cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing `capture_enable` cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, `capture_enable`, or `capture_enable` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K51

### Message Text

DFTMAX LBIST Rule K51: Invalid shift\_counter\_shadow shift operation

N shift\_counter\_shadow cells (input I of P/G) had invalid shift values during X procedure.

Severity:Error

### Severity

Error

### Description

Note the following:

N indicates number of failing shift\_counter\_shadow cells

P indicates position of first failing shift\_counter\_shadow cell

G indicates gate ID of first failing shift\_counter\_shadow cell

X indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the shift\_counter\_shadow cells are checked to ensure that they are successfully shifting data during the RESEED\_PARTIAL\_OVERLAP\_SHIFT procedures. This checking is done by first simulating the JTAG\_RESEED\_SETUP procedure to set values on the JTAG controller. This is followed by performing parallel (32) pattern simulation using the simulated values of the RESEED\_PARTIAL\_OVERLAP\_SHIFT procedure, and placing XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. Shift\_counter\_shadow cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used. Only one violation is issued for a given shift\_counter\_shadow chain.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that shift\_counter\_shadow cells correctly shift during the RESEED\_PARTIAL\_OVERLAP\_SHIFT procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing shift\_counter\_shadow cell with all the gates in the traceback cone of the input which was



believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or `shift_counter_shadow` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K52

### Message Text

```
DFTMAX LBIST Rule K52: shift_counter_shadow cell disturb
```

```
Shift_counter_shadow cell (G) was disturbed during X procedure.
```

### Severity

Error

### Description

Note the following:

G indicates gate ID of `shift_counter_shadow` cell

X indicates name of procedure

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the `shift_counter_shadow` cells are checked to ensure that they hold state during the `SHADOW_TO_CAREPRPG` and `SHADOW_TO_XTOLPRPG` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. `Shift_counter_shadow` cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `shift_counter_shadow` cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing `shift_counter_shadow` cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or `shift_counter_shadow` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K53

### Message Text

DFTMAX LBIST Rule K53: Invalid shift\_counter reload operation

N shift\_counter cells (input I of P/G) had invalid reload values during X procedure.

### Severity

Error

### Description

Note the following:

N indicates number of failing shift\_counter cells

I indicates input ID of failing shift\_counter cellP

P indicates position of first failing shift\_counter cell

G indicates gate ID of first failing shift\_counter cell

X indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the shift\_counter cells are checked to ensure that they successfully load data from their associated shift\_counter\_shadow cells during the SHADOW\_TO\_CAREPRPG and SHADOW\_TO\_XTOLPRPG procedures. This checking is done by first simulating the JTAG\_RESEED\_SETUP procedure to set values on the JTAG controller. This is followed by performing parallel (32) pattern simulation using the simulated values of the failing procedure, and placing XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. Shift\_counter cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used. Only one violation is issued for a given shift\_counter chain.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that shift\_counter cells capture their expected values from their associated shift\_counter\_shadow cells during the failing procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing shift\_counter

cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or `shift_counter_shadow` cells.

This rule violation might be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K54

### Message Text

```
DFTMAX LBIST Rule K54: Invalid shift_counter count operation
```

```
N shift_counter cells (P/G) had invalid count values during LBIST  
shifting.
```

### Severity

Error

### Description

Note the following:

`N` indicates number of failing `shift_counter` cells

`P` indicates position of first failing `shift_counter` cell

`G` indicates gate ID of first failing `shift_counter` cell

`Shift_counter` cells are checked to ensure that their count operation is correctly performed during the internal scanchain shift process. This checking is done by first initializing the state of the circuit to their values at the beginning of the SHIFT procedure. Then the `capture_enable` cell is set to its `capture_enable_value`, `seed_type` cell is set to its `CarePRPG_select_value`, `shift_counter` cells set to a value equal to the number shifts per load, `shift_counter_shadow` cells set to a value equal to the number shifts per load, and the `pattern_counter` cells are set to a value of 2. Finally, the SHIFT procedure is simulated a number of times equal to the number of shifts per load plus the minimum number of ATE clock cycles. The shift counter should end up at a value equal to the number of shifts per load. `Shift_counter` cells which are not at their correct value will result in a violation of this rule. This rule is only checked if JTAG LBIST is used. Only one violation is issued for a given `shift_counter` chain.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `shift_counter` cells count properly during LBIST shifting. The shift counter and pattern

counter values for each shift can be displayed during the rules checking by using the `-compressor_debug_data` option of the `set_drc` command. The automated violation analysis can be also used to help debug this problem. This analysis will display all of the cells of the failing shift counter with all connections between the cells. The pindata is set to the simulated values during the `number_shifts_per_load + 1` shift cycle.

The `-cells` option of the `report_compressors` command can be used to report the `shift_counter` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K55

### Message Text

```
DFTMAX LBIST Rule K55: shift_counter cell disturb
```

```
Shift_counter cell (G) was disturbed during X procedure.
```

```
Severity:Error
```

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing `shift_counter` cell

X indicates name of procedure

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the `shift_counter` cells are checked to ensure that they hold state during the `JTAG_RESEED_SETUP` and `JTAG_MISR_SHIFT` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. `Shift_counter` cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `shift_counter` cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing `shift_counter` cell with all the gates in the traceback cone of the input which was believed

to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or `shift_counter` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K56

### Message Text

DFTMAX LBIST Rule K56: Invalid pattern\_counter shift operation

N pattern\_counter cells (input I of P/G) had invalid shift values during X procedure.

### Severity

Error

### Description

Note the following:

N indicates number of failing pattern\_counter cells

I indicates input ID of failing pattern\_counter cell

P indicates position of first failing pattern\_counter cell

G indicates gate ID of first failing pattern\_counter cell

X indicates name of procedure

When the `XTOL_enable` and `Power_enable` cells are set to their off state, the pattern\_counter cells are checked to ensure that they are successfully shifting data during the `PATCOUNTER_SHIFT` procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the `PATCOUNTER_SHIFT` procedure, and placing `XTOL_enable` and `Power_enable` cells to their off state, and all other register types to random values. Pattern\_counter cells which do not successfully shift their expected value for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST is used. Only one violation is issued for a given shift\_counter\_shadow chain.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that pattern\_counter cells correctly shift during the failing procedure. The automated violation

analysis can be used to help debug this problem. This analysis will display the failing pattern\_counter cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or pattern\_counter cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K57

### Message Text

```
DFTMAX LBIST Rule K57: Invalid pattern_counter count operation  
  
N pattern_counter cells (P/G) had invalid count values during LBIST  
shifting.
```

### Severity

Error

### Description

Note the following:

**N** indicates number of failing shift\_counter cells

**P** indicates position of first failing shift\_counter cell

**G** indicates gate ID of first failing shift\_counter cell

Pattern\_counter cells are checked to ensure that their count operation is correctly performed during the internal scanchain shift process. This checking is done by first initializing the state of the circuit to their values at the beginning of the SHIFT procedure. Then the capture\_enable cell is set to its capture\_enable\_value, seed\_type cell is set to its CarePRPG\_select\_value, shift\_counter cells set to a value equal to the number shifts per load, shift\_counter\_shadow cells set to a value equal to the number shifts per load, and the pattern\_counter cells are set to a value of 2. Finally, the SHIFT procedure is simulated a number of times equal to the number of shifts per load plus the minimum number of ATE clock cycles. The pattern counter should end up at a value equal to 1. Pattern\_counter cells which are not at their correct value will result in a violation of this rule. This rule is only checked if JTAG LBIST is used. Only one violation is issued for a given pattern\_counter chain.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that pattern\_counter cells count properly during shifting. The shift counter and pattern counter values for each shift can be displayed during the rules checking by using the -Compressor\_debug\_data option of the set\_drc command. The automated violation analysis can be also used to help debug this problem. This analysis will display all of the cells of the failing pattern\_counter with all connections between the cells. The pindata is set to the simulated values during the number\_shifts\_per\_load + 1 shift cycle.

The -cells option of the report\_compressors command can be used to report the pattern\_counter cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K58

### Message Text

```
DFTMAX LBIST Rule K58: pattern_counter cell disturb
```

```
Pattern_counter cell (G) was disturbed during X procedure.
```

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing pattern\_counter cell

X indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the pattern\_counter cells are checked to ensure that they hold state during the RESEED\_PARTIAL\_OVERLAP\_SHIFT, SHADOW\_TO\_CAREPRPG, SHADOW\_TO\_XTOLPRPG, JTAG\_RESEED\_SETUP, and JTAG\_MISR\_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. Pattern\_counter cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. This rule is only checked if JTAG LBIST or JTAG ULTRA is used.

### What Next

To correct this problem, the netlist or the STL procedure file must be changed so that `pattern_counter` cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing `pattern_counter` cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The `pindata` is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or `pattern_counter` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K59

### Message Text

```
DFTMAX LBIST Rule K59: Scan cells with same erroneous signatures
```

```
Scan cells N1 (G1) and N2 (G2) have same signature.
```

### Severity

Error

### Description

Note the following:

N1 indicates name of scan cell

G1 indicates gate ID of scan cell

N2 indicates name of scan cell

G2 indicates gate ID of scan cell

Uniqueness of erroneous signatures is required improving mapping features back to scan cells for debugging and diagnosis. In certain cases, the MISR might fail to satisfy this condition and two or more failing scan cells might generate the same erroneous signature. In this case, the failing scan cell cannot be uniquely determined even when a single scan cell fails for a particular pattern.

### What Next

Need to redesign the MISR to meet this condition.



---

## DRC Rule K60

### Message Text

DFTMAX LBIST Rule K6: chain\_mask cell disturb

N chain\_mask cells (P/G) in compressor S were disturbed during X procedure.

### Severity

Error

### Description

Note the following:

N indicates number of failing chain\_mask chain

P indicates position of failing chain\_mask chain

G indicates gate ID of failing cell in the chain\_mask chain

S indicates name of CODEC associated with failing chain\_mask chain

X indicates name of procedure

When the XTOL\_enable and Power\_enable cells are set to their off state, the chain\_mask cells are checked to ensure that they hold state during the SHIFT, RESEED\_WITH\_MISR\_SHIFT, RESEED\_SHIFT, RESEED\_OVERLAP\_SHIFT, RESEED\_PARTIAL\_OVERLAP\_SHIFT, SHADOW\_TO\_CAREPRPG, SHADOW\_TO\_XTOLPRPG, JTAG\_RESEED\_SETUP, JTAG\_MISR\_SHIFT, RESEED\_STREAMING\_SHIFT, and OCC\_SHIFT procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL\_enable and Power\_enable cells to their off state, and all other register types to random values. Chain\_mask cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given chain\_mask chain.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that chain\_mask cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing chain\_mask chain with all the gates in the fanin cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report\_compressors command can be used to report the XTOL\_enable, Power\_enable, or chain\_mask cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise the OCC\_chain load operation to be incorrect.

---

## DRC Rule K61

### Message Text

DFTMAX LBIST Rule K61: Invalid xchains\_register reload operation

N xchains\_register cells (input I of P/G) in compressor S had invalid value during X procedure.

### Severity

Error

### Description

Note the following:

N indicates number of failing xchains\_register cells

I indicates input ID of first failing cell in the xchains\_register

P indicates position of failing cell in the xchains\_register

G indicates gate ID of failing cell in the xchains\_register

S indicates name of CODEC associated with failing cell in the xchains\_register

X indicates name of procedure

When the XTOL\_ENABLE and POWER\_ENABLE cells are set to their off state, the xchains\_register cells are checked to ensure that they are successfully capturing reseed data from the PRPG\_SHADOW during the SHADOW\_TO\_XTOLPRPG procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_ENABLE/POWER\_ENABLE cells to their off state, and all other register types to random values. Xchains\_register cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for cells are determined by the values from the associated cell in the PRPG\_SHADOW. Only one violation is issued for a given xchains\_register.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that xchains\_register cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing xchains\_register register. If the failure occurred

on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or `xchains_register` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise the scan chain load operation from PRPGs to be incorrect.

---

## DRC Rule K62

### Message Text

```
DFTMAX LBIST Rule K62: xchains_register cell disturb
```

```
N xchains_register cells (P/G) in compressor S were disturbed during X  
procedure.
```

### Severity

Error

### Description

Note the following:

`N` indicates number of failing `xchains_register` cells

`P` indicates position of failing cell in the `xchains_register`

`G` indicates gate ID of failing cell in the `xchains_register`

`S` indicates name of CODEC associated with failing cell in the `xchains_register`

`X` indicates name of procedure

When the `XTOL_ENABLE` and `POWER_ENABLE` cells are set to their off state and codec enables are set to their active state, the `xchains_register` cells are checked to ensure that they hold state during the `SHIFT`, `SHADOW_TO_CAREPRPG`, `RESEED_WITH_MISR_SHIFT`, `RESEED_SHIFT`, `RESEED_PARTIAL_OVERLAP_SHIFT1`, `RESEED_PARTIAL_OVERLAP_SHIFT2`, `RESEED_LOAD`, `JTAG_RESEED_SETUP`, `JTAG_MISR_SHIFT`, `RESEED_OVERLAP_SHIFT`, `RESEED_PARTIAL_OVERLAP_SHIFT1`, `RESEED_PARTIAL_OVERLAP_SHIFT2`, `RESEED_STREAMING_SHIFT1`, `RESEED_STREAMING_SHIFT2`, and `OCC_SHIFT` procedures.

When the `xtol_enables` are set active, `POWER_ENABLE` cells are set inactive, and codec enables are set active, the `xchains_register` cells are also checked to ensure that they hold state during the `SHADOW_TO_XTOLPRPG` procedure.

This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `XTOL_ENABLE/POWER_ENABLE` cells to their off state, codec enables are set to their active state, and all other register types to random values. `xXchains_register` cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given procedure.

When `xchains_registers` are used, some procedures are also checked considering CODEC disabling variations. This checks all combinations of a single CODEC disabled (used as an `xchain` and only observable in single observe mode) and all other codecs enabled. When failures occur during checking with a codec disabled, the error message will include an extra line indicating which codec was disabled.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that `xchains_register` cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing `xchains_register` with all the gates in the traceback cone of the input which was believed to have caused the problem. The `pindata` is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or `xchains_register` cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns which exercise the `xchains_register` operation to be incorrect.

---

## DRC Rule K63

### Message Text

```
DFTMAX LBIST Rule K6: Invalid value from jtag_reseed_setup
```

```
Register T cell (G) had incorrect value (V) at end of jtag_reseed_setup  
procedure.
```

### Severity

Error

## Description

Note the following:

T indicates type of failing register cell

G indicates gate ID of failing register cell

V indicates the incorrect value that was set by the `jtag_reseed_setup` procedure

At the end of the `jtag_reseed_setup` procedure, the address, broadcast, and increment\_address cells are checked to ensure that they are at the correct value. The address cells must indicate a value of 0, the broadcast cell must be at the defined broadcast value, and the increment address cell must be at the complement of the increment\_address\_value. Cells which are not at their correct value at this time will result in a violation of this rule.

## What Next

To correct this problem, the netlist or the test procedure file must be changed so that the failing cells are at their intended value at the end of the `jtag_reseed_setup` procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell with all the gates in its traceback cone. The pindata is set to the simulated values of the `jtag_reseed_setup` procedure.

The `-cells` option of the `report_compressors` command can be used to report the address, broadcast (with broadcast values), or increment\_address cells (with increment\_address values).

The `-summary` option of the `report_compressors` command can be used to report the `codec_enables` with their associated enable values.

The `jtag_reseed_setup` option of the `set_pindata` command can be used to report the stored simulated values of the `jtag_reseed_setup` procedure.

This rule violation can be downgraded to warning, but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K64

### Message Text

DFTMAX LBIST Rule K64: Invalid instruction\_register reload operation

N instruction register cells (input I of P/G) had invalid value during X procedure.

## Severity

Error

## Description

Note the following:

**N** indicates number of failing failing instruction register cells

**I** indicates input ID of first failing cell in the instruction register

**P** indicates position of failing cell in the instruction register

**G** indicates gate ID of failing cell in the instruction register

**X** indicates name of procedure

When the xt看 enable cells are set off and POWER\_ENABLE cells are set off and idu\_enable cells are set active and codec enables are set active, the instruction register cells are checked to ensure that they are successfully loading reseed data during the RESEED\_SHIFTA, RESEED\_SHIFTB, and RESEED\_PARTIAL\_OVERLAP\_SHIFT2 procedures. The instruction register contains the cells of the broadcast, address, seed\_type, capture\_enable\_shadow, and shift\_counter\_shadow register types. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_ENABLE/POWER\_ENABLE cells to their off state, idu\_enable/codec enable to their active state and all other register types to random values. Instruction register cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for cells are determined by the values from the associated cell in the PRPG\_SHADOW. Only one violation is issued for a given procedure.

## What Next

To correct this problem, the netlist or the test procedure file must be changed so that xchains\_register cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing xchains\_register register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The -cells option of the report\_compressors command can be used to report the XTOL\_ENABLE, power enable, codec\_enable, and instruction register cells. The -cells option can also be used to report the individual register types used in the instruction register. These register types include broadcast, address, seed\_type, capture\_enable\_shadow, and shift\_counter\_shadow.

The `-summary` option of the `report_compressors` command can be used to report the `idu_enables` with their associated enable values.

This rule violation can be downgraded to warning, but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K65

### Message Text

```
DFTMAX LBIST Rule K65: Address cell disturb  
N address cells (P/G) were disturbed during X procedure.
```

### Severity

Error

### Description

Note the following:

`N` indicates number of failing address cells

`P` indicates position of failing cell in the address register

`G` indicates gate ID of failing cell in the `xchains_register`

`S` indicates name of CODEC associated with failing cell in the address register

`X` indicates name of procedure

When the `XTOL_ENABLE` and `POWER_ENABLE` cells are set to their off state and `codec_enables` are set to their active state, the address cells are checked to ensure that they hold state during the `SHIFT`, `RESEED_SHIFT`, `RESEED_LOAD`, `RESEED_PARTIAL_OVERLAP_SHIFT1`, `SHADOW_TO_CAREPRPG`, and `SHADOW_TO_XTOLPRPG` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `XTOL_ENABLE/POWER_ENABLE` cells set to their off state, `codec_enables` set to their active state, and all other register types set to random values. Address cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given procedure.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that address cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell for the failing address cell with all the gates in the traceback cone of the input which was

believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_ENABLE`, `POWER_ENABLE`, `codec_enable`, or address cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K66

### Message Text

```
DFTMAX LBIST Rule K66: Broadcast cell disturb
```

```
Broadcast cell (G) was disturbed during X procedure.
```

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing broadcast cell

X indicates name of procedure

When the `XTOL_ENABLE` and `POWER_ENABLE` cells are set to their off state and `codec_enables` are set to their active state, the broadcast cell is checked to ensure that it holds state during the `SHIFT`, `RESEED_SHIFT`, `RESEED_LOAD`, `RESEED_PARTIAL_OVERLAP_SHIFT1`, `SHADOW_TO_CAREPRPG`, and `SHADOW_TO_XTOLPRPG` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `XTOL_ENABLE`/`POWER_ENABLE` cells set to their off state, `codec_enables` set to their active state, and all other register types set to random values. Broadcast cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given procedure.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so the broadcast cell holds state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing broadcast cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.



The `-cells` option of the `report_compressors` command can be used to broadcast cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K67

### Message Text

DFTMAX LBIST Rule K6: Invalid `increment_address` set operation

`Increment_address` cell (G) had invalid set value (V1/V2) during X procedure.

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing cell in the `xchains_register`

V1 indicates simulated value of failing `increment_address` cell

V2 indicates expected value of failing `increment_address` cell

X indicates name of procedure

At the end of an application of certain shift procedures which are intended to set the `increment_address` cell, the `increment_address` cell is checked to ensure that it is at the correct value. The `increment_address` cell must be set to the complement of its `increment_address_value` for the `RESEED_SHIFTA` and `RESEED_SHIFTB` procedures. The `increment_address` cell must also be set to the its `increment_address_value` for the `RESEED_SHIFTC`, `RESEED_PARTIAL_OVERLAP_SHIFT1`, and `RESEED_LOAD` postamble procedures. Cells which are not at their correct value will result in a violation of this rule.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that the failing cells are at their intended value at the end of the failing procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell with all the gates in its traceback cone. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `increment_address` cells (with `increment_address` values).

The `-summary` option of the `report_compressors` command can be used to report the `codec_enables` with their associated enable values.

The `RESEED_SHIFTA`, `RESEED_SHIFTB`, `RESEED_SHIFTC`, `RESEED_PARTIAL_OVERLAP_SHIFT1`, and `RESEED_LOAD` options of the `set_pindata` command can be used to report the stored simulated values of the failing procedure.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K68

### Message Text

```
DFTMAX LBIST Rule K68: Increment_address cell disturb
```

```
Increment_address cell (G) was disturbed during X procedure.
```

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing `increment_address` cell

X indicates name of procedure

When the `XTOL_ENABLE` and `POWER_ENABLE` cells are set to their off state and codec enables are set to their active state, the `increment_address` cells are checked to ensure that they hold state during the `SHIFT`, `RESEED_LOAD`, `RESEED_PARTIAL_OVERLAP_SHIFT2`, `SHADOW_TO_CAREPRPG`, and `SHADOW_TO_XTOLPRPG` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `XTOL_ENABLE/POWER_ENABLE` cells set to their off state, codec enables set to their active state, and all other register types set to random values. `Increment_address` cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given procedure.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that `increment_address` cell holds state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing `increment_address` cell with all the gates in the traceback cone of the input which

was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `XTOL_enable`, `Power_enable`, or address cells.

The `-summary` option of the `report_compressors` command can be used to report the `codec_enables` with their associated enable values.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K69

### Message Text

DFTMAX LBIST Rule K69: Invalid `codec_enable` set decode value operation  
  
N `codec_enable` cells (input I of P/G) had invalid decode value during X  
procedure.

### Severity

Error

### Description

Note the following:

N indicates number of failing `codec_enable` cells

I indicates input ID of first failing cell in the `codec_enable` register

P indicates position of failing cell in the `codec_enable` register

G indicates gate ID of failing cell in the `codec_enable` register

X indicates name of procedure

When the `XTOL_ENABLE` and `POWER_ENABLE` cells are set off, the `codec_enable` cells are checked to ensure that they successfully perform the decode operation during the `RESEED_LOAD` (third preamble shift) and `RESEED_PARTIAL_OVERLAP_SHIFT2` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing `XTOL_ENABLE`/  
`POWER_ENABLE` cells to their off state, and all other register types to random values. `Codec_enable` cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for `codec_enable` cells are calculated considering the values that were randomly assigned to the `increment_address`, `address`, and `broadcast` cells. For patterns where the

increment\_address is set to its increment\_address\_value, the expected value is calculated as follows:

The expected value for the first cell is 0.

Else if expected value of preceding codec\_enable cell was 1, the expected value is 1.

Else expected value is value assigned to preceding codec\_enable cell.

For patterns where the increment\_address is not set to its increment\_address\_value, the expected value is calculated as follows:

Calculate the address and broadcast values for given pattern.

For cells whose ID<address, the expected value is 0.

For cells whose ID=address, the expected value is 1.

For cells whose ID>address, if broadcast=broadcast\_value, the expected value is 1, else expected value is 0.

Only one violation is issued for a given procedure.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that the failing codec\_enable cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing codec\_enable register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The -cells option of the report\_compressors command can be used to report the XTOL\_ENABLE, power enable, codec\_enable, increment\_address, address, and broadcast cells

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K70

### Message Text

```
DFTMAX LBIST Rule K70: codec_enable cell disturb
```

```
N codec_enable cells (P/G) were disturbed during X procedure.
```

### Severity

Error

### Description

Note the following:

**N** indicates number of failing codec\_enable cells

**P** indicates position of failing cell in codec\_enable register

**G** indicates gate ID of failing cell in codec\_enable register

**X** indicates name of procedure

When the XTOL\_ENABLE and POWER\_ENABLE cells are set to their off state, the codec\_enable cells are checked to ensure that they hold state during the RESEED\_SHIFTA, RESEED\_SHIFTB, RESEED\_SHIFTC, and RESEED\_PARTIAL\_OVERLAP\_SHIFT1 procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL\_ENABLE/POWER\_ENABLE cells to their off state, and all other register types to random values. Codec\_enable cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given procedure.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that codec\_enable cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing codec\_enable register with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_enable, Power\_enable, or codec\_enable cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K71

### Message Text

DFTMAX LBIST Rule K71: Invalid cumulative\_codec\_enable set decode value operation

`N cumulative_codec_enable cells (input I of P/G) had invalid decode value during X procedure.`

### Severity

Error

### Description

Note the following:

`N` indicates number of failing `cumulative_codec_enable` cells

`I` indicates input ID of first failing cell in the `cumulative_codec_enable`

`P` indicates position of failing cell in the `cumulative_codec_enable`

`G` indicates gate ID of failing cell in the `cumulative_codec_enable`

`X` indicates name of procedure

When the `XTOL_ENABLE` and `POWER_ENABLE` cells are set off, the `cumulative_codec_enable` cells are checked to ensure that they successfully perform the decode operation during the `RESEED_LOAD` (third preamble shift) procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing `XTOL_ENABLE/POWER_ENABLE` cells to their off state, and all other register types to random values. `Cumulative_codec_enable` cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected values for `cumulative_codec_enable` cells are calculated considering the values that were randomly assigned to the `increment_address`, `address`, and `broadcast` cells. For patterns where the `increment_address` is set to its `increment_address_value`, the expected value is calculated as follows:

The expected value for the first cell is 0.

Else if expected value of preceding `cumulative_codec_enable` cell was 1, the expected value is 1.

Else expected value is value assigned to `codec_enable` cell whose ID is 1 less than ID of the `cumulative_codec_enable` cell.

For patterns where the `increment_address` is not set to its `increment_address_value`, the expected value is calculated as follows:

Calculate the address and broadcast values for given pattern.

For cells whose `ID < address`, the expected value is 0.

For cells whose `ID = address`, the expected value is 1.

For cells whose ID>address, if broadcast=broadcast\_value, the expected value is 1, else expected value is 0.

Only one violation is issued for a given procedure.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that the failing cumulative\_codec\_enable cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing cumulative\_codec\_enable register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The -cells option of the report\_compressors command can be used to report the XTOL\_ENABLE, power enable, cumulative\_codec\_enable, increment\_address, address, and broadcast cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K72

### Message Text

```
DFTMAX LBIST Rule K72: Invalid cumulative_codec_enable set accumulate  
value operation)
```

```
N cumulative_codec_enable cells (input I of P/G) had invalid accumulate  
value during X procedure.
```

### Severity

Error

### Description

Note the following:

N indicates number of failing cumulative\_codec\_enable cells

I indicates input ID of first failing cell in the cumulative\_codec\_enable register

P indicates position of failing cell in the cumulative\_codec\_enable register

G indicates gate ID of failing cell in the cumulative\_codec\_enable register

x indicates name of procedure

When the XTOL\_ENABLE and POWER\_ENABLE cells are set off, the cumulative\_codec\_enable cells are checked to ensure that they successfully perform the accumulate operation during the RESEED\_PARTIAL\_OVERLAP\_SHIFT2 procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing XTOL\_ENABLE/POWER\_ENABLE cells to their off state, and all other register types to random values. Cumulative\_codec\_enable cells which do not successfully capture their expected value for all of the simulated parallel patterns will result in a violation of this rule. The expected value for a cumulative\_codec\_enable cell is calculated by ORing its randomly assigned value with the randomly assigned value of its associated codec\_enable cell. Only one violation is issued for a given procedure.

### What Next

o correct this problem, the netlist or the test procedure file must be changed so that the failing cumulative\_codec\_enable cells are properly capturing data during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing cumulative\_codec\_enable register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

The -cells option of the report\_compressors command can be used to report the XTOL\_ENABLE, power enable, cumulative\_codec\_enable, and codec\_enable cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K73

### Message Text

DFTMAX LBIST Rule K73: Cumulative\_codec\_enable cell disturb)

N cumulative\_codec\_enable cells (P/G) were disturbed during X procedure.

### Severity

Error



## Description

Note the following:

**N** indicates number of failing xchains\_register cells

**I** indicates input ID of first failing cell in the xchains\_register

**P** indicates position of failing cell in the xchains\_register

**G** indicates gate ID of failing cell in the xchains\_register

**S** indicates name of CODEC associated with failing cell in the xchains\_register

**X** indicates name of procedure

When the XTOL\_ENABLE and POWER\_ENABLE cells are set to their off state, the cumulative\_codec\_enable cells are checked to ensure that they hold state during the RESEED\_SHIFTA, RESEED\_SHIFTB, RESEED\_SHIFTC, RESEED\_PARTIAL\_OVERLAP\_SHIFT1, and RESEED\_LOAD (postamble) procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL\_ENABLE/POWER\_ENABLE cells to their off state, and all other register types to random values. Cumulative\_codec\_enable cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule. Only one violation is issued for a given procedure.

## What Next

To correct this problem, the netlist or the test procedure file must be changed so that cumulative\_codec\_enable cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing cumulative\_codec\_enable register with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_ENABLE, power enable, or cumulative\_codec\_enable cells.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K74

### Message Text

```
DFTMAX LBIST Rule K74: XTOL_ENABLE inversion difference
```

```
Seq compressors S1 and S2 had different inversion parity to XTOL_ENABLE  
port P.
```

### Severity

Error

### Description

Note the following:

`S1` indicates name of first seq\_compressor associated with failure

`S2` indicates name of second seq\_compressor associated with failure

`P` indicates name of port associated with XTOL\_ENABLE

For all CODECs, the inversion parity between the XTOL\_ENABLE port and the connecting PRPG\_SHADOW register cell must be the same. The inversion parity of the first CODEC is compared to the inversion parity of all other CODECs. A violation is issued for each CODEC which has an inversion difference.

### What Next

To correct this problem, the netlist must be changed so that the inversion parity is the same for all CODECs. The automated violation analysis can be used to help debug this problem. This analysis will display the PRPG\_SHADOW cells associated with the XTOL\_ENABLE of the two failing CODECS and their connection path to the XTOL\_ENABLE port. There is no pindata used for this analysis.

The `-cells Prpg_shadow` option of the `report_compressors` command can be used to report the XTOL\_ENABLE cell for a given CODEC.

The `-scan_input_ports` option of the `report_compressors` command can be used to report the XTOL\_ENABLE port. This port is the first port in the port list.

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K75

### Message Text

```
DFTMAX LBIST Rule K75: Invalid pattern_start set operation
```

```
Pattern_start cell (G) had invalid set value (V1/V2) during X procedure.
```

### Severity

Error

## Description

Note the following:

G indicates gate\_id of failing register cell

V1 indicates the simulated value of failing cell for failing procedure

V2 indicates the expected value of failing cell for failing procedure

X indicates name of failing procedure

At the end of the test\_setup and shadow\_to\_careprpg (with capture\_enable set to capture\_enable value) procedures, the pattern\_start cell is checked to ensure that it is at the pattern\_start value. At the end of the jtag\_reseed\_setup (with pattern\_start cell initialized to pattern\_start value) and jtag\_reseed\_setup2 (with pattern\_start cell initialized to complement of pattern\_start value) procedures, the pattern\_start cell is checked to ensure that it is at the complement of pattern\_start value. Cells which are not at their correct value at the end of checked procedure will result in a violation of this rule.

## What Next

To correct this problem, the netlist or the test procedure file must be changed so that the failing cells are at their intended value at the end of the failing procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell with all the gates in its traceback cone. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report\_compressors command can be used to report the pattern\_start cell with its associated pattern\_start value. The command usage is:

```
report_compressors <-all | comp_name> [-cells <PATTERN_Start>]
```

The test\_setup, jtag\_reseed\_setup, jtag\_reseed\_setup2, and shadow\_to\_careprpg options of the set\_pindata command can be used to report the stored simulated values of the failing procedure. The command usage is:

```
set_pindata [test_setup | jtag_reseed_setup | jtag_reseed_setup2 |  
shadow_to_careprpg]
```

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K76

### Message Text

```
DFTMAX LBIST Rule K76: Pattern_start cell disturb
```

```
Pattern_start cell (G) was disturbed during X procedure.
```

## Severity

Error

## Description

Note the following:

G indicates gate\_id of failing increment\_address cell

X indicates the name of the procedure

When the XTOL\_ENABLE and POWER\_ENABLE cells are set to their off state and codec enables are set to their active state, the pattern\_start cell is checked to ensure that it holds state during the SHIFT, RESEED\_SHIFTA, RESEED\_SHIFTB, RESEED\_SHIFTC, RESEED\_SHIFTD, RESEED\_SHIFTE, RESEED\_LOAD, RESEED\_PARTIAL\_OVERLAP\_SHIFT1, RESEED\_PARTIAL\_OVERLAP\_SHIFT2, SHADOW\_TO\_CAREPRPG (with capture\_enable set to complement of capture\_enable value), JTAG\_MISR\_SHIFT, OCC\_SHIFT, and capture procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL\_ENABLE/POWER\_ENABLE cells set to their off state, codec enables set to their active state, and all other register types set to random values. Pattern\_start cells which do not hold state for all of the simulated parallel patterns will result in a violation of this rule.

## What Next

To correct this problem, the netlist or the test procedure file must be changed so that increment\_address cell holds state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing increment\_address cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The -cells option of the report\_compressors command can be used to report the XTOL\_ENABLE, power enable, or pattern\_start cells. The command usage is:

```
report_compressors <-all | comp_name> [-cells <-All | comp_name> [-Cells  
  <XTOL_Enable | PWR_Enable | PATTERN_Start>]]
```

The -summary option of the report\_compressors command can be used to report the codec\_enables with their associated enable values. The command usage is:

```
report_compressors -summary
```

This rule violation can be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K77

### Message Text

DFTMAX LBIST Rule K77: TE last cell of chain connected to TE MISR  
TE last cell (G) of chain C was connected to TE MISR in compressor S.

### Severity

Error

### Description

Note the following:

G indicates gate\_id of failing register cell

C indicates the name of the failing chain

S indicates the name of compressor associated with failing chain

If the MISR captures data from the scan chains on the trailing edge of MISR clock, then the scan cells closest to each chain output are checked to ensure they capture data on the leading edge of the scancell shift clocks. If a chain output cell captures on its trailing edge when the MISRs capture on its trailing edge, a violation is issued.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that either the MISR captures on the leading edge or the failing scancells capture data on the leading edge. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell with pin data set to the simulated values of the SHIFT procedure.

The `-cells` option of the `report_compressors` command can be used to report the MISR cells. The command usage is:

```
report_compressors <-all | comp_name> [-cells < Misr>]
```

The `shift` option of the `set_pindata` command can be used to display the simulated values of the SHIFT procedure. The command usage is:

```
set_pindata [shift]
```

This rule violation can be downgraded to warning, but it is not recommended since this will likely cause the ATPG patterns to be incorrect due to simulation assuming MISRs capture the newly captured values for all shift cycles.

---

## DRC Rule K78

### Message Text

DFTMAX LBIST Rule K78: *T* cell (*G*) using cycle *C* failed load operation from port *P*.

### Severity

Error

### Description

Note the following:

*T* indicates the register type of the failing cell. It may be a PRPG shadow input, OCC SERDES cell, MISR\_shadow SERDES (serializer/deserializer) cell, or Pattern\_counter SERDES cell)

*G* indicates the gate ID of the failing register cell

*C* indicates the name of the failing chain

*P* indicates the name of the port that loads the data for the failing cell

When `external_cycles_per_shift` is set to a nonzero number, the SERDES circuitry (which includes pipeline and serializer cells) associated with several register types is checked to ensure that it is loaded correctly from its port inputs. The checking inserts data on the inputs of the SERDES circuitry, simulates the associated shift procedure and the adequate number of cycles, and then ensures that the SERDES outputs achieve the intended values at the right time. The following SERDES register types are checked:

PRPG\_shadow register – Uses the `RESEED_PARTIAL_OVERLAP_SHIFT1` procedure and can store patterns in the `SERDES_PRPG_INPUT_SHIFT`

OCC register – Uses the `OCC_SHIFT` procedure and can store patterns in the `SERDES_OCC_INPUT_SHIFT`

MISR\_shadow register – Uses the `MISR_SHADOW_SHIFT` procedure and can store patterns in the `SERDES_MISR_SHADOW_INPUT_SHIFT`

Pattern\_counter register – Uses the `PAT_COUNTER_SHIFT` procedure and can store patterns in the `SERDES_PAT_COUNTER_INPUT_SHIFT`

The inputs of the SERDES circuitry are stored in the associated port list and are reported using the `report_compressors -scan_input_ports` command. The outputs of the SERDES circuitry are the segment outputs associated with the register types and can be displayed using the `report_compressors -chain_segments` command.

Simulation is performed by first setting all gates to the values at the beginning of the shift procedure associated with the selected register type. This shift procedure is then simulated for three slow cycles ( $3 * N$  applications). Selected binary data is placed on the SERDES inputs at the beginning of each fast shift cycle. After the first three slow cycles, the SERDES inputs are set to X.

The simulation of the shift procedure is repeated until all loaded binary values are fully propagated to the SERDES outputs and compared with expected values. The SERDES outputs that do not achieve the intended value at the appropriate time cause a violation of the K78 rule. The patterns that were simulated during this checking can be stored during DRC using the following command:

```
set_drc -store_full_cycles pattern_type_for_selected_register
```

This data is then available for display using the following command:

```
set_pindata -serdes_full_cycle_period_display begin_period end_period
```

### What Next

To correct this problem, you need to change the netlist or the test procedure file so that the failing SERDES load operation operates correctly. You can use automated violation analysis to help debug this problem. This analysis displays the SERDES input and output gates associated with the failure with pin data set to the simulated values of the associated SHIFT procedure used for the simulation.

Since `external_cycles_per_shift` is set to a nonzero number, this simulation is for a full slow cycle of  $N$  applications of the shift procedure, but the displayed data only includes three time periods: the initial value of the first application of the shift procedure, the clock-on time of the application of the shift procedure expected to capture, and the final time period of the last application of the shift procedure. All slow cycle simulation data used during the analysis can be stored and displayed using the `-store_full_cycles` option of the `set_drc` command and the `-serdes_full_cycle_period_display` option of the `set_pindata` command. The syntax for these options is as follows:

```
set_drc -store_full_cycles <serdes_prpg_input_shift |  
serdes_occ_input_shift | serdes_patcounter_input_shift |  
serdes_misr_shadow_input_shift>
```

```
set_pindata -serdes_full_cycle_period_display begin_period end_period
```

This rule violation can be downgraded to a warning but it is not recommended since this will likely cause corrupted patterns due to invalid loading of data through the SERDES circuitry.

---

## DRC Rule K79

### Message Text

DFTMAX LBIST Rule K79: SERDES MISR cell (G) using cycle C failed unload operation to port P.

### Severity

Error

### Description

Note the following:

G indicates gate ID of the failing register cell

C indicates the name of the failing chain

P indicates the name of the port that loads the data for the failing cell

When `external_cycles_per_shift` is set to a nonzero number, the SERDES circuitry (which includes pipeline and serializer cells) associated with several register types is checked to ensure that it is loaded correctly from its port inputs. The checking inserts data on the inputs of the SERDES circuitry, simulates the associated shift procedure and an adequate number of cycles, and then ensures that the SERDES outputs achieve the intended values at the right time.

Simulation is performed by first setting all gates to the values at the beginning of the shift procedure associated with the selected register type. This shift procedure is then simulated for three slow cycles (3 \* N applications). Selected binary data is placed on the SERDES inputs at the beginning of each fast shift cycle. After the first three slow cycles, the SERDES inputs are set to X.

The simulation of the shift procedure is repeated until all loaded binary values are fully propagated to the SERDES outputs and compared with expected values. The SERDES outputs that do not achieve the intended value at the appropriate time cause a violation of the K78 rule. The patterns that were simulated during this checking can be stored during DRC using the following command:

```
set_drc -store_full_cycles serdess_output_shift
```

This data is then available for display using the following command:

```
set_pindata -serdes_full_cycle_period_display begin_period end_period
```

### What Next

To correct this problem, you need to change the netlist or the test procedure file so that the failing SERDES MISR load operation operates correctly. You can use automated violation



analysis to help debug this problem. This analysis displays the SERDES input and output gates associated with the failure that is the pin data set to the simulated values of the associated MISR\_SHIFT procedure used for the simulation.

Since `external_cycles_per_shift` is set to a nonzero number, this simulation is for a full slow cycle of  $N$  applications of the shift procedure, but the displayed data only includes three time periods: the initial value of the first application of the shift procedure, the clock-on time of the application of the shift procedure expected to capture, and the final time period of the last application of the shift procedure. All slow cycle simulation data used during the analysis can be stored and displayed using the `-store_full_cycles` option of the `set_drc` command and the `-serdes_full_cycle_period_display` option of the `set_pindata` command. The syntax for these options is as follows:

```
set_drc -store_full_cycles serdes_output_shift
```

```
set_pindata -serdes_full_cycle_period_display begin_periodend_period
```

This rule violation can be downgraded to a warning but it is not recommended since this will likely cause corrupted patterns due to invalid loading of data through the SERDES circuitry.

---

## DRC Rule K80

### Message Text

```
DFTMAX LBIST Rule K80: Extended test_setup simulation mismatch
```

```
T cell (G) in position P has extended test_setup simulation mismatch  
(V1/V2).
```

*T* indicates register type of failing cell.

*G* indicates `gate_id` of failing cell.

*P* indicates position of failing cell in the CODEC DRC chain.

*V1* indicates simulated value of the failing cell.

*V2* indicates expected value of the failing cell.

### Severity

Error

### Description

After simulating the extended test setup using the CODEC-DRC load and shift procedures, certain CODEC-DRC cells are checked to ensure they have attained their intended values. Cells which do not have the intended value that violate this rule.

## What Next

To correct this problem, the netlist or the test procedure file must be changed to ensure that the `CODEC_DRC` load process is working correctly. Use the automated violation analysis to debug this problem. This analysis displays the failing cell and all gates in its traceback cone with pindata set to the simulated values of the `CODEC_DRC LOAD` procedure.

## More debug info for For IDU2

The following command can be used to display more debugging information:

```
Set_drc -analyze_drc_violation {K80 gate_id first_shift last_shift}
```

It displays the total number of `codec_drc_shifts` simulated for K80 check. It displays the `Payload_cycles_counter` value, and the `Local_cycles_counter` value for every `Seq_compressor_IDU` just after decoding the `Codec_drc` procedure. It also displays the `codec_drc_shift` values (`codec_drc_init_values` are the intended values) for all the shifts, and for all the scan-ins.

This displays the value of the gate (`gate_id` provided as an argument for this command) whenever the value changes, and it also displays the `shift_number` at which the value changes. It also stores the `codec_drc_shift` simulation cycles(shift cycles between the `first_shift` and `last_shift`) under `CODEC_DRC_SHIFTA` pattern. These `first_shift` and `last_shift` are provided as arguments for this command.

`Codec_drc_init_values` has lot of X's as there are several registers whose value at the end of `extended_test_setup` can be don't care. With X's loading into the chain, it's difficult to know at which position on the `codec_drc_chain`, the `codec_drc_shifting` is not correct. The following command can be used to use binary values as the intended values.

```
Set_drc -binary_K80_check
```

This gives the nearest position where the `codec_drc_shifting` is not correct. This option is just for analyzing K80(and not be used during regular DRC checking). To reset these options, you can use the following command:

```
Set_drc -noanalyze_drc_violation -nobinary_K80_check
```

This rule violation may be downgraded to warning but it is not recommended since this likely causes patterns to be incorrect due to invalid loading of the CODEC DRC chain which is a critical part of the state initialization process.

---

## DRC Rule K81

### Message Text

DFTMAX LBIST Rule K81: *N* misr\_shadow cells (input *I* of *P/G*) had invalid update values during *X* procedure.

### Severity

Error

### Description

Note the following:

*N* indicates the number of failing misr\_shadow cells in the failing MISR (multiple-input signature register) shadow register

*I* indicates the input that caused problem of first failing cell

*P* indicates the position of the first failing cell in the MISR\_shadow register

*G* indicates the gate ID of the failing register cell

*X* indicates the name of the procedure

The misr\_shadow cells are checked to ensure that they successfully capture data from the MISR register during the MISR\_SHIFT procedure. This check performs parallel pattern simulation using the simulated values of the MISR\_SHIFT procedure and placing random values on the MISR cells. Misr\_shadow cells that do not successfully capture the expected value for all of the parallel patterns simulated cause a violation of this rule. The expected values for a misr\_shadow cell are the values assigned to the associated cell in the MISR register. Only one violation is issued for each misr\_shadow register.

### What Next

To correct this problem, you need to change the netlist or the test procedure file so that either the MISR captures on the leading edge or the failing scancells capture data on the leading edge. You can use automated violation analysis to debug this problem. This analysis displays the failing cell with the pin data set to the simulated values of the SHIFT procedure.

You can use the `-cells` option of the `report_compressors` command to report the MISR shadow cells. The syntax is as follows:

```
report_compressors <-all | comp_name> [-cells misr_shadow]
```

You can downgrade this rule violation to a warning, but it is not recommended since this will likely cause the patterns to be incorrect due to invalid measures associated with unloading misr\_shadow\_status\_cells.

---

## DRC Rule K82

### Message Text

DFTMAX LBIST Rule K82: *N* misr\_shadow cells ( *P/G*) in compressor *S* were disturbed during *X* procedure.

### Severity

Error

### Description

Note the following:

*N* indicates the number of failing misr\_shadow cells in the failing MISR\_shadow register

*P* indicates the position of the first failing cell in the MISR\_shadow register

*G* indicates gate ID of the failing MISR\_shadow register cell

*S* indicates the name of the sequential compressor associated with the failing misr\_shadow cell

*X* indicates the name of the procedure

The misr\_shadow register cells are checked to ensure that they hold state during the MISR\_UNLOAD procedure and all other procedures in which the MISR (multiple-input signature register) must hold state that are not associated with MISR\_SHADOW and MISR\_SHADOW\_STATUS operations. This check is done by performing parallel pattern simulation using the simulated values of the appropriate procedure. The misr\_shadow cells that do not hold state for all of the simulated parallel patterns cause a violation of this rule. Only one violation is issued for a given misr\_shadow register.

### What Next

To correct this problem, you need to change the netlist or the test procedure file so that misr\_shadow cells hold state during the specified procedure. You can use automated violation analysis to debug this problem. This analysis displays the failing cell for the failing misr\_shadow register with the pin data set to the simulated values of the failing procedure.

You can use the `-cells` option of the `report_compressors` command to report the MISR shadow cells. The syntax is as follows:

```
report_compressors <-all | comp_name> [-cells misr_shadow]
```

You can downgrade this rule violation to a warning, but it is not recommended since this will likely cause the patterns to be incorrect due to invalid measures associated with unloading misr\_shadow\_status\_cells.

---

## DRC Rule K83

### Message Text

DFTMAX LBIST Rule K83: `misr_shadow_status` cell (input *I* of *G*) had invalid update value during *X* procedure.

### Severity

Error

### Description

Note the following:

*I* indicates the input that caused the problem of the first failing cell

*G* indicates the `gate_id` of the failing `MISR_shadow_status` cell

*X* indicates the name of the procedure

The `misr_shadow_status` cells are checked to ensure that they are successfully capturing data from the `misr_shadow` register during the last clock pulse of the `MISR_SHADOW_STATUS_UPDATE` procedure. This checking is done by performing parallel pattern simulation using the simulated values of the last clock pulse in the `MISR_SHADOW_STATUS_UPDATE` procedure and placing random values on the `MISR_SHADOW` cells. One pattern of the 32 random values assigned to the `misr_shadow` cells is adjusted to be all zeros. The expected value for a `misr_shadow_status` cell of a CODEC is the result of ORing all the values that were assigned to the `misr_shadow` register cells of that CODEC. The `misr_shadow_status` cells that do not successfully capture their expected value for all of the parallel patterns simulated causes a violation of this rule.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that the `misr_shadow_status` cells are properly updating data from the associated `misr_shadow` cells during the `MISR_SHADOW_STATUS_UPDATE` procedure. You can use automated violation analysis to debug this problem. This analysis displays the failing `misr_shadow_status` cell with the pin data set to the simulated values of the failing procedure.

You can use the `-cells` option of the `report_compressors` command to report the `misr_shadow_status` cells. The syntax is as follows:

```
report_compressors <-all | comp_name> [-cells misr_shadow_status_cells]
```

This rule violation can be downgraded to a warning, but it is not recommended since this will likely cause the patterns to be incorrect due to invalid measures associated with unloading `misr_shadow_status_cells`.

---

## DRC Rule K84

### Message Text

DFTMAX LBIST Rule K84: `Misr_shadow_status` cell (G) in seq compressor C was disturbed during X procedure.

### Severity

Error

### Description

Note the following:

G indicates `gate_id` of failing `MISR_shadow_status` cell

C indicates the name of the compressor containing failing `MISR_shadow_status` cell

X indicates name of procedure

The `misr_shadow_status` register cells are checked to ensure that they hold state during the `MISR_SHIFT` procedure and all other procedures where the MISR must hold state that are not associated with `MISR_SHADOW` and `MISR_SHADOW_STATUS` operations. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure. `Misr_shadow_status` cells which do not hold state for all of the parallel patterns simulated will result in a violation of this rule.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that `misr_shadow_status` cells hold state during the specified procedure. Automated violation analysis can be used to help debug this problem. This analysis will display the failing `misr_shadow_status` cell with `pindata` set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `misr_shadow_status` cells. The command usage is:

```
report_compressors <-all | comp_name > [-cells misr_shadow_status ]
```

This rule violation can be downgraded to warning, but it is not recommended since this will likely cause the patterns to be incorrect due to invalid measures associated with unloading `misr_shadow_status_cells`.

---

## DRC Rule K85

### Message Text

DFTMAX LBIST Rule K85: Invalid idu\_enable value during reseed\_load preamble

IDU enable (G) had invalid value (V1/V2) during reseed\_load preamble.

### Severity

Error

### Description

Note the following:

G indicates gate\_id of failing register cell

V1 indicates simulated value of the failing cell

V2 indicates expected value of the failing cell

During the simulation of the clock-on time of the the last cycle of the preamble of the RESEED\_LOAD procedure, the IDU enable gate is checked to ensure it is at the idu\_enable active value. If it is not at the expected value it will result in a violation of this rule.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that the IDU enable is at the correct value. The automated violation analysis can be used to help debug this problem. This analysis will display the IDU enable gate with pindata set to the RESEED\_LOAD procedure.

The `-cells` option of the `report_compressors` command can be used to report the MISR cells. The command usage is:

```
report_compressors <-all | comp_name> [-cells < Misr>]
```

This rule violation can be downgraded to warning, but it is not recommended since this will likely cause the ATPG patterns to be incorrect due to simulation assuming MISRs capture the newly captured values for all shift cycles.

---

## DRC Rule K86

### Message Text

DFTMAX LBIST Rule K86 (CODEC DRC chain cell disturb)

`CODEC DRC chain cell G` was disturbed during `T` time `D` of `X` procedure.

`N` DRC chain cells (`P/G2`) in seq compressor `S` were disturbed during `X` procedure.

### Severity

Warning

### Description

Note the following:

`G` indicates the `gate_id` of the failing `CODEC_DRC` chain cell

`T` indicates the time during which the failure occurred (preamble or postamble)

`X` indicates the name of the procedure

`N` indicates the number of the failing cells in failing `PRPG_SHADOW` register

`P` indicates the position of the first failing cell in the failing `PRPG_SHADOW` register

`G2` indicates the `gate_id` of the first failing cell

`S` indicates the name of the sequential compressor associated with the failing cell

There are two distinct variations of this rule:

The slow `CODEC DRC` chain cells are checked to ensure that they hold state during the preamble cycles and non-shifting postamble cycles of the `CODEC_DRC_LOAD_UNLOAD` procedure. This checking is done using the pre-calculated simulated values of the `CODEC_DRC_LOAD_UNLOAD` procedure. Slow `CODEC DRC` chain cells which do not hold state for all of the checked time periods result in a violation of this rule.

The `PRPG_SHADOW` cells are checked to ensure that they hold state during the `SHADOW_TO_CAREPRPG` and `SHADOW_TO_XTOLPRPG` procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the selected procedure. `PRPG_SHADOW` cells which do not hold state for all of the parallel patterns simulated will result in a violation of this rule. Only one violation is issued for a given `PRPG_SHADOW` register.

### What Next

If `K-86` DRC violation is present, `codec_drc` self-test patterns may fail VCS simulation. To correct this problem, the netlist or the test procedure file must be changed so that failing cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the failing cell with pin data set to the simulated values of the failing procedure.



---

## DRC Rule K87

### Message Text

DFTMAX LBIIST Rule K87: invalid fast shifting

Misr\_shadow\_status cell (G) in seq compressor C was disturbed during X procedure.

Scan cell (G) in chain C for seq compressor S had invalid fast shifting during X procedure.

### Severity

Error

### Description

Note the following:

G indicates the gate\_id of failing internal scan cell

C indicates the name of the failing chain

S indicates the name of compressor associated with failing chain

X indicates the name of the procedure

When external\_cycles\_per\_shift (N) is set to a nonzero number, all scan cells are checked to identify how many times they are pulsed during N applications of the SHIFT procedure. Scan cells which are pulsed more than once will result in a violation of this rule. Only one violation is issued for an internal scan chain.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that internal scan cells shift once per N applications of the SHIFT procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell with pindata set to the normally stored simulated values of the SHIFT procedure. When external\_cycles\_per\_shift is set to a nonzero number, this data only includes 3 time periods which are the initial value of the first application of the shift procedure, the clock-on time of the application of the shift procedure expected to capture, and the final time period of the last application of the shift procedure. To view all time periods of all N cycles, the `-store_full_cycles` option of the `set_drc` command can be used prior to running DRC to collect this simulation data. These values can then be displayed using the `-serdes_full_cycle_period_display` option of the `set_pindata` command. The command usages are:

```
set_drc -store_full_cycles serdes_internal_shift
```

```
set_pindata -serdes_full_cycle_period_display begin_period end_period
```

This rule violation may be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect due to invalid loading and unloading of data into and out of the internal scan chains.

---

## DRC Rule K88

### Message Text

```
DFTMAX LBIST Rule K88: MISR_SHADOW_STATUS cell disturb  
MISR_SHADOW_STATUS cell G was disturbed during T time D of X procedure.
```

### Severity

Error

### Description

Note the following:

G indicates gate\_id of failing MISR\_SHADOW\_STATUS cell

T indicates time during which failure occurred (preamble or postamble)

X indicates the name of the procedure

The slow MISR\_SHADOW\_STATUS cells are checked to ensure that they hold state during the preamble cycles and nonshifting postamble cycles of the MISR\_SHADOW\_STATUS\_UNLOAD procedure. This checking is done using the precalculated simulated values of the MISR\_SHADOW\_STATUS\_UNLOAD procedure. Slow MISR\_SHADOW\_STATUS cells which do not hold state for all of the checked time periods will result in a violation of this rule.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that failing cells hold state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the failing cell with pindata set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the MISR\_SHADOW\_STATUS cells. The command usage is:

```
report_compressors -cells misr_shadow_status
```

This rule violation may be downgraded to warning but it is not recommended since this will likely cause the IP test patterns to be incorrect.

---

## DRC Rule K89

### Message Text

DFTMAX LBIST Rule K89: Invalid ShadowMISR shift operation

N ShadowMISR cells (input I of P/G) in seq compressor S had invalid shift values during X procedure.

### Severity

Error

### Description

Note the following:

N indicates the number of failing ShadowMISR cells

I indicates the input ID of the first failing cell in the ShadowMISR register

P indicates the position of the failing cell in the ShadowMISR register

G indicates the gate ID of the failing cell in the ShadowMISR register

S indicates the name of the DFTMAX Seq CODEC associated with the failing ShadowMISR register

X indicates the name of the procedure

The misr\_shadow\_load procedure is used to ensure that ShadowMISR cells are shifting data successfully.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that ShadowMISR cells are properly shifting during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing cell of the failing ShadowMISR register. If the failure occurred on a clock/set/reset input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the source of its capture data are also displayed.

This rule violation may be downgraded to warning but it is not recommended since this will likely cause patterns to be incorrect.

---

## DRC Rule K90

### Message Text

DFTMAX LBIST Rule K90: Codec "*name1*" differs from MasterCODECAddress *n*("name2") by condition (*n1*/*n2*)

### Severity

Error

### Description

When MasterCODECAddress keywords are used to identify an expected relationship between two codecs, DRC validates the required structural relationships between the two codecs. If the structural requirements are not satisfied the MasterCODEC relationship will not be supportable. The list of checked conditions are below. The error message also identifies the difference in value between the first code (the "slave", reported as value *n1*) and the second codec (the "master" reported as value *n2*).

The condition is one of the following:

number of multiple observe modes

full observe mode value

full observe mode mask

number of full-observable scanchains

the length of scan chains on connection *n* (chains "*ch\_name1*"/"*ch\_name2*")

the parity of scan chains on connection *n* (chains "*ch\_name1*"/"*ch\_name2*")

the number of taps on connection *n*

the tap positions on connection *n*, index *n*

### What Next

To correct this problem, the codecs in the netlist and the SPF description must be aligned, or the MasterCODECAddress reference in the SPF can be removed to not match these two codecs as similar.

This rule violation can be downgraded to warning, but it is not recommended as this will likely impact performance and QoR as the ATPG flow will attempt to generate patterns to apply to both codes and will not be successful.

---

## DRC Rule K91

### Message Text

DFTMAX LBIST Rule 91: Invalid initialization of MISRs

N misr (M) cells (C/G) in seq compressor S had incorrect value at end of test\_setup procedure.

### Severity

Error

### Description

Note the following:

N indicates the number of failing cells in the failing MISR

M indicates the name of the failing MISR (only used if multiple MISRs per CODEC)

C indicates the cell ID of the first failing cell in MISR

G indicates the gate ID of the first failing cell in MISR

S indicates the name of the compressor associated with the failing chain

When the MASK\_ALL\_CHAINS cell is defined and extended test setup is not used, all MISRs are checked to ensure they are initialized to 0 at the end of the test setup procedure. MISR cells which are not at 0 will result in a violation of this rule. Only one violation is issued for a MISR.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that MISR cells are set to 0 at the end of the TEST\_SETUP procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing MISR cell with pindata set to the stored simulated values of the TEST\_SETUP procedure. This problem can also be corrected by supporting the usage of the extended test setup procedure in the test procedure file.

This rule violation may be downgraded to warning but it is not recommended since this will likely cause the first BIST pattern interval to be incorrect.

---

## DRC Rule K92

### Message Text

DFTMAX LBIST Rule K92: Invalid initialization of MASK\_ALL\_CHAINS cell

`MASK_ALL_CHAINS` cell (G) had incorrect value (V) at end of the `test_setup` procedure.

### Severity

Error

### Description

Note the following:

G indicates gate ID of failing cell

V indicates the gate ID of the failing chain

When the `MASK_ALL_CHAINS` cell is defined and extended test setup is not used, the `MASK_ALL_CHAINS` cell is checked to ensure it is initialized to the complement of the `mask_all_chains_value` at the end of the test setup procedure. An incorrect value will result in a violation of this rule.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that the `MASK_ALL_CHAINS` cell is set to the `mask_all_chains_value` at the end of the `TEST_SETUP` procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the `MASK_ALL_CHAINS` cell with pindata set to the stored simulated values of the `TEST_SETUP` procedure. This problem can also be corrected by supporting the usage of the extended test setup procedure in the test procedure file.

This rule violation may be downgraded to warning but it is not recommended since this will likely cause the first SEQ pattern to be incorrect.

---

## DRC Rule K93

### Message Text

DFTMAX LBIST Rule K93: Invalid setting of `MASK_ALL_CHAINS` cell

`MASK_ALL_CHAINS` cell (G) had incorrect set value (V) at end of P procedure.

### Severity

Error

### Description

Note the following:

G indicates the gate\_id of failing cell

V indicates the incorrect value at end of procedure

P indicates the name of the failing procedure

When the MASK\_ALL\_CHAINS cell is defined, the MASK\_ALL\_CHAINS cell is checked to ensure it is set to the mask\_all\_chains\_value at the end of the PATCOUNTER\_LOAD procedure and is set to its complement value at the end of the LOAD procedure. An incorrect value will result in a violation of this rule.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that the MASK\_ALL\_CHAINS cell is set to the correct value at the end of the PATCOUNTER\_LOAD and the LOAD procedure. The automated violation analysis can be used to help debug this problem. This analysis will display the MASK\_ALL\_CHAINS cell with pindata set to the stored simulated values of the failing procedure.

This rule violation may be downgraded to warning but it is not recommended since this will likely cause BIST pattern intervals to be incorrect.

---

## DRC Rule K94

### Message Text

DFTMAX LBIST Rule K94: Invalid MISR hold for first unload

N misr (M) cells (C/G) in seq compressor S had incorrect value at end of test\_setup procedure.

N misr (M) cells (C/%G) in seq compressor S did not hold value during first unload.

### Severity

Error

### Description

Note the following:

N indicates the number of failing cells in failing MISR

M indicates the name of the failing MISR (only used if multiple MISRs per CODEC)

C indicates the cell ID of first failing cell in MISR

**G** indicates the gate ID of first failing cell in MISR

**S** indicates the name of the seq compressor associated with the failing MISR

When the MASK\_ALL\_CHAINS cell is defined, all MISRs are checked to ensure they hold their value during the first scanchain unload of a BIST pattern interval. MISR cells which do not hold will result in a violation of this rule. Only one violation is issued for a MISR.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that MISR cells hold their value during the first scanchain unload of a BIST pattern interval. The automated violation analysis can be used to help debug this problem. This analysis will display the first failing MISR cell with pindata set to the stored simulated values of the TEST\_SETUP procedure.

This rule violation may be downgraded to a warning but it is not recommended since this will likely cause the BIST pattern intervals to be incorrect.

---

## DRC Rule K95

### Message Text

Invalid setting of FIRST\_SHIFT cell.

FIRST\_SHIFT cell (G) had incorrect set value (V) during P procedure.

### Severity

Error

### Description

#### Note:

- **G** indicates the gate ID of the failing cell
- **V** indicates incorrect set value during the failing procedure
- **P** indicates the name of the failing procedure

If the FIRST\_SHIFT cell is defined, the FIRST\_SHIFT cell is checked to ensure that it is set to the first\_shift value at the beginning of the LOAD\_UNLOAD procedure and to its complement value at the end of the SHIFT procedure. An incorrect value results in a violation of this rule.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that the FIRST\_SHIFT cell is set to the correct value at the beginning of the LOAD\_UNLOAD



procedure and the end of the SHIFT procedure. You can use the automated violation analysis to debug this problem. This analysis displays the trace backward gates of the FIRST\_SHIFT cell with pin data set to the stored simulated values of the failing procedure.

---

## DRC Rule K96

### Message Text

Invalid capture of XTOL\_ENABLE cell for MASK\_ENABLE full\_observe-on/off mode.

XTOL\_ENABLE cell (input I of G) in seq compressor S had invalid capture for MASK\_ENABLE full\_observe\_on mode.

XTOL\_ENABLE cell (input I of G) in SEQ compressor S had invalid capture for MASK\_ENABLE full\_observe\_off mode.

### Severity

Error

### Description

#### Note:

- **I** indicates the input ID of failing XTOL\_ENABLE cell (0-set, 1-reset, 2-clock, 3-D pin)
- **G** indicates the gate ID of the failing XTOL\_ENABLE cell
- **S** indicates name of the DFTMAX Seq CODEC associated with failing XTOL\_ENABLE cell

This check is only performed if MASK\_ENABLE cells are used. This checks both the full\_observe\_on and full\_observe\_off observe modes.

When the XTOL\_ENABLE cells are set to their off state, the XTOLPRPG outputs are set to select the full\_observe\_on mode and mask\_enable cells are set to their on state, the XTOL\_ENABLE cells are checked to ensure that they successfully capture the on state for XTOL\_ENABLES. This checking is done by performing parallel pattern simulation using the simulated values of the SHIFT procedure and placing MASK\_ENABLES on, XTOL\_ENABLE cells to their off state, and XTOLPRPG outputs to select the full\_observe\_on mode. XTOL\_ENABLE cells that do not capture the correct value for all of the simulated parallel patterns results in a violation of this rule.

When the XTOL\_ENABLE cells are set to their on state and the XTOLPRPG outputs are set to select the full\_observe\_off mode and mask\_enable cells are set to their on state, the XTOL\_ENABLE cells are checked to ensure that they successfully capture the off state for XTOL\_ENABLES. This checking is done by performing parallel pattern simulation

using the simulated values of the SHIFT procedure and placing MASK\_ENABLES on, XTOL\_ENABLE cells to their on state, and XTOLPRPG outputs to select the full\_observe\_off mode. XTOL\_ENABLE cells which do not capture the correct value for all of the simulated parallel patterns results in a violation of this rule.

### What Next

To correct this problem, you must change the netlist or STIL protocol file so that the failing XTOL\_ENABLE cell captures the correct value in full\_observe\_on and full\_observe\_off modes.

You can use automated violation analysis to debug this problem. This analysis displays the failing XTOL\_ENABLE cell, XTOLPRPG output, XTOL\_ENABLE values, Mask\_ALL\_CHAINS values, XCHAIN\_REGISTERS\_CELL, and MASK\_ENABLE\_CELLS. The displayed pin data is set to the simulated values of the SHIFT procedure and setting all the conditions used for this checking.

If the failing ID is 2(clock input), then clock on shift pattern values are being displayed.

If the failing ID is 2(clock input), the `-shift` option of the `set_pindata` command helps to identify the problem with the clock input.

Use the `-cells` option of the `report_compressors` command to report the XTOL\_ENABLE, MASK\_ENABLE, or MISR cells.

Use the `-lfsr_connections` option of the `report_compressors` command to report the XTOLPRPG output values.

Use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

You can downgrade this rule violation to a warning. However, it is not recommended since this is likely to cause incorrect patterns that exercise the full\_observe\_on and full\_observe\_off modes.

---

## DRC Rule K97

### Message Text

```
Invalid MISR capture value for MASK_ENABLE full_observe-on/off mode.
```

```
N MISR cells (P/G) in seq compressor S had invalid capture data for  
full_observe_on mode.
```

```
N MISR cells (P/G) in seq compressor S had invalid capture data for  
full_observe_off mode.
```

## Severity

Error

## Description

Note the following:

**N** indicates the number of failing MISR cells

**P** indicates the position of the first failing cell in the MISR

**G** indicates the gate ID of the first failing cell in the MISR

**s** indicates the name of the DFTMAX Seq CODEC associated with the failing MISR

This check ensures that the data inputs of MISR cells successfully capture full\_observe mode values without xchain credit.

Checking is done by performing parallel pattern simulation using the simulated values of the SHIFT procedure

The data inputs of MISR cells are first checked to ensure that they successfully capture full\_observe mode values without xchain credit under the following conditions:

- MISR cells are set to 0
- The XTOL\_ENABLE cells are set to the off state
- The XTOLPRPG outputs are set to select the full\_observe\_on mode
- The MASK\_ENABLE cells are set to their on state

The data inputs of MISR cells are next checked to ensure that they successfully capture full\_observe mode values without xchain credit under the following conditions:

- MISR cells are set to 0
- The XTOL\_ENABLE cells are set to the on state
- The XTOLPRPG outputs are set to select the full\_observe\_off mode
- The MASK\_ENABLE cells are set to their on state

Data inputs of MISR cells which are not at the correct full\_observe value for all of the simulated parallel patterns cause a violation of the K97 rule. Only one violation is issued for a given MISR.

## What Next

To correct this problem, you must change the netlist or STIL procedure file so that the data input of the failing MISR cell is set to the correct value in the full\_observe\_on and full\_observe\_off modes.

You can use automated violation analysis to debug this problem. This analysis displays the first failing cell for the failing MISR. The displayed pin data is set to the simulated values of the SHIFT procedure and sets all the conditions used for this checking. Also displayed are the gates in the paths between the XTOLPRPG outputs used to select the full\_observe\_on and full\_observe\_off modes and the failing MISR cell.

Use the `-cells` option of the `report_compressors` command to report the XTOL\_ENABLE, MASK\_ENABLE, or MISR cells.

Use the `-lfsr_connections` option of the `report_compressors` command to report the XTOLPRPG output values.

Use the `-mode_data` option of the `report_compressors` command to report the XTOLPRPG output values for all observe modes.

You can downgrade this rule violation to a warning. But it is not recommended since this likely causes patterns which exercise the full\_observe\_on and full\_observe\_off modes to be incorrect.

---

## DRC Rule K98

### Message Text

Invalid MASK\_ENABLE reload operation

Mask\_enable cell (input I of G) in seq compressor S had invalid value during P procedure.

### Severity

Error

### Description

Note the following:

**I** indicates the input ID of first failing MASK\_ENABLE cell

**G** indicates the gate ID of failing MASK\_ENABLE cell

**S** indicates the name of seq compressor associated with failing MASK\_ENABLE cell

**P** indicates the name of procedure

When the XTOL\_ENABLE and POWER\_ENABLE cells are set to their off state, this check ensures that the MASK\_ENABLE cells successfully capture reseed data from the PRPG\_SHADOW during the SHADOW\_TO\_XTOLPRPG procedure.

This check performs parallel pattern simulation using the simulated values of the appropriate procedure, and places the XTOL\_ENABLE and POWER\_ENABLE cells to their off state and all other register types to random values.

All MASK\_ENABLE cells which do not successfully capture their expected value for all of the simulated parallel patterns cause a violation of the K98 rule. The expected values for the cells are determined by the values from the associated cells in the PRPG\_SHADOW.

### What Next

To correct this problem, you must change the netlist or the STIL procedure file so the MASK\_ENABLE cells are properly capturing data during the specified procedure.

You can use automated violation analysis to debug this problem. This analysis displays the failing MASK\_ENABLE cell.

If the failure occurs on a clock, set, or reset input, the pin data is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are displayed. Otherwise, the pin data displays one of the parallel pattern values that failed and the gates in the path between the failing cell and the source of its capture data are also displayed.

Use the `-cells` option of the `report_compressors` command to report the XTOL\_ENABLE, POWER\_ENABLE, or MASK\_ENABLE cells.

You can downgrade this rule violation to a warning. But it is not recommended since this will likely cause the BIST pattern intervals to be incorrect.

---

## DRC Rule K99

### Message Text

```
mask_enable cell disturb.
```

```
mask_enable cell (G) in seq compressor S was disturbed during P  
procedure.
```

### Severity

Error

### Description

Note the following:

G indicates the gate ID of failing mask\_enable cell

S indicates the name of seq compressor associated with failing mask\_enable cell

P indicates the name of procedure

This check ensures the mask\_enable cells hold state during the SHIFT, SHADOW\_TO\_CAREPRPG, RESEED\_WITH\_MISR\_SHIFT, RESEED\_SHIFT, and RESEED\_PARTIAL\_OVERLAP\_SHIFT1 procedures.

When the xtol\_enable cells are set to active, the power\_enable cells are set to inactive, and the codec enables are set active, the mask\_enable cells are also checked to ensure that they hold state during the SHADOW\_TO\_XTOLPRPG procedure.

When xchains\_registers are used, some procedures are also checked to consider CODEC disabling variations. This checks all combinations with a single disabled codec (it is used as an X-chain and is only observable in single observe mode) and all other codecs enabled. When failures occur during checking with a codec disabled, an error message indicates which codec was disabled.

### What Next

To correct this problem, you must change the netlist or the STIL procedure file so the mask\_enable cells hold state during the specified procedure.

You can use automated violation analysis to debug this problem. This analysis displays all the gates in the traceback cone of the input which appear to cause the problem. The pindata is set to the simulated values of the failing procedure.

Use the -cells option of the report\_compressors command to report the xtol\_enable, power\_enable, or mask\_enable cells.

Use the -summary option of the report\_compressors command to report the codec\_enable cells with their associated enable values.

You can downgrade this rule violation to a warning. But it is not recommended since this will likely cause the pattern intervals to be incorrect.

---

## DRC Rule K100

### Message Text

Expected seven random bits cells, found N

This error is reported when 'N' is more than seven.

### Severity

Error

### Description

Random-bits registers are used for randomly selecting different sets of clocks for XLBIST. Current architecture supports up to seven registers which supply random values to clock the weight decoder logic.

### What Next

To solve this problem, the netlist and the test procedure file must be changed, and random-bits registers count should be made seven or less.

This rule violation cannot be downgraded to warning.

---

## DRC Rule K101

### Message Text

N random bits cells (P/G) were disturbed during X procedure, where  
N indicates the number of failing random-bits cells  
P indicates the position of the failing cell in the random-bits register  
G indicates the gate ID of the failing cell in the random-bits register  
X indicates the name of the procedure

### Severity

Error

### Description

When the `xtol_enable` and `power_enable` cells are set to their off state, and codec is set to their active state, and the random-bits cell is checked to ensure that it holds state during the `SHIFT`, `RESEED_SHIFTA`, `RESEED_SHIFTB`, `RESEED_SHIFTC`, `RESEED_SHIFTD`, `RESEED_SHIFTE`, `RESEED_LOAD`, `RESEED_PARTIAL_OVERLAP_SHIFT1`, `RESEED_PARTIAL_OVERLAP_SHIFT2`, `SHADOW_TO_CAREPRPG` (with `capture_enable` set to complement of `capture_enable` value), `JTAG_MISR_SHIFT`, `OCC_SHIFT`, and capture procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, `xtol_enable`/`power_enable` cells set to their off state, codec enables set to their active state, and all other register types set to random values. Random-bits cells that do not hold state for all of the simulated parallel patterns result in a violation of this rule. Random-bits registers are used for randomly selecting different sets of clocks for XLBIST.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that random-bits cell holds state during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the failing random-bits cell with all the gates in the traceback cone of the input that were causing the problem. The `pindata` is set to the simulated values of the failing procedure.

This rule violation may be downgraded to a warning but it is not recommended since this likely causes patterns to be incorrect.

---

## DRC Rule K102

### Message Text

Set/Reset is not OFF for clock control cell 'S'.

Clock control cell 'S' has invalid capture

Where 'S' is the name of the BistWeightEnable gate.

### Severity

Warning

### Description

Clock weighted XLBIIST functionality depends upon proper functioning of the registers specified in protocol file with tag 'BistWeightEnable'. If clocking or other asynchronous inputs of these registers are not functioning properly, clock weights mechanism would not work as expected.

### What Next

To correct this problem trace back from the clock pin of the reported BistWeightEnable gate and look for the cause of clock not pulsing. You can use automated violation analysis to debug this problem. This analysis displays the failing 'BistWeightEnable' cell. The displayed pin data is set to the simulated values of the SHIFT procedure and sets all the conditions used for this checking.

---

## DRC Rule K103

### Message Text

Forbidden clocks 'C1' and 'C2' are grouped together.

Where 'C1' and 'C2' are the gate-ids of clock input pin.

### Severity

Error

### Description

This message indicates clocks which are forbidden to be pulsed together are being pulsed. When clock weights are enabled for XLBIIST, different sets of clocks are pulsed



per pattern depending upon the randomized values of random bit registers. K103 indicates two clocks were pulsed together and these two clocks were marked as disturbed and there are too many cells that might get an 'X' during capture if these clocks are pulsed together. Pulsing these two clocks would cause huge number of cells to capture Xs and this would cause most/all chains being masked by XLBIST. This would cause very low test coverage.

#### What Next

Use the `report_clocks -matrix` command and see the report about which clocks are groupable. You may have to redo DFT with proper clock grouping specifications.

You may downgrade this error to warning, but it would cause very low test coverage.

---

## DRC Rule K104

### Message Text

Disturbed clocks 'C1' and 'C2' are grouped together.

C1 and C2 are gate-ids of clocks.

### Severity

Warning

### Description

When clock weights are enabled for XLBIST, different sets of clocks are pulsed per pattern depending upon the randomized values of random bit registers. K104 indicates two clocks were pulsed together and these two clocks were marked as disturbed and there are cells that might get an 'X' during capture if these clocks are pulsed together. Pulsing these two clocks would cause cells to capture Xs and this would cause chains being masked by XLBIST. This would cause low test coverage.

### What Next

Use the `report_clocks -matrix` command and see the report about which clocks are groupable. You may have to redo DFT with proper clock grouping specifications. You may ignore this warning, but it would cause low test coverage.

---

## DRC Rule K105

### Message Text

Clock 'C' never applied.

Where 'C' is the gate-id of the reported clock.

### Severity

Warning

### Description

In clock weighted XLBIST, different set of clocks are chosen at random for each pattern. Total percentage of time a clock is chosen is determined by user specification during DFT. This warning indicates that some clocks were never pulsed. This could cause some clock domain not being tested at all, causing overall low test coverage.

### What Next

This might be an issue with DFT. You may check what clock specifications were there during DFT.

---

## DRC Rule K106

### Message Text

```
Groupable clocks 'C1' and 'C2' are not applied.  
'C1' and 'C2' are clock gate-ids.
```

### Severity

Warning

### Description

Clocks which are groupable should be pulsed together in some or other pattern. Pulsing all possible clocks together gives better coverage with lesser patterns. This warning indicates clocks which were found to be safe to be pulsed together (allowed clock groups) were not pulsed in any pattern. It could be an issue with how DFT grouped the clocks with clock-weights.

### What Next

This might be an issue with DFT. You may check what clock specifications were there during DFT.

---

## DRC Rule K107

### Message Text

```
Expected seven random bits cells, found only N  
'N' is the number of random bit register.
```

### Severity

Warning

### Description

Clock weighted XLBIST supports maximum seven random bit registers. In this case less number of registers were used. Using less number of registers restricts randomization.

### What Next

Check whether user specified clocks and percentages in DFT can be handled by less than seven random bits.

---

## DRC Rule K108

### Message Text

NEW\_ADDRESS/NEW\_ADDRESS\_SHADOW cell disturb.

NEW\_ADDRESS/NEW\_ADDRESS\_SHADOW cell (G) of IDU S was disturbed during X procedure.

### Severity

Error

### Description

#### Note:

- G indicates the gate ID of failing NEW\_ADDRESS/NEW\_ADDRESS\_SHADOW cell
- S indicates the IDU name
- X indicates the name of a procedure

When the XTOL\_ENABLE and POWER\_ENABLE cells are set to their off state and CODEC\_ENABLES are set to their active state, the NEW\_ADDRESS/NEW\_ADDRESS\_SHADOW cell is checked to ensure that it holds state during the SHIFT, RESEED\_SHIFTB, RESEED\_SHIFTD, RESEED\_LOAD, SHADOW\_TO\_CAREPRPG, and SHADOW\_TO\_XTOLPRPG.

This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL\_ENABLE/POWER\_ENABLE cells set to their off state, CODEC\_ENABLES set to their active state, and all other register types set to random values. NEW\_ADDRESS/NEW\_ADDRESS\_SHADOW cells which do not hold state for all of the simulated parallel patterns results in a violation of this rule.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that NEW\_ADDRESS/NEW\_ADDRESS\_SHADOW cell holds state during the specified procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing NEW\_ADDRESS/NEW\_ADDRESS\_SHADOW cell with all the gates in the trace-back cone of the input which was believed to have caused the problem. The pin data is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_ENABLE, power enable shadow cells.

The `-cells` option of the `report_idus` command can be used to report the NEW\_ADDRESS or NEW\_ADDRESS\_SHADOW cells and CODEC\_ENABLES with their associated enable values.

This rule violation may be downgraded to warning but it is not recommended since this likely causes patterns to be incorrect.

---

## DRC Rule K109

### Message Text

```
FINAL_SEGMENT cell disturb
```

```
FINAL_SEGMENT cell (G) of IDU S was disturbed during X procedure.
```

### Severity

Error

### Description

#### Note:

- `G` indicates gate ID of failing FINAL\_SEGMENT cell
- `S` indicates the IDU name
- `X` indicates the name of a procedure

When the XTOL\_ENABLE and POWER\_ENABLE cells are set to their off state and CODEC\_ENABLES are set to their active state, the FINAL\_SEGMENT cell is checked to ensure that it holds state during the SHIFT, RESEED\_SHIFTB, RESEED\_SHIFTD, RESEED\_LOAD, SHADOW\_TO\_CAREPRPG, and SHADOW\_TO\_XTOLPRPG procedures.

This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, XTOL\_ENABLE and POWER\_ENABLE cells set to

their off state, CODEC\_ENABLES set to their active state, and all other register types set to random values. FINAL\_SEGMENT cells which do not hold state for all of the simulated parallel patterns result in a violation of this rule.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that NEW\_ADDRESS/NEW\_ADDRESS\_SHADOW cell holds state during the specified procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing NEW\_ADDRESS/NEW\_ADDRESS\_SHADOW cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pin data is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_ENABLE, power enable shadow cells.

The `-cells` option of the `report_idus` command can be used to report the NEW\_ADDRESS or NEW\_ADDRESS\_SHADOW cells and CODEC\_ENABLES with their associated enable values.

This rule violation may be downgraded to warning but it is not recommended since this likely causes patterns to be incorrect.

---

## DRC Rule K110

### Message Text

IDU cells disturb during RESEED\_LOAD

N IDU register X cells (P/G) in seq compressor IDU S were disturbed during RESEED\_LOAD procedure.

### Severity

Error

### Description

#### Note:

- `G` indicates the gate ID of the failing cell in the IDU
- `S` indicates the seq compressor IDU name
- `X` indicates the IDU register name
- `N` indicates the number of failing IDU register X cells
- `P` indicates the position of failing cell in the IDU register

When the XTOL\_ENABLE and POWER\_ENABLE cells are set to their off state and CODEC\_ENABLES are set to their active state, the IDU cells are checked to ensure that they hold state during the RESEED\_LOAD procedure.

This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and XTOL\_ENABLE and POWER\_ENABLE cells set to their off state, CODEC\_ENABLES set to their active state, and all other register types set to random values. The IDU cells which do not hold state for all of the simulated parallel patterns result in a violation of this rule.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that IDU cells holds state during the RESEED\_LOAD procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing IDU register cell with all the gates in the traceback cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_ENABLE, and POWER\_ENABLE.

The `-cells` option of the `report_idus` command can be used to report the failings register and CODEC\_ENABLES with their associated enable values.

---

## DRC Rule K111

### Message Text

IDU cells disturb during MISR\_UNLOAD

N IDU register X cells (P/G) in seq compressor IDU S were disturbed during MISR\_UNLOAD procedure.

### Severity

Error

### Description

#### Note:

- `G` indicates the gate ID of the failing cell in the IDU
- `S` indicates the seq compressor IDU name
- `X` indicates the IDU register name
- `N` indicates the number of failing IDU register X cells
- `P` indicates the position of failing cell in the IDU register

When the XTOL\_ENABLE and POWER\_ENABLE cells are set to their off state and CODEC\_ENABLES are set to their active state, the IDU cells are checked to ensure that they hold state during the MISR\_UNLOAD procedure.

This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and XTOL\_ENABLE and POWER\_ENABLE cells set to their off state, CODEC\_ENABLES set to their active state, and all other register types set to random values. The IDU cells which do not hold state for all of the simulated parallel patterns result in a violation of this rule.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that IDU cells holds state during the MISR\_UNLOAD procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing IDU register cell with all the gates in the trace-back cone of the input which was believed to have caused the problem. The pindata is set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the XTOL\_ENABLE, and POWER\_ENABLE.

The `-cells` option of the `report_idus` command can be used to report the failings register and CODEC\_ENABLES with their associated enable values.

---

## DRC Rule K112

### Message Text

Incorrect value of Payload\_cycles\_counter register or  
Local\_cycles\_counter register

*N* Payload(Local)\_cycles\_counter\_reg cells(*P/G*) of IDU *S* had incorrect  
values (*g/e*) during procedure *Z*, where,

*N* indicates number of failing Payload(Local)\_cycles\_counter register  
cells

*P* indicates position of first failing Payload(Local)\_cycles\_counter cell

*G* indicates gate ID of first failing Payload(Local)\_cycles\_counter cell

*g* indicates the value of Payload(Local)\_cycles\_counter register

*e* indicates the expected value

*S* is the Seq compressor IDU name

*Z* indicates the procedure

## Severity

Error

## Description

Right after IDU decodes a procedure, `Payload_cycles_counter` register gets loaded from the corresponding `cycles_reg` register as follows:

| Procedure                 | Corresponding cycles_reg                                                                                               |
|---------------------------|------------------------------------------------------------------------------------------------------------------------|
| Reseed_load               | NewAddressCyclesReg if new_address_shadow cell value is 1. IncrAddressCyclesReg if new_address_shadow cell value is 0. |
| Occ_load                  | OCCCyclesReg                                                                                                           |
| Misr_load                 | MISRLoadCyclesReg                                                                                                      |
| Misr_unload               | MISRCyclesReg                                                                                                          |
| Misr_shadow_status_unload | ShadowUnloadCyclesReg                                                                                                  |
| MISR_status_unload        | StatusUnloadCyclesReg                                                                                                  |
| Codec_drc_load_unload     | -                                                                                                                      |

`Payload_cycles_counter` cells are checked to ensure that they are loaded with correct value at the end of procedure decoding for a given procedure, and their count operation is correct(should get decremented by 1 for a shift) during shift procedures. This checking is done by first initializing the state of the circuit to the previous procedure simulation values at the beginning of a particular procedure. At the end of procedure decoding, the `Payload_cycles_counter` register should have a value which is equal to the value in the corresponding `cycles_reg` of a procedure if procedure is decoded correctly. For the shift procedure, the `Payload_cycles_counter` register should have value 1 less than the value it had before shifting. `Payload_cycles_counter` register cells which are not at their correct value results in a violation of this rule.

### Check for codec\_drc\_load\_unload Procedure

For `codec_drc_load_unload` procedure we check `Payload_cycles_counter` cell. It must be non-zero at the end of procedure decoding and zero during procedure decoding.

Similarly this check is done for `Local_cycles_counter` register also which is shown in the following table:

| Procedure | Corresponding local_cycles_reg |
|-----------|--------------------------------|
| Occ_load  | OCCLocalCyclesReg              |



| Procedure        | Corresponding local_cycles_reg |
|------------------|--------------------------------|
| Misr_shadow_load | ShadowLoadLocalCycles          |

## What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that `Payload(Local)_cycles_counter` register has a correct value after procedure decoding. The automated violation analysis can be used to help debug this problem. This analysis displays the first failing `Payload(Local)_cycles_counter` cell and the corresponding `cycles_reg` cell. The pindata is set to the simulated values of the failing procedure. Tracing the `Payload(Local)_cycles_counter` cell further gives you information as to why it is not loaded correctly. Many a times state machine flops get X during the earlier procedure simulation which causes incorrect decoding of the current procedure. The following command can be used to know the details about previous procedure simulation data:

```
Set_drc -analyze_drc_violation {K112 gate_id}
```

It displays the gate (provided by `gate_id` argument) value at the end of every procedure that is simulated, and it also stores the simulation data under pindata "`k112_debug_data`". The following option can be used to see the simulation data of a particular procedure:

```
Set_pindata -k112_debug_data {start_cycle end_cycle}
```

Start\_cycles and end\_cycles of procedures are printed when "`set_drc -analyze_drc_violation`" command is enabled for K112.

The `-cells` option of the `report_idus` command can be used to report the `Payload(Local)_cycles_counter` cells and various `cycles_registers`.

### *Hint to fix Issue With `codec_drc_load_unload` Procedure*

Even if you load correct value in `Payload_cycles_counter` CDR bits, you might still get K112 error. One of the reasons could be value of `InternalLoadDelay` present in protocol file and netlist are not same.

For example:

Case 1 : If you get the following error

```
Error: payload_cycles_counter_reg cells(2/65131) of IDU <idu_name> had
incorrect
values(got=<non_zero_value>/exp=0) at delay_cycle = 2 during procedure
codec_drc_load_unload.(K112-1)
```

It is highly possible that

```
InternalLoadDelay on netlist < InternalLoadDelay on spf
```

Case 2 : If you get the following error an error:

```
Error: payload_cycles_counter_reg cells(1/65131) of IDU <idu_name> had
incorrect
values(got=0/exp=Non-zero positive value) at delay_cycle = 1 during
procedure
codec_drc_load_unload. (K112-1)
```

It is highly possible that

```
InternalLoadDelay on netlist > InternalLoadDelay on spf
```

---

## DRC Rule K113

### Message Text

Incorrect New(or Increment)\_address\_cycles\_reg value

NewAddressCycles register value (d1) of IDU S is not equal to the number of cycles required to load PRPG and IR (d2).

(OR)

IncrAddressCycles register value (d3) of IDU S is not equal to the number of cycles required to load PRPG and Partial\_IR (i.e. only x,b bits) d4.

### Severity

Error

### Description

#### Note:

- d1 indicates the value in the NewAddrssCycles register
- d2 indicates seq compressor IDU name
- d3 indicates the value in the IncrAddrssCycles register
- d4 indicates the number of cycles required to load PRPG and Partial\_IR(i.e. only x, b bits)
- s indicates Seq compressor IDU name

At the end of test\_setup or extended\_test\_setup (if present), the NewAddressCyclesReg and IncrAddressCyclesReg registers are checked to ensure that they are loaded with the values d1 and d2 respectively, where d1 is the number of cycles required to load the PRPG and IR, d2 is the number of cycles required to load the PRPG and Partial\_IR (i.e. only x,b bits). The NewAddressCyclesReg or IncrAddressCyclesReg which are not at their correct value results in a violation of this rule.

### What Next

To correct this problem, STIL protocol file must be changed so that NewAddressCyclesReg and IncrAddressCyclesReg have correct values. You can use the automated violation analysis to check the value in the NewAddressCyclesReg and IncrAddressCyclesReg. The automated violation analysis displays the first NewAddressCyclesReg or IncrAddressCyclesReg cell. The pindata is set to the simulated values of the CODEC\_DRC\_SHIFT procedure.

The `-cells` option of the `report_idus` command can be used to report the NewAddressCyclesReg and IncrAddressCyclesReg.

This rule violation may be downgraded to a warning but it is not recommended since this likely to cause patterns to be incorrect.

---

## DRC Rule K114

### Message Text

Incorrect loading of new\_address reg

Incorrect value at pin P of new\_address\_reg (Gate G of IDU S) during procedure X.

### Severity

Error

### Description

#### Note:

- P indicates the input pin of failing NEW\_ADDRESS cell
- G indicates the gate ID of failing NEW\_ADDRESS cell
- S indicates the name of IDU
- X indicates the name of procedure

The NEW\_ADDRESS cells are checked to ensure that they are successfully loading data from the NEW\_ADDRESS\_SHADOW during RESEED\_SHIFTA procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure, and placing PAYLOAD\_CYCLES\_COUNTER to the value of 2. NEW\_ADDRESS cells which do not successfully capture their expected value for all of the simulated parallel patterns result in a violation of this rule. The expected values for a NEW\_ADDRESS cell are the values assigned to the associated NEW\_ADDRESS\_SHADOW cell.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that NEW\_ADDRESS cells are properly capturing data from its NEW\_ADDRESS\_SHADOW during the specified procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing NEW\_ADDRESS cell. If the failure occurred on a clock/set/reset input, then the pin data is set to the simulated values for the indicated procedure and all gates in the trace-back cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing cell and the associated cell in the NEW\_ADDRESS\_SHADOW are also displayed.

The `-cells` option of the `report_idus` command can be used to report the NEW\_ADDRESS or NEW\_ADDRESS\_SHADOW cells.

This rule violation may be downgraded to a warning but it is not recommended since this is likely to cause patterns to be incorrect.

---

## DRC Rule K115

### Message Text

```
Invalid misr_status update operation
```

```
misr_status cell (input I of G) had invalid update value during X  
procedure.
```

### Severity

Error

### Description

#### Note:

- `I` indicates the input which caused problem of first failing cell
- `G` indicates the gate ID of failing MISR\_STATUS cell
- `X` indicates the name of procedure

The MISR\_STATUS cells are checked to ensure that they are successfully capturing data from the MISR register during the last clock pulse of the MISR\_SHADOW\_STATUS\_UPDATE procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the last clock pulse in the MISR\_SHADOW\_STATUS\_UPDATE procedure and placing random values on the MISR cells. The expected value for a MISR\_STATUS cell of a CODEC is the result of ORing all the values that were assigned to the MISR register cells of that CODEC. MISR\_STATUS

cells which do not successfully capture their expected value for all of the parallel patterns simulated results in a violation of this rule.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that MISR\_STATUS cells are properly updating data from the associated MISR cells during the MISR\_SHADOW\_STATUS\_UPDATE procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the failing MISR\_STATUS cell with pindata set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the MISR\_STATUS cells.

The command usage is: `Report Compressors <-All | comp_name> [-Cells <MISR_STATUS>]`

This rule violation may be downgraded to a warning but it is not recommended since this is likely to cause patterns to be incorrect due to invalid measures associated with unloading MISR\_STATUS\_CELLS.

---

## DRC Rule K116

### Message Text

MISR\_STATUS cell disturb

MISR\_STATUS cell (G) in seq compressor C was disturbed during X procedure.

### Severity

Error

### Description

#### Note:

- G indicates the gate ID of failing MISR\_STATUS cell
- C indicates the name of compressor containing failing MISR\_STATUS cell
- X indicates the name of procedure

The MISR\_STATUS register cells are checked to ensure that they hold state during the MISR\_LOAD\_SHIFT procedure and all other procedures where the MISR must hold state that are not associated with MISR\_STATUS operations. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate

procedure. MISR\_STATUS cells which do not hold state for all of the parallel patterns simulated results in a violation of this rule.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that MISR\_STATUS cells hold state during the specified procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing MISR\_STATUS cell with pin data set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the MISR\_STATUS cells.

The command usage is: `Report Compressors <-All | comp_name> [-Cells <MISR_STATUS>]`

This rule violation may be downgraded to a warning but it is not recommended since this is likely to cause patterns to be incorrect due to invalid measures associated with unloading MISR\_STATUS\_CELLS.

---

## DRC Rule K117

### Message Text

MISR\_STATUS cell disturb

MISR\_STATUS cell G was disturbed during T time %d of X procedure.

### Severity

Warning

### Description

#### Note:

- G indicates the gate ID of failing MISR\_STATUS cell
- T indicates the time during which failure occurred (preamble or postamble)
- X indicates the name of procedure

The slow MISR\_STATUS cells are checked to ensure that they hold state during the preamble cycles and non-shifting postamble cycles of the MISR\_STATUS\_UNLOAD procedure. This checking is done using the precalculated simulated values of the MISR\_STATUS\_UNLOAD procedure. Slow MISR\_STATUS cells which do not hold state for all of the checked time periods result in a violation of this rule.

### What Next

To correct this problem, the netlist or the test procedure file must be changed so that failing cells hold state during the specified procedure. You can use the automated violation analysis to debug this problem. This analysis displays the failing cell with pindata set to the simulated values of the failing procedure.

The `-cells` option of the `report_compressors` command can be used to report the `MISR_STATUS` cells.

The command usage is: `<-All | comp_name> [-Cells <MISR_STATUS>]`

This rule violation may be downgraded to a warning but it is not recommended since this is likely to cause the IP test patterns to be incorrect.

---

## DRC Rule K118

### Message Text

Invalid shift\_counter\_length

Shift Counter Length L1 is not sufficient to accommodate scan\_chain X of length L2.

### Severity

Error

### Description

#### Note:

- L1 indicates the length of the Shift Counter register
- L2 indicates the length of the scan\_chain X
- x indicates the name of the scan\_chain

Shift Counter is loaded with the value equal to the number of shifts required to load the scan chains. Shift Counter of length  $L$  can be loaded with maximum  $2^L$ . If the `scan_chain` length is greater than  $2^L$ , it cannot be fully loaded using shift counter of length  $L$ .

### What Next

To correct this problem, the netlist must be changed to have a shift counter of sufficient length.

---

## DRC Rule K119

### Message Text

N silencer\_DLATs (P/G) in seq compressor S failed hold during X procedure.

where:

N indicates number of failing silencer\_DLATs

P indicates position of the care\_shadow cell(of the corresponding silencer\_DLAT) in the CARE\_shadow register

G indicates gate ID of failing silencer\_DLAT

S indicates name of DFTMAX Seq CODEC associated with failing DLAT

X indicates name of procedure

### Severity

Error

### Description

Silencer\_DLATs are checked to ensure that they are successfully holding state during the SHIFT, RESEED\_SHIFT, JTAG\_RESEED\_SETUP and MISR\_UNLOAD procedures. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedures. Silencer\_DLATs which do not successfully hold their value for all the simulated parallel patterns results in a violation of this rule. Only one violation is issued for all the DLATs.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that silencer\_DLATs are holding values during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the first failing silencer\_DLAT of the silencer\_DLATs of the associated CARE\_shadow register. If the failure was due to a problem on a clock input, then the pindata is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed.

The -cells option of the report\_compressors command can be used to report the CARE\_Shadow cells.

This rule violation can be downgraded to a warning but it is not recommended since this likely causes patterns which exercise the silencer\_DLAT to be incorrect.



---

## DRC Rule K120

### Message Text

N `silencer_DLATs` (input I of P/G) in seq compressor S had invalid value during X procedure.

where:

N indicates number of failing `silencer_DLATs`

I indicates input ID of failing `silencer_DLAT`

P indicates position of the `care_shadow` cell(of the corresponding `silencer_DLAT`) in the `CARE_shadow` register

G indicates gate ID of failing `silencer_DLAT`

S indicates name of DFTMAX Seq CODEC associated with failing DLAT

X indicates name of procedure

### Severity

Error

### Description

`Silencer_DLATs` are checked to ensure that they are successfully loading data from the respective `CARE_Shadow` cells during `OCC_LOAD` procedure. This checking is done by performing parallel (32) pattern simulation using the simulated values of the appropriate procedure. `Silencer_DLATs` which do not successfully capture their expected value for all of the simulated parallel patterns results in a violation of this rule. The expected values for a `silencer_DLAT` are the values assigned to the associated `CARE_Shadow` cell in the `CARE_Shadow` register. Only one violation is issued for all the `silencer_DLATs` in a compressor.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that `silencer_DLATs` are properly capturing data from its `CARE_Shadow` cells during the specified procedure. The automated violation analysis can be used to help debug this problem. This analysis displays the first failing `silencer_DLAT` and the respective `CARE_Shadow` cell. If the failure occurred on a clock input, then the pin data is set to the simulated values for the indicated procedure and all gates in the traceback cone of this input are also displayed. Otherwise, the displayed pin data is set to one of the parallel pattern values that failed and gates in the path between the failing `silencer_DLAT` and the associated cell in the `CARE_Shadow` register are also displayed.

The `-cells` option of the `report_compressors` command can be used to report the `CARE_Shadow` cells.

This rule violation may be downgraded to warning but it is not recommended since this likely causes patterns which exercise the `silencer_DLAT` to be incorrect.

# 6

## Category N - Netlist Rules

---

### See Also

- [N1 - parsing error](#)
- [N2 - unsupported construct](#)
- [N3 - maximum string size exceeded](#)
- [N4 - redefined cell](#)
- [N5 - redefined module](#)
- [N6 - duplicate definition](#)
- [N7 - missing definition](#)
- [N8 - invalid construct](#)
- [N9 - nameless instance](#)
- [N10 - ambiguous vector net direction](#)
- [N11 - redefined UDP entry](#)
- [N12 - invalid UDP entry](#)
- [N13 - X\\_DETECTOR found](#)
- [N14 - illegal UDP](#)
- [N15 - incomplete UDP](#)
- [N16 - overspecified UDP](#)
- [N17 - UDP failed validation](#)
- [N18 - net has name of port](#)
- [N19 - unused construct](#)
- [N20 - underspecified UDP](#)
- [N21 - unsupported UDP entry](#)

- [N22 - duplicate pin](#)
- [N23 - inconsistent UDP](#)
- [N24 - UDP cannot hold state](#)
- [N25 - redefined instantiated module](#)
- [N26 - empty model](#)
- [N27 - incomplete memory definition](#)
- [N28 - unsupported priority](#)
- [N29 - tied inout net](#)
- [N30 - inconsistent memory definition](#)
- [N31 - Name length exceeded maximum and was truncated](#)
- [N32 - Behavioral syntax detected in netlist](#)
- [N33 - Undefined port in netlist.](#)

---

## DRC Rule N1

### Message Text

Netlist Rule N1: Line *L* (*filename*), parsing error (*additional\_info*).

### Severity

Error or Fatal Error

### Description

A syntax error was encountered while reading the netlist. Either the netlist has a syntax problem, or there is a feature of the Verilog language not supported by TestMAX ATPG.

### What Next

The line number *L* in the specified *filename* identifies the area where the problem was detected. In general, the netlist file must be modified to overcome the problem.

The *additional\_info* text provides clues as to the potential cause of the problem.

Sometimes the design modules contain BEHAVIORAL syntax. Only STRUCTURAL design descriptions are supported. If the syntax is correct and is structural, contact Synopsys support with a sample of the syntax you believe should be supported.

Specifying Lists in Tcl Mode

---

## DRC Rule N2

### Message Text

Netlist Rule N2: Line L (filename), unsupported entry (<additional info>).

### Severity

Warning

### Description

The netlist contains an unsupported construct (such as a behavioral construct).

### What Next

You can ignore the violation if the information is not necessary for model build, or if the module is not referenced by your design.

---

## DRC Rule N3

### Message Text

Netlist Rule N3: Line L (filename), maximum size exceeded for string (D...).

### Severity

Fatal Error

### Description

A netlist string exceeded the maximum allowed size.

D is the first 16 characters of the string.

### What Next

You can correct this by increasing the allowed size using the `set_workspace_sizes` command.

---

## DRC Rule N4

### Message Text

Netlist Rule N4: Line L (filename), redefined cell (<additional info>).

### Severity

Warning

### Description

An EDIF cell has already been defined in the same or another EDIF file.

### What Next

Edit the EDIF files to remove the duplicate cell.

Use `set_netlist` command to define the behavior to apply when duplicate cells are encountered.

---

## DRC Rule N5

### Message Text

Netlist Rule N5: Line L (filename), redefined module (M).

### Severity

Warning

### Description

A module defined in any netlist should not be defined more than once. Duplicate definitions of modules might be indicative of a netlist database problem.

### What Next

You can see a list of modules with multiple definitions using the `report_violations` command.

You can select that the handling of the violation use the first or the last occurrence of the definition of the module. Make the selection using the `set_netlist` command.

Edit netlist or list of files read by TestMAX ATPG to remove the redundancy if you feel the result is inaccurate.

---

## DRC Rule N6

### Message Text

Netlist Rule N6: Line L (filename), duplicate definition (<additional info>).

**Severity**

Fatal Error

**Description**

A module must not contain more than one entity with the same name.

**What Next**

This is a fatal error condition that requires changing the netlist.

---

## DRC Rule N7

**Message Text**

Netlist Rule N7: Line L (filename), missing definition (M/N).

**Severity**

Fatal Error

**Description**

A module referenced an undefined entity such as an instance or a net.

M is the module name and N is the missing entity.

**What Next**

This is a fatal error condition that requires changing the netlist.

---

## DRC Rule N8

**Message Text**

Netlist Rule N8: Line L (filename), invalid construct (<additional info>).

**Severity**

Fatal Error

**Description**

A netlist construct is used in an invalid context or with invalid arguments.

**What Next**

This is a fatal error condition that requires changing the netlist.

If the additional info reports "Incorrect address width of memory read port" the problem might be due to a current limitation of the TestMAX ATPG RAM syntax which does not support a RAM with an address bus of width = 1. In other words, a RAM with two addressable words. A workaround for this is to modify the RAM description to declare the address bus a vector of width 1. For example: `input [0:0] address_bus;`

If the additional info reports "Control net "N1" cannot differ from net "N2" for a RAM model, then the problem might be the order of the nets in the sensitivity list of a read or write port description. The second net, N2, which appears in the qualifying `if` clause must be the same net as the first net in the sensitivity list for *always*. For example:

```
// UNSUPPORTED sensitivity list order causes N8 Violation

always @(ADDR or READ or WRITE_OP)
  if (READ) data_out = memory[ADDR];

// After correction

always @(READ or ADDR or WRITE_OP)
  if (READ) data_out = memory[ADDR];
```

---

## DRC Rule N9

### Message Text

Netlist Rule N9: Line L (filename), nameless instance (<additional info>).

### Severity

Ignored

### Description

A module instance is not given a name (this is an N1 error in netlist formats that require a name).

### What Next

A unique name for the nameless instance is automatically generated. You can view all instance names with the `-verbose` option of the `report_modules` command.

---

## DRC Rule N10

### Message Text

Netlist Rule N10: Line L (filename), ambiguous vector net direction (<additional info>).



### Severity

Fatal Error

### Description

Some netlists, such as Verilog, allow implicit definition of internal nets. Such nets might be parts of a vector net, whose bit direction (ascending or descending) is not known.

### What Next

In most cases, the direction of the implicitly defined vector net is not required for unambiguous connectivity information and the rule violation does not occur. However, if the vector net is used as a vector entity to connect to another vector net, the direction of its bits is important and can lead to incorrect connectivity. If the rule setting is not error, a default direction is chosen and you need do nothing.

---

## DRC Rule N11

### Message Text

```
Netlist Rule N11: Line L (filename), redefined UDP entry (<additional info>).
```

### Severity

Ignored

### Description

A given input matches more than one UDP table entry.

### What Next

The first table entry is kept and all new ones are ignored, consistent with Verilog UDP definition. In some cases, this violation might be caused by a serious UDP functionality flaw, which you should correct.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using SETUP to select a primitive view and then using the SHOW button and specifying a port name or the gate ID of 0. Then click the net expansion diamonds until all gates have been drawn.

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

---

## DRC Rule N12

### Message Text

Netlist Rule N12: Line L (filename), invalid UDP entry (<additional info>).

### Severity

Warning

### Description

A table entry is invalid.

### What Next

The extracted gate representation can still be correct, but it is not correctly defined and part of the intended functionality might be missing.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use `add_display_gates-all`.

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

---

## DRC Rule N13

### Message Text

Netlist Rule N13: Line L (filename), X\_DETECTOR found (<additional info>).

### Severity

Warning

### Description

A Boolean combination of input values in the UDP table results in the output being at a Boolean value, but changing one or more inputs to X results in the output being the opposite Boolean value. For example, in the following UDP table, the two entries, 01:0 and x1:1, describe an X-detector. This behavior cannot be modeled with Boolean gates, for which X is simply the worst case (unknown) between 0 and 1.

```
in1 in2 : out
--- --- : ---
```

```
0    1    : 0
```

```
x    1    : 1
```

### What Next

This behavior cannot be modeled with Boolean gates. The model is replaced with a TIEX gate.

If this module is referenced by your design you will need to come up with a suitable ATPG specific model.

---

## DRC Rule N14

### Message Text

```
Netlist Rule N14: Line L (filename), illegal UDP (<additional info>).
```

### Severity

Error

### Description

The UDP table contains an entry that is not allowed.

### What Next

The extracted gate representation is very likely missing some of the intended UDP functionality.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using SETUP to select a primitive view and then using the SHOW button and specifying a port name or the gate ID of 0. Then click the net expansion diamonds until all gates have been drawn.

---

## DRC Rule N15

### Message Text

```
Netlist Rule N15: Line L (filename), incomplete UDP (<additional info>).
```

### Severity

Ignored

## Description

The UDP table had a missing line which the model translator was expecting. This line would have defined a non-X output value for an input combination.

Failure to satisfy this rule is not hazardous if the default `-xmodeling` option of `set_netlist` command has not been altered. However, the original Verilog UDP table should be reviewed to understand the cause of the message.

## What Next

By definition, all unspecified input combinations in a UDP table result in the output being X. However, for some UDPs a boolean value can be inferred from other table entries. For example in the table below an input of "1 1" is missing and by definition means the output should be an X for this input condition. However, in translation this table into gate primitives it may be inferred that this table represent an OR function and the missing entry should indicate a 1 on the output for an input of "1 1".

```
in1 in2 : out
--- --- : ---
0    0  : 0
0    1  : 1
1    0  : 1
```

The ATPG tool issues the N15 violation for the missing "1 1" entry and based upon the `set_netlist` option for X modeling, either produces an accurate (but complex) internal model which produces an X for an input of "1 1", or produces an optimized boolean gate mapping using an OR primitive.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using SETUP to select a primitive view and then using the SHOW button and specifying a port name or the gate ID of 0. Then click the net expansion diamonds until all gates have been drawn.

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

---

## DRC Rule N16

### Message Text

```
Netlist Rule N16: Line L (filename), overspecified UDP (<additional
info>).
```

## Severity

Warning

## Description

The UDP table specified an entry with a 0 or 1 as an output but the derived ATPG gate level model produces an X. The line number (L) will usually reference the "endtable" marker and not indicate the actual line number which contributed to the rule violation.

This usually is an indication of defined simulation behavior that has no exact match as a gate level equivalent. The UDP behavior for simulation has a different goal in mind than the needs of the ATPG algorithm and it is not uncommon for this condition to occur.

The extracted ATPG gate level representation is not incorrect, it is just more pessimistic than the UDP table as it predicts an X on an output where the original table produces a non-X.

Violation of this rule will cause no danger of creating bad patterns and generally the test coverage will not be reduced.

## What Next

You can ignore these warnings for ATPG efforts. If you wish, you can use the `set_rules` command to change the severity level to ignore.

If you are a library developer or simulation model builder you might be interested in reviewing the original UDP table. Perhaps a few edits will eliminate the warning messages.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using `SETUP` to select a primitive view and then using the `SHOW` button and specifying a port name or the gate ID of 0. Then click on the net expansion diamonds until all gates have been drawn.

```
# An example which results in N16 violations.
```

```
primitive MUX ( Q, A, B, SL );  
output Q;  
input A, B, SL;
```

```
table  
#   A B SL: Q  
# -----  
    0 ? 0 : 0; # 1:  
    1 ? 0 : 1; # 2:  
    ? 0 1 : 0; # 3:  
    ? 1 1 : 1; # 4:  
    0 0 x : 0; # 5:  
    1 1 x : 1; # 6:  
    0 1 x : 1; # 7: causes N16
```

```
x 1 x : 1; # 8: causes N16
endtable
endprimitive
```

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

---

## DRC Rule N17

### Message Text

```
Netlist Rule N17: Gate-level model of UDP failed validation.

UDP not validated (<additional info>).
```

### Severity

Error

### Description

The symbolic comparison between the circuit as defined in the UDP table and the extracted Boolean gate representation failed.

### What Next

The extracted gate representation can still be correct, but too complex or not correctly defined.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using SETUP to select a primitive view and then using the SHOW button and specifying a port name or the gate ID of 0. Then click the net expansion diamonds until all gates have been drawn.

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

---

## DRC Rule N18

### Message Text

```
Netlist Rule N18: Line L (filename), Internal net and port have the same
name (additional_info).
```

### Severity

Warning

### Description

An internal net has the same name as an external port of the same module.

### What Next

If the setting is not error, the internal net is renamed by suffixing "\_intern". If a net by this name already exists, a unique number is also appended to the name.

Note that renaming the net name due to this violation might cause the following issues:

Bridging fault annotation might not match.

Milkyway database might not match netlist in diagnostics.

---

## DRC Rule N19

### Message Text

Netlist Rule N19: Line L (filename), unused construct (<additional info>).

### Severity

Ignored

### Description

The netlist contains a construct not used for model build (such as a timing construct).

### What Next

TestMAX ATPG ignores all timing, so no user action is required.

---

## DRC Rule N20

### Message Text

Netlist Rule N20: Line L (filename), underspecified UDP (Expected *string* got *string*).

### Severity

Warning

### Description

The UDP table defined an input combination for which the output produced is X but the ATPG derived gate-level model for the table predicts a non-X value. The line number (L) will reference the "endtable" line from the UDP.

The most common cause of this rule violation is a missing table entry. By definition, any input combination not explicitly given in the UDP table is assumed to produce an X for an output. When a violation occurs the derived ATPG gate-level model predicts a non-X output where the UDP table predicts an X either by an explicit line or by omitting a line.

Violation of this rule will not affect test coverage but can result in patterns created which fail in simulation. When the ATPG model predicts a non-X but the simulation model predicts X there is a potential for a mismatch. The patterns can actually work fine on real silicon but this is hard to validate if the simulation is failing.

### What Next

Usually the derived gate-level is correct with respect to silicon and the UDP table has omitted some entries either intentionally or unintentionally that are needed to reduce modeling pessimism.

It is rare that patterns will fail simulation. You should generate some test patterns to check the outcome. If simulation of patterns is failing, discuss with your model supplier the particular UDPs which experience these N20 violations and see if some of the pessimism can't be removed.

If you are a library developer or simulation model builder, you might be interested in reviewing the original UDP table. Perhaps a few edits will eliminate the warning messages.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using SETUP to select a primitive view and then using the SHOW button and specifying a port name or the gate ID of 0. Then click on the net expansion diamonds until all gates have been drawn.

```
# An example which results in N20 violations.

primitive MUX ( Q, A, B, SL );
output Q;
input A, B, SL;
  table
    #   A B SL: Q
    # -----
        0 ? 0 : 0;
        1 ? 0 : 1;
        ? 0 1 : 0;
        ? 1 1 : 1;

    # omitting the following lines produces N20 violations of:
    # underspecified UDP (Expected "??x:t" got "x"). (N20-1)

    # 0 0 x : 0;
    # 1 1 x : 1;
  endtable
endprimitive
```



# A second example which results in N20 violations where the intended functionality was a 2-input majority logic gate but a number of entries to reduce pessimism were missing.

```
primitive MAJORITY (Z, A, B, C);
  output Z;
  input A, B, C;
  table
    # A B C : Z
    # -- -- -- : ----
      0 0 0 : 0 ;
      0 0 1 : 0 ;
      0 1 0 : 0 ;
      0 1 1 : 1 ;
      1 0 0 : 0 ;
      1 0 1 : 1 ;
      1 1 0 : 1 ;
      1 1 1 : 1 ;
  endtable
endprimitive
```

# The revised model to remove pessimism and eliminate N20.

```
primitive MAJORITY (Z, A, B, C);
  output Z;
  input A, B, C;
  table
    # A B C : Z
    # -- -- -- : ----
      0 0 ? : 0 ;
      0 ? 0 : 0 ;
      ? 0 0 : 0 ;
      1 1 ? : 1 ;
      1 ? 1 : 1 ;
      ? 1 1 : 1 ;
  endtable
endprimitive
```

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

---

## DRC Rule N21

### Message Text

Netlist Rule N21: Line L (filename), unsupported UDP entry (Entry *string*)

### Severity

Warning

## Description

A UDP table entry is unsupported and is ignored when deriving an ATPG gate-level representation of the UDP. The line number (L) indicates the line in the file of the table entry that is unsupported.

This violation occurs most frequently when a non-functional table entry is detected. This is common when the "notify register" technique is used for UDP table definitions. The notify register is a dedicated input which is encoded such that any event on this input causes the output to change state. This input is usually triggered by a setup/hold or other timing check that sees a violation and wishes the output to change to X. There is no gate level representation which would provide this functionality but that is OK as this input never changes in an ATPG environment and is considered non-functional.

Violation of this rule will not affect test coverage or result in patterns which fail simulation.

## What Next

In most cases, no corrective action is required. However, if you are a purist, you will review the original UDP table to make sure the notify register technique exists. If it is not there, there is some other sort of problem which must be investigated.

You can eliminate these warnings by using the `set_rules` command to change the severity level to ignore.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use `add_display_gates-all`.

```
# An example that produces an N21 violation
primitive FLOP ( Q, D, CK, NR );
output Q;
input D, CK, NR;
reg Q;
table
  # D CK NR : Q- : Q+
  #-----
    0 (01) ? : ? : 0;
    1 (01) ? : ? : 1;
    ? 0 ? : ? : -;
    0 (0x) ? : 0 : 0;
    1 (0x) ? : 1 : 1;
    ? ? * : ? : x; # this is unsupported, N21
endtable
endprimitive
```

---

## DRC Rule N22

### Message Text

Netlist Rule N22: Line L (filename), duplicate pin (<additional info>).

### Severity

Warning

### Description

A netlist module has two or more external pins (such as two bidirectionals) with the same name.

### What Next

If this is intended functionality and not a typographical error, no action is required. All but the first pin is renamed to create unique names, and all these pins are connected together bidirectionally.

---

## DRC Rule N23

### Message Text

Netlist Rule N23: Line L (filename), inconsistent UDP entry (<additional info>).

### Severity

Warning

### Description

An entry in the UDP table conflicts with clock information derived from previous entries. This entry is ignored in deriving the ATPG gate-level model. The line number (L) indicates the line in the file on which the violation occurred.

A common cause of this violation is an entry which is inconsistent with prior entries defining clock or asynchronous set/reset behavior. For example, from previous entries it has been inferred that input *S* is the asynchronous set of a latch and that input *R* is its asynchronous reset. If there exists an entry that shows the state of the latch being unchanged when *S* and *R* are simultaneously active then this is considered inconsistent with prior inferred behavior. This entry would not be considered inconsistent if a previous entry in the UDP table had explicitly defined an input combination with *S* and *R* simultaneously on.

A violation of this rule will not affect test coverage but might be an indication that patterns generated could fail simulation. This is possible whenever the derived ATPG gate-level model does not match the simulation model.

### What Next

The extracted gate representation can still be correct. However, the inconsistent entry represents a potential problem and can indicate that the UDP is flawed. Generate some sample ATPG patterns and then simulate them to see if there is a problem. If so, consult with your library supplier on the UDP in question.

You can review the derived ATPG gate model which is used by using the `run_build_model` command to specify the module name in question. Then use the graphical schematic viewer to draw the module. Start by using SETUP to select a primitive view and then using the SHOW button and specifying a port name or the gate ID of 0. Then click the net expansion diamonds until all gates have been drawn.

```
# An example which produces an N23 violation of:
# (Entry ?1*?:- conflicts with clocks offstate)

primitive N23_SAMPLE ( Q, D, CK, V );
input D, CK, V;
output Q;
reg Q ;
table
  # D    CK    V    : Q- : Q+
  #-----
  (?0) 1    ?    : ?    : 0 ; # CK=1, latch open
  (?1) 1    ?    : ?    : 1 ;

  *    0    ?    : ?    : - ; # CK=0, hold

  0 (?1)    ?    : ?    : 0 ; # open latch
  1 (?1)    ?    : ?    : 1 ;

  ? (?0)    0    : ?    : - ; # close latch, hold

  0 *        0    : 0    : 0 ; # reducing pessimism
  1 *        0    : 1    : 1 ;

  ? 1        *    : ?    : - ; # N23, logic flaw??
endtable
endprimitive
```

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

---

## DRC Rule N24

### Message Text

Netlist Rule N24: Line L (filename), UDP cannot hold state (<additional info>).

### Severity

Warning

### Description

A sequential UDP's table has no entry that holds state. Thus, the UDP does not represent a state element (latch or flip-flop) with optional combinational gates around it. The gate representation extracted does not include state elements, but can include feedback paths.

### What Next

In many cases, such UDP represents a pulse processor in the clock logic. In these cases, the extracted representation is adequate for ATPG. If, however, the UDP intends to represent a state element, particularly a scan cell, the table is erroneous and the extracted gate representation will fail scan trace rules.

You can have to create your own ATPG specific model for this library cell.

---

## DRC Rule N25

### Message Text

Netlist Rule N25: Line L (filename), Module (M) has been previously defined and instantiated.

### Severity

Error

### Description

This violation is raised when all of the following conditions are met:

module A has been defined and has at least one vector port

module B instantiates module A by name

module A is redefined after it has been defined and instantiated

the setting of `set_netlist` is `-redefined_modulelast`.

The reason for the violation is that the new definition of module A might redefine some vector port differently than its first definition, and the first definition has already been used to create the connectivity of module B.

By default, the severity is error and the first definition is not replaced.

#### What Next

Any one of the following resolves the error:

Remove all but one of the definitions for module A and reread the netlist.

Use the command `set_netlist-redefined_module first`.

If the vector ports are the same in the new and old definitions or if module A is not used in the simulation model, set rule N25 to warning or ignored.

---

## DRC Rule N26

#### Message Text

Netlist Rule N26: Line L (filename), empty model (<additional info>).

#### Severity

Warning

#### Description

Reading and analysis of a library module resulted in an empty ATPG model. If the module is referenced by the design an empty model will produce other problems such as TIEX nets, undriven nets, and often scan chain blockages.

An empty model can result from such items as:

violation of a rule with severity of error

violation of rule N13 (XDETECTOR), independent of the severity of N13

violation of rule N15 (incomplete UDP) and a combination model.

#### What Next

Review the model and original source module to see if a change to the source might be helpful. You might be required to write your own ATPG model to overcome this problem.

In rare UDP cases, some benefit can occur by setting rule N15 to ignore.

---

## DRC Rule N27

### Message Text

Netlist Rule N27: Line L (filename), incomplete memory definition (<additional info>).

### Severity

Warning

### Description

This violation occurs when a behavioral Verilog memory model is recognized as a TestMAX ATPG supported model, but it misses some features or has some unexpected or unsupported syntax.

The occurrence message of this message is detailed in specifying the cause of the violation (for example, old memory models did not include the write-to-read update notification done by signaling an event). The occurrence message is displayed with the `report_violation` command.

It is also possible for this message to occur because of unrecognized or unexpected syntax. In these cases, the detailed text might be inaccurate and indicate the problem is a "Net XXX should be declared as REG" when in fact some other problem is the real cause. In these situations, you should view carefully near the line numbers of the file indicated in the N27 violation. Compare the syntax near that line with the supported syntax for memory modeling.

In some cases, non-TestMAX ATPG memory models can raise some N27 violations before they are recognized as non-TestMAX ATPG models. After they are identified as non-TestMAX ATPG models, you should see N2 violations but no further N27 violations.

### What Next

Review the original library module definition and consider making the changes suggested.

---

## DRC Rule N28

### Message Text

Netlist Rule N28: Line L (filename), unsupported priority (<additional info>).

### Severity

Warning

### Description

This violation occurs when a Verilog UDP model describes behavior in which the simultaneous application of set and reset, or set and clock, or reset and clock produces known results. The TestMAX ATPG primitive for a DLAT does not fully support this priority behavior for all ATPG algorithms.

### What Next

This message is mainly information of the current limitations of the ATPG tool for modeling set/reset priorities and other types of priorities. There is really very little you can do. The pessimistic ATPG model can result in a lower test coverage, but does not present a danger of creating patterns that fail in simulation.

Most common forms of set/rst/clk prioritization are supported for fault grading of functional patterns. This warning can be ignored as its effect has a potential lowering of test coverage.

See also Interpreting UDP Warning Messages in the TestMAX ATPG and TestMAX Diagnosis Messages Guide

---

## DRC Rule N29

### Message Text

```
Netlist Rule N29: Line L (filename), inout net tied to supply
(<additional info>).
```

### Severity

Error

### Description

This violation occurs when module port defined to be a bidirectional port is connected to a strong driver such as a supply0 or supply1 net. This usually indicates a netlist error.

### What Next

After reviewing the source of the error you might want to modify the netlist. If you wish to ignore this error and continue, then change the severity of this rule to either warning or ignore using the `set_rules` command and retry the `run_build_model` command.

---

## DRC Rule N30

### Message Text

```
Netlist Rule N30: Inconsistent memory definition
```



## Severity

Error

## Description

TestMAX ATPG has limited support for a single model that supports both simulation and ATPG by ignoring the #nnn delays when they appear. In order to get the simulation of a RAM model to work, it is necessary to have non-zero delay between the write port clock and the data appearing at the read port outputs. This is accomplished by putting "#nnn" delays into the model. TestMAX ATPG can limitly support a single model that supports both simulation and ATPG by ignoring the #nnn delays when they appear.

## What Next

### Examples and explanations:

```
"Edge-triggered read port", clk->name(), "cannot capture new memory data"
"Previous edge-triggered read port", rconn->net->name(), "cannot capture new memory data"
```

Edge-triggered ports *a/ways* capture old data; that's the nature of edge-triggered.

You need to modify your memory model.

```
"Write port delays reflect dominance, not X-behavior"
```

Mismatch between the read/write delays and the explicitly declared behavior (`define). You need to modify your memory model.

```
"LS read always reads new data and its delay should be > write port delay %d", maxWdelay
"LS read always reads new data and its delay %d should be > write port delay %d", minLSRdelay, maxWdelay
"Edge-read delay should be > write delay %d for read_write=new", minWdelay
"Edge-read delay %d should be > write delay %d for read_write=new", maxERdelay, minWdelay
"Write delay should be > edge-read delay %d for read_write=mixed", minERdelay
"Write delay %d should be > edge-read delay %d for read_write=mixed", maxWdelay, minERdelay
```

Definition of the error is provided in the statements above.

---

## DRC Rule N31

### Message Text

Netlist Rule N31: Line *L* (*filename*), Name length exceeded maximum and was truncated (the *name\_length* is longer than the *supported\_length*).

### Severity

Warning

### Description

The length of a netlist module instance name is longer than the length supported for internal names. Long instance names are truncated because they consume significant memory and affect all TestMAX ATPG operations before reading the netlist. The name-truncation process assigns the instance a different name from the original name in the design.

### What Next

The line number (*L*) in the file name (*filename*) identifies the location of the detected problem. Modify the netlist to reduce the length of the instance name and all references to that name.

---

## DRC Rule N32

### Message Text

Netlist Rule N32: Line *L* (*filename*), Behavioral syntax detected in netlist (*additional information*).

### Severity

Warning

### Description

The netlist contains behavioral syntax/constructs (complex assign statements). Other behavioral constructs are flagged by DRC Rule N2.

### What Next

You can ignore the violation if the information is not necessary for model build, or if the module is not referenced by your design.

---

## DRC Rule N33

### Message Text

Netlist Rule N33: Undefined port in netlist.

### Severity

Warning

### Description

In the netlist module, report a warning message for pins that are not connected with the net name. Use `report_violations N33` to get the pin names.

### What Next

You can eliminate these warnings by using the `set_rules` command to change the severity level to ignore.

An example that produces a N33 violation is as follows:

```
=====
module CORE ( CLK, SE, SI, DIN, DOUT );
    input CLK, SE, SI, DIN;
    output DOUT;
    SDFQX1 REG_1 ( .CK(CLK), .SE(SE), .SI(SI), .D(DIN), .Q(DOUT) );
endmodule
module TOP ( CLK, SE, SI, DIN, DOUT );
    input CLK, SE, SI, DIN;
    output DOUT;
    CORE
    ( .CLK(CLK), .SE(SE), .SI(SI), .DIN(DIN), .DOUT(DOUT), .N33_PORT() );
Endmodule
=====
```

In the `CORE` module, `N33_PORT()` is not connected to any net name.

# 7

## Category P - Path Delay Rules

---

### See Also

- [P1 - parsing error](#)
- [P2 - missing delay path name](#)
- [P3 - duplicate delay path name](#)
- [P4 - invalid pin pathname](#)
- [P5 - path contains unconnected combinational node](#)
- [P6 - path traverses a sequential cell](#)
- [P7 - capture earlier than path cycle time](#)
- [P8 - capture later than path cycle time](#)
- [P9 - invalid transition indicator](#)
- [P10 - path contains less than 2 nodes](#)
- [P11 - duplicate delay path](#)
- [P12 - multiple source delay path](#)
- [P13 - multiple destination delay path](#)
- [P14 - ambiguous path segment](#)
- [P15 - ambiguous launch clock](#)
- [P16 - ambiguous capture clock](#)
- [P17 - path not connected to valid capture node](#)
- [P18 - path contains duplicate node](#)
- [P19 - tied value blocks path propagation](#)
- [P20 - constrained value blocks path propagation](#)
- [P21 - path node values not satisfiable](#)

- [P22 - off-path node values not satisfiable](#)
- [P23 - path conditions not satisfiable with path values](#)
- [P24 - path traverses tristate enable with unresolved Z](#)
- [P25 - path contains sensitizable feedback loop](#)

---

## DRC Rule P1

### Message Text

Path Delay Rule P1: Line L (file), parsing error (<additional info>).

### Severity

Fatal Error

One example, many variations are possible:

Listing of delay paths file ckt.path with syntax error on lines 4 and 5:

```
$path {  
    $transition {  
        reg_fdl/Q v  
        XOR_2/Z ^  
    }  
}
```

In this case, the following error is delivered:

```
Error: Line 4 (ckt.path), parsing error (missing ';' delimiter). (P1-1)
```

---

## Example

### Description

The file must satisfy parsing rules. The problem was first identified on line L but can have occurred before this line in the file. Also note that there might be other errors following the first.

### What Next

Identify the location of the error and correct the syntax.

---

## DRC Rule P2

### Message Text

Path Delay Rule P2: Line L (file), missing delay path name (assigned name = "pathP").

### Severity

Warning

### Description

The path defined near line L of the delay path definition file was not named. Each delay path definition is assigned a name. If the \$name construct is missing from the path definition, a default name of "pathP" is assigned (where P is the next available delay path definition number).

See also set\_rules in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

The assigned delay path name might be used, or a \$name construct might be added to the path definition. Set this rule severity to ignore if path names are not important.

---

## DRC Rule P3

### Message Text

Path Delay Rule P3: Line L (file), duplicate delay path name (P).

### Severity

Error

One example, many variations are possible:

Listing of delay paths file ckt.path with duplicate path name error on line 9:

```
$path {  
  $name my_path ;  
  $transition {  
    reg_fd1/Q v ;  
    XOR_2/Z ^ ;  
  }  
}  
$path {  
  $name my_path ;  
  $transition {  
    reg_fd2/Q v ;  
    XOR_4/Z ^ ;  
  }  
}
```

```
}
```

In this case, the following error is delivered:

```
Error: Line 9 (ckt.path), duplicate delay path name (my_path). (P3-1)
```

---

## Example

### Description

The delay path defined on line L of the delay path definition file contained a duplicate delay path name P. Each delay path must have a unique name. If any delay path definition tries to reuse a name, a violation error is issued.

Note: When a delay path definition contains no \$name construct, a name of the form pathP is assigned by TestMAX ATPG. Subsequent delay path definitions should not use this form to name paths, or a naming collision can occur.

### What Next

Change the names of the delay path definitions so that they don't reuse existing delay path definition names. Or, if this is indeed a duplicate delay path definition, remove the duplicate delay path definition.

See also set\_rules in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

---

## DRC Rule P4

### Message Text

```
Path Delay Rule P4 : Line L (file), invalid pin pathname (<pin  
pathname>).
```

### Severity

Warning

One example, many variations are possible:

Listing of delay paths file ckt.path with bad pin pathname error on line 4:

```
$path {  
    $transition {  
        XOR_2/A v ;  
        XOR_2/Z ^ ;  
    }  
}
```

In this case, the following error is delivered:

Warning: Line 4 (ckt.path), invalid pin pathname (XOR\_2/Z). (P4-1)

## Example

### Description

The pin pathname defined by delay path entry on line L of the delay path file was incorrect or unrecognized. Every node defined along the delay path definition must be a valid node in the circuit. Any unrecognized nodes is ignored for the path definition.

### What Next

Verify that the node defined in the delay path definition has the correct naming convention, and is recognized by TestMAX ATPG. Use the `report_instances` command on the disputed instance pathname (pin pathname without the final pin name). In the previous example, "XOR\_2" would be the instance name used. TestMAX ATPG will print a list of pins associated with this instance, if it finds the instance in the built database. Insure that the pin name in the delay path definition is the same.

See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## DRC Rule P5

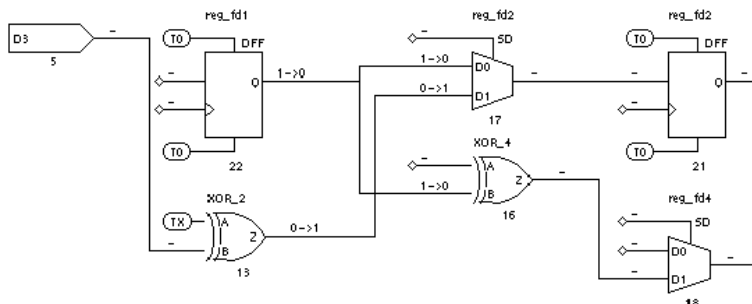
### Message Text

Path Delay Rule P5 : Path P contains unconnected combinational node <pin pathname> (G).

### Severity

Error

One example, many variations are possible:





```
Error: Path path2 contains unconnected combinational node XOR_2/A (13).
(P5-1)
```

### Description

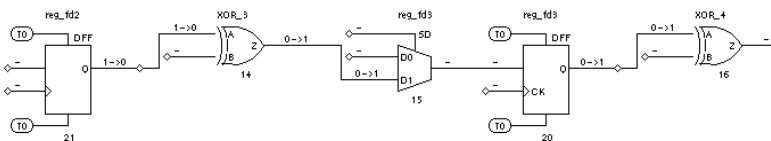
## What Next

Either the delay path definition or the circuit model should be changed to compensate for this violation. It is possible that modeling issues encountered during the build operation have resulted in this error. Check N and B rule violations encountered during netlist read and build operations.

## Message Text

## Severity

One example, many variations are possible:



### Description

289

## What Next

A violation can be analyzed using the graphical schematic viewer (GSV) by selecting its violation ID number from the Analyze button dialog box. This changes the pindata reporting to delay data and displays the delay path segment effecting this rule violation. Notice the highlighted sequential element, gate ID G. The path passes through the highlighted sequential gate.

TestMAX ATPG only supports combinational logic delay paths. The delay path definition should be altered or removed from the delay path definition file.

See also set\_rules in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

---

## DRC Rule P7

### Message Text

Path Delay Rule P7 : Capture time (T1ns) for path P occurred T2ns earlier than path cycle time.

### Severity

Error

One example, many variations are possible:

The following path requires a cycle time of 20 ns and is launched from the rising edge of CK, and is captured on the falling edge of CK:

```
$path {
  $name too_early ;
  $cycle 20.00 ns ;
  $transition {
    XOR_4/B v ;
    reg_fd4/Q ^ ;
  }
}
```

However, the STL procedure file only specifies a "\_default\_WFT\_" WaveformTable as follows:

```
Timing {
  WaveformTable "_default_WFT_" {
    Period '2ns';
    Waveforms {
      "all_inputs" { 01Z { '0ns' D/U/Z; } }
      "all_outputs" { X { '0ns' X; } }
      "all_outputs" { HLT { '0ns' X; '2ns' H/L/T; } }
      "CK" { P { '0ns' D; '4ns' U; '6ns' D; } }
    }
  }
}
```

```
    }  
  }  
}
```

This means that the launch will occur 4 ns into the launch cycle, and the capture will occur 2 ns later. This is much faster than the path was analyzed with, and will likely fail on the tester.

The message supplied by TestMAX ATPG for this violation is as follows:

```
Error: Capture time (2ns) for path too_early occurred 18ns earlier than  
path cycle time. (P7-1)
```

---

## Example

### Description

This error calls attention to the fact that path P has been analyzed using a particular cycle time, T1, but the timing information given to TestMAX ATPG indicates that the timing of the launch and capture events will likely cause this path to fail on the tester because the \$cycle indicated for the path is T2 ns longer than the cycle used for the test vectors required for this fault.

### What Next

Either change the timing information in the STL procedure file to allow a later capture to occur, or use `remove_delay_paths` commands to eliminate paths having long cycle times.

See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

---

## DRC Rule P8

### Message Text

Path Delay Rule P8 : Capture time (T1ns) for path P occurred T2ns later than path cycle time.

### Severity

Warning

One example, many variations are possible:

The following path requires a cycle time of 50 ns:

```
$path {  
  $name too_late;  
  $cycle 50 ns;  
  $transition {
```

```
        reg_fd3/Q v;  
        XOR_4/Z ^;  
    }  
}
```

However, the STL procedure file specifies a "\_default\_WFT\_" WaveformTable as follows:

```
Timing {  
    WaveformTable "_default_WFT_" {  
        Period '1000ns';  
        Waveforms {  
            "all_inputs" { 01Z { '0ns' D/U/Z; } }  
            "all_outputs" { X { '0ns' X; } }  
            "all_outputs" { HTL { '0ns' X; '400ns' H/T/L; } }  
            "CK" { P { '0ns' U; '450ns' D; '550ns' U; } }  
        }  
    }  
}
```

This means that the launch will occur 450 ns into the launch cycle, and the capture will occur 1000 ns later. This is slower than the path was analyzed with originally.

The message supplied by TestMAX ATPG for this violation is as follows:

```
Warning: Capture time (1000ns) for path too_late occurred 950ns later  
        than path cycle time. (P8-1)
```

---

## Example

### Description

This error calls attention to the fact that path P has been analyzed with a particular cycle time, but the timing information given to TestMAX ATPG indicates that the timing of the launch and capture events will likely be slower. The test could pass on the tester when there is a significant delay fault on the path.

If an on-chip clock controller is used to launch or capture this path, then the clock timing used for the P8 check does not match the actual clock timing and this warning is irrelevant and can be ignored. The only clock timing used for the P8 check is in the SPF Timing block, and there is no way to specify OCC clock timing so that the P8 check will use it.

### What Next

Nothing needs to be done to fix this issue; however, delay faults along this path will likely not be caught on the tester. ATPG will proceed, but it could be possible that the paths file or the STL procedure file are inappropriately matched to one another. The warning message calls attention to this possibility.

This warning occurs most often when TestMAX ATPG uses the "\_default\_WFT\_" because a specific delay WFT has not been defined. Check your timing protocol statements for a valid WFT definition. The "\_launch\_WFT\_", "\_capture\_WFT\_", and "\_launch\_capture\_WFT\_" could be defined inappropriately, or might be inheriting the timing from the "\_default\_WFT\_" if not provided.

See also set\_rules in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## DRC Rule P9

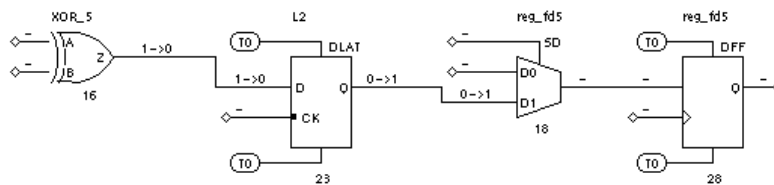
### Message Text

Path Delay Rule P9 : Path P has an invalid transition indicator on node <pin pathname>.

### Severity

Error

One example, many variations are possible:



```
$path {
  $name path1 ;
  $transition {
    D3 v ;
    BUF_1/Z ^ ;
    BUF_2/Z v ;
    reg_fd5/D1 ^ ;
  }
}
```

In this case, the following error is delivered:

Error: Path path1 had an invalid transition indicator on node reg\_fd5/D1.  
(P9-1)

---

## Example

### Description

The delay path definition named P specifies a node transition that conflicts with the transition indicated on the previous node.

### What Next

A violation can be analyzed using the graphical schematic viewer (GSV) by selecting its violation ID number from the Analyze button dialog box.

This changes the pindata reporting to delay data and displays the delay path segment effecting this rule violation. Notice the highlighted circuit element containing the gate ID G. The delay path transition indicators show the on-path transitions for the delay path definition in question. Looking back from this gate, notice the logic transition direction. Apparently, the path in question does not support the sequence of transitions requested in the delay path definition.

An inverting logic can have been assumed when a non-inverting logic has since been implemented along the path.

The delay path definition should be altered or removed from the delay path definition file.

See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

---

## DRC Rule P10

### Message Text

Path Delay Rule P10: Path defined on line L (file) contains less than 2 nodes.

### Severity

Error

One example, many variations are possible:

Listing of delay paths file `ckt.path` with P10 error:

```
$path {  
  $transition {  
    XOR_2/A v;  
  }  
}
```

In this case, the following error is delivered:

```
Error: Path defined on line 1 (ckt.path) contains less than 2 nodes.  
(P10-1)
```

---

## Example

### Description

A delay path definition defined on line L of the delay paths file contained less than two nodes. In order for a delay path test to be created, a delay path must be defined between at least two nodes in the circuit. One node path might be referring to a transition fault node, not a delay fault path. This path is invalid and will not be added to the internal delay path list.

### What Next

Either remove this path from the delay path definition file or add more circuit nodes to the delay path definition.

If both P4 and P10 messages exist, P4 might be the source of the P10 message. One example could be that the hierarchical delimiter is "." instead of "/". The "/" is the default hierarchical delimiter in TestMAX ATPG.

---

## DRC Rule P11

### Message Text

Path Delay Rule P11 : Path defined on line L (file) identical to path P.

### Severity

Warning

One example, many variations are possible:

Listing of delay paths file ckt.path with duplicate delay path definitions:

```
$path {  
    $name path0 ;  
    $transition {  
        reg_fdl/Q v ;  
        XOR_2/A v ;  
    }  
}  
$path {  
    $name path2 ;  
    $transition {  
        reg_fdl/Q v ;
```

```
        XOR_2/A v ;  
    }  
}
```

In this case, the following error is delivered:

Warning: Path defined on line 8 (ckt.path) identical to path path0.  
(P11-1)

## Example

### Description

The delay path definition file contained a duplicate delay path definition defined at line L. It had the same delay path definition as delay path definition P.

Each delay path should only be defined once in the delay path definition files. Any duplicate delay path definitions is ignored.

### What Next

Remove the duplicate definition from the delay path definition file.

See also set\_rules in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## DRC Rule P12

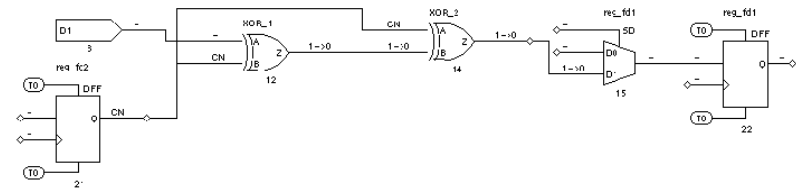
### Message Text

Path Delay Rule P12 : Path P has multiple sources (G1 and G2).

### Severity

#### Warning

One example, many variations are possible:







## Example

### Description

The delay path definition named P might be observed by more than one capture site denoted by gate IDs G1 and G2. Either of the terminating gate IDs might be used to capture the result of the path delay test. Different delays might be tested depending on which destination gate is used.

### What Next

A violation might be analyzed using the graphical schematic viewer by selecting its violation ID number from the Analyze button dialog box. This changes the pindata reporting to delay data and displays the delay path segment effecting this rule violation. Notice the highlighted circuit element. Looking forward in the circuit from this element, notice the 2 gate IDs listed might be used to capture the final transition specified in the delay path definition named P.

See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## DRC Rule P14

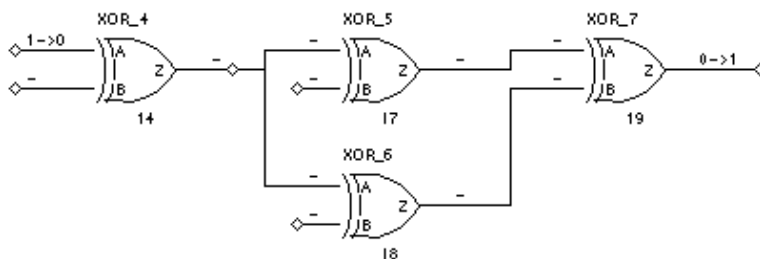
### Message Text

Path Delay Rule P14 : Path P has an ambiguous path connection to node <pin pathname> (G).

### Severity

Warning

One example, many variations are possible:





## Example

### Description

The delay path definition named P can be launched either by clock C1 or clock C2. A violation will also occur if the clock for the launch site does not match the \$launch clock defined in the delay path definition, if the launch site is a clock unstable cell, or if the launch clock has been disabled.

### What Next

A violation can be analyzed using the graphical schematic viewer (GSV) by selecting its violation ID number from the Analyze button dialog box. This changes the pindata reporting to delay\_data and displays the delay path segment and clocks effecting this rule violation. Clicking the node diamonds feeding this path segment should reveal the connectivity to flops capable of launching the delay path vector. Notice that the flops are driven by more than one clock in the circuit.

Clock ambiguity could result in different timing, depending on which launch clock is used. Violations can be ignored if you don't care which clock is used for the test.

See also set\_rules in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## DRC Rule P16

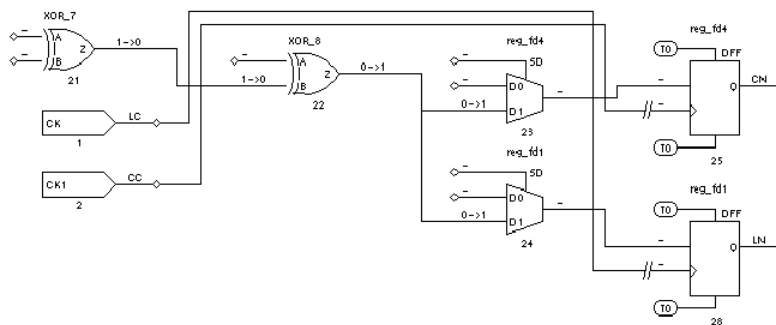
### Message Text

Path Delay Rule P16 : Path P has an ambiguous capture clock (C1 and C2).

### Severity

### Warning

One example, many variations are possible:



## Example

### Description

The delay path definition named P can be captured either by clock C1 or clock C2. A violation will also occur if the clock for the capture site does not match the \$capture clock defined in the delay path definition, if the capture site is a clock unstable cell, or if the capture clock has been disabled.

### What Next

A violation can be analyzed using the graphical schematic viewer (GSV) by selecting its violation ID number from the Analyze button dialog box. This changes the pindata reporting to delay\_data and displays the delay path segment and clocks effecting this rule violation. Clicking the node diamonds on the right side of this path segment should reveal the connectivity to flops capable of capturing the delay path vector. Notice that the flops are driven by more than one clock in the circuit.

Clock ambiguity could result in different delays tested, depending on which capture clock is used. Violations can be ignored if you don't care which clock is used for the test.

See also set\_rules in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## DRC Rule P17

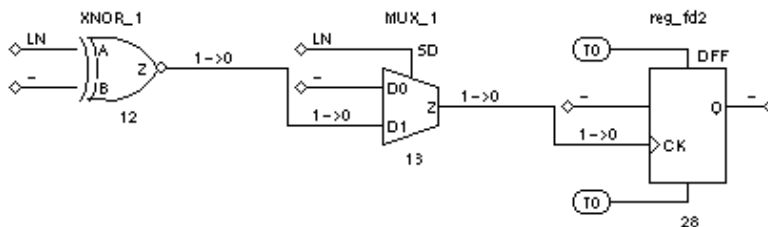
### Message Text

Path Delay Rule P17 : Path not connected to valid capture node.

### Severity

Error

One example, many variations are possible:



## Example

### Description

The delay path definition named P defines a path which terminates at the clock input of a state element.

### What Next

A violation can be analyzed using the graphical schematic viewer (GSV) by selecting its violation ID number from the Analyze button dialog box. This changes the pindata reporting to delay\_data and displays the delay path segment terminating at a clock input pin of a design element or at the input of a black box module. This path is invalid and will not be added to the internal delay path list.

This type of delay path is not supported by TestMAX ATPG and should be removed from the delay path definition file.

See also set\_rules in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## DRC Rule P18

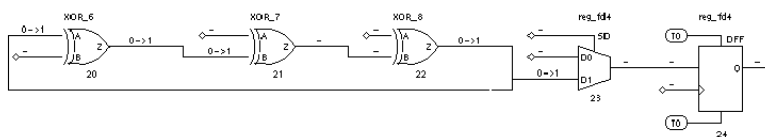
### Message Text

Path Delay Rule P18 : Path P has a duplicate node <pin pathname> (G).

### Severity

Error

One example, many variations are possible:



Listing of delay paths file ckt.path with duplicate node error:

```
$path {
  $name path0 ;
  $transition {
    XOR_8/Z v ;
    XOR_6/Z ^ ;
    XOR_8/Z ^ ;
  }
}
```

In this case, the following error is delivered:

```
Error: Path path0 had a duplicate node XOR_8/Z (22). (P18-1)
```

---

## Example

### Description

The delay path definition named P listed a node connected to gate ID G more than once. This path is invalid and will not be added to the internal delay path list.

### What Next

A violation can be analyzed using the graphical schematic viewer (GSV) by selecting its violation ID number from the Analyze button dialog box. This changes the pindata reporting to delay\_data and displays the delay path segment effecting this rule violation. Another method to view the violation is to use the command `report_delay_pathsP`, where P is the offending delay path definition. You should then be able to see the duplicate node defined in this delay path definition.

A duplicate node means that a feedback path has been specified. TestMAX ATPG does not support delay path definitions containing a combinational feedback loop.

If the path file is written by PrimeTime, there is no feedback loop, since PrimeTime breaks all loops before analysis. In this case, the violation occurs because a node is written multiple times as it crosses hierarchical boundaries. To work around this, you should downgrade the P18 error to a warning. To force the violated paths back to the “not detected” status, specify the `update_faults -reset_au` command after the `add_faults` command, but before the `run_atpg` command. This causes the paths to be handled correctly by ATPG.

See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

---

## DRC Rule P19

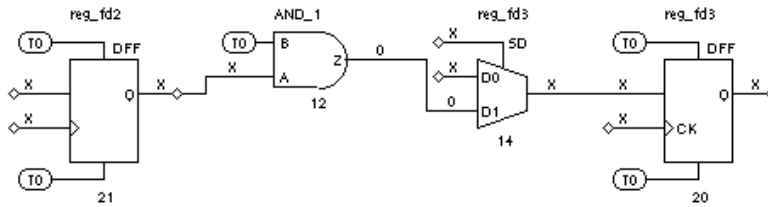
### Message Text

```
Path Delay Rule P19 : Path P has a propagation blockage on node <pin  
pathname> (G) due to tied values.
```

### Severity

Warning

One example, many variations are possible:



## Example

### Description

The delay path definition named P defined a path through a node on gate ID G which was blocked from propagating through that node due to a tied node on another pin.

### What Next

A violation can be analyzed using the GSV by selecting its violation ID number from the Analyze button dialog box. This changes the pindata reporting to tied values and displays the delay path segment effecting this rule violation. Notice the tied nodes in the circuit which might block the passage of the delay path vector.

The circuit will need modification to remove the tied blockage, or the delay path definition can be removed from the delay path definition file. If the delay path definition is not removed, the associated path delay faults is classified as undetectable tied (UT) and will not be targeted for testing.

To see more precisely what this problem is and why it cannot be resolved, use the `analyze_faults` command for the corresponding path delay fault.

See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## DRC Rule P20

### Message Text

Path Delay Rule P20 : Path P had a propagation blockage on node <pin pathname> (G) due to constrained values.

### Severity

Warning



### Description

## What Next

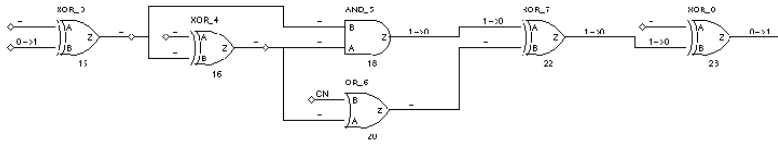
See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## Message Text

## Severity

### Warning

One example, many variations are possible:



## Example

### Description

While trying to satisfy transition values along the delay path definition named P, the transition specified for gate ID G could not be satisfied while also satisfying other transition values. Delay paths that violate this rule are combinational false paths.

### What Next

A violation can be analyzed using the graphical schematic viewer (GSV) by selecting its violation ID number from the Analyze button dialog box. This changes the pindata reporting to delay data and displays the delay path segment effecting this rule violation. Notice the highlighted circuit element containing the gate ID G. Clicking back on the gate inputs should reveal logic which precludes the requested delay path definition from being satisfied.

If the delay path definition is not removed, the associated path delay faults is classified as undetectable redundant (UR) and will not be targeted for testing.

To see more precisely what this problem is and why it cannot be resolved, please use the `analyze_faults` command for the corresponding path delay fault.

See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## DRC Rule P22

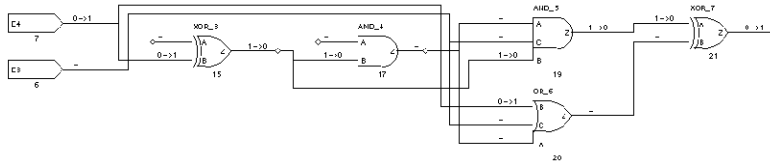
### Message Text

Path Delay Rule P22 Off-path node values not satisfiable for path P.

### Severity

Warning

One example, many variations are possible:



## Example

### Description

A test for the delay path definition named P cannot be generated because the off-path values cannot all be satisfied, or cannot be satisfied while also satisfying all transition values along the delay path.

### What Next

A violation can be analyzed using the GSV by selecting its violation ID number from the Analyze button dialog box. This changes the pindata reporting to delay data and displays the delay path segment effecting this rule violation. Notice the highlighted circuit element containing the gate ID G. The delay path transition indicators show the on-path transitions for the delay path definition in question. The off-path nodes around the highlighted gate should be checked to make sure the defined delay path can be sensitized to allow the data to flow through the delay path.

Off-path nodes can be forced to a path-controlling value by redundant timing paths that reconverge through an off-path input. Use the `set_delay-allow_reconverging_paths` option to target redundant timing paths.

To see more precisely what this problem is and why it cannot be resolved, please use the `analyze_faults` command for the corresponding path delay fault.

See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## DRC Rule P23

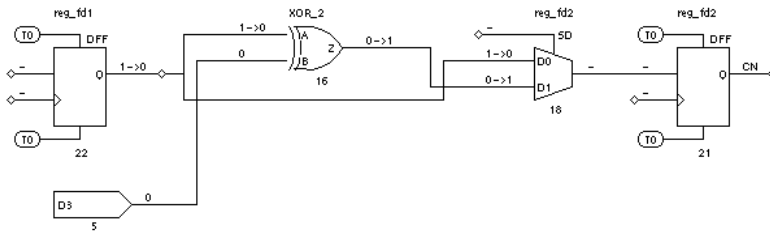
### Message Text

Path Delay Rule P23: Path conditions not satisfiable with path values on node G for path P.

### Severity

Warning

One example, many variations are possible:



## Example

### Description

The delay path ancillary node or transition values requested for the delay path definition named P in the \$condition block could not be satisfied at gate ID G while also satisfying all delay path transition values.

### What Next

A violation can be analyzed using the graphical schematic viewer (GSV) by selecting its violation ID number from the Analyze button dialog box. This changes the pindata reporting to delay data and displays the delay path segment effecting this rule violation. Notice the highlighted circuit element containing the gate ID G. The delay path transition indicators show the on-path transitions for the delay path definition in question. The nodes defined in the \$condition block around the highlighted gate should be checked to make sure the defined delay path can be sensitized to allow the data to flow through the delay path. The GSV will show the required condition to satisfy delay path sensitization. These might be different than the request made in the \$condition block.

The delay path definition can be altered or removed from the delay path definition file.

To see more precisely what this problem is and why it cannot be resolved, please use the `analyze_faults` command for the corresponding path delay fault.

See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## DRC Rule P24

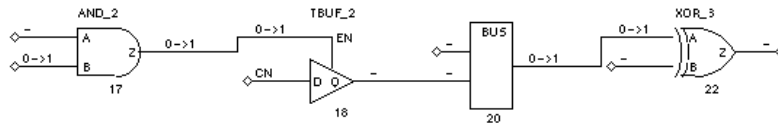
### Message Text

Path Delay Rule P24: Path P traverses enable of tristate gate (G) with unresolved Z.

### Severity

Warning

One example, many variations are possible:



## Example

### Description

The delay path definition named P is not fully testable because the bus driven by G can reach a floating condition.

### What Next

A violation can be analyzed using the graphical schematic viewer (GSV) by selecting its violation ID number from the Analyze button dialog box. This changes the pindata reporting to delay data and displays the delay path segment effecting this rule violation. Notice the highlighted circuit element containing the gate ID G and enable signal. The delay path transition indicators show the on-path transitions for the delay path definition in question. Since the delay path will pass through the enable signal to the output of the TSD, it is possible that the transition to or from the Z state cannot be properly propagated and detected.

Ideally a bus keeper or a weak bus driver, such as a pullup or pulldown device, is added to the bus. Otherwise, the path might not be testable and can never be robustly tested.

See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

## DRC Rule P25

### Message Text

Path Delay Rule P25: Path P contains sensitizable feedback loop L through gate G.

### Severity

Error

### Description

The delay path definition named P contains a gate G that is in a c (loop ID L). There are many potential problems with attempting to generate a test for a path delay fault through such a feedback loop.

### What Next

A violation can be analyzed using the graphical schematic viewer (GSV) by selecting its violation ID number from the Analyze button dialog box. This changes the pin data reporting to delay data and displays the delay path segment effecting this rule violation. Notice the highlighted circuit element with gate ID G. The delay path transition indicators show the on-path transitions for the delay path definition in question. Since there is no clear statement of what a path delay test should look like for a path that contains part of a feedback loop, it would be potentially very misleading for TestMAX ATPG to attempt to generate such a test. The best option in this case would be to select another path that will meet the same needs as path P.

In some cases, it might be possible to block the feedback path of the loop while generating a test. The P25 violation might be downgraded to a Warning and such tests attempted, but the simulation values produced by any tests generated for such paths should be examined extremely closely to ensure that they are consistent with the goals of path delay testing.

See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide.

---

## DRC Rule P26

### Message Text

Path Delay Rule P26: Clock C is missing a Q waveform in the W WaveformTable for path N.

### Severity

Error

### Description

The clock signal C is required to launch or capture a delay fault in path N, but does not have an appropriate pulsed waveform defined in the WaveformTable W. Without a pulsed waveform this path delay cannot be properly defined.

This error will cause additional path delay timing check errors (P7 or P8 messages) if the path has a specified delay, because the timing of the pulse is not defined and subsequent checks for the delay timing is incorrect.

In most circumstances, the signal will require a P waveform. However, if mux-clock waveforms are present and the signal is being used to launch a test, an E waveform is required on this signal. The error message indicates whether an E or P waveform is needed to satisfy this test.

### **What Next**

Review the definition of the referenced WaveformTable, and define an appropriate P or E waveform for this signal in this WaveformTable.

See also `set_rules` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide

# 8

## Category R - Compressor Rules

---

### See Also

- R1 - Compressor references an undefined chain.
- R2 - Chain input not connected to a load compressor.
- R3 - Chain output not connected to an unload compressor.
- R4 - Load compressor U\_decompressor does not have an internal connection for output X during mode Y.
- R5 - Missing load compressor input connection to a port.
- R6 - Missing unload compressor output connection to a port.
- R7 - Load compressor U\_decompressor failed propagation checking for chain X during mode Y.
- R8 - Incorrect unload compressor operation
- R9 - Two chains always have the same load value.
- R10 - Two chains always have the same unload signature
- R11 - X on chain affects observe ability of other chains.
- R12 - Simultaneously satisfiable modes for a compressor.
- R13 - Single observe mode connection for chain to output gate of unload compressor can be affected by X on other chains.
- R14 - X source T gate (G1) is propagatable to N scancells.
- R15 - X source T gate (G1) is unblockable to N scancells.
- R16 - Unidentified unload compressor input.
- R17 - Unidentified unload compressor output.
- R18 - Load compressor pipeline cell not holding state.
- R19 - Missing unload compressor pipeline lockup latch.



- R20 - Missing load compressor pipeline lockup latch.
- R21 - X-tolerant chain connection to output port for unload mode with load\_mode was not found.
- R22 - Chain chain\_name has no x-tolerant observation.
- R23 - Unload mode enable <mode\_name> at 0 does not condition full compression on output port of port\_name.
- R24 - Unable to infer output of chain C for unload compressor
- R25 - No X-tolerant observation for chain during a load mode
- R26 - Invalid PLL clock chain connections for load compressor
- R27 - Load compressor pipeline cell not holding or shifting
- R28 - Clock chain connected to multiple ports
- R29 - Clock chain port used for X-tolerance control
- R30 - Two nonclock chains not independent ignoring clock chain inputs
- R31 - Incorrect defined multi chain X-tolerant connection to output N of unload compressor X using mode L/U
- R32 - Unload compressor had invalid capturing edge on cell from load compressor cell
- R33 - Unable to find output of unload compressor
- R34 - Multiple candidates for output of unload compressor
- R35 - Unable to find input of load compressor
- R36 - Multiple candidates for input of load compressor
- R37 - Scan cell was clocked during serializer nonshifting cycle
- R38 - Scan cell was not clocked during serializer shifting cycle
- R39 - Unload compressor decompressor failed propagation checking for chain in unload mode
- R40 - Could not find flop at index of Load Serializer
- R41 - Incorrect single fanout connection for a chain
- R42 - Load compressor pipeline cell ID disturbed at time T of P procedure
- R43 - Load compressor pipeline cell ID disturbed during serializer initial shift cycles at time T of P procedure

- R44 - Pipeline cell ID between load serializer and load compressor does not hold state during capture
- R45 - Load compressor pipeline cell ID must not be constrained on PI\_NAME
- R46 - Serializer output compressor pipeline discrepancy
- R47 - Nonscan cell cannot control PLL internal clocking
- R48 - Load pipeline path cannot have multiple outputs with inconsistent inversions on different paths
- R49 - Load compressor is driven by a scan input with only dynamic bits
- R50 - Incorrect SPC chain structure
- R51 - Incorrect SPC update latch asynchronous control
- R52 - Incorrect SPC load compressor propagation
- R53 - SPC control register not holding state
- R54 - SPC control register is not capturing a constant value
- R55 - SPC chain is not in one to one mode
- R56 - Decompressor or compressor has different tail or pipeline stages
- R57 - Compressor scan assignment is missing in the load\_unload procedure shift vector
- R58 - ScanEnable not constrained to LOGICAL LOW
- R59 - Failed to extract SPC register due to incorrect number of pipeline stages
- R60 - Unload compressor output pipeline cell name on output\_signal does not hold during capture
- R61 - Invalid SPC register connection to cell\_name
- R62 - Ultra register Cell-name (Cell-id) disturbed during pre-shift of load/unload.
- R63 - Two chains always have the same unload signature was violated N times
- R64 - LE to TE transition from serial flop to the first scan flop.

---

## DRC Rule R1

### Message Text

Compressor Rule R1: Compressor references an undefined chain.

**Severity**

Fatal Error

**Description**

Indicates an Error in protocol file. A chain referred to in CompressorStructures section is not defined as scan chain in the ScanChain section

**What Next**

If the protocol file is generated by TestMAX DFT, please submit a testcase.

---

## DRC Rule R2

**Message Text**

Compressor Rule R2: Chain input not connected to a load compressor.

**Severity**

Fatal Error

**Description**

Indicates an error in the netlist file. An internal chain input is not connected to the decompressor.

**What Next**

If the netlist is generated by the TestMAX DFT, please submit a testcase.

---

## DRC Rule R3

**Message Text**

Compressor Rule R3: Chain output not connected to an unload compressor.

**Severity**

Fatal Error

**Description**

Indicates an error in the netlist file. An internal chain output is not connected to the compressor.

**What Next**

If the netlist is generated by the TestMAX DFT, please submit a testcase.

---

## DRC Rule R4

### Message Text

Compressor Rule R4: Load compressor U\_decompressor does not have an internal connection for output X during mode Y.

Unload compressor U\_compressor does not have an internal connection for output X.

### Severity

Fatal Error

### Description

The first version of this violation indicates that DRC failed for decompressor (also called the load compressor). This failure indicates that the decompressor internal connections in the netlist do not match what is specified in the STIL procedure file.

The second version of this violation indicates that DRC failed for compressor named U\_compressor (also called the unload compressor). This failure indicates that the compressor internal connections in the netlist do not match what is specified in the STIL procedure file.

There is no GUI support for an R4 violation.

### What Next

This error occurs when you use the `-hookup` option of the `set_scan_signal` DFT Compiler command during implementation, but do not sensitize the path between scan-input pin and hookup path properly with the constraints. If you manually modified the netlist or the STIL procedure file, recheck your changes.

This violation is related to an issue in TestMAX DFT. See the TestMAX DFT documentation for more details.

---

## DRC Rule R5

### Message Text

Compressor Rule R5: Missing load compressor input connection to a port.

### Severity

Fatal Error

### Description

An R5 violation indicates that the decompressor input specified in the protocol file is not connected to any port.

### What Next

If you used DFT compiler to generate the netlist and protocol file, please submit a testcase to Synopsys.

---

## DRC Rule R6

### Message Text

Compressor Rule R6: Missing unload compressor output connection to a port.

### Severity

Fatal Error

### Description

This violation indicates that the compressor output specified in the protocol file is not connected to any port.

### What Next

If you used DFT compiler to generate the netlist and protocol file, please submit a testcase to Synopsys.

---

## DRC Rule R7

### Message Text

Compressor Rule R7: Load compressor U\_decompressor failed propagation checking for chain X during mode Y.

### Severity

Fatal Error

### Description

An R7 violation indicates that DRC failed while verifying internal connections of the decompressor or the load compressor. There is no GUI support for this violation.

## What Next

If M624 warnings are generated, you should directly identify the combination of problem gates and problem states for the mode. This tracing methodology is explained in detail in the description of the M624 message in TestMAX ATPG Help.

Alternatively, specify the following commands:

```
set_rules r7 warning
set_pindata -shift
run_drc
```

This methodology enables you to continue the DRC process and access TEST mode. From TEST mode, list the R7 violations using the `report_violations r7` command, then debug one of the violations. You can report the contents of the chain violations using the `report_scan_cells x` command, where “x” is the chain listed as violated in the R7 violation report.

Since the R7 violations highlight a discrepancy between a chain and the load-compressor definition, identify the chain “X” cell with the most significant scan cell order number. This is the cell closest to the scan-in side (at the bottom of the list of cells reported by `report_scan_cells`). Open the GSV and add the scan cell to the display using the `add_display_gates C` command, where “C” is the name of the scan cell closest to scan in.

Click on the data input to the flip-flop. Make sure it connects to a scan input port and the mode select pins in the STIL procedure file.

Open the STIL procedure file and search for the unload compressor name, “U\_decompressor,” listed in the R7 rule violation. It might look like this:

```
CompressorStructures {
    Compressor
    "DFT__des_chip_core_U_decompressor_ScanCompression_mode" {
        ModeGroup mode_group;
        LoadGroup load_group;
        CoreGroup core_group;
        Modes 3;
        Mode 0 {
            ModeControls {
                "ci_i_rdata_de[7]" = 0;
                "spare_test_si" = 0;
            }
            Connection 0 "2" "9" "15" "20" "26" "32" "38" "45" "51" "56"
            "63"
                "69" "74" "80" "85" "91" "96";
            ...
        }
    }
}
```

Search for chain “X” in the list of connections listed under the mode “Y” specified in the R7 rule violation. The mode control pins are listed in the `ModeControls` block.

Connection 0 (as in the example, above) refers to “sccompin0” which points to the pin “ci\_i\_rdata\_de[1]”, as shown below:

```
ScanChain "sccompin0" {  
    ScanIn "ci_i_rdata_de[1]";  
}
```

In the example, "Connection 0" would list the chain “X” called out by the R7 violation. The “0” of “Connection 0” is the same as the “0” of “sccompin0”. Make sure you use the Connection and Mode with chain “X” listed in it, and called out by the R7 violation.

---

## DRC Rule R8

### Message Text

```
Compressor Rule R8: Incorrect unload compressor operation Default  
Severity: Error
```

### Severity

Error

### Description

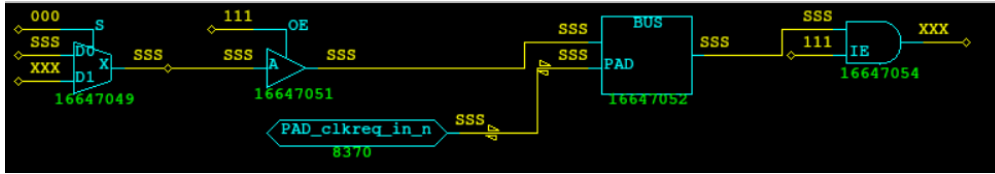
This violation indicates that the compressor connections specified in protocol file do not match the connections in the netlist.

### What Next

If you used DFT compiler to add compressors and generate the netlist, submit a test case to Synopsys. For more information, see the `-set_unload_mode_ports_to_x` option of the `set_drc` command.

For R8 violation, the `analyze_violation` command cannot give clear information for debugging in the following case.

One possible cause may be associated with the bus device, which is a gate inserted by TestMAX ATPG during ATPG design building to resolve multidriver nets. Both potential sources for the bus inputs appear to be driving, and both have values of X. One of the sources that has an X value is the `PAD_clkreq_in_n` bidirectional port. You can correct this by driving the port to a Z state. You edit the STIL file to add the declaration `PAD_clkreq_in_n = Z` to one of the `V{..}` vectors at the start of the `load_unload` procedure.



### Message Text

## Severity

### Description

The following table shows the threshold levels for the number of scan inputs and the maximum number of any two load-independent chains for a high X-tolerance codec:

| Number of Scan Inputs* | Maximum Number of Any Two Load-Independent Chains |
|------------------------|---------------------------------------------------|
| 2                      | 1                                                 |
| 3                      | 2                                                 |
| 4                      | 4                                                 |
| 5                      | 16                                                |
| 6                      | 81                                                |
| 7                      | 256                                               |
| 8                      | 625                                               |
| 9                      | 1296                                              |
| 10                     | 2401                                              |
| 11                     | 4096                                              |



| Number of Scan Inputs* | Maximum Number of Any Two Load-Independent Chains |
|------------------------|---------------------------------------------------|
| 12                     | 6561                                              |
| 13                     | 10000                                             |
| 14                     | 14641                                             |
| 15                     | 20736                                             |
| 20                     | 83521                                             |
| 25                     | 234256                                            |
| 30                     | 531441                                            |

\* In a serializer flow, the first column ("Number of Scan Inputs") refers to the number of combinational decompressor scan inputs driven by the deserializer, as specified by the `-inputs` option of the `set_scan_compression_configuration` command in DFT Compiler.

#### What Next

Either increase the number of scan input pins or decrease the number of chains in the load decompressor.

---

## DRC Rule R10

### Message Text

Compressor Rule R10: Two chains always have the same unload signature was violated X times.

### Severity

Warning

### Description

This message indicates that the threshold was exceeded for the maximum number of chains that can be used to create an unload compressor without aliasing (no two chains have errors that cancel out each other). This violation is dependent on the compressor architecture and not the design; it might reduce diagnostic effectiveness and lower diagnostic precision.

The following table shows the thresholds for the number of scan outputs and the maximum number of chains for creating an unload compressor with no aliasing:

| Number of Scan Outputs* | Maximum Number of Chains With No Aliasing |
|-------------------------|-------------------------------------------|
| 2                       | 3                                         |
| 3                       | 7                                         |
| 4                       | 15                                        |
| 5                       | 31                                        |
| 6                       | 63                                        |
| 7                       | 127                                       |
| 8                       | 255                                       |
| 9                       | 510                                       |
| 10                      | 1012                                      |
| 11                      | 1980                                      |
| 12                      | 3796                                      |
| 13                      | 7098                                      |
| 14                      | 12910                                     |
| 15                      | 22818                                     |
| 16 and higher           | 32000                                     |

\* In a serializer flow, the first column ("Number of Scan Outputs") refers to the number of combinational compressor scan outputs that drive the serializer, as specified by the `-outputs` option of the `set_scan_compression_configuration` command in DFT Compiler.

### What Next

An R10 violation is usually caused by a high compression ratio. This violation also might affect diagnostic accuracy. To eliminate this violation, you can either increase the number of scan output pins or decrease the number of affected chains in the unload compressor.

## DRC Rule R11

### Message Text

Compressor Rule R11: X on chain affects observability of other chains

### Severity

Warning

### Description

This message indicates that the threshold was exceeded for the maximum number of chains that can be used to create an unload compressor in which a single X does not mask observation on any other chain in a cycle (that is, no two chains can have errors that cancel out each other).

This condition might affect coverage in the default X-tolerant flow, and might affect the compression capability in any X-tolerant flow.

The following table shows the threshold levels for the number of scan output pins and the maximum number of chains that can be used to create an unload compressor in which a single X does not mask observation on any other chain in a cycle:

| Number of Scan Outputs* | Maximum Number of Chains With Single X-Tolerance |
|-------------------------|--------------------------------------------------|
| 2                       | 2                                                |
| 3                       | 3                                                |
| 4                       | 6                                                |
| 5                       | 10                                               |
| 6                       | 20                                               |
| 7                       | 35                                               |
| 8                       | 56                                               |
| 9                       | 84                                               |
| 10                      | 120                                              |
| 11                      | 165                                              |
| 12                      | 220                                              |
| 13                      | 286                                              |

| Number of Scan Outputs* | Maximum Number of Chains With Single X-Tolerance |
|-------------------------|--------------------------------------------------|
| 14                      | 364                                              |
| 15                      | 455                                              |
| 16                      | 560                                              |
| 17                      | 680                                              |
| 18                      | 816                                              |
| 19                      | 969                                              |
| 20                      | 1140                                             |
| 21                      | 1330                                             |
| 22                      | 1540                                             |
| 23                      | 1771                                             |
| 24                      | 2024                                             |
| 25                      | 2300                                             |
| 26                      | 2600                                             |
| 27                      | 2925                                             |
| 28                      | 3276                                             |
| 29                      | 3654                                             |
| 30                      | 4060                                             |

Number of  
Scan Outputs\*

\* In a serializer flow, the first column ("Number of Scan Outputs") refers to the number of combinational compressor scan outputs that drive the serializer, as specified by the `-outputs` option of the `set_scan_compression_configuration` command in DFT Compiler.

### What Next

Either increase the number of scan output pins or decrease the number of affected chains in the unload compressor.

---

## DRC Rule R12

### Message Text

Compressor Rule R12

Simultaneously satisfiable modes for a compressor

### Severity

Fatal Error

### Description

The load compressor modes must be mutually exclusive; that is, only one mode is active at the time. A violation of this rule typically indicates an error in the STL procedure file. For example:

```
Mode 0 {  
    ModeControls {  
        "test_si14" =0;  
        "test_si15" =0;  
    }  
Mode 1 {  
    ModeControls {  
        "test_si14" =0; # same as in Mode 0 !!  
    }  
}
```

### What Next

You need to correct the STL procedure file (and the netlist if needed). Keep in mind that the STL procedure file must match the netlist, otherwise other rule violations will occur.

---

## DRC Rule R13

### Message Text

Compressor Rule R13: Single observe mode connection for chain  
<chain\_name> to output gate <gate\_id> of unload compressor  
<compressor\_name> using mode <load\_mode>/<unload\_mode> can be affected by  
X on other chains.

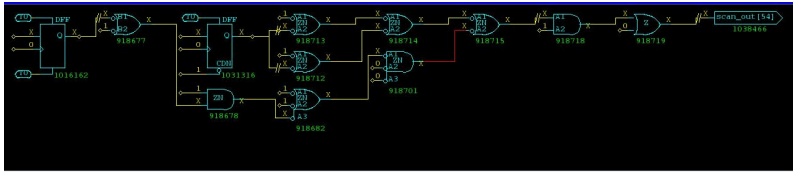
### Severity

Error

## Description

One-to-one modes are X-tolerant compression modes in which each scan output port is connected to only one compressed scan chain. A violation of this rule means that the path from the last scan register of the compressed scan chain to the scan output port is not sensitized at each logic level. This is often caused by reconvergent logic on another compressed scan chain's path through the compressor, such that the one-to-one mode is sensitized when the other scan chain output is either 0 or 1, but the simulation resolves to X when it is X.

## Example



The figure shows an R13 violation on the one-to-one mode path from the DFF on the left side to the scan output port on the right side. Data from the DFF in the center fans out to satisfy different compressor modes, but then reconverges to block the one-to-one mode path at the gate driven by the highlighted red net. The data on this DFF can be X for a number of reasons including capturing data from nonscan cells or from timing exception paths.

## What Next

The reconvergent logic is not in the original compressor design, but is added by synthesis optimizations. The compressor should be inserted again, with synthesis optimizations disabled for it.

Downgrading this violation will result in Verilog simulation mismatches, but only when serial simulation is performed. (Parallel simulation does not simulate the data path through the compressor). These mismatches will occur when the specified one-to-one mode is selected and the other compressed scan chain value is at X during the same shift cycle.

## DRC Rule R14

### Message Text

Compressor Rule R14: X source T gate (G1) is propagatable to N scancells (G2) .

### Severity

Warning

### Description

This message indicates that the potential for unknown values (Xs) to propagate to scan cells exists. Xs can cause lower test coverage, increased pattern count, and lower diagnostic accuracy.

T is the gate type of the X-source gate, G1 is its gate id, N is the number of scan cells to which it can propagate, and G2 is the gate id of one of the scan cell which can receive the X value.

The check is done assuming Basic-Scan algorithm, so some X sources (such as memories) might be controllable with a different algorithm. Xs might be due to incomplete design data (blackboxes), constraints, floating buses or uninitialized state. Buses that can have contention are not considered X sources because ATPG patterns will attempt to avoid bus contentions (by default).

### What Next

No action is required on your part if the coverage is satisfactory, or high diagnostic accuracy is not required.

The `analyze_violation` command can be used to understand the violation. The schematic display will show all gates between the X source and the identified scan cell, with pin data set to be constrain values.

To correct this problem, you might consider defining blackboxed modules, updating constraints, or enhancing the design to reduce Xs.

---

## DRC Rule R15

### Message Text

```
Compressor Rule R15: X source T gate (G1) is unblockable to scancells (G2) .
```

### Severity

Warning

### Description

This message indicates that it is not possible to block propagation of an unknown value (X) from an X source to a scan cell. Xs can cause lower test coverage, increased pattern count and lower diagnostic accuracy.

T is the gate type of the X source gate, G1 is its gate id, and G2 is the gate id of one of the scan cell which can receive the unblockable X value.

This check works best if all signal values required for capture mode operation are fully specified by PI constraints or in the STL procedure file. For example, scan-enable should be constrained to the capture value; if scan-enable is not constrained, this DRC check is free to assume the shift mode value to avoid X capture.

The check is done assuming Basic-Scan algorithm, so some X sources might be controllable with a different algorithm. Xs might be due to incomplete design data (blackboxes), constraints, floating buses or uninitialized state. Buses that can have contention are not considered X sources because ATPG patterns will attempt to avoid bus contentions (by default).

### What Next

No action is required on your part if the coverage is satisfactory, or high diagnostic accuracy is not required.

The `analyze_violation` command can be used to understand the violation. The schematic display will show all gates between the X source and the identified scan cell, with pin data set to be constrain values.

To correct this problem, you might consider defining black boxed modules, updating constraints, or enhancing the design to reduce Xs or allowing blocking of Xs by ATPG.

---

## DRC Rule R16

### Message Text

Compressor Rule R16

Unidentified unload compressor pipeline input. Unable to identify unload compressor pipeline input for port Y.

### Severity

Fatal Error

### Description

During the process of tracing back from the defined output port of the unload compressor, the tool was unable to uniquely identify the corresponding input of the pipeline stage.

### What Next

This error is likely due to a wrong definition of the compressor ports (in the STL procedure file) or the number of pipeline stages. Before making any changes to your design or the SPF, you can try using the `-max_trace_gates` option of the `set_workspace_sizes` command to increase the gate-tracing limit and determine if this change removes the R16 error.



## DRC Rule R17

### Message Text

Compressor Rule R17

Unidentified load compressor pipeline output. Unable to identify load compressor pipeline output for port Y.

### Severity

Fatal Error

### Description

During the process of tracing forward from the defined input port of the load compressor, the tool was unable to uniquely identify the corresponding output of the pipeline stage.

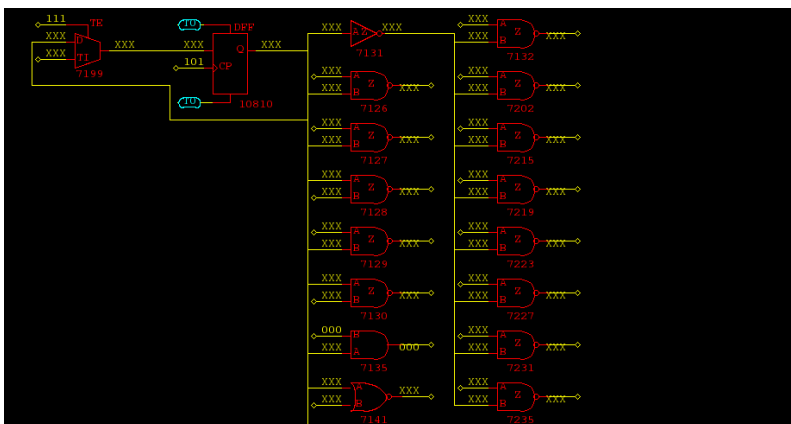
### What Next

This error is likely due to a wrong definition of the compressor ports (in the SPF) or the number of pipeline stages. Before making any changes to your design or the SPF, you can try using the `-max_trace_gates` option of the `set_workspace_sizes` command to increase the gate-tracing limit and determine if this change removes the R17 error. You can use the `analyze_violation` command to display the failing path. If you suspect that the tracing of head pipelines is leading to false R17, see Using Pipeline Guidance section for more debug steps.

### Example

```
>> analyze_violation R17-1
Error: Unable to identify load compressor pipeline output for port
test_si6. (R17-1)
The gate_report data is now set to "shift".
```

Open the GSV and gates at which the TestMAX ATPG blocks are displayed.



---

## DRC Rule R18

### Message Text

Compressor Rule R18

Load compressor pipeline cell not holding state. Load compressor pipeline cell Y does not hold state during capture.

### Severity

Warning

### Description

The load pipeline cell must hold state during capture for optimal results.

### What Next

If the load pipeline cell captures its own data during capture cycles, the violation can be corrected by constraining scan\_enable to its capture value. If the load pipeline cell has a separate clock, the violation can be corrected by constraining the clock off.

The presence of an R18 violation results in a small loss of coverage or increased pattern count. However, there is no danger of generating patterns that could fail simulation.

### See Also

- [R27 and M674](#)

---

## DRC Rule R19

### Message Text

Compressor Rule R19 : Missing unload compressor pipeline lockup latch. Lockup latch is missing between chain Y and unload pipeline.

### Severity

Error

### Description

This rule is violated if the last scan cell of a chain shifts on the leading edge, and the first stage of an unload pipeline cell shifts on the trailing edge, and there is no lockup latch in between; as a result, the pipeline cell could capture the new value of the scan cell.

### What Next

The simulator will assume the old value is captured, so this rule should be downgraded to warning only if enough data delay or enough clock skew exists from the last scan cell to the first unload pipeline stage to ensure that the old value is captured.

---

## DRC Rule R20

### Message Text

Compressor Rule R20 : Missing load compressor pipeline lockup latch.  
Lockup latch is missing between load pipeline and chain Y.

### Severity

Error

### Description

This rule is violated if the first scan cell of a chain shifts on the trailing edge, and the last stage of a load pipeline cell shifts on the leading edge, and there is no lockup latch in between; as a result, the scan cell could capture the new value of the pipeline cell.

### What Next

The simulator will assume the old value is captured, so this rule should be downgraded to warning only if enough data delay or enough clock skew exists from the last load pipeline stage to the first scan cell to ensure that the old value is captured.

---

## DRC Rule R21

### Message Text

Compressor Rule R21: X-tolerant chain connection to output port *port\_name* for unload mode <mode\_name1> with load\_mode <mode\_name2> was not found.

### Severity

Warning

### Description

This violation appears when, for a given load mode, some combination of inputs does not select an X-tolerant mode or a single-fanout XOR mode. Typically, R21 violations have no implication on QoR, although in some cases a small increase in pattern count might result.

### What Next

In general, R21 violations indicate that the compressor could be enhanced to provide more opportunities for observability.

You can enhance the compressor to avoid R21 violations by using all unused unload modes to select either a direct observation mode or a single-fanout XOR mode. TestMAX DFT and the `analyze_compressors` command can be used for this purpose. However, depending on the configuration (`#inputs`, `#outputs`, `#chains`), it might not be possible to eliminate all R21s.

Annotated schematic data for the first R21 violation (`report_violationR21-1`) can be viewed in the schematic viewer by setting: `set_pindata-drcdata {unload_mode <unload_mode>}`

In this case, the schematic is annotated with `unload_mode` data, that is, shift data plus unload mode pins for the unload mode given as an argument (`<unload_mode>`).

This data is for the load-mode and unload-mode specified in `report_violationR21-1`.

The same data is viewed by selecting: `analyze_violationR21-1`.

---

## DRC Rule R22

### Message Text

Compressor Rule R22 : Chain `chain_name` has no x-tolerant observation.

### Severity

Error

### Description

A violation of this rule indicates that the specified chain cannot be uniquely (non-XORed) observed in any unload mode. Downgrading this rule to pass DRC might result in loss of observability on the chain(s) that failed, and possibly loss of test coverage.

### What Next

Regenerate the unload compressor using more input/output pins or fewer internal chains to ensure all chains have X-tolerant observation and thus no R22 message.

When running DRC (before ATPG) on circuits which include blocks that have both default and high X-tolerant architectures, specify the following `set_rules` command:

```
set rules R22 -warn
```

This will downgrade a check done for fully X-tolerant designs built by DFT MAX which were actually built with blocks that include both default X-tolerant architectures and high X-tolerant architecture

---

## DRC Rule R23

### Message Text

Compressor Rule R23 : Unload mode enable *port\_name* at off\_state does not condition full compression on output port of *port\_name* (description)

Note that the description can be one of the following:

1. No sensitized path from chain *chain\_name*.
2. No sensitized XOR path from chain *chain\_name*.
3. Unexpected sensitized path from chain *chain\_name* blocked by X value.
4. Blocked by X value.

### Severity

Error

### Description

DFTMAX compression with enhanced X-tolerance may operate in two modes:

The default mode is Full XOR. In this mode, the DFTMAX circuitry does not use any additional X masking and operates as if classic DFTMAX is being used.

In selective mask mode, DFTMAX compression enables additional X masking. Most shift cycles do not require a selective mode for masking and could achieve higher observability in full compression (full XOR) mode. The unload mode enable should create the full compression condition on a per-shift basis.

Note that downgrading this rule to pass DRC might result in increased pattern count

In the description #1 version of the message, the value placed on the named chain has NO effect on the named output. Only the first problem chain is listed.

In the description #2 version of the message, the value placed on the named chain has an effect on the named output, but the effect is not that of an XOR. Only the first problem chain is listed.

In the description #3 version of the message, the value placed on the named chain has an effect on the named output, but no connection was expected based on the protocol file description of the compressor. Only the first problem chain is listed.

In the description #4 version of the message, the named output is blocked by an X value, so it does not reflect the XOR of the chain values.

### What Next

Specify the following:

```
set_pindata -drcdata unload_enable
```

Browse in the GSV and see what happens during shift with unload\_enable at off\_state: it should be full-XOR mode. Follow the shift path (X) from the DFF on the output of chains to the output ports; or back from the output ports to the DFF.

Note that the schematic is annotated with a value per gate derived from the shift procedure, plus the unload\_enable primary inputs at their off\_state. The output gates of internal scan chains are placed at X, so that you can trace the sensitized path in the GSV -- except in the case of "Blocked by X value." In this case, the output gates of internal scan chains are placed at random binary values -- thus enabling you to use the GSV to trace X-blockage.

For more information, see the -set\_unload\_mode\_ports\_to\_x option of the set\_drc command.

---

## DRC Rule R24

### Message Text

```
Compressor Rule R24 : Unable to infer output of chain C for unload  
compressor
```

### Severity

Fatal Error

### Description

TestMAX ATPG DRC initially tries to infer the connection between scan chains and the unload compressor. It traces from the scanout ports back through the unload compressor until it reaches a scan flip-flop. Since each scanout port is fed by multiple scan chains, tracing back from a scanout port creates a collection of scan chains that drive that particular scanout port. From this information, DRC infers the scanout ports driven by each scan chain. The CompressorStructures block in the STL procedure file used for DRC contains similar information. When TestMAX ATPG infers the scanout ports driven by a scan chain, the DRC process does not match the same ports as specified in the protocol file, and an R24 message is printed.

An R24 error can also be reported in the following cases while running DFTMAX Ultra compression:

- The deserializer chain was incorrectly traced.
- A scan chain tracing failure occurred because of incomplete shift DRC tracing.

### What Next

1. Specify the following:

```
set_drc -compressor_debug_data
```

2. Execute the `run_drc` command to produce verbose output for debugging.
3. Add the missing element to the graphical schematic viewer (GSV).
4. Set the pindata to shift.
5. Analyze the violation.

If the ScanDataOut pin for a ScanChain is specified in a design's ScanStructures section of the protocol, make sure that the Place-and-Route tool does not make any changes from that pin. Failure to do this can result in pattern mismatches in simulation and on the ATE.

```
Begin compressor chain I/O checking: #undefined_io=6576 ...

Warning: #candidate chains (3287) and #connecting chains (3288)
for unload compressor SUB_AudioVideo_U_compressor are different. (M620)

Error: Unable to infer output of chain 1877 for unload compressor.
(R24-1)

Begin R24 debug data: Chain=1877, connection_ports = test_so54 test_so91
-----

output_id  instance_name of cell on chain output
           connections
-----
-----

364962
c8compo1_7100_top_0/i_compo_gen/i2_vp_md1/i_vp_reg_md1/vp_reg_coeff1r_re
gx17x  test_so1 test_so23
```

There are many lines similar to the previous example. In the previous example, gate id 364962 has been identified as the output of an internal chain that connects, through the

unload compressor, to POs test\_so1 test\_so23; this chain is then matched to a chain defined in the STL procedure file that connects to the same outputs.

To see the connections, run the `set_pindata shift` command and trace through the GSV.

---

## DRC Rule R25

### Message Text

Compressor Rule R25 : Chain A has no x-tolerant observation for load mode B

### Severity

Warning

### Description

This violation appears when the number of unload mode internal signals ( $= \log(\#chains / \#outputs)$ ) is larger than the number of load decompressor scan inputs ( $= \text{total inputs} - \text{unload\_enable\_pin} - \text{load\_mode\_pins}$ ). The presence of R25 violations might result in increased pattern count.

This violation is similar to [R22](#), except that it refers to a specific load mode. The chain considered might have X-tolerant observation in other load modes. If it has no X-tolerant observation in any load mode, then [R22](#) is also violated. Violation of R25 might result in test coverage loss because it increases the probability that a fault cannot be observed in the load mode in which it was sensitized.

---

## DRC Rule R26

### Message Text

Compressor Rule R26 : PLL clock chain A with control cell B has invalid connections for load mode C of compressor D

### Severity

Error

### Description

This message checks that the cells that control PLL internal clocks do NOT share scanin data with any other scan cells.

The clock control chain is all care bits all the time, so no compression of its data is possible.



### What Next

Downgrading an R26 message allows the generation of correct patterns; the simulator rejects bad patterns, if any. However, loss of coverage or increased pattern count might result.

---

## DRC Rule R27

### Message Text

Compressor Rule R27 : Load compressor pipeline cell *name* neither holds or shifts during capture.

### Severity

Error

### Description

The load pipeline cell must hold state during capture for optimal results (see the description of the [R18](#) violation). Alternately, the load pipeline cell can shift during capture, or it can either shift or hold during capture, depending on the value of some input (e.g., `scan_enable`). The `report_compressors-pipeline` command identifies the three types of load pipeline cells as "hold", "shift," or "both," respectively.

Load pipeline cells in the "hold" category pass both [R18](#) and R27.

Load pipeline cells in the "shift" or "both" categories fail [R18](#), but pass R27; this can result in a small loss of coverage and/or increased pattern count (see [R18](#)).

An R27 violation indicates that a load pipeline cell is in neither "hold", "shift," or "both".

Note that all chains must go through the compressors. Even if a chain (such as the clock chain) is not compressed, it is still recognized by TestMAX ATPG as going through the compressors, albeit with a 1:1 "compression." All compressor inputs must have the same pipeline depth and all compressor outputs must have the same pipeline depth.

### What Next

If possible, use the `add_pi_constraints` command to add constraints to pass without an R27 violation. Otherwise, you must modify the design, including balancing the pipeline depth on all chains.

To verify if the compressors have the same pipeline depths for inputs and output, temporarily downgrade the R27 message to a warning. Then run the `report_compressors-pipeline -verbose` command so you can view additional information on the pipeline registers that might help identify the cause of the violation.

*Note:* Downgrading an R27 violation from an error to a warning to pass DRC can result in patterns that will fail simulation and on the tester.

---

## DRC Rule R28

### Message Text

Compressor Rule R28: Clock chain connected to multiple ports

### Severity

Error

### Description

This message indicates that the clock chain (that is, a scan chain that contains one or more scan cells used to control internal clocking) must connect to the same scan input and must be independent of the load decompressor mode. Note that the clock cells are always care bits, so they must be set to 0 or 1 in every pattern. If this requirement is not met, there might be a loss of test coverage and pattern inflation because of the additional load decompressor dependencies between the clock cells and other scan cells.

Because of the potential for test coverage degradation and pattern inflation, you should not downgrade this rule to pass DRC. Instead, you should change the DFT implementation. In this case, DFTMAX constructs and connects a compressor to prevent this rule violation -- but only if it is specified with a clock chain.

A common error in the OCC controller recognition flow is to omit the `set_scan_path -class occ` command to force the DFT architect to recognize the clock chain. It is important to specify this command at the hierarchical level in which the OCC controller is first declared. In a hierarchical flow, this problem might occur only at a higher level.

In all cases, the `preview_dft` command report should show that the clock chain is a segment with the "occ" property. For example:

```
Scan chain '1' contains 4 cells
Active in modes: ScanCompression_mode :
CORE/clock_chain (s) (o) (m) (CORE/duto/clk, 45.0, rising)
```

The string "(o)" refers to a clock chain. All clock chain segments should have this tag. If it is missing, then DFTMAX compression might stitch the clock chain to cause a rule violation.

---

## What's Next

---

### DRC Rule R29

#### Message Text

Compressor Rule R29 : Clock chain port used for X-tolerance control

#### Severity

Error

#### Description

This message indicates that the chain (that is, a scan chain that contains one or more scan cells used to control internal clocking) must connect to a scan input which is not used to control X-tolerance. Note that the clock cells are always care bits, so they must be set to 0 or 1 in every pattern. If this requirement is not met, there might be a loss of test coverage and pattern inflation because of the additional load decompressor dependencies between clock cells and X-tolerance controls.

#### What Next

Because of the potential for test coverage degradation and pattern inflation, you should not downgrade this rule to pass DRC. Instead, you should change the DFT implementation. In this case, DFTMAX constructs and connects a compressor to prevent this rule violation -- but only if it is specified with a clock chain.

A common error in the OCC controller recognition flow is to omit the `set_scan_path -class occ` command to force the DFT architect to recognize the clock chain. It is important to specify this command at the hierarchical level in which the OCC controller is first declared; in a hierarchical flow, the problem might only occur at a higher level.

In all cases, the `preview_dft` command report should show that the clock chain is a segment with the "occ" property. For example:

```
Scan chain '1' contains 4 cells
Active in modes: ScanCompression_mode :
CORE/clock_chain (s) (o) (m) (CORE/duto/clk, 45.0, rising)
```

The string "(o)" refers to a clock chain. All clock chain segments should have this tag. If it is missing, then DFTMAX compression might stitch the clock chain to cause a rule violation.

---

## DRC Rule R30

### Message Text

Compressor Rule R30 : Nonclock chains <name1> and <name2> are not independent when clock chain inputs are ignored.

### Severity

Warning

### Description

The load decompressor must be designed to minimize dependencies between scan chains. At a minimum, any two non-clock chains must be independent in at least one load mode. Because clock cells are always “care bits” (they must be set to 0 or 1 in every pattern), non-clock chains should be independent of each other and of clock chain scan inputs. Failure to satisfy this requirement will result in additional load decompressor dependencies between non-clock chains. The clock chain bits is set properly, so this rule violation does not indicate that bad patterns are generated.

### What Next

If this rule fails, during ATPG the results might be a loss of test coverage and pattern inflation due to the additional load decompressor dependencies.

---

## DRC Rule R31

### Message Text

Compressor Rule R31

Incorrect defined multi chain X-tolerant connection to output N of unload compressor X using mode L/U.

Incorrect defined X-tolerant connection for chain C to output N of unload compressor X using mode L/U.

### Severity

Error

### Description

This rule checks the correctness of the defined connections (if they are defined) between chain outputs and the outputs of an unload compressor for a given X-tolerant mode, as specified by a load and unload mode value.

There are two variations of this rule: the first is for modes with a single chain connection to a compressor output; the second is for modes with multiple chain connections to a chain output.

Note the following:

C is the name of the chain

N is the ID number of the output of the unload compressor

X is the name of the unload compressor

L is the load mode value

U is the unload mode value

This rule is checked by simulating the conditions set up by the shift procedure augmented by the conditioning of a given load and unload mode (including the X-tolerant enable pin set to its active value) and with the chain outputs set to random values. The simulated value of an output of an unload compressor is compared with its expected value as determined by its defined connections. If this rule fails, the connection definition might be incorrect or the conditioning set up by the shift procedure might be incomplete. There is no automated violation analysis available for this violation to assist in debug.

This rule can be downgraded to a warning to pass DRC, however there is a high risk that the patterns might not work at the tester.

### What Next

To debug a selected violation, specify the following command before `run_drc`:

```
set_drc -store_unload_mode_data L
```

In this case, data for up to 32 unload modes is stored for the selected load mode, L. Next, specify the following:

```
set_pindata -drcdata {unload_mode U}
```

At this point, you need to open the GSV and trace back the sensitized (X) path from the output N of the unload compressor to determine why it does not connect to the expected scan chain output(s).

For more information, see the `-set_unload_mode_ports_to_x` option of the `set_drc` command.

---

## DRC Rule R32

### Message Text

Compressor Rule R32

```
Unload compressor <unl_comp_name> had invalid capturing edge on cell  
<unl_gate_id> from load compressor cell <load_gate_id>.
```

### Severity

Error

### Description

This rule checks the edges of load and unload compressor pipelines for X-tolerant compressors. A violation occurs if a load compressor cell, indicated by <load\_gate\_id>, is a leading-edge (LE) cell and an unload compressor cell, indicated by <unl\_gate\_id>, is a trailing-edge (TE) cell. The LE-TE data transfer could occur in a single cycle, thus patterns might fail.

Downgrading this error to pass DRC could result in patterns that fail simulation and on the tester

### What Next

The edge of the pipeline cells must be changed, or lockup latches must be inserted to eliminate all LE-TE paths.

---

## DRC Rule R33

### Message Text

```
Compressor Rule R33
```

```
Unable to find output output_id of unload compressor name.
```

### Severity

Fatal Error

### Description

When unload serializers are used, random pattern simulation of the events in the load and shift procedures is performed to identify unload compressor output nodes. This violation occurs when there is a failure to identify a DFF (D flip-flop) gate containing data input with the expected value at the appropriate cycle.

### What Next

This violation is a fatal error, and must be corrected to pass DRC. There is no automated violation analysis for this rule. You can use the `report_serializers -all` command to display the serializer data and identify unload serializer outputs. Normally, the problem is due to an incorrect netlist or test procedure file.

This error can occur if the top-level serializer protocol is generated by the spfgen utility or manually created, and the ScanEnable signal is not constrained. In this case, you must constrain the ScanEnable signal to 0 using the `add_pi_constraints 0` command.

If both R33 and R35 errors are reported, you should investigate all R35 errors first.

If you encounter only R33 errors, and no R35 errors, you should perform the following steps:

Specify the `set_drc -store_initial_shifts` command.

Specify the `set_pindata -shift` command

Execute the `run_drc` command.

Perform the following examinations:

Use the GSV to make sure the clock behavior is correct by checking clock pins of unload serializer registers.

Make sure no unexpected blockage occurs on unload serializer registers that traverse data paths to scan output ports.

To understand the pindata values, refer to the “Debugging TestMAX ATPG Serializer DRC Errors” section in the *Adaptive Scan with Serializer User Guide*.

If there are no problems on the signals, there might be problems on the internal scan chains or inside the compressor logic.

The following steps enable TestMAX ATPG to recognize user-provided unload serializer registers without undergoing the identification process during DRC. This allows you to move on to subsequent DRC processes.

Specify unload serializer register names in the STIL procedure file. For more information, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

Reset the `set_drc -nostore_initial_shifts` command.

Specify the `set_drc -nocheck_user_serializer_bits` command.

Execute the `run_drc` command.

By applying these steps, you can move on to subsequent DRC processes in which S and R violations (other than R33 to R38) might appear. You can usually debug these violations using the GSV. This enables you to skip serializer-specific DRC and move on to scan DRC and DFTMAX DRC. You will most likely encounter S or R violations (other than R33 to R38), and you should debug them.

## DRC Rule R34

### Message Text

Compressor Rule R34

Multiple candidates *gate\_id1*, *gate\_id2* for output *output\_id* of unload compressor *name*.

### Severity

Error

### Description

When unload serializers are used, the unload compressor output nodes are identified via random pattern simulation of the events in the load and shift procedures. This violation occurs when more than one D flip-flop (DFF) gate is identified containing a data input with the expected value at the appropriate cycle.

### What Next

You can downgrade the severity of this violation and continue the DRC process. If multiple candidates are identified for an unload compressor output, the first candidate identified is selected. Additional DRC failures will occur if you downgrade the severity, and the first candidate is incorrect.

If you encounter an R34 error, you can edit the STIL procedure file to specify the correct unload serializer registers, as shown in the following example:

```
UserKeywords SerializerStructures CompressorStructures;
SerializerStructures {
    InternalShiftStart 6;
    UnloadDataStart 7;
    ExternalCyclesPerShift 5;
    LoadSerializer "abc_top_U_deserializer_abc" {
        Length 5;
        ActiveScanChains load_group;
    }
    UnloadSerializer "abc_top_U_serializer_abc" {
        Length 5;
        ActiveScanChains unload_group;
        ParallelInputs {
            0 "abc_top_U_compressor_abc/serial_reg_0_" no
            1 "abc_top_U_compressor_abc/serial_reg_1_" no
            2 "abc_top_U_compressor_abc/serial_reg_2_" no
            3 "abc_top_U_compressor_abc/serial_reg_3_" no
            4 "abc_top_U_compressor_abc/serial_reg_4_" no
        }
    }
}
```



```
}  
}
```

For more information, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

---

## DRC Rule R35

### Message Text

Compressor Rule R35

Unable to find input *input\_id* of load compressor *name*.

Unable to find xtol-enable input of unload compressor *name*.

Unable to find mode-port input with serializer connection *port\_nameposition*.

### Severity

Fatal Error

### Description

When load serializers are used, random pattern simulation of the events in the load and shift procedures is performed to identify the load compressor input nodes, xtol-enable input nodes, and mode-port input nodes. Failure to identify any D flip-flop (DFF) gates with an output containing the expected value at the appropriate cycle will result in this violation.

### What Next

This violation is a fatal error, and you must fix it to pass DRC. There is no automated violation analysis for this rule. You can use the `report_serializers -all` command to display serializer data and identify load serializer inputs.

If you encounter any R35 errors, you should perform the following steps:

Specify the `set_drc -store_initial_shifts` command.

Specify the `set_pindata -shift` command

Execute the `run_drc` command.

Perform the following examinations:

Use the GSV to make sure the clock behavior is correct by checking clock pins of the load serializer registers and update stage registers (if they are used).

Make sure no unexpected blockage occurs on load serializer registers that traverse data paths to scan input ports.

To understand the pindata values, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

After you finish the debugging process, make sure you reset the `set_drc -nostore_initial_shifts` command.

---

## DRC Rule R36

### Message Text

Compressor Rule R36

Multiple candidates `gate_id`, `gate_id` for input `input_id` of load compressor `name`.

Multiple candidates `gate_id`, `gate_id` for xtol-enable input of unload compressor `name`.

Multiple candidates `gate_id`, `gate_id` for mode-port input with serializer connection `port_nameposition`.

### Severity

Error

### Description

When load serializers are used, random pattern simulation of the events in the load and shift procedures is performed to identify load compressor input nodes, xtol-enable input nodes, and mode-port input nodes. This violation occurs when more than one D flip-flop (DFF) gate is identified with an output that has an expected value at the appropriate cycle.

### What Next

You can reduce the severity of this violation and continue the DRC process. If multiple candidates are identified for a load-compressor, xtol-enable, or mode-port input, the first candidate identified is selected. If you reduce the severity and this candidate is not the correct choice, then other DRC failures will occur.

If you encounter an R36 error, you can edit the STIL procedure file to specify the correct load serializer registers, as shown in the following example:

```
UserKeywords SerializerStructures CompressorStructures;  
SerializerStructures {  
    InternalShiftStart 6;
```

```
UnloadDataStart 7;
ExternalCyclesPerShift 5;
LoadSerializer "abc_top_U_deserializer_abc" {
    Length 5;
    ActiveScanChains load_group;
    ParallelOutputs {
        0 "abc_top_U_compressor_abc/serial_reg_0_" no
        1 "abc_top_U_compressor_abc/serial_reg_1_" no
        2 "abc_top_U_compressor_abc/serial_reg_2_" no
        3 "abc_top_U_compressor_abc/serial_reg_3_" no
        4 "abc_top_U_compressor_abc/serial_reg_4_" no
    }
}
UnloadSerializer "abc_top_U_serializer_abc" {
    Length 5;
    ActiveScanChains unload_group;
}
}
```

For more information, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

---

## DRC Rule R37

### Message Text

Compressor Rule R37: Scan cell `gate_id` was clocked during serializer nonshifting cycle.

### Severity

Error

### Description

When serializers are used, random pattern simulation of the events in the load and shift procedures is performed to verify that D flip-flops (DFFs) are not clocked in the same nonshift cycle. The pattern simulation is done at the same time the load compressor inputs and unload compressor outputs are identified. An R37 error is issued if the DFFs are clocked during a nonshift cycle. Later in the DRC process, after the scan cells have been identified, the scan cell master cells are checked to make sure they are not clocked during nonshift cycles.

When using the `set_drc -store_initial_shifts` command, the first InternalShiftStart shift cycles are stored and can be inspected when the pindata is set to shift. However, this applies strictly to situations when you are debugging R violations that occur before tracing the internal scan chains (since storing all cycles in the shift will not allow the tracing of internal scan chains). An M464 warning message is issued in this case.

### What Next

You can downgrade the severity of this violation, and continue the DRC process. If you reduce the severity, the patterns generated during ATPG will not work at the tester. Normally, the source of the problem is an incorrect netlist or test procedure file.

You can debug an R37 error by performing the following steps:

Specify the `set_drc -store_initial_shifts` command.

Specify the `set_pindata -shift` command

Execute the `run_drc` command.

Use the GSV to display the gate ID reported with the R37 violation, and examine the pindata on the clock line.

Trace back the clock signal to determine why it is clocked in a nonshift cycle, and identify any differences from other scan cells without the R37 violations.

To understand the pindata values, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

After you finish the debugging process, make sure you reset the `set_drc -nostore_initial_shifts` command.

---

## DRC Rule R38

### Message Text

Compressor Rule R38

Scan cell `gate_id` was not clocked during serializer shifting cycle.

### Severity

Error

### Description

When serializers are used, random pattern simulation of the events in the load and shift procedures is performed to verify that the D flip-flops (DFFs) are clocked in a shift cycle. This pattern simulation is performed at the same time the load compressor inputs and unload compressor outputs are identified. An R38 violation is issued if the DFFs are not clocked in a shift cycle. Later in the DRC process, after the scan cells have been identified, the scan cell master cells are checked to make sure they are clocked during shift cycles.

When using the `set_drc -store_initial_shifts` command, the first InternalShiftStart shift cycles are stored and can be inspected when the pindata is set to shift. However, this applies strictly to situations when you are debugging R violations that occur before tracing the internal scan chains (since storing all cycles in the shift will not allow the tracing of internal scan chains). An M464 warning message is issued in this case.

### What Next

You can downgrade the severity of this violation, and continue the DRC process. If you reduce the severity, the patterns generated during ATPG will not work at the tester. Normally, the problem is due to an incorrect netlist or test procedure file.

You can debug an R38 error by performing the following steps:

Specify the `set_drc -store_initial_shifts` command.

Specify the `set_pindata -shift` command

Execute the `run_drc` command.

Use the GSV to display the gate ID reported with the R38 violation, and examine the pindata on the clock line.

Trace back the clock signal to determine why it is not clocked in a shift cycle, and identify any differences from other scan cells without R38 violations.

To understand the pindata values, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

After you finish the debugging process, make sure you reset the `set_drc -nostore_initial_shifts` command.

---

## DRC Rule R39

### Message Text

Compressor Rule R39

Unload compressor decompressor failed propagation checking for chain *n* in unload mode *M*.

### Severity

Error

### Description

During tracing from the ScanOut port, the expected mask bits were not found for the identified unload mode.

### What Next

Specify the following:

Execute the `run_drc` command to produce verbose output for debugging.

---

## DRC Rule R40

### Message Text

Compressor Rule R40

Could not find flop at index (N) of Load Serializer (M)

### Severity

Error

### Description

Deserializer chain tracing was incomplete. The possible causes are as follows:

1. Clock not pulsing or netlist/STL procedure file was manually edited.
2. When serializer is implemented in the design, staggered scan clocks have timing that either occur later than trailing edge of the deserializer register clocks or earlier than leading edge of the serializer register clocks.

### What Next

1. Specify the following:

```
set_drc -compressor_debug_data
```

2. Execute the `run_drc` command to produce verbose output for debugging.
3. Add the missing gate to the graphical schematic viewer (GSV).
4. Set the pindata to shift.
5. Analyze the violation.
6. If R40 is reported in a design where serializer is implemented, then you must ensure that timing of the scan cells capturing data from deserializer and update registers must not occur later than the trailing edge of the deserializer clocks. Similarly, the internal scan cells that launch the data through compressor to serializer registers must not occur earlier than leading edge of the serializer register clock.

Refer to "Staggered Scan Clocks" in the TestMAX DFT User Guide in for more details.

---

## DRC Rule R41

### Message Text

Compressor Rule R41

Error: Unload compressor (M): Single fanout connection error for chain (C). (R41-3)

### Severity

Error

### Description

R41 indicates a wrong single fanout connection for a chain (C). Chains are assigned a single fanout output based on the order of appearance of the chains in the core group definition. A chain that is at the first position in the core group should have zero as the chain single fanout output.

### What Next

R41 might be caused due to the wrong netlist or wrong SPF. First, check if the SPF is wrong. If the order of chains in the core group is different from the ordering of chains in unload 'Mode 0', then it is an SPF issue. If the ordering is correct, then the error could be due to issues in the netlist. If the ordering is different, you can modify the chain ordering in the core group definition.

---

## DRC Rule R42

### Message Text

Compressor Rule R42

Load compressor pipeline cell ID disturbed at time  $T$  of  $P$  procedure.

### Severity

Error

### Description

The load pipeline cells must be in a hold state during certain procedures, such as load\_unload preamble.

A violation of this rule indicates that the load compressor pipeline cell ID is not in a hold state (it was disturbed) at time  $T$  of the procedure  $P$ .

Rules [R18](#) and [R27](#) check load pipeline behavior during capture. However, unlike these rules, R42 checks other procedures, such as the load\_unload preamble.

You can downgrade the R42 error to a warning to pass DRC. However, this can result in patterns that will fail simulation and on the tester.

### What Next

You can analyze this violation in the graphical schematic viewer (GSV) by displaying the pipeline cell ID with the pin data set to the offending procedure `P`. You should correct the protocol or the netlist to pass this rule.

---

## DRC Rule R43

### Message Text

Compressor Rule R43

Load compressor pipeline cell ID disturbed during serializer initial shift cycles at time `T` of `P` procedure.

### Severity

Error

### Description

When serializers are used, the first internal shift clock pulse occurs after a certain number of serializer shift clock pulses. For example, for the normal serializer duty cycle, the first internal shift clock pulse occurs at the InternalShiftStart cycle.

The load pipeline cells must be in a hold state during initial shift cycles of the serializer Shift procedure.

A violation of this rule indicates that the load compressor pipeline cell ID is not in a hold state (it was disturbed) at time `T` of the Shift procedure.

Rules [R18](#) and [R27](#) check the load pipeline behavior during capture. However, unlike these rules, [R42](#) and R43 check the load pipeline behavior during scan shift. [R42](#) checks the load pipeline behavior during the load\_unload preamble, while R43 checks the initial shift cycles for the Shift procedure.

You can downgrade the R43 error to a warning to pass DRC. However, this can result in patterns that will fail simulation and on the tester.



### What Next

You can debug an R43 violation by performing the following steps:

Specify the `set_drc -store_initial_shifts` command.

Specify the `set_pindata -shift` command

Execute the `run_drc` command.

Use the GSV to display the pipeline cell ID reported with the R43 violation, and examine the pindata on the clock line.

Determine why the load pipeline is clocked during the initial shift cycles before the first internal shift.

Correct the netlist to pass DRC.

To understand the pindata values, refer to the “DFTMAX Compression with Serializer” section in the *TestMAX DFT, DFTMAX, and DFTMAX Ultra User Guide*.

After you finish the debugging process, make sure you reset the `set_drc -nostore_initial_shifts` command.

---

## DRC Rule R44

### Message Text

Compressor Rule R44

Pipeline cell ID between load serializer and load compressor does not hold state during capture.

### Severity

Error

### Description

When serializers are used, pipeline cells between load serializer and load compressor must be in a hold state during capture. Failure to satisfy this rule results in patterns that fail simulation.

### What Next

You can add a self loop around the pipeline cell by using a multiplexer with the select line connected to the scan enable so the data is capture during the capture cycles.

Alternatively, if the load pipeline cell has a separate clock, the violation can be corrected by constraining the clock off.

### See Also

- [R27](#)
- [R42](#)
- [R43](#)

---

## DRC Rule R45

### Message Text

Compressor Rule R45

Load compressor pipeline cell *cell\_ID* must not be constrained on *PI\_NAME*

### Severity

Error

### Description

The load pipeline cell (*cell\_ID*) does not hold its state during capture, although it might be a shift pipeline. In this context, the primary input (*PI\_NAME*) driving this pipeline must not be constrained. When the primary input is constrained, TestMAX ATPG generates patterns that fail the simulation and tester operation. This occurs because dependencies exist between the pattern requirements and the ability to control the state on the primary input.

### What Next

Remove the primary input constraint from the specified input and make sure this change passes DRC. Note that primary input constraints might mask subsequent [R27](#) issues on these pipeline cells. These issues must be resolved separately.

### See Also

- [R18](#)
- [R27](#)

---

## DRC Rule R46

### Message Text

Compressor Rule R46

Serializer output compressor pipeline discrepancy, should be *number* more elements deep.

### Severity

Warning

### Description

The number of shifts performed in the load\_unload procedure does not match the data length generated for the serializer context. This indicates the serializer output pipelines were incorrectly specified in the `SerializerOutputPipelineStages` statement in the SPF. Patterns are generated in this context, however the input data for the load\_unload procedure will exceed the data limit for the load\_unload constructs, and may affect test translation flows.

### What Next

Remove the primary input constraint from the specified input and make sure the input passes DRC. The primary input constraints might mask subsequent [R27](#) issues on these pipeline cells, so you must resolve these issues separately.

### See Also

- [R18](#)
- [R27](#)

---

## DRC Rule R47

### Message Text

Compressor Rule: R47:

Nonscancell *ID* used to control PLL internal clocking.

### Severity

Error

### Description

This message is issued when the *ID* nonscan cell controls PLL internal clocking. However, nonscan cells cannot be controlled through scan load. This means the ability to control clocks might be severely degraded, which causes poor test coverage.

This message is also issued for an incorrectly configured compressed (DFTMAX) design if pipelining is used and the clock chain does not have pipelines.

All chains must go through the compressors. Even if a chain is not compressed, such as the clock chain, it is still recognized by TestMAX ATPG as such (it is "compressed" at a 1:1 ratio). Also, all compressor inputs must share the same pipeline depth, and all compressor outputs must share the same pipeline depth.

For example, in a design with two stages of output pipeline on the unload compressor, the two cells on the output side of the clock chain are considered pipeline cells (see the `report_compressors-pipeline` command). These cells are not part of the chain because they are nonscan cells. This situation causes the R47 violation. The expected usage is to add two additional DFFs on the output of the clock chain.

### What Next

You must modify the design so that all chains, including non-compressed chains, have the same input and the same output pipeline depth.

---

## DRC Rule R48

### Message Text

Compressor Rule R48

Load pipeline path from *name\_1* (*cell\_ID*) has [inverted | non-inverted] polarity and is connected to cell *name\_2* (*cell\_ID*) with [non-inverted | inverted] polarity.

### Severity

Error

### Description

The load pipeline path from the *name\_1* cell has multiple outputs (a forked output path in the pipeline) with inconsistent inversions on the different paths. Some of these paths might not be primary pipeline paths. However, the cells on these paths have relationships with each other; for example, as master-slave combinations. Therefore, dependencies exist between the cells. TestMAX ATPG does not support complex pipeline paths with path-specific inversions.

### What Next

Eliminate the inversion differences in the pipeline fork so the paths generate properly. You may reduce this violation to a warning. However, the generated patterns contain incorrect states for one of the paths in this context and might fail during the `run_simulation` command or application at test. For information on analyzing a violation in the graphical schematic viewer (GSV), see "Analyzing DRC Violations in the GSV."

---

## DRC Rule R49

### Message Text

Compressor Rule R49

Load compressor *decompressor\_name* is driven by scan-input *scan\_input* (*gate\_id*) with only dynamic bits. Re-run DFT with a less number of inputs at top-level.

### Severity

Error

### Description

This error occurs when the number of inputs required to drive a load compressor exceeds the available number of dedicated top-level inputs. When scan inputs are dedicated for a DFTMAX Ultra load compressor (also called a decompressor), each input consists of data and control flip-flops. Data bit values are stored inside the scan chains as part of the scan load. These bits are also referred to as dynamic bits because data streams through the bits to the scan chain when a pattern is loaded through the load compressor. Control bits are stored in a cache at the end of the pattern and are static since they do not change for a pattern. If a decompressor requires more inputs than are available, DFTMAX Ultra compression separates the data and control bits, and TestMAX ATPG issues an R49 error.

### What Next

Rerun DFTMAX Ultra compression using a decompressor that uses no more than the available number of top-level inputs.

---

## DRC Rule R50

### Message Text

Compressor Rule R50

Ignore: Incorrect SPC chain structure at *SPC\_serial\_reg\_instance* (*gate\_id*).

### Severity

Ignore

## Description

TestMAX ATPG expects a value of 0 in the shift power controller (SPC) register in order to turn off a group of chains. This rule violation indicates inversions leading up to the SPC control register. As a result, the register receives a value of 1 instead of 0.

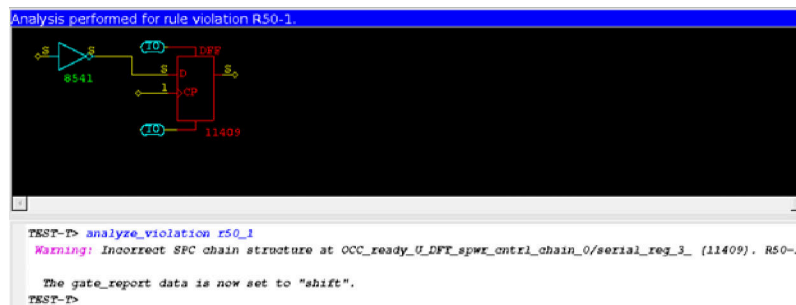
## What Next

This is an informational message only. If the violation is not corrected, you can run ATPG and generate patterns as normal.

## Example

```
TEST-T> report_violation r50_1
Ignore: Incorrect SPC chain structure at
OCC_ready_U_DFT_spwr_cntrl_chain_0/serial_reg_4_ (11408). R50-1
```

You can use the `analyze_violation` command to highlight the SPC register that violated the R50 rule, as shown in the following figure.



---

## DRC Rule R51

### Message Text

Compressor Rule R51

Error: Incorrect SPC update latch asynchronous control in  
`SPC_update_latch_instance (gate_id)`.

### Severity

Error

### Description

This rule checks the clock/set/reset inputs of the shift power controller (SPC) update latches. A violation indicates any of the following:

The set input is not off

The reset input is not off

The clock/enable input is not OFF during shift procedures

The clock/enable input is not ON during capture

### What Next

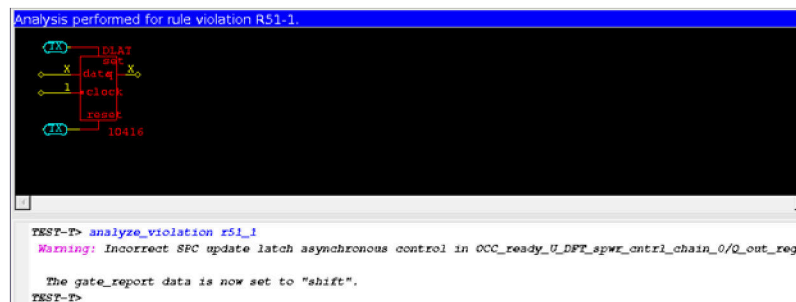
This rule can be downgraded to expedite debugging. If the violation is not corrected, you can run ATPG generate patterns as normal. However, there is a high probability that the resulting patterns will fail the Verilog resimulation.

Shift pindata is selected by the `analyze_violation` command so that violations can be debugged during the scan shifting operation (when the latch must hold state). However, it is possible that the actual violation is in capture mode (when the latch must accept new data). If the shift pindata looks correct, use the Setup dialog box to change the pindata to "Constrain Value". The latch enable must be constrained to the state that makes the latch transparent, and the latch data input must be shown as X, which means that ATPG can set it to any value.

### Example

```
TEST-T> report_violation r51_1
Error: Incorrect SPC update latch asynchronous control in
OCC_ready_U_DFT_spwr_cntrl_chain_0/Q_out_reg_2_ (10416). R51-1
```

Use the `analyze_violation` command to highlight the SPC update latch that is violating the rule, as shown in the following figure.



---

## DRC Rule R52

### Message Text

Compressor Rule R52

Ignore: Incorrect SPC load compressor propagation from SPC control register `SPC_serial_reg_instance (gate_id)` to `scan_cell_instance (gate_id)`.

## Severity

Ignore

## Description

This rule violation indicates that a value of 0 in the shift power controller (SPC) control register cannot propagate to the internal chain. The value might be blocked due to various reasons based on the gate types in the path from the SPC register to the internal chain. This violation implies that the SPC register is unable to turn off the chain.

## What Next

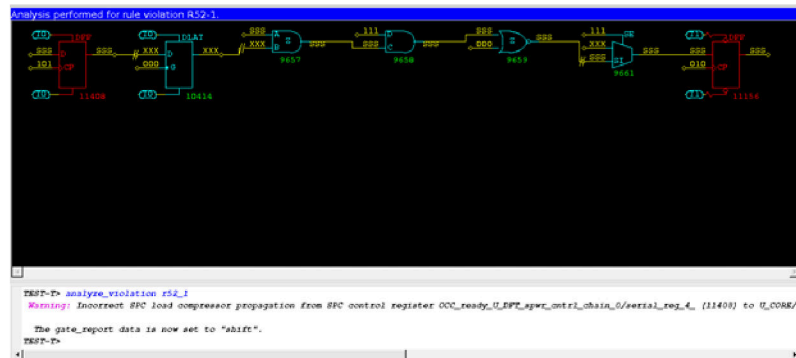
No action is required. ATPG takes the circuit structure into account when generating and formatting patterns.

## Example

```
TEST-T> report_violation r52_1
```

Ignore: Incorrect SPC load compressor propagation from SPC control register OCC\_ready\_U\_DFT\_spwr\_cntrl\_chain\_0/serial\_reg\_4\_ (11408) to U\_CORE/de\_encrypt/kdin\_reg\_20\_ (11156) R52-1

You can use the `analyze_violation` command to display the entire path from the SPC register to the first cell of the internal chain, that is violating the rule. The SPC register and the first cell of the internal chain will be highlighted, as shown in the following figure.





## Severity

Error

## Description

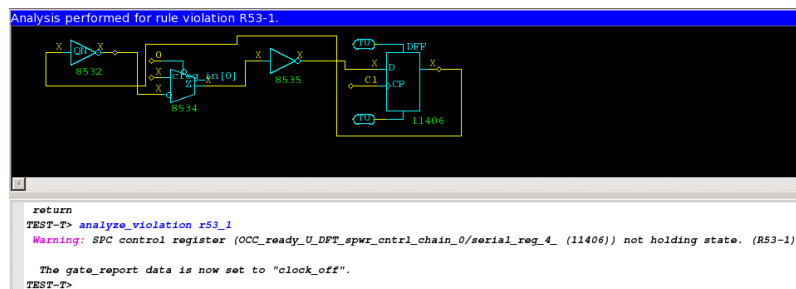
This rule can be downgraded to expedite debugging. If the violation is not corrected, you can run ATPG generate patterns as normal. However, there is a high probability the resulting patterns will fail Verilog resimulation. This rule indicates that the SPC register does not hold state during capture. This could be caused by inversions in the path that connects the SPC register output to its own input.

## What Next

You can use the `analyze_violation` command to display and highlight the SPC register that violates the rule.

## Example

```
DRC-T> report_violation r53
Error: SPC control register
(OCC_ready_U_DFT_spwr_cntrl_chain_0/serial_reg_4_ (11406)) not holding
state. (R53-1)
```



## DRC Rule R54

### Message Text

SPC control register is not capturing a constant value.

### Severity

Error

### Description

This rule violation applies to Serializer DFTMAX architectures when the shift power controller (SPC) cell architecture is modified for Serializer DFTMAX. The modification causes the update stage to become a delay flip-flop (DFF), and the control register captures a constant value (TIE0, TIE1 or scan enable). This modification mitigates pattern

inflation caused by having the SPC control chain as an internal chain with its output connected to the unload compressor.

A violation of this rule indicates that the SPC update stage is a DFF and the SPC control register is not capturing a constant value. Downgrading this error to warning can result in pattern inflation.

#### What Next

Modify your design so that the SPC registers capture a constant value in capture mode.

---

## DRC Rule R55

### Message Text

SPC chain is not in one to one mode

### Severity

Error

### Description

This rule applies to Serializer DFTMAX architectures when the shift power controller (SPC) control chain is an internal chain, with the output of this chain connected to the unload compressor. It is expected that the input of the chain is always in one to one mode. Downgrading this rule to warning can result in patterns that will fail simulation and on the tester.

### What Next

You must modify the design so the decompressor or compressor have the same head or tail pipeline stages.

---

## DRC Rule R56

### Message Text

Compressor Rule R56: [Decompressor | Compressor] *name* has different [head | tail] pipeline stages *pipeline\_length* and *seg\_pipeline\_length* for its [inputs | outputs].

### Severity

Error

### Description

Indicates that a particular decompressor or compressor (*name*) contains different head or tail pipeline stage lengths for its inputs or outputs. The pipeline length and segment pipeline length are indicated by the `pipeline_length` and `seg_pipeline_length` parameters.

For example, if a decompressor has two inputs, the head pipeline stages for both inputs should be the same, but different decompressors can have different head pipeline stage lengths. Compressors can also have different tail pipeline stage lengths.

### What Next

Modify your design so the decompressor or compressor has the same head or tail pipeline stage lengths.

---

## DRC Rule R57

### Message Text

Compressor Rule R57: Compressor Scan <Input | Output> *signal\_name*=#  
assignment is missing in the load\_unload procedure Shift Vector.

### Severity

Error

### Description

Indicates the parameters specified in the load\_unload procedure are incomplete and cannot support a call from the patterns. The output pattern data references the *signal\_name*. However, the pattern data cannot pass into the test without a matching reference to the indicated signal in the load\_unload procedure.

### What Next

Review the set of signals in the Shift block vector of the load\_unload procedure. Typically, these signals are referenced by the “\_si” or “\_so” group. In this case, a group is likely missing the indicated signal. If this signal is not intended to be included in the passed data, you can use the `set_rules` command to downgrade this rule violation to a warning to pass DRC.

---

## DRC Rule R58

### Message Text

Compressor Rule R58: ScanEnable name not constrained to LOGICAL LOW.

### Severity

Warning

### Description

If the ScanEnable `name` is not constrained to "LOGICAL LOW," serial simulation might fail.

### What Next

If this warning is unnecessary, you can remove it by adding the `add_pi_constraints` command to the script, as shown in the following example:

```
add_pi_constraints 0 scan_en
```

---

## DRC Rule R59

### Message Text

```
Failed to extract SPC register due to incorrect number of pipeline stages.
```

### Severity

Error

### Description

If the number of pipeline stages in the shift power controller (SPC) control chain is different than the number of internal chains, the SPC registers could be misidentified and incorrectly classified as pipeline and SPC control registers during scan chain extraction. This can lead to a crash or incorrect behavior during DRC.

### What Next

Make sure the number of pipeline stages in the SPC control chain matches the number of internal chains.

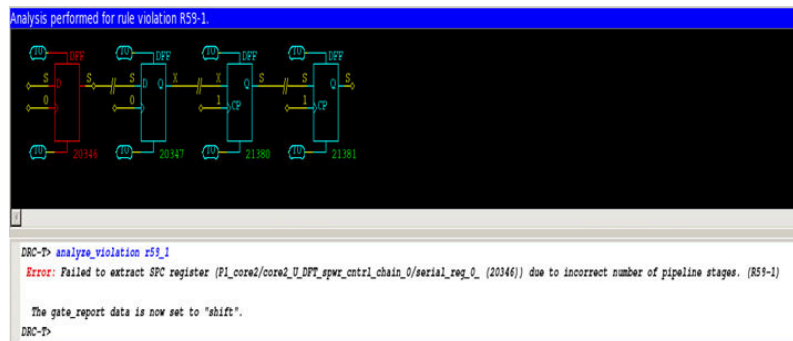
### GSV Support

Use the `analyze_violation` command to display and highlight the SPC register that violates the R59 rule.

### Example

The example below shows a design in which SPC register 20346 is followed by three tail pipeline registers: 20347, 21380 and 21381. Note that the clock input of 21380 is inverted with respect to 20347, which causes the scan chain extraction to consider the pair as master and slave. This creates a mismatch between the number of pipeline stages and

the number of internal chains. As a result, SPC register 20346 is classified as a pipeline register to meet the expectation of three tail pipeline registers.



## DRC Rule R60

### Message Text

Unload compressor output pipeline cell name (*cell\_ID*) on output\_signal does not hold during capture.

### Severity

Error

### Description

Output pipeline cells must be hold pipelines when the `set_drc -nouse_pipelines` command is applied. Hold pipelines must maintain their current state (they cannot update) during capture operations. You can define hold pipelines by suppressing the clock to the pipeline cell during the capture operation or by recirculating the state of the element during capture.

The R60 rule is only checked then the `-nouse_pipelines` option of the `set_drc` command is applied. Rule R60 is not a restriction for any other flow. While you can downgrade this rule, all violations will fail DRC when the `set_drc -nouse_pipelines` command is set.

### What Next

Modify the pipelines to behave as hold pipelines during the capture operations, or remove the `set_drc -nouse_pipelines` command to generate patterns when the pipeline behavior is not required to hold.

## DRC Rule R61

### Message Text

```
Invalid SPC register(cell_ID) connection to cell_name(cell_ID).
```

### Severity

Error

### Description

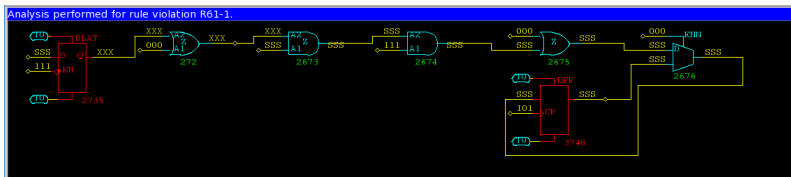
The purpose of Shift Power Controller (SPC) is to manage shift power by loading constant values directly from the SPC update registers into the scan chain inputs. Therefore, in shift mode, SPC update registers should only be connected to the first scan cell of a chain. This check is performed by tracing the SPC update registers forward in shift mode and generating an error any time the trace reaches a primary output or a register that is not the first scan cell of a chain.

### What Next

The `analyze_violation` command can be used to understand this violation. The schematic display will show the connection between the SPC registers and cell that is not part of the an internal chain with pin data set to be shift.

This rule can be downgraded to expedite debugging with `set_rule R61 warning`. If the violation is not corrected, you can run ATPG generate patterns as normal. However, there is a high probability the resulting patterns fail Verilog resimulation.

To prevent simulation failures it's recommended to fix the netlist.



## DRC Rule R62

### Message Text

```
Compressor Rule R62
```

```
Compressor register Cell-name (Cell-id) disturbed during pre-shift of  
load_unload.
```

### Severity

Error

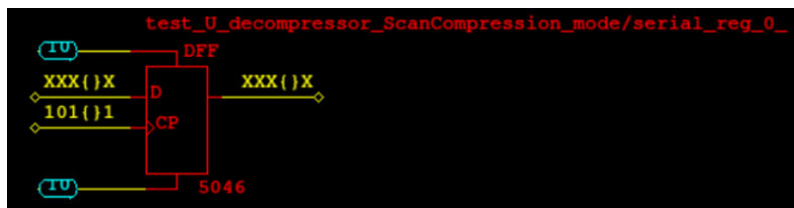
### Description

DFTMAX Ultra serial registers cannot be disturbed during the pre-shift or post-shift procedures. Any disturbance of these registers would cause wrong pattern generation that leads to simulation failures.

### What Next

The `analyze_violation` command can be used to understand this violation. The schematic displays the cell with the `R62` violation, pin-data is set to `load_unload`.

For example, in the following figure, it is evident that `serial_reg_0` has the clock pulsing in pre-shift. Clock pins of serial registers should not be pulsed (disturbed) during pre-shift. Try to resolve the error by preventing the disturb from happening by editing the protocol file or fixing the implementation. If this error is downgraded without fixing it would lead to simulation failures.



---

## DRC Rule R63

### Message Text

Compressor Rule R63: Two chains always have the same unload signature that was violated N times.

### Severity

Warning

### Description

This indicates that the `N` scan chain pairs will alias for a sub-set of its scan cells. It gives an early indication for potential deficiency in diagnosis.

### What Next

Alter the connectivity of the scan chain outputs for each pair of such scan chains. Increase the number of serializer flops or number of serializer segments to resolve this violation.

---

## DRC Rule R64

### Message Text

Compressor Rule R64: LE to TE transition from the load serial flop instance\_name to the first scan flop scan\_instance\_name of chain chain\_name may result in simulation mismatches.

Compressor Rule R64: LE to TE transition from the last scan flop scan\_instance\_name of chain chain\_name to the unload serial flop instance\_name may result in simulation mismatches.

### Severity

Warning

### Description

This violation can occur in both load and unload side.

When there is a transition from positive edge (LE) load decompressor serial register to a scan chain whose first scan flop is negative edge (TE), it creates a manager-worker condition. This results in missing the first shift cycle and cause simulation mismatches.

Similar issue can occur in unload side when there is a transition from LE last scan flop to TE unload side compressor serial register.

### What Next

Check if the re-synchronizer flop was disabled or scan segment bypass erroneously created negative edge head element.

---

## DRC Rule R65

### Message Text

Compressor Rule R65: Inversion in between serial reg reg\_instance\_name1 and reg\_instance\_name2 can caused simulation fail.

### Severity

Error

### Description

Inversion between two serial registers is not supported.

### What Next

Remove the inverter that causes the error.



# 9

## Category S - Scan Chain Rules

---

### See Also

- [S1 - scan chain blockage](#)
- [S2 - no scan cells in scan chain](#)
- [S3 - invalid scan chain input](#)
- [S4 - adjacent latches active at same time](#)
- [S5 - gate traced twice during chain trace](#)
- [S6 - invalid force scan input time](#)
- [S7 - invalid chain input propagation](#)
- [S8 - invalid measure scan output time](#)
- [S9 - missing master\\_observe procedure](#)
- [S10 - unused master\\_observe procedure](#)
- [S11 - unobservable master](#)
- [S12 - unused shadow\\_observe procedure](#)
- [S13 - mismatch of entered and actual #shifts](#)
- [S14 - excessive state elements in scan cell](#)
- [S15 - invalid scan cell constraint](#)
- [S16 - multiple constraints on a scan cell](#)
- [S17 - invalid single shift chain length](#)
- [S18 - scan cell disturb](#)
- [S19 - nonscan cell disturb](#)
- [S20 - invalid user #shifts](#)
- [S21 - too many apply single shifts](#)

- S22 - multiple clocked scan chain
- S23 - unobservable potential TLA
- S24 - clock connected to data input of potential TLA
- S25 - hot set/reset line on potential TLA
- S26 - no usable clock port for potential TLA
- S27 - clock\_po connection for potential TLA
- S28 - fail TE port connection for potential TLA
- S29 - invalid dependent slave operation
- S30 - unstable RAM during test procedure operation
- S32 - Scancell set to fixed binary value after load was violated
- S34 - Specified SPC chain *spc\_chain* is invalid. (S34-1)
- S35 - Specified chain C contains non-hold pipeline cells in position P (I).
- S36 - Scancell G shifts data on trailing edge.
- S37 - Scancell G captures data from shift path.

---

## DRC Rule S1

### Message Text

Scan Chain Rule S1: Chain C blocked at T gate I (G) after tracing N cells.

### Severity

Fatal Error

### Description

The scan chain path must be sensitized by the conditions that exist during the application of the shift procedure.

C is the name of the scan chain; T is the type of gate; I is the instance pathname of the gate; G is its corresponding gate ID; and N is the number of scan chain cells traversed before the blockage occurred. Counting starts with zero at the scan output port and increases as the analysis moves toward the scan input port.

This is a fatal error condition that must be corrected by changing the shift procedure or changing other procedures or settings that condition the shift procedure such as `test_setup`, PI constraints, or restricted clocks using `set_drc-clock`.

This check is performed by placing an "X" on the scan input and simulating the clocking of the shift procedure until all logic stabilizes. Then, using the event sequence of the simulated shift procedure, the scan output is traced backward through combinational and sequential gates that have a sensitized path (value=X) until the scan input is reached.

A rule violation occurs if the path cannot be determined or cannot pass through a sequential element.

### What Next

A violation can be analyzed using the GSV by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to the simulated shift procedure data and displays all gates in the backtrace cone of the blocked gate. By reviewing the shift data near the blockage point, you might be able to determine the cause of the problem.

If the scan blockage occurs after tracing 0 cells a common problem is that a tristate enable is off for tristateable or bidirectional scan outputs. For bidirectional ports used as scan outputs you should also explicitly force a Z on this port in the `load_unload` procedure.

Be aware that the TestMAX ATPG scan chain trace algorithm places an "X" value on the scan data path. In this case an "X" is a good thing! TestMAX ATPG works from the scan output pin back to the scan input pin. It follows the location of the "X" to help it figure out the scan data path. When the data path is a constant value of 1 or 0 instead of X this means there is some PI constraint or other constant value which is interfering with the scan data path by forcing the net to a constant 1 or 0.

Look for clock pins which are either X or are not being clocked. This might be because the proper port is not assigned a "port=P;" in the Shift procedure, the port is not defined as a clock, or because the clock path is blocked or uncertain as it passes through MUXes or other gates. If a clock passes through a MUX, the select line of the MUX must be constrained to a constant value either directly from a top level port, or indirectly from nonscan logic which is stable during ATPG.

Look for asynchronous set and reset pins which are either "X" or being pulsed. Either of these conditions will result in the scan blockage.

Look for data paths which are incorrect because the scan enable control is either at the wrong state, or connected with the wrong polarity.

When you run DRC, do you get a S1 violation that appears as if the clock is not clocking? Are you using the Quick STIL tab in the DRC dialog box to add clocks and scan chain/enable information to create a TestMAX ATPG STIL procedure file?

You can have used the add clock command or GSV to add your clocks, but could have forgotten to specify that clocks should be used as a SHIFT clock. In doing so, the generated STIL procedure file is a missing "clk"=P; statement in the shift procedure.

You must use the add clock -shift option to specify a clock that is used for shift on the command line; for example,

```
add_clocks 0 clk -shift -timing 100 50 80 40
```

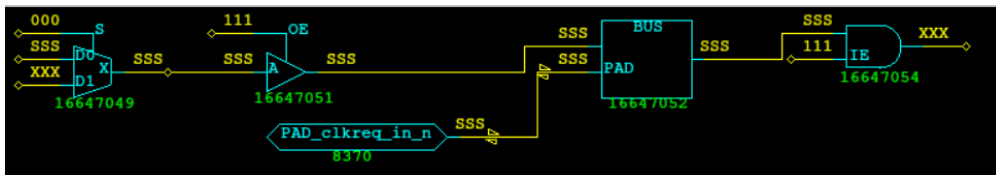
When using the GSV Edit Clocks dialog box, make sure to select the Shift checkbox to the right of the port name declaration. This port name you specify is used as a shift clock. Now the STIL procedure file will have the needed information and the DRC will pass.

A correct shift procedure could look similar to this:

```
Shift {
  V { _si=\r1 #; _so=\r1 #;
    "clk"=P; } # clk should be pulsed during shift
}
```

For S1 violation, the analyze\_violation command cannot give clear information for debugging in the following case.

One possible cause may be associated with the bus device, which is a gate inserted by TestMAX ATPG during ATPG design building to resolve multidriver nets. Both potential sources for the bus inputs appear to be driving, and both have values of X. One of the sources that has an X value is the PAD\_clkreq\_in\_n bidirectional port. You can correct this by driving the port to a Z state. Edit the STIL file to add the declaration PAD\_clkreq\_in\_n = Z to one of the V{..} vectors at the start of the load\_unload procedure.



## DRC Rule S2

### Message Text

Scan Chain Rule S2: No scan cells identified in scan chain C.

### Severity

Fatal Error

### Description

A scan chain must contain at least one scan cell.

### What Next

This is a fatal error condition that requires removing the scan chain from the scan chain list defined in the DRC file.

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the shift procedure simulated data and displays all gates in the path from the defined scan input to the output.

The `report_scan_chains` command can be used to list the scan chains along with their input and output ports.

---

## DRC Rule S3

### Message Text

Scan Chain Rule S3: Traced input P1 of chain C does not match entered value P2.

### Severity

Fatal Error

### Description

The traced scan chain input pin must be the same as the defined input pin of the chain.

P1 is the port which was found to be the scan input by the simulation of the shift procedure. P2 is the port defined in the DRC file to be the scan input, and C is the scan chain name.

### What Next

You can analyze a violation in the schematic viewer by selecting its violation ID number using the `ANALYZE` button. This changes the pin data reporting to the simulated shift procedure data and displays all gates from the traced scan in port to the adjacent scan cell. By reviewing this circuitry, you might be able to determine the cause of the problem.

If the correct scan-in port is the port identified by the analysis, edit the test protocol file and rerun DRC.

If the displayed port is not the correct scan input, then the port assignments in the `load_unload` procedure before the shift procedure should be reviewed and possibly changed to sensitize the path to the desired scan input instead of the one identified by TestMAX ATPG.

You can also analyze an S3 violation by using the `set_drc -chain_trace chain_name` command and rerunning DRC to trace the shift data propagation on the affected scan chain.

---

## DRC Rule S4

### Message Text

Scan Chain Rule S4: Adjacent latches I1 (G1) and I2 (G2) in chain C load data at same time.

### Severity

Warning

### Description

Adjacent latches in a scan chain path must not be active at the same time during the application of the shift procedure.

This check is performed during the tracing of the scan chain through a sensitized path using the simulated values of each time period of the shift procedure.

### What Next

You can use the graphical schematic viewer (GSV) to analyze this violation by selecting its violation ID number and clicking the ANALYZE button. This sets the gate reporting to the shift procedure simulated data and displays all gates between the two latches. In some situations, DRC will not report adjacent latches. However, you can use the `report_lockup_latches` command to list adjacent latches with a M832 message.

---

## DRC Rule S5

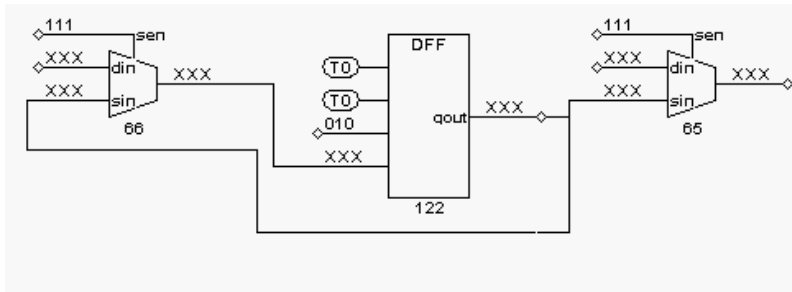
### Message Text

Scan Chain Rule S5: N (G) in chain C previously used in chain trace.

### Severity

Fatal Error

One example, many variations are possible:



## Example

### Description

A latch or flip-flop gate can only be traced once during the scan chain tracing process. The indicated cell was either traced twice for the same scan chain or is part of a previously traced scan chain and marked as part of the other scan chain already.

### What Next

This is a fatal error condition that requires either removing the scan chain from the chain list or changing the chain conditioning. You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the shift procedure simulated data and displays as a schematic the multiply traced gate.

## DRC Rule S6

### Message Text

Scan Chain Rule S6: Input force of chain C must be loaded by first clocking of N (G).

### Severity

Error

### Description

The forcing of the scan chain inputs in the shift procedure must occur so it can be loaded by the first clocking of the scan cell connected to the chain input.

### What Next

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the shift procedure simulated data and displays all gates from the traced scan-in pin to the adjacent scan cell.

Also, you can check the WaveformTable and make sure the bidirectional delay is 0.

---

## DRC Rule S7

### Message Text

Scan Chain Rule S7: Invalid propagation of input of chain C to scan cell N (G) .

### Severity

Error

### Description

The shift data path between the scan chain input and the first scan chain cell (state element) must be sensitized (able to propagate data) at the time of the shift clock. In addition, a scan chain output must be measured before the shift clock changes that value.

### What Next

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the shift procedure simulated data and displays all gates from the traced scan-in pin to the adjacent scan cell.

One commonly occurring cause of this violation is attempting to define an end-of-cycle protocol which does a pre-measure of "\_so=#" before the Shift procedure without defining a timing block that calls for a measure time on outputs that comes after the clock. Double check the defined timing to make sure the measure time for outputs occurs after clocks.

---

## DRC Rule S8

### Message Text

Scan Chain Rule S8: Scan cell I (G) of chain C can change before output measure time.

### Severity

Error

### Description

The measure of scan chain outputs by the Shift procedure must occur before the scan cell connected to the chain output is clocked again.

I is the instance pathname of the scan cell in violation, G is the corresponding gate\_ID number, and C is the name of the scan chain.



Failure to satisfy this rule results in the scan chain being unusable for ATPG.

### What Next

A common cause of this violation is an incorrectly defined clock shifting sequence in the Shift procedure. The event order of the Shift sequence is controlled at two levels: at a macro level it is controlled by the number of  $\forall\{ \dots \}$ , vector statements used in the Shift procedure. At the micro level the event order within each vector statement is controlled by the defined pin timing found in the Timing block. Both areas should be reviewed to check the event order of the Shift procedure.

Another possible cause of this violation is forgetting to have the scan out pre-measure required before the Shift procedure when the end-of-cycle measure protocol is used. When the measure times defined in the Timing definition of the DRC procedure file occur after clock pulses, then this implies the end-of-cycle protocol is to be used. For more information and examples of this protocol see the "Creating End-of-Cycle Measures in ATPG Patterns" topic.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to the simulated shift procedure data and displays gates between the scan output port and the scan cell with the problem. By reviewing the clocking patterns of the scan cell, you might be able to determine the problem.

---

## DRC Rule S9

### Message Text

Scan Chain Rule S9: Unobservable master (G) in chain C due to missing master\_observe procedure.

### Severity

Error

### Description

A master\_observe procedure must be defined if a MASTER cannot be observed by the load\_unload procedure.

G is the gate\_ID of the scan cell considered to be a MASTER, and c is the name of the scan chain.

### What Next

Try defining a master\_observe procedure in the STIL procedure files and rerunning DRC. Your goal is to capture or "observe" the value in the MASTER cell identified into some other scan chain cell by performing one or more clocking operations.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to the simulated shift procedure data and displays all gates in the chain path between the failing MASTER gate and the scan cell output gate.

---

## DRC Rule S10

### Message Text

Scan Chain Rule S10: No master gates identified using master\_observe procedure.

### Severity

Warning

### Description

A master\_observe procedure was defined but was not necessary. To be necessary at least one MASTER gate must be unobservable without the master\_observe procedure.

### What Next

This violation has no automated schematic viewing analysis. If all master gates are observable, the master\_observe procedure is redundant and you should remove it from the test protocol file. If some master gates were found to be unobservable (rule S11), analyze those violations. This might identify a problem with the content of the master\_observe procedure.

---

## DRC Rule S11

### Message Text

Scan Chain Rule S11: I (G) not successfully observed by master\_observe procedure.

### Severity

Error

### Description

A master\_observe procedure must successfully observe master gates that cannot be observed by the load-unload procedure.

I is the instance pathname of the scan cell in violation, and G is the corresponding gate\_ID number.

### What Next

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the master\_observe procedure simulated data and displays all gates in the chain path from the failing master gate to the scan cell output gate.

---

## DRC Rule S12

### Message Text

Scan Chain Rule S12: No observable shadows identified using shadow\_observe procedure.

### Severity

Warning

### Description

A shadow\_observe procedure was defined but no SHADOW devices were observed by it. A shadow\_observe procedure must observe at least one shadow gate or it is unneeded.

### What Next

This violation has no automated schematic viewing analysis. If there are no shadow gates, the shadow\_observe procedure is redundant and you should remove it from the test protocol file. If there are shadow gates that were intended to be observable, perform an analysis by displaying that gate with pin data selected to be "shadow\_observe". Navigate through circuitry searching for why the shadow gate doesn't propagate to the output of its associated scan cell. This can identify a problem with the content of the shadow\_observe procedure.

---

## DRC Rule S13

### Message Text

Scan Chain Rule S13: Traced number shifts (N1) for group G does not match entered value (N2).

### Severity

Ignored

### Description

The traced number of shifts must equal the entered number of shifts for a scan chain group. If a mismatch occurs, the traced number of shifts is used.

### What Next

This violation has no automated schematic viewing analysis. A violation only indicates that the calculated number of shifts to load the scan chain did not match the number of applied shifts entered in the test protocol file. Normally, this is not a problem and the calculated number of shifts is used.

---

## DRC Rule S14

### Message Text

Scan Chain Rule S14: Excessive scan cell states due to *cell\_type* *instance\_name* (*gate\_id*) in chain *scan\_chain*.

### Severity

Fatal Error

### Description

A scan cell cannot contain state elements other than a master (with an optional dependent slave), a slave (with an optional dependent slave), and any number of shadows.

The *cell\_type* is the cell type of the DFF or DLAT. The *instance\_name* is the instance pathname of the affected scan cell. The *gate\_id* is the corresponding gate ID, and the *scan\_chain* is the name of the scan chain.

### What Next

This is a fatal error condition that requires you to either remove the scan chain from the chain list, or change the chain shifting or conditioning, or modify the design.

You can analyze an S14 violation using schematic analysis by selecting its violation ID number. This sets the gate reporting to the shift procedure simulated data and displays the failing gate.

When this DRC violation occurs, all subsequent DRC checks are halted. You should specify the `report_rules-fail` command to list other DRC problems. You can obtain additional information for understanding a S14 violation by reviewing related DRC violations.

One common cause for this violation occurs when there are two or more latches in a series separated by combinational gates, and the controlling gates for each latch are simultaneously active. This is a timing hazard. In this case, you should use the ANALYZE button to select the S14 violation, then start your analysis on the gate farthest away from the input and analyze backward in the logic cloud for other latches.

Since an S14 violation is a fatal error, only the first violation is reported. If more than one S14 violation exists in a design, the debugging process can be intensive. If you suspect

there are several S14 violations, specify the `set_rules S14 -verbose` command before the `run_drc` command. This will print all the S14 violations. You can then analyze these violations using the `analyze_violation` command.

---

## DRC Rule S15

### Message Text

Scan Chain Rule S15: Invalid cell constraint defined for N.

### Severity

Fatal Error

### Description

You can assign a cell constraint only to a valid scan cell.

### What Next

This is a fatal error that requires you to either remove or correct the cell constraint.

To obtain a list of legal scan cell names:

Remove all cell constraints so the `run_drc` command processes completely.

Use the `report_scan_cells` command to list either the scan cells for a specific scan chain or all scan chains.

An ambiguous instance pathname usually occurs when the library cell contains more than one sequential element (such as an LSSD scan style), or when multiple scan cells are defined under a single module with a ``celldefine` construct. In this case, you should specify the gate ID of the scan cell on which to place the cell constraint, or specify the scan chain name and scan cell bit position. The gate ID can change if the design changes or if a flattened design environment changes (such as a gate optimization change).

Other options to eliminate the S15 in this context include:

Remove the ``celldefine` construct for the module.

Do not use the `-lib` option of the `read_netlist` command to maintain the internal scan cell names in the module.

If the violation message identifies a gate ID, use the `analyze_violation` command to graphically display the violation. This command sets the gate reporting to "none" and displays the invalid gate. If the cell constraint is applied to a nonscan device, you can also use the `add_capture_masks` command.

---

## DRC Rule S16

### Message Text

Scan Chain Rule S16: Multiple cell constraints defined for N.

### Severity

Fatal Error

### Description

No more than one cell constraint can be placed on the same scan cell.

This is a fatal error condition that requires either removing or correcting the cell constraint.

### What Next

Enter the following report violations command to obtain details of all s16 violations:

```
TEST> report_violations s16
```

A violation due to a cell constraint identified by an invalid gate can be analyzed using the schematic analysis by selecting its violation ID number. This sets the gate reporting to "none" and displays the invalid gate.

---

## DRC Rule S17

### Message Text

Scan Chain Rule S17: Chain C length (L1) using single shift not equal to chain length (L2).

### Severity

Fatal Error

### Description

The chain length calculated for a trace of a single shift application must be the same length as that calculated for the general shift application.

### What Next

This is a fatal error condition that requires either removing the scan chain from the chain list or changing the chain conditioning in the shift or load\_unload procedures.

---

## DRC Rule S18

### Message Text

Scan Chain Rule S18: Scan C I (G) disturbed during time T of P procedure.

### Severity

Error

### Description

The values loaded into scan cells by the `load_unload` or shift procedure, or captured in scan cells by the capture procedures, must not be disturbed by the other procedures.

C is the cell type of DFF or DLAT, I is the instance path-name of the affected scan cell, G is its corresponding `gate_ID`, P is the name of the procedure, and T is the time within the procedure when the cell was disturbed.

Failure to satisfy this rule can result in inaccurate simulation results and the generation of ATPG patterns which fail in simulation, and also unreliable coverage results and fault classifications.

This check is performed by using the simulated values of each time period of the test procedures. The rule violation occurs if any clock input of any scan cell state element allows data to be captured at an inappropriate time.

### What Next

A common cause of this problem is often an inappropriately applied clock pulse or active asynchronous set/reset within a procedure.

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the failing procedure pattern data and displays all gates in the back trace cone of the disturbed gate. You might need to toggle between Load pin data and Shift pin data to see the whole load operation, in case an extra clock edge is caused by a change in the clock-off state.

It is common for this violation to occur on designs using JTAG/TAP controllers. Particularly in the area of the TDO output register, as it tends to be clocked on the trailing edge of TCK while the rest of the scan chain is clocked on the rising edge.

Sometimes the source of the violation is at a sequential cell identified as a shadow register to a scan cell. If this is the case you can see some benefit in using the `-serial_shadows_only` or `-noshadows` option of the `set_drc` command to change shadow register recognition. You need to rerun DRC after making this change.

Use the `-mask` option of the `set_rules` command to compensate for the DRC violation. If you set the rule severity to warning and enable the `-mask` option, the ATPG algorithm

masks off the control and observe values to ensure that any ATPG pattern generated passes simulation. The tradeoff in doing this is that there is a reduction of test coverage.

Sometimes a clock chain for an on-chip clock controller has S18 violations, especially if the ATE clock is used as the clock for the clock chain. These cells must be masked using the command `add_cell_constraints 0X` to allow them to control the on-chip clock controller without their final value being strobed. This does not result in any coverage loss because the clock chain does not capture new data during capture cycles.

## DRC Rule S19

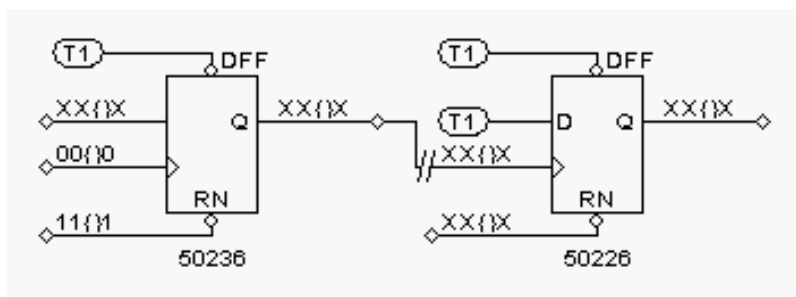
### Message Text

Scan Chain Rule S19: Nonscan Cell I (G) disturbed at time T of P procedure.

### Severity

Warning

One example, many variations are possible:



## Example

### Description

Nonscan cells must hold their state during the application of the load\_unload procedure.

C is the cell type of DFF or DLAT, I is the instance pathname of the affected scan cell, G is its corresponding gate\_ID, P is the name of the procedure, and T is the time within the procedure when the nonscan cell was disturbed.

Passing this rule allows the fault simulator and test generator to use nonscan cell values that exist before loading and unloading scan chains to advantage and can increase test coverage. Failing this rule means the nonscan cells are treated as containing X values, and can't be used to advantage when creating ATPG patterns. There is no danger of generating patterns which fail simulation.



This check is performed using the simulated values of each time period of the load\_unload test procedure. A rule violation occurs if any nonscan cell clock/set/reset line becomes potentially active. For a given nonscan cell, only the first occurrence of a violation is reported.

### What Next

It is quite common for designs which contain both scan and nonscan sequential cells to have the nonscan cells disturbed. This DRC violation should only be of concern if you had expected the nonscan cells to be stable by design. If this is the case, then further investigation should be performed.

One common cause of this S19 violation is the DFF devices which form the output registers on synchronous RAMs. As TestMAX ATPG creates a derived ATPG model for the RAM it inserts DFF devices for the RAM data output ports. These devices are nonscan and can trigger the S19 violation.

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the failing procedure pattern data and displays all gates in the back trace cone of the disturbed gate.

It's important to note that the loadable nonscan cells feature is limited in its effectiveness. It will not be of much benefit if the load\_unstable cells form a deep sequential circuit, have sequential loops, or are fed by X-generators. Since this feature has runtime penalties in DRC, ATPG, and simulation, you should make sure you are getting a net benefit when using it. You can check its effectiveness by comparing the number of S19 violations to the following line of `run_drc` output (which is only printed when the `set_drc -load_nonscan_cells` command is specified):

```
Nonscan load cell identification completed: #load_nonscan_cells=6  
(nonx=4)
```

Note that the loadable nonscan cells feature is beneficial if the `#load_nonscan_cells` value is large and/or this value comprises a large percentage of S19s violations.

For more information, see the following:

Using Loadable Nonscan Cells in TestMAX ATPG

```
set_drc -load_nonscan_cells
```

```
set_simulation -shift_cycles
```

---

## DRC Rule S20

### Message Text

Scan Chain Rule S20: Selected number of shifts L1 is less than chain C length L2.

### Severity

Error

### Description

The user-selected number of shifts for a group of scan chains must not be less than the length of the longest scan chain of the group. If the severity is not error, the number of shifts is set to the length of the largest scan chain.

### What Next

This violation has no automated schematic viewing analysis. A violation indicates that the calculated number of shifts to load the scan chain was less than the user selected number of shifts. A scan chain must be capable of being fully loaded during a load operation, and requires at least a number of shifts equal to the length of the longest scan chain. Either change the selected number of shifts to meet this requirement or change the severity to warning or ignore, which results in using the calculated number of shifts.

---

## DRC Rule S21

### Message Text

Scan Chain Rule S21: Number of single shifts (L1) is not less than chain C length (L2).

### Severity

Fatal Error

### Description

For a chain group, the number of single shift applications must be less than the length of the longest scan chain.

### What Next

This is a fatal error condition that requires the removal of some single shift applications by adjusting procedures.

## DRC Rule S22

### Message Text

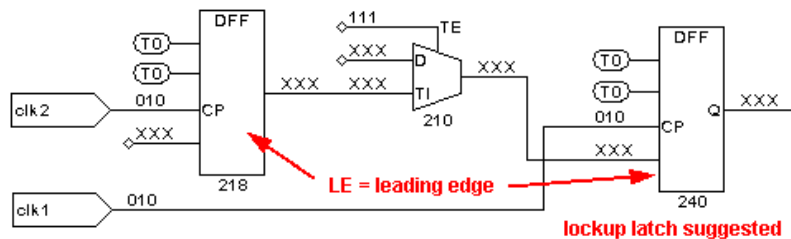
Scan Chain Rule S22: Multiple clocks (*S2C2*) were used to shift scan chain *chain\_name*.

### Severity

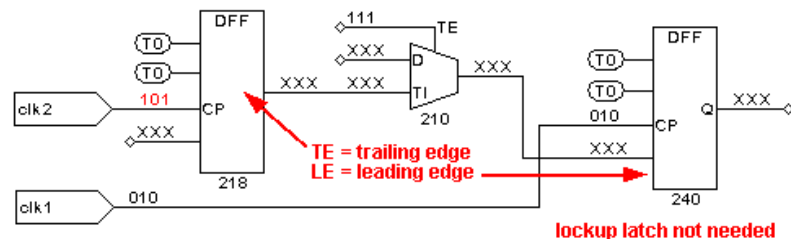
Warning

### Examples

First example, LE to LE devices, LOCKUP latch suggested:



Second example, TE to LE devices, ignore S22 warning:



### Description

All master gates of scan chain cells in the same scan chain should use a single shift clock. This rule is a safeguard to try to avoid a situation where chip tester clock skew results in tester pattern failures during the shifting of the scan chains. If a single clock is used then no clock skew due to the tester can occur. If more than one clock is used then any skew can result in the rejection at the tester of potentially good devices.

*S2* and *C2* are the clock port names involved, and *chain\_name* is the symbolic name of the scan chain as defined in the DRC procedure file used with the *run\_drc* command.

This check is performed by inspecting the clocks sources of the state elements for all scan cells of a scan chain without regard to the polarity of the clocks or the phase relationship between them.

A rule violation occurs when a scan chain uses more than one clock to shift data. A potential problem can exist, but you will need to further investigate the affected clock polarities. Only the first occurrence of an S22 violation on each scan chain is identified, so investigating one violation does not mean there are not others on the same scan chain. You can specify the `set_rules s22 -verbose` command to print all instances and clocks of the violating transition.

Note that the S22 message is not reported on clock domain crossings that are identified with a DSLAVE lockup latch. This is only reported on clock domain crossings that have direct paths between two scan Master flops. If this checking is not sufficient, you can use the `report_lockup_latches` command.

### What Next

You can use the `report_violationss22` command to list the name of the scan chain and the clocks that have been identified as shifting the scan chain.

A violation can also be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to "shift" and displays two scan elements from the same scan chain along with their different clock sources.

When the clocks of the two scan chain elements involved are of the same phase and polarity (both devices are LE) you should consider adding a lockup latch between the two scan cells, using the inverted clock of the first element to enable the latch. This will protect the scan chain from shift failures due to tester skew of the clocks involved.

When the clocks of the two scan chain elements are of opposite phase so that a TE to LE condition exists, you might investigate whether the clock of the first device might be defined with the opposite polarity. This can often have a beneficial effect on test coverage. If this is not possible, then it is generally safe to ignore the S22 violation for this specific clocking arrangement.

If the clocks can be safely pulsed at the same time for both scan chain shifting and normal operation, you might want to declare the clocks as equivalent using the `add_pi_equivalences` command. This will result in both clocks always being operated simultaneously and will eliminate the S22 violation due to these clocks. It can also have a benefit in reducing total pattern count. However, before declaring two or more clocks as being equivalent the clocks should have identical timing and you should be sure that these clocks can be safely operated together under normal design operation as well as scan data shifting.

See Also `report_lockup_latches` in the TestMAX ATPG and TestMAX Diagnosis Commands Guide

---

## DRC Rule S23

### Message Text

Scan Chain Rule S23: Latch I (G) not transparent due to not observable.

### Severity

Warning

### Description

A potential transparent latch gate must have a path to a primary output, a scan cell, another transparent latch, or a sequential-scan cell to be observable.

I is the instance pathname of the DLAT primitive and G is its corresponding gate\_ID.

Failure to satisfy this rule results in the data through the latch not being observable. Consequently fault sites involved with this path cannot be detected and a reduction of test coverage can occur. There is no danger of generating ATPG patterns which fail in simulation.

### What Next

If a significant number of faults associated with the latch are untestable, it might be worthwhile to add an internal scan cell to observe the value coming out of the latch.

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the clocks-off simulated data and displays the forward trace cone of the failing gate.

---

## DRC Rule S24

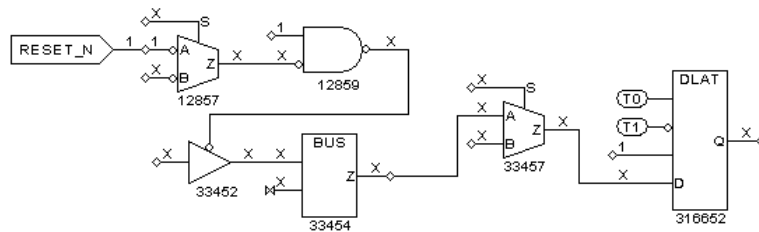
### Message Text

Scan Chain Rule S24: Latch I (G) not transparent due to data input (N) connected to clock.

### Severity

Warning

One example, many variations are possible:



## Example

### Description

A potential transparent latch gate must not have a path from any of its data inputs to a clock. Failure to satisfy this rule prevents the latch from being used as a transparent latch and can lead to some reduction of test coverage.

$\mathbb{I}$  is the instance pathname of the DLAT primitive,  $G$  is its corresponding gate\_ID, and  $N$  is the DLAT primitive input pin number (0=set, 1=reset, 2=clock, 3=data, and so forth.).

### What Next

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the clocks-off simulated data and displays the backward trace cone of the failing gate to clock pins.

## DRC Rule S25

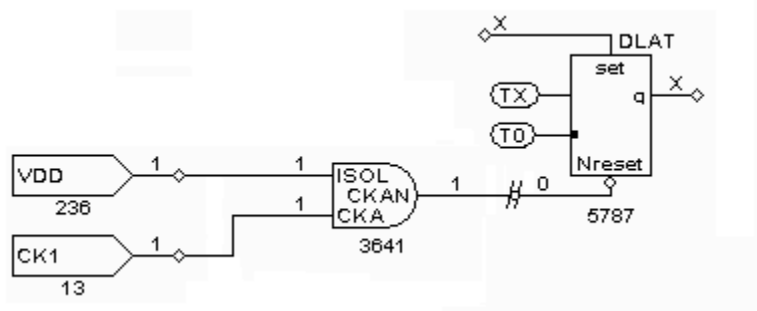
### Message Text

Scan Chain Rule S25: Latch N (G) not transparent due to hot SET/RESET line.

### Severity

Warning

One example, many variations are possible:



## Example

### Description

A potential transparent latch gate must not have an active set or reset input when all defined clocks are off.

Failure to satisfy this rule keeps the latch from being used in a transparent mode.

A violation of this rule introduces no danger of generating bad patterns. The latch cannot be made transparent and is treated as a constant X value. This in turn, can cause a reduction of test coverage.

### What Next

If the number of untestable faults associated with the latch is significant, you might want to look at design changes to eliminate the active set/reset during ATPG test mode.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to "clock off" simulated data and displays the backward trace cones of the set/reset input of the gate with the violation.

## DRC Rule S26

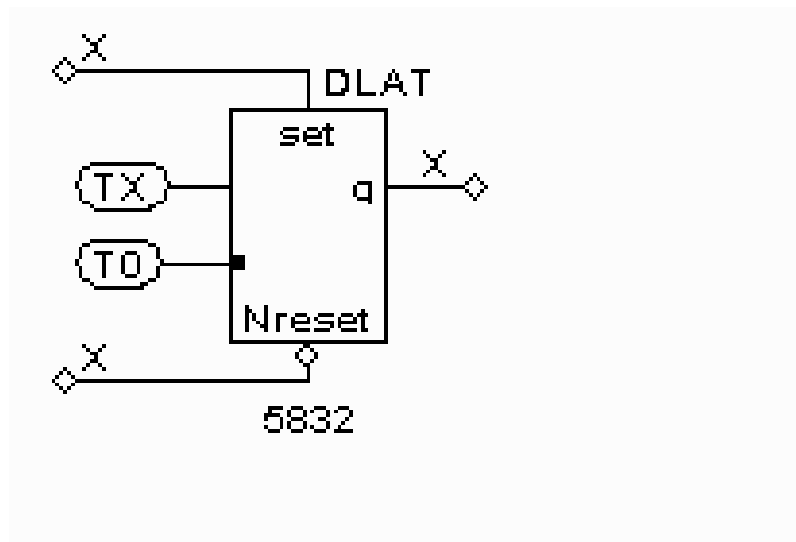
### Message Text

Scan Chain Rule S26: Latch N (G) not transparent due to no usable clock ports.

### Severity

Warning

One example, many variations are possible:



## Example

### Description

A latch gate must have at least one clock port that can be made active when all defined clocks are off to be considered a transparent latch.

Failure to satisfy this rule keeps the latch from being used in a transparent mode.

A violation of this rule introduces no danger of generating bad patterns. The latch cannot be made transparent and is treated as a constant X value. This in turn, can cause a reduction of test coverage.

### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to "clock off" simulated data and displays the backward trace cones of all clock inputs of the gate with the violation.

You should also investigate the derived ATPG gate level model for the latch by selecting the latch in question in the graphical view and picking "Go Down Hierarchy" from the RMB menu. Sometimes the functional model for a latch produces a derived model where the SET and RESET of the DLAT primitive are used but the clock and data inputs are not. If you find this to be the case but your original vendor library module did not intend this functionality then you might need to create a replacement ATPG model to use.



## DRC Rule S27

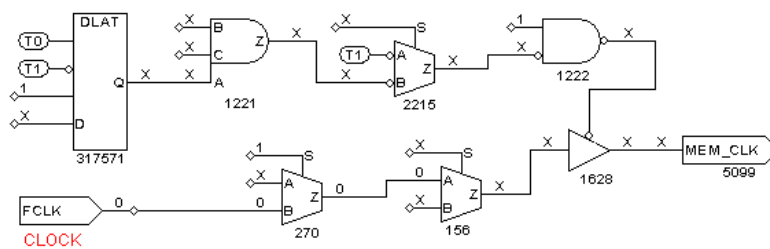
### Message Text

Scan Chain Rule S27: Latch N (G) not transparent due to connection to clock\_po (P).

### Severity

Ignored

One example, many variations are possible:



### Example

#### Description

A potential transparent latch gate must not have a path to a clock primary output (PO). A clock PO output is one which has a combinational path from a defined clock to this output. Faults along this path which require the clock to be asserted for detection require a special type of pattern not supported by default in TestMAX ATPG. When the potential transparent latch has a path to a clock PO output it can't effectively work as a transparent latch when the clock is off.

A violation of this rule introduces no danger of generating bad patterns but is a warning that a slight reduction of test coverage can result.

#### What Next

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the **ANALYZE** button. This changes the pin data reporting to "none" and displays the gates in the path between the failing gate and a clock\_po output port.



## Severity

Warning

## Description

A dependent slave device must always carry the same value as its master device. I1 and I2 are instance pathnames. G1 and G2 are the corresponding gate ID numbers.

If this rule is violated, the dependent slave device might have a value different than its master device either at the end of the shift procedure or after a capture clock. This might lead to the generation of ATPG patterns that fail during simulation. However, TestMAX ATPG takes steps to avoid a potentially bad patterns as described in the What Next section.

TestMAX ATPG performs this check using shift data. It considers the first device to be loaded during shift as the master and the second device loaded during shift as the dependent slave.

## What Next

You can continue by changing the severity of this rule to either warning or ignore using the `set_rules` command and then repeating the `run_drc` command. The ATPG algorithm marks the affected master or slave device as an observe X device to avoid generating patterns that fail in simulation. However, this observe X condition might result in lower test coverage.

To analyze a violation using the schematic viewer, select its violation ID number from the ANALYZE button. This changes the pin data reporting to shift data and displays the gates in the path from the master device to the dependent slave device.

The following are common causes of the S29 scan chain rule violation:

A dependent slave device does not have exactly the same asynchronous set and reset controls as its master device. If the dependent slave has different values than its master, erroneous simulation data is created.

You are using the wrong polarity for clocks. TestMAX ATPG classifies a device as master or dependent slave (DSLAVE) based on which device captures first during a shift procedure. If you define clocks with the wrong polarity, TestMAX ATPG classifies a DSLAVE device as a MASTER device and associates it with a downstream sequential device instead of the upstream device. You should reverse the polarity of all clocks associated with the scan chain on which the S29 violation occurs.

A dependent slave device has a different clock than its master device with the opposite off state (phase). When this occurs, you might be able to redefine the polarity of a single clock for one of the devices (usually the dependent slave). When both devices use the same phase clock you eliminate the dependent slave/master relationship and have two masters instead. Your scan chain length increases and S22 violations (multiple clocked

scan chain) occur. An S22 violation can also result in patterns that fail simulation, unless you use techniques such as lockup latches to prevent failures due to clock skew.

You can use the `-mask` option of the `set_rules` command to compensate for the DRC violation. If you set the severity to Warning and enable the `-mask` option, the ATPG algorithm masks the control or observe functions (or both) to ensure that any ATPG pattern generated passes simulation. However, it reduces test coverage.

---

## DRC Rule S30

### Message Text

Scan Chain Rule S30: RAM I (G) unstable during time T of P procedure.

### Severity

Warning

### Description

A RAM must hold state during all test procedure operations, except for the test setup procedure, if the RAM is to be used in read-only mode. "I" is the instance name of the RAM, "G" is its corresponding gate ID, "T" is an event time in test procedure "P".

This violation is detected during the simulation of the test procedure file. It indicates one of the write port controls or perhaps an optional asynchronous set or reset is not at an off state.

A violation of this rule introduces no danger of generating bad patterns but is a warning that a reduction of test coverage can result. The RAM cannot be used in read-only mode and its contents is set to all X's. This cannot be important if the RAM had no initialization file, the initialization file contained all X's, or the RAM is still controllable during Fast-Sequential pattern generation.

### What Next

In general, it is better to have a RAM for which Fast-Sequential patterns can be generated than it is to have a RAM that operates in read-only mode (behaves like a ROM). If your RAM can be used during Fast-Sequential pattern generation, then you can change this rule to warning or ignore and continue.

The `report_memory` and `report_primitives` commands can be used to review whether the RAM is considered controllable during Fast-Sequential ATPG. Check that the RAM is has a stable clock and is stable during scan chain loads. If the RAM is considered usable during Fast-Sequential ATPG pattern generation, then test coverage cannot suffer at all, and in fact might be greater than treating the RAM as a read-only device.

This violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "pattern data" and displays the gates in the backward trace cone of the RAM's write, set, or reset control input for which the violation was reported.

Based upon interactive investigation, the solution to eliminating the violation can involve changing one of the test procedures such as the `load_unload`, or `shift`, or changing logic, or perhaps finding a control input that can be constrained to avoid the RAM instability.

If no changes to the design or to test procedures are desired you can continue by using the `set_rules` command to change the severity of the rule to warning or ignore and the repeating the `run_drc` command.

---

## DRC Rule S32

### Message Text

Scan Chain Rule S32: Scancell set to fixed binary value after load was violated *number* times

### Severity

Error

### Description

PLL shift clocks are internal nodes used for shift. These clocks might need to be defined when scan cells are set to binary values during various test procedures. This might result in incorrect identification of load0, load1 cells, which might lead to a loss of coverage.

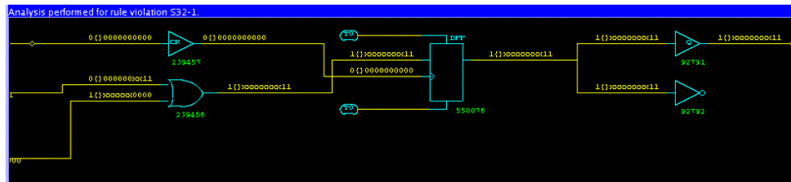
The S32 rule checks for unconstrained scan cells identified as load0, load1. If DRC passes without defined PLL shift clocks, it implies that the definition of PLL shift clocks is not necessary.

When analyzing an S32 violation, both the value at the end of `load_unload` and the constrain value must be inspected for the failing scan cell. An S32 failure will occur if the value at the end of load is non-X (that is, 0 or 1), and the cause can be traced back in the GSV with pin data set to `load_unload`. An S32 failure can also occur when the value at the end of load is X — if the constrain value is non-X. For example, a D flip-flop (DFF) might have its reset line constrained active, so the loaded value is immediately replaced at the beginning of the capture cycle. In this case, you should set the pin data to constrain the value to trace the source of the problem in the GSV.

The `load_unload` pindata might show a non-X value at the end of load even though no clock pulse is displayed. When internal clocks are used, this situation can be caused by a sensitized path from a PLL clock source. The PLL clock pulses are simulated, but the GSV does not display them. To verify this, you can identify the clock used for the

DFF using the `report_scan_cells -vbose` command, then use the `report_clocks -intclock` command to find the corresponding PLL source. The `analyze_violation s32-1` command displays the DFF with the load pindata. You can then view the PLL source separately, and use the GSV to see how they're connected and determine if the path is sensitized.

The following figure shows an example of this situation:



## What Next

S32 violations must be analyzed and fixed. If you downgrade an S32 violation to pass DRC, a loss of test coverage might result, including increased test data and time. This might also result in generation of ATPG patterns that fail VCS simulation.

If the S32 violation is caused by PLL clocks, then removing all PLL clock and PLLSource definitions will prevent the undesired pulses. TestMAX ATPG DRC only requires these constructs for OCC controllers that require PLL pulses to enter shift mode. For other types of OCC controllers, it is safe to remove them.

## DRC Rule S34

### Message Text

Scan Chain Rule S34: Specified SPC chain *spc\_chain* is invalid. (S34-1)

### Severity

Error

### Description

The chain name specified by the `set drc -spc_chain chain_name` command does not exist in the SPF.

The chain name specified by the `set_drc -spc_chain chain_name` command exists in the SPF, but it is not an SPC chain.

The design is using the DFTMAX Ultra and the SPC bit is disabled to bypass the SPC extraction, but `set_drc -spc_chain chain_name` was still set before `set_drc`.

### What Next

A violation of this rule is a fatal error and its severity cannot be downgraded. Use the `set_drc -spc_chain chain_name` command to specify the correct SPC chain or a chain that exists in the SPF and as an SPC chain.

If using DFTMAX Ultra and the SPC bit/port is being disabled, then do not specify the `set_drc -spc_chain chain_name` before running DRC.

---

## DRC Rule S35

### Message Text

Specified chain C contains non-hold pipeline cells in position P (I).

### Severity

Error

### Description

The pipeline cell must hold state during capture for optimal results in pipeline independent flow.

C is the name of the scan chain; P is the type of gate; I is the instance pathname of the gate;

---

## DRC Rule S36

### Message Text

Scan Chain Rule S34: Scancell G shifts data on trailing edge. G indicates the gate ID of the failing scancell.

### Severity

Error

### Description

When the *CAREShadow* shifts data on the leading edge (Message during DRC: *CAREShadows* is treated as capturing the leading edge). Scancells(closer to *CAREShadow*) are checked to ensure that they shift the data on the leading edge to make sure that *CAREShadow* and scancells do not shift the data in the same cycle during the `SHIFT_PROC` procedure.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that failing scancells shift the data on the leading edge. Use the command `set_pindata -shift` can be used to check on which edge CAREShadow/scancells shifts the data. This rule violation may be set to warning but it is not recommended since this could cause patterns to be incorrect.

---

## DRC Rule S37

### Message Text

Scan Chain Rule S37: Scancell G captures data from shift path. G indicates the gate ID of the failing scancell.

### Severity

Warning

### Description

Scancells closer to *CAREShadow* are checked if they capture data from the shift path. When scancell captures data from shift path, `noseq_comp_first_reseed_start` switch of the `set_patterns` command cannot be used. Using `noseq_comp_first_reseed_start` command may cause patterns to fail simulation as it changes the capture value of the scancell.

### What Next

To correct this problem, the netlist or the STIL protocol file must be changed so that failing scancell does not capture the data from `shift_path`.

The command `set_pindata -shift` can be used to check if the scancell can capture data from shift path.

---

## DRC Rule S38

### Message Text

Scan Chain Rule S38: Non-state holding pipeline cell C I (G) disturbed during time T of P procedure.

### Severity

Warning



### Description

The S38 violation message is triggered exclusively for non-state holding pipeline cells operating in an uncompressed mode. In such instances, the spf may contain the keyword `PipelineStateHolding 'False'`.

### What Next

Mask the non-state holding pipeline cells to facilitate VCS passage. The masking process occurs automatically if the spf contains the keyword `PipelineStateHolding 'False'`.

# 10

## Category V - Vector Rules

---

### See Also

- [V1 - Parsing error](#)
- [V2 - Redefined item](#)
- [V3 - Superfluous block](#)
- [V4 - Missing definition](#)
- [V5 - Unresolved reference](#)
- [V6 - Unused item](#)
- [V7 - Unsupported construct](#)
- [V8 - Illegal context](#)
- [V9 - Invalid time](#)
- [V10 - Miscounted items](#)
- [V11 - Repeated entry](#)
- [V12 - Unexpected item](#)
- [V13 - Invalid state](#)
- [V14 - Missing state](#)
- [V15 - Illegal write patterns](#)
- [V16 - Miscounted arguments](#)
- [V17 - Clock forced on](#)
- [V18 - Mixed timing aggregate](#)
- [V19 - Missing clock definition](#)
- [V20 - Unsupported event](#)
- [V21 - Unsupported old syntax](#)

- [V22 - Invalid waveform](#)
- [V23 - Strobe conflict](#)
- [V24 - Unsupported pattern event](#)
- [V25 - Unload mode enable not defined](#)
- [V26 - Unload mode enable port incorrectly set](#)
- [V27 - Unload mode control port incorrectly set](#)
- [V28 - Pipelining not supported](#)
- [V29 - Too many events or too small a resolution in procedure](#)
- [V31 - Waveform simulation events](#)
- [V32 - Incorrect event sequence](#)
- [V33 - Missing constrained signal](#)
- [V34 - Invalid internal clock timing](#)
- [V35 - Invalid clock constraint](#)
- [V36 - Failed to map external chain](#)
- [V37 - External chain used default mapping to a codec](#)
- [V39 - Line L \(filename\), missing state in load\\_unload](#)

---

## DRC Rule V1

### Message Text

Vector Rule V1: Line L (filename), parsing error (<additional info>).

### Severity

Fatal Error

### Description

Parsing rules were not adhered to in the STIL procedure file. The problem was first identified on line `L` but issues might have occurred before this line. Note that any missing quotation marks (reported with the message `expected ""`) might not be identified until the end of the file. To find the origination of the syntax error, you might need to search the entire text of the file before the reported location.

### What Next

Identify the location of the error, correct it, and try again.

---

## DRC Rule V2

### Message Text

Vector Rule V2: Line L (filename), redefined item (<additional info>).

### Severity

Error

### Description

Redefined items would have replaced the original definition.

### What Next

Remove the duplicate definition and try again.

One possible condition which creates this violation is a change to the defined clock list and/or the PI equivalences list. Suppose you had three clocks A, B, and C and created a DRC file which had three corresponding capture procedures "capture\_A", "capture\_B", and "capture\_C". Later you decide that you can declare these clocks as equivalent using the `add_pi_equivalences` command. Your next attempt at running DRC checks will produce a V2 violation with a message similar to:

```
redefined item (Procedure "capture_A" redefined)
```

The solution to this problem is to adjust the DRC file to contain only a single "capture\_A" procedure which pulses all three clocks. Eliminate the other two capture procedures.

You can change the severity of this rule to a warning or ignore using the `set_rules` command but it is highly recommended that you attempt to eliminate the violation by identifying the cause and adjusting the DRC file so that the violation is eliminated.

---

## DRC Rule V3

### Message Text

Vector Rule V3: Line L (filename), superfluous block (<additional info>).

### Severity

Error

### Description

You must not redefine clocks that are already correctly defined and linked. The superfluous definition is ignored.

### What Next

Remove the duplicate definition and try again or change the severity of this rule to warning or ignore using `set_rules` and try again.

---

## DRC Rule V4

### Message Text

Vector Rule V4: missing definition (*additional information*).

### Severity

Fatal Error

### Description

A required item is missing and must be defined.

### What Next

Provide the missing item or remove the reference to it. Then specify the `run_drc` command again.

One possible cause for this violation is that a bidirectional scan input or output was referenced by name. If so, and you use the "actual\_bidi\_port\_name"=#; syntax, then TestMAX ATPG cannot determine if the BIDI port should be forced or measured. Instead, try using predefined signal groups: `_si` for forcing scan inputs or `_so` for measuring scan outputs. For more information, see "Predefined Signal Groups in STIL."

Another possibility is that you defined a PI Constraint with a value of X. The default STIL Timing definition cannot define the N character. The N character is used in STIL to define an X value as input (the X character means don't measure an output). You can fix this by editing the Timing block of STIL to define the N character for inputs. The following example shows a typical timing definition with the additional N character definition line highlighted.

```
Timing {
  WaveformTable "_default_WFT_" {
    Period '100ns';
    Waveforms {
      "_default_In_Timing_" { 0 { '0ns' D; } }
      "_default_In_Timing_" { 1 { '0ns' U; } }
      "_default_In_Timing_" { Z { '0ns' Z; } }
      "_default_In_Timing_" { N { '0ns' N; } }
      "_default_Clk0_Timing_" { P { '0ns' D; '50ns' U; '80ns' D; } }
    }
  }
}
```

```
"_default_Clk1_Timing_" { P { '0ns' U; '50ns' D; '80ns' U; } }
"_default_Out_Timing_" { X { '0ns' X; } }
"_default_Out_Timing_" { H { '0ns' X; '40ns' H; } }
"_default_Out_Timing_" { T { '0ns' X; '40ns' T; } }
"_default_Out_Timing_" { L { '0ns' X; '40ns' L; } }
}
}
```

You can also specify the `report_violations` command and review any [V16](#) messages in more detail.

---

## DRC Rule V5

### Message Text

Vector Rule V5: Line L (filename), unresolved reference (<additional info>).

### Severity

Fatal Error

### Description

With a few exceptions, all items must be defined before they are referenced.

One commonly occurring cause of this violation is the use of "clock=P" in the DRC file without either having a Waveform definition block or defining the port as a clock with the `add_clock` command.

Another possibility is that you have a bidirectional clock or asynchronous set/reset port. In the case of a bidirectional port which has been defined as a clock using `add_clock` command you must also provide the timing definition for this port in the Waveform definition block of the DRC file.

### What Next

Provide the missing definition or remove the reference to it, then run DRC again.

---

## DRC Rule V6

### Message Text

Vector Rule V6: Line L (filename), unused item (<additional info>).

### Severity

Warning

### Description

There was an item in the DRC procedure file which was not used or ignored. The line number, `L`, will indicate on which line in the file the item occurred. The additional info will suggest a reason.

### What Next

This is a warning that can be corrected by adjusting the line indicated.

The message:

```
Unused event "P" defined for signal(s)  port_name
```

will occur if a port or group of ports has timing defined which includes the ForcePrior or "P" waveform character. This "P" character is often necessary for proper WGL pattern output. However, the algorithm does not actually create patterns based on the "prior" signal value and this V6 violation is a warning to this difference between what is requested in the STIL timing and what the tool uses. The "P" is processed when creating the WGL pattern output file so if you are trying to generate WGL, then you should leave the DRC file alone and perhaps set this rule to ignore using the `set_rules` command.

The message:

```
unused item (Measures not allowed in test_setup; signal "XYZ")
```

will occur if you are trying to initialize a port to an `x`. The STIL character for forcing an undefined input is `N`. An `x` is considered an output measure that is masked. TestMAX ATPG does not support any measures of outputs in the `test_setup` procedure. Check your `test_setup` and change all X's to N's.

---

## DRC Rule V7

### Message Text

```
Vector Rule V7: Line L (filename), unsupported construct (<additional info>).
```

### Severity

Warning

### Description

Only a subset of the constructs in the vector format language can be used. The unsupported construct is ignored.

### What Next

No action is necessary if the ignored construct will not negatively impact the usefulness of the data. Otherwise, restructuring the input file might be necessary.

---

## DRC Rule V8

### Message Text

Vector Rule V8: illegal context (<additional info>).

### Severity

Fatal Error

### Description

A construct can only be used in certain contexts. For example, STIL vectors cannot mix "#" and other waveform characters in the same statement.

The illegal construct is ignored and this can quickly lead to other syntax errors.

### What Next

This violation is a fatal error, so it must be corrected to pass DRC.

You can specify the `report_violationsV8` command to view the full text of each reported violation. If there are many violations, or to see a specific violation only, use the following syntax:

```
report_violations V8-N
```

where "N" is the number of a specific violation to review.

The `report_violations` command will report additional detail about each violation that you can use to identify the situation to be addressed -- especially if the rule is a warning or has been downgraded to a warning.

The V8 violation can sometimes occur when a bidirectional scan input or output is used. If so, and you use the `"bidi_port"=#;` syntax then TestMAX ATPG cannot tell if the bidi port is to be forced or measured. Instead, try the predefined symbols `_si` for forcing or `_so` for measuring.



This violation can also occur when the defined timing for forcing inputs, measuring outputs, and pulsing clocks produces an event sequence that is not supportable by the current ATPG algorithm. Common causes of this are:

Attempting to use end-of-cycle measure timing when the scan out premeasure (`_so=#`) does not immediately precede the `load_unload` procedure just before the Shift macro. In this case, the waveform definition calls for a measure of scan output after the shift clock pulse, but the initial scan\_out premeasure that TestMAX ATPG requires to support end-of-cycle is missing. See also: End-of-Cycle Measures.

Attempting to use the preferred preclock measure timing, but with an extra scan-out pre-measure (`_so#`). In this case, you have mixed the requirements of end-of-cycle with preclock. Remove the scan out premeasure and measure scan outs only within the Shift macro, or if your intent was to support end-of-cycle measure, you still need to adjust the measure time in the waveform definitions.

Messages containing references to “force-all-clocks” refer to multiple-cycle generic capture procedures. For more information, see “Creating Generic Capture Procedures” in the *TestMAX ATPG User Guide*.

An example message is as follows: `Error: illegal context (Procedure "multiclock_capture" is missing a force-all-clocks parameter). (V8-1)`

As is the case with standard capture procedures, the `multiclock_capture` procedure can consist of multiple vectors. In this case, you need to specify an additional argument to hold the variable clock-pulse information, as shown in the following example:

```
Procedures { "multiclock_capture" { // 2-cycle W "TS1"; C { "_po"=\r9
X ; } V { "_pi"=\r11 # ; "_po"=\r9 # ; } C { "_po"=\r9 X ; } V {"_clks"=
###; } } "multiclock_capture" { // 3-cycle W "TS1"; C { "_po"=\r9 X ; }
V { "_pi"=\r11 # ; } V { "_po"=\r9 # ; } C { "_po"=\r9 X ; } V {"_clks"=
###; } } }
```

You should review the pin timing of each waveform definition, and the event order of every procedure, including "capture" procedures.

The V8 violation can also occur if the `-bidi_control_pin` option of the `set_drc` command has been used and the DRC file does not contain any capture procedures. If this is the case, use the `write_drc_file` command to create a file which defines the capture clock procedures in the LSI reflect IO template and then copy them into your original DRC file and try the `run drc` command again. See also: Creating LSI Compatible WGL

The V8 violation can occur when reading Extended VCD pattern files if the state codes in the data are inconsistent with the port direction in the top module. For example, if the top-level port "A" is an input, then TestMAX ATPG would not expect to see any of the state codes {L,H,X,T,I,h} which are for outputs. Likewise, if the top-level port "B" is an output, then TestMAX ATPG would not expect to see the state codes {D,U,N,Z,d,u} which are for inputs.

---

## DRC Rule V9

### Message Text

Vector Rule V9: Line L (filename), invalid time (<additional info>).

### Severity

Fatal Error

### Description

A valid time value (such as for defining signal timing) must be within certain limits.

### What Next

Correct the error and run DRC again.

---

## DRC Rule V10

### Message Text

Vector Rule V10: Line L (filename), miscounted items (<additional info>).

### Severity

Fatal Error

### Description

The number of items in a list must correspond to previous definitions of that list.

One frequently occurring cause for this message is a mismatch in the number of '#' characters used for the scan in and scan out sections of the Shift procedure.

### What Next

Extra items are automatically ignored but missing items must be supplied. Correct the error in the DRC procedure file and run DRC again.

---

## DRC Rule V11

### Message Text

Vector Rule V11: Line L (filename), repeated entry (<additional info>).

### Severity

Warning

## Description

An item is repeated on line *L*.

Frequently, this occurs during the assignment of values to ports because a port is assigned as part of a symbolic group and also assigned explicitly by name. An example would be:

```
V { _io=\r32 Z; bidi2 = 1; }
```

If the port "bidi2" is also a member of the symbolic group "\_io" then a conflict in assigned values exists. The IEEE STIL specification indicates that the order from left-to-right of assignments within the vector should not affect the outcome. Rather than stop with an error, TestMAX ATPG uses the final value encountered reading from left to right for it's simulation and analysis and issues a V11 warning.

Note: For some forms of patterns generated, both values is assigned to the same pin at the same simulation time, to honor what is requested by the line in the procedure file. For some simulators, this will cause a glitch on the affected pin.

## What Next

It is highly suggested that you adjust the indicated line in the procedure file to eliminate the duplicate assignment. This corrects the potential ambiguity of what value should be applied and also avoids the potential for a glitch during simulation.

Don't forget that the symbolic group "\_pi" contains all inputs, including bidirectional pins and clocks.

You can find it convenient to define a custom symbolic group by copying an existing definition such as `_io` or `_pi`, deleting a few ports that you will explicitly set differently, and then using this new symbolic name instead of the original. An example would be:

```
V { MY_CUSTOM_LIST=\r31 Z; bidi2 = 1; }
```

This would correspond to the example earlier in which the symbolic group of `_io` was copied and the port `bidi2` was eliminated from this list. Now the line assigns 31 values to `MY_CUSTOM_LIST` and a single value to `bidi2`, without causing a multiple assignment. An example of using a symbolic group or `SignalGroup` can be found in Controlling Bidirectional Ports in STIL.

---

## DRC Rule V12

### Message Text

```
Vector Rule V12: Line L (filename), unexpected item (additional info).
```

### Severity

Warning

### Description

An unexpected definition was encountered, for example a waveform character 'Q'. Other possibilities include definition of waveform characters of {0,1,Z,N} which are input characters, for ports that are pure outputs. Or the reverse situation of defining output waveform characters {H/L/T/X} for ports which are pure inputs.

### What Next

Note the following scenarios:

The V12 message reports: "Scanport names need to be changed for VCS simulation; use `-top_scanports` for VCS simulation."

In this case, the models used for the scan cell description do not allow direct parallel simulation because they are Verilog primitives. You should consider adding ``celldefine` references to your modeling flow. If modeling constructs are required, you need to write a special pattern file for simulation using the `-top_scanports` option of the `write_patterns` command.

The V12 message reports: "Scanport names changed for VCS simulation; use `-notop_scanports` for pattern read-back flows."

In this case, the pattern file is usable for simulation but cannot be read back into TestMAX ATPG, (for example, to support diagnostic flows with the pattern set).

To correct the violation indicated in the message, edit the STIL procedure file and repeat the `run_drc` command until the violation no longer appears. Failure to fix this violation can lead to unpredictable results later in the flow, including problems trying to write or read patterns.

If the message contains the text "STIL scan pad", see STIL Scan Padding Behavior.

---

## DRC Rule V13

### Message Text

Vector Rule V13: Line L (filename), invalid state (<additional info>).

### Severity

Fatal Error

### Description

An invalid state was found, for example a clock that should be at its off state at the end of a procedure was left on.

### What Next

Correct the error and run DRC again.

---

## DRC Rule V14

### Message Text

Vector Rule V14: Line L (filename), missing state (<additional info>).

### Severity

Warning

### Description

A missing state was found, for example a constrained pin was not set to its constrained value in the `test_setup` macro.

If you are reading in functional patterns with the `set_patterns` command you can have forgotten to specify the `-strobe` option to be applied to the Extended VCD patterns.

If you are using the `-strobe` option, the port name you specified cannot be found in the design or in the Extended VCD pattern file. Double check spelling and case of the port name.

To review the context of a V14 message, specify the command `report_violations V14`. If TestMAX ATPG displays the following violation message:

```
(Missing alias in load_unload for <signal> in chain  name,
group  name; cannot write proper STIL for scangroups)
```

Then, adjust the scan references in the Vector of the Shift block pertaining to the group stated in the warning message. This adjustment is only required if you are generating STIL pattern data for this test. See "Multiple Scan Groups" in the *TestMAX ATPG User's Guide* for more details.

### What Next

If this violation occurs during `run_drc` then the missing state is automatically added. You can write out the corrected DRC file using the `write_drc_file` command and compare it to the original.

If this violation occurs for the set patterns command, retry the command and specify one or more `-strobe` options.

If the message contains the text `STIL scan pad`, see STIL Scan Padding Behavior.

---

## DRC Rule V15

### Message Text

Vector Rule V15: No pin name for parallel load.

Parallel STIL format doesn't support multi cycle Shift for `scan_compression`.

### Severity

Error

### Description

TestMAX ATPG cannot write patterns in the selected format using the specified pattern options.

### What Next

Try a different format, or if you were trying a parallel form, try the serial form instead.

If you are attempting to use both forms by writing STIL patterns (`-format stil`) after modifying procedures for STIL99 (`-format stil99`), first write all the STIL patterns you need and then write the STIL99 patterns.

If you are attempting to write parallel Verilog patterns when the message occurred and the violation message suggested that "N+1" shifts were required where "N" is the length of the scan chain the most likely your design contains a multi bit library cell with no accessible internal instance names. The presence of a ``celldefine` directive around the definition of the multi bit module or use of the `-library` option of the `read_netlist` command restricts access and use of the internal instance names necessary for parallel simulation. You might be able to workaround the issue by remodeling the multi bit cell to remove the ``celldefine` and make the internal instance names available for simulation.

If you are attempting to write WGL patterns for LSSD-style designs, the problem might be due to a multi cycle Shift procedure. WGL only supports a single cycle shift procedure. Make sure you have a Timing block defined in the STIL Procedure file. Review the timing of the master and slave clocks to ensure that they can be applied successfully in a single tester cycle. Then adjust your Shift procedure to use a single test cycle. Pulse both master and slave clocks in this cycle.

Note that WGL patterns are not supported for a serialized compressed core without a serializer clock controller.

This problem might also occur because you specified the `set_scan_ability` command earlier in your flow. You cannot write patterns to a file if any of the patterns were generated with instances on the scan ability list. Any such patterns would be invalid for the current design.

---

## DRC Rule V16

### Message Text

Vector Rule V16: Line L (filename), miscounted arguments (<additional info>).

### Severity

Warning

### Description

A procedure definition uses an incorrect number of arguments.

This frequently occurs because the number of "#s" used in the reference to scan inputs or scan outputs does not match the number of scan chains defined. It can also occur in various capture procedures if the number of #s for force PI or measure PO does not match the number of inputs or outputs in the design.

### What Next

If the rule severity is not set to error, then the correct number is substituted. You can write out the corrected DRC file using the `write_drc_file` command and compare it to the original.

---

## DRC Rule V17

### Message Text

Vector Rule V17: Line L (filename), clock *name* should be pulsed, not forced on.

### Severity

Error

### Description

DRC procedures must pulse clocks by assigning them the value "P" or hold the clocks off by assigning them their off state.

TestMAX ATPG does not support holding a clock to its on state for one or more tester cycles. The reason is that this would require a tester channel definition that would be RZ(RO) on one cycle, but then NRZ(NRO) on the next. Very few ASIC vendors or testers like to have the signal type switch dynamically from cycle to cycle.

### What Next

You can downgrade the severity of the violation to warning or ignore to continue, but when you write patterns you will see inconsistencies across the various pattern formats. Some formats (Verilog, VHDL, STIL) will support the force "on" across multiple test cycles while others WGL, FTDL, TDL91, TSTL2 will convert the force on into a pulse in each tester cycle. The suggested solution is to change the clock forced to its on state of 1/0, into a force to "P" or pulse, or a force to its off state if that is acceptable. Following are some additional ideas you might want to consider:

Does the clock or asynchronous set/reset need to be applied? Many users feel they should apply a global reset for good measure or apply at least one clock in the `test_setup` procedure but ATPG generally doesn't care if the design is initialized or not unless nonscan cells need to be initialized to enable other ATPG operations. So you might be able to remove the reset or clock event. Try replacing the on value with an off value and see if the scan chain check of "run drc" still passes and what happens to the various contention checks such as Z4.

Another item to review is whether the set, reset, or clock is really used outside of the `test_setup` procedure? If you have a PI constraint on the set/rst/clock pin so that it is always off (except for `test_setup`), then there is really no reason to declare this pin as a clock and you can remove it from the defined clock list. After this is done, you are then free to force the value either to 1 or 0 in `test_setup`, as long as you restore the PI constraint value by the end of `test_setup`.

If you have a JTAG design, or for some other reason you are using the `set_drc-clock <some_pin>`, then you have the same condition as where a reset declared as a clock will not be used for patterns. You can then remove the pin from the clock list, and add it to a PI Constraint list and are then free to force it 0 or 1 in `test_setup`.

The option `set_drc-allow_unstable_set_resets` supports asynchronous set/rst pins without defining them as clocks. By selecting this option you can omit 'rst' from the clock list. TestMAX ATPG then creates patterns using this pin as an NRZ/NRO pin and you are free to force it to 0 or 1 in `test_setup`.

If the reset is actually used for pattern generation, then another alternative is to use two STIL timing definitions, one for `test_setup` and another for all other patterns. Suppose the standard test cycle is 100ns but you wish to initialize reset for 2000ns. Then define a second timing definition that is used in the `test_setup` procedure in which the cycle period is 2100ns and the reset is pulsed from 50ns to 1950ns. *Note* that many vendors using WGL as a pattern format support only one timing definition.



```
#
# Example using two timing definitions to achieve long reset
#
Timing {
  WaveformTable "NORMAL" {
    Period '100ns';
    Waveforms {
      "grp1" { 01NZ { '0ns' D/U/N/Z; } }
      "grp2" { 01NZ { '5ns' D/U/N/Z; } }
      "clk" { P { '0ns' D; '45ns' U; '55ns' D; } }
      "rst" { P { '0ns' U; '45ns' D; '55ns' U; } }
      "outpins" { LHXT { '0ns' X; '95ns' L/H/X/T; } }
    }
  }
  WaveformTable "LONG" {
    Period '2100ns';
    Waveforms {
      "grp1" { 01NZ { '0ns' D/U/N/Z; } }
      "grp2" { 01NZ { '5ns' D/U/N/Z; } }
      "clk" { P { '0ns' D; '50ns' U; '2050ns' D; } }
      "rst" { P { '0ns' U; '50ns' D; '2050ns' U; } }
      "outpins" { LHXT { '0ns' X; '2095ns' L/H/X/T; } }
    }
  }
}
:
:
```

```
MacroDefs {
  "test_setup" {
    W "LONG"; # use 2100ns cycles
    V { clk=0; rst=P; } # apply long reset pulse

    W "NORMAL"; # switch to normal 100ns cycles
    V { clk=P; rst=1; in=0; oe=1; } # normal 'clk' pulse
    V { clk=0; }
  }
}
```

set\_drc, set\_rules in the TestMAX ATPG and TestMAX Diagnosis Commands Guide

---

## See Also

---

## DRC Rule V18

### Message Text

Vector Rule V18: Mixed timing (<additional info>).

## Severity

Warning

## Description

In the WaveformTable section of the STIL procedures where port timing is defined it is possible to have as many different timing combinations as there are ports. Groups of ports are defined to have different timing offsets for force and/or for measure. The V18 violation is issued when the ports do not all have a common force time and a common measure time.

The violation is due to the fact that the ATPG algorithm uses only one event sequence for its internal algorithm, despite the differing defined times that can occur for input, output, and bidirectional ports. This fundamental ATPG event sequence is:

- 1) force inputs
- 2) measure outputs
- 3) pulse capture clock or shift clock

Because of this algorithm and despite the defined timing, at least internally all of the non-clock inputs is forced at the same time, all of the measures will occur at the same time, and all of the clocks will occur at the same time.

Failure to satisfy this rule will have no affect on test coverage but there is a slight chance that patterns generated will fail simulation.

## What Next

When the rule severity is not set to error the rules checking will continue and will use for an event time the largest timing value for the group of pins in question: forcing inputs, measuring outputs, pulsing clocks.

When patterns are written, the timing as originally defined is used so you have a condition where the pattern generation algorithm used a timing different than that used in the final patterns.

For test purists, you might want to adjust the defined timing so that the events are all common. This will ensure the timing in the patterns matches the timing assumed by the pattern generation algorithm. For the rest of the world, the patterns will generally work. If simulation passes, great! If simulation fails, try adjusting the timing to eliminate V18.

---

## DRC Rule V19

### Message Text

Vector Rule V19: Line L (filename), clock <port> undefined in test protocol file.

### Severity

Warning

### Description

The indicated port was previously defined as a clock, but was not defined as a clock in the test procedure file used during DRC.

If the severity is ignore or warning (the default), then the port is treated as a clock and DRC processing continues. If the severity has been changed to error, then processing stops.

### What Next

Adjust the procedure file or your command sequence so that the inconsistency is removed by either removing the port from the clock list or by defining the port as a clock in the procedure file.

---

## DRC Rule V20

### Message Text

Vector Rule V20: unsupported event (more info here).

### Severity

Error

### Description

The defined order of events in a capture procedure is not supported or there are extra events in the procedure.

Failure to pass this rule check indicates that a procedure has defined events in an order which is not supported by the Basic-Scan or Fast-Sequential ATPG algorithms. Capture procedures must define events in the following order:

force values on inputs

measure values on outputs

pulse one defined clock and all equivalent ports (optional)

This rule violation occurs if the events have been defined in an order which does not match this sequence or if extra events (such as explicitly setting a bidirectional control pin) have been defined which are not expected.

The order of events is defined both by the number of "V{...}" structures used as well as the port timing defined in the timing section. For forces and measures within the same "V", the port timing determines the order of events.

Use of Full-Sequential ATPG provides more flexibility in the event order of capture procedures. When using the Full-Sequential ATPG algorithm you can make use of the Sequential Capture Procedure.

### What Next

Review the capture procedures indicated and consider changing the order of events or the defined timing to match the sequence above, or considering use of Full-Sequential and the 'sequential\_capture' procedure.

If you wish to proceed without changes, you can adjust the severity of this rule from an error to a warning using the `set_rules` command. However, be advised that you do this at your own risk and that the unsupported events or extra events are not part of the ATPG algorithm and are not considered when checking contention and performing other DRC checks. Patterns can be generated with undetected contention or can fail in simulation.

With V20 set to warning or ignore TestMAX ATPG will generate patterns using the following environment for Basic-Scan or Fast-Sequential patterns:

All data change events will occur at the same time.

Next, all data measure events will occur at the same time.

Next, all clock events will occur at the same time. So multiple clocks defined as "PI Equiv" with different timing will pulse simultaneously, but not with the unique timing the user has asked for.

Then, when patterns are written to a specific output format, the user desired timing is used to create the patterns. So the patterns are written with a different timing than what was actually used to generate those patterns and this could result in patterns which fail in simulation.

---

## DRC Rule V21

### Message Text

Vector Rule V21: unsupported old syntax (more info here).

### Severity

Error

### Description

You are using old syntax which is no longer supported.

### What Next

You will need to identify and modify the old syntax to proceed.

If the parse error is reported in the DRC file, one common possibility is the file contains the old syntax for defining the Equivalence relationships. This old syntax was not IEEE 1450.0 or P1450.1 compliant and should now be changed to be P1450.1 compliant:

Old non-compliant syntax:

```
E { "POS"=0; "NEG"=1; } # inverting relation

E { "POS"=1; "NEG"=0; }

E { "P1"=0; "P2"=0; "P3"=0; } # non-inverting relation

E { "P1"=1; "P2"=1; "P3"=1; }
```

New STIL P1450.1 syntax:

```
E "POS" \m "NEG" ; # inverting relation

E "P1" "P2" "P3" ; # non-inverting relation
```

Perhaps the easiest way to make these changes is to use command line options to define PI equivalences, comment out all the old `E` lines in the DRC file, run `drc`, and then use the `write_drc_file` command to write a new file. This new file will use the new P1450.1 syntax for the equivalence statements.

---

## DRC Rule V22

### Message Text

Vector Rule V22: Invalid waveform (more info here).

### Severity

Error

### Description

There has been an inconsistency or illegal waveform definition attempt in the DRC/STL procedure file procedure file. This can occur if input WaveformChars {0,1,Z,N} are defined for output pins, or output WaveformChars {L,H,T,X} are defined for input pins, or from other causes.

### What Next

You can downgrade the severity of this violation to a warning. However, this is not advised because the affect upon writing patterns cannot be predicted. In some cases, pattern syntax might be corrupted and you can have either bad patterns or patterns which will not read back in.

It is advisable to correct the cause of the V22 before continuing. Use the `report_violations` command to get explicit information for a particular V22 violation. The additional info text in the message should assist in identifying which signal definition has the problem and which line in the file the problem is near. Double check consistency between defined pin directions and the WaveformChars definitions, which are listed.

---

## DRC Rule V23

### Message Text

Vector Rule V23: Line `L` (file), strobe conflict (Measure point coincides with events at time = `T`). (V23-1)

### Severity

Warning

### Description

It is always best to try to measure circuit outputs at times when the circuit is stable and there are no inputs or outputs changing. TestMAX ATPG generates this violation when a calculated measure strobe occurs at the same time as an Extended VCD event.

For example, if the Extended VCD file contains an event at '#12500' and you specified a strobe period when reading the file of 500 nS, then an integer multiple of strobe time occurs exactly at time 12500 (500\*25). If the event at time #12500 updates the circuits inputs and/or changes the expected value of the circuits outputs then TestMAX ATPG has no way of knowing whether it should make the measure using the previous values before time #12500, or process the new input changes and output expect values and then use these new values to make a measure.

In this situation TestMAX ATPG will always insert a measurePO event using the circuit state before any changes described by the new event. Since this cannot be the correct assumption for the patterns, a violation message is issued. The violation message indicates the line number `L` in the VCDE file at which strobe conflict occurs as well as the event time `T`.

### What Next

If the behavior of inserting a measurePO using values before the event is acceptable, then you can ignore the violation.

You can also try to adjust either the strobe period or the strobe offset to try to shift the points in time at which measure strobes occur. You might be able to move the strobe time by a few time units and avoid the ambiguity of the strobe time collision. For example, using a `-strobe offset 490 ns` in conjunction with a strobe period of 500 ns causes the first strobe at offset 490 nS instead of 500. Subsequent strobes will occur at times of  $T = 490 + (500 \cdot N)$  and the strobe that was originally at 12500 will shift to 12490.

---

## DRC Rule V24

### Message Text

Vector Rule V24: Time T, PO measures with clocks active not supported for pattern mapping. (V24)

Time T, PI changes with clocks active not supported for pattern mapping. (V24)

Time T, Staggered clocks not supported for pattern mapping. (V24)

Time T, Overlapped clocks not supported for pattern mapping. (V24)

### Severity

Error

### Description

Pattern mapping requires certain restrictions on the input patterns in order for the mapping algorithm to have the best chance of succeeding. Certain pattern event ordering is not currently supported:

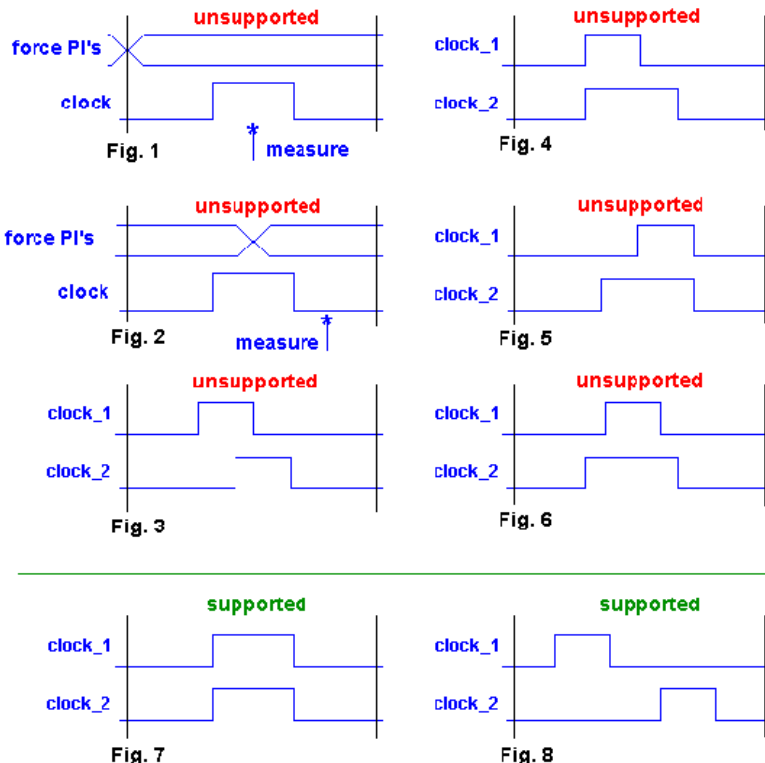
output measures with clocks active (fig. 1)

inputs changing with clocks active (fig. 2)

staggered clocks (fig 3., fig 5.)

coincident clocks with different trailing edges (fig 4.)

overlapped clocks with different durations (fig. 6)



When an unsupported sequence is detected the time,  $T$ , near where the problem is detected is extracted from the Extended VCDE file and reported. Depending upon the type of problem, this reported time might be at or just near the problem spot in the incoming patterns.

Pattern mapping effort is aborted when a V24 is detected.

### What Next

The patterns must be modified to match supported input styles.

For an output measure occurring with an active clock it might be possible to shift the measure time by adjusting the strobe options used with the run mapping command.

For inputs that change with an active clock, or the variations of staggered and overlapping clocks that do not have identical edges and durations, the patterns must be changed before they can be successfully processed into internal forms and mapping attempted.

## DRC Rule V25

### Message Text

Vector Rule V25 : Line L (filename), unload mode enable not defined.



### Severity

Error

### Description

A per\_load mode is active and an unload mode enable port has not been defined. (An unload mode enable port is not required in per\_shift mode.)

### What Next

Downgrading this rule to pass DRC and run\_atpg can result in increased pattern count for the same test coverage.

---

## DRC Rule V26

### Message Text

Vector Rule V26 : Line L (filename), unload mode enable port *port\_name* incorrectly set.

### Severity

Error

### Description

Incorrect patterns might be generated because the unload mode enable port is incorrectly set.

### What Next

In per\_shift mode, the unload mode enable port (if defined) must be set to 1 in the load\_unload preamble.

In per\_load mode, the unload mode enable port (which must be defined, see [V25](#)), must be set to 0 in the load\_unload preamble. Failure to do so can result in incorrect patterns when per-shift data for the unload mode enable is not part of the pattern.

---

## DRC Rule V27

### Message Text

Vector Rule V27: Line L (filename), unload mode control port *port\_name* incorrectly set.

### Severity

Error

### Description

Incorrect patterns might be generated because the unload mode control port is incorrectly set.

### What Next

In per\_shift mode, the unload mode control ports must be set to 0 in the load\_unload preamble. Failure to do so can result in incorrect patterns when per-shift data for the unload mode controls is not part of the pattern.

In per\_load mode, the unload mode control ports must not be set, or else the patterns will be incorrect.

---

## DRC Rule V28

### Message Text

Vector Rule V28 : Line L (filename), pipelining (load\_depth=<depth>, unload\_depth=<depth>) not supported.

### Severity

Error

### Description

The specified pipelining per-load patterns are not supported.

### What Next

See limitations on incompatibility of pipelining with per-load patterns.

---

## DRC Rule V29

### Message Text

Vector Rule V29

("resolution loss"): Too many events or too small a resolution in procedure *name*.

### Severity

Warning

## Description

This message appears when the timing information in the STL procedure file, STIL, or Verilog file exceeds the available integer precision (for example, 32 bits). In most cases, this violation does not affect DRC or ATPG since only the ordering of the timing is affected and not the magnitude. However, a V29 message is issued when scaling causes a loss of resolution. If this message is generated when procedure definitions are read in the STL procedure file, the procedure definitions might be incomplete.

Overflow cases are handled as follows:

32-bit time delays are supported.

33-bit and higher delays are supported on 64-bit platforms. For 32-bit platforms, the delay is computed based on procedure information, which might cause a loss of resolution. All times are divided by 10 as often as needed to adjust them to a 32-bit value.

123000000000000 is transformed into a 32-bit value with no loss of resolution while using the appropriate timescale.

1230000000000001 causes a V29 warning.

## What Next

Review the context of the V29 message and change the timescale if this prevents the time values from exceeding the 32-bit value. If this message is generated when reading a procedure (such as the test\_setup procedure), the DRC and ATPG issues might be caused by truncated procedure events.

If the events in the test\_setup procedure exceed the 32-bit time value limit, all subsequent TestMAX ATPG operations (such as run-time and memory requirements) are affected. This condition is sometimes caused by loops with a large number of events intended for locking PLLs. In this situation, you can insert the `DontSimulate ATPGDRC` construct before each loop, and insert the `UserKeywords DontSimulate` construct before the `MacroDefs` block. These constructs prevent the loop events from affecting the TestMAX ATPG operations and preserve the loops throughout the flow. Loops are written in the patterns even though the loop events bypass other TestMAX ATPG operations. PLL models are black boxes in TestMAX ATPG, and events sent exclusively to these models are not pertinent to TestMAX ATPG operations.

---

## DRC Rule V31

### Message Text

Vector Rule V31:

Waveform simulation events (H and L Waveforms across Timing have a different number of leading Z events(n,n); check consistency) Waveform

simulation events (Both multiple and single-cycle capture procedures are defined; (name) is [single|multi]-cycle. Affects bidirectional handling in external simulation Waveform simulation events (Waveforms [H,L,T,X] need Z event in single-cycle procedure (name) Waveform simulation events (Waveforms [H,L,T,X] must not have a Z event in multiple cycle procedure (name)

## Severity

Warning

## Description

V31 rule violations flag potential issues with external simulation of the STIL data, with the events defined in the waveforms associated with bidirectional signals in the design. These are warnings only; there might be environments where the waveforms cannot be specified for these simulation behaviors, and these warnings will not affect TestMAX ATPG operation. However it is important to note these warnings if simulation of the STIL data is to be performed.

All of these rules are related to the effect of an initial drive-off event in measure waveforms on bidirectional signals, as the first event. This is typically expressed as `0ns:Z` in the waveform definition. For proper simulation, this event is required for capture procedures that contain 1 cycle; when the capture procedure is defined as multiple cycles this event must not be present.

There are four messages generated under the V31 rule:

The message H and L Waveforms across Timing have a different number of leading Z events(n,n); check consistency identifies that there were a different number of leading Z events defined across all H and L waveforms. This might indicate a consistency problem in the waveform definitions, and should be reviewed.

The message Both multiple and single-cycle capture procedures are defined identifies that there are a mixture of capture behaviors - some capture procedures are defined with a single capture Vector, others are defined with multiple capture Vectors. This will result in additional violations if the same timing is used across the different capture procedures, and it indicates a potential problem for simulation of the bidirectionals. If possible, all capture environments should be uniform, but this is not a requirement. If each capture environment defines separate timing then each capture procedure can be unique, and this warning can be ignored.

The other two Waveforms messages indicate specific waveforms that are either missing the leading Z event which is required for simulation (for single-cycle captures), or there is a leading Z event that should not be present (for multiple-cycle captures).

## What Next

If an external simulation of the STIL data is not intended, these messages can be ignored.

For the Waveforms messages, consider adding or deleting the leading `z` event as specified in the message. If there are mixed single-cycle and multiple-cycle capture procedures, it might be necessary to define unique timing (WaveformTables) for each capture procedure to eliminate the Waveforms messages (in this context, you will always get a V31 message advising that there are mixed environments for capture).

When the message Both multiple and single-cycle capture procedures are defined is generated, the procedure name referenced is the first procedure where the number of cycles changed from the prior definitions. This might not be the only procedure; all procedures defined should be reviewed to eliminate this situation if possible.

Note that the warning generated for multi cycle capture procedures and the `Z` event on measured outputs (H.L, and T) will only cause a problem if the patterns are written when the `write_patterns-measure_forced_bidis` option is specified. If this option is not used, then any warnings associated with these states can be ignored. However, there are no issues with correcting the waveforms in any circumstance; nor is there a need to address these warnings for all potential output formatting options.

---

## DRC Rule V32

### Message Text

Vector Rule V32:

```
Incorrect event sequence (<wft-name>; PI time (<integer>) must be less  
than PO time (<integer>)
```

```
Incorrect event sequence (<wft-name>; PO time (<integer>) must be less  
than clock LE (<integer>) on <signal-name>
```

### Severity

Warning

### Description

The WaveformTables used for path delay testing ("`_launch_WFT_`", "`_capture_WFT_`", and "`_launch_capture_WFT_`") must follow certain timing restrictions. V32 messages are generated during DRC when these restrictions are violated.

These are the same messages generated for M336; please see M336 for details (an M336 message is generated if the fault model is set to transition after DRC has run).

### What Next

See M336 for details.

---

## DRC Rule V33

### Message Text

Vector Rule V33: Missing constrained signal ( *name* is constrained to <0|1|X|Z> for capture, not defined in load\_unload)

### Severity

Warning

### Description

This message is generated for each signal that has been set to a constrained value, but is never applied in the load\_unload procedure. By default, these signals will assume their constrained values during TestMAX ATPG application of the load\_unload and shift operations, in particular during DRC analysis. This behavior might not propagate through STIL test flows unless this data is explicitly provided. In some situations, the constrained state is necessary for the function to operate properly.

### What Next

If you are using WGL output, the required states are provided and this warning can be ignored. If you are using STIL data, then it is recommended that you review the missing signals and/or provide for an explicit state on these signals in the load\_unload procedure.

You may use the command `set_rulesV33 -autofix` to automatically insert these missing assignments in the load\_unload procedure during STIL write. This option will suppress generation of the V33 rules, so non-STIL-based flows may use this option to remove this warning. Only STIL output is affected, including STL procedure file generated from `write_drc` commands.

If you do not want the load\_unload procedure to be modified, you can use the command `set_drc-noconstraints`, and check the behavior of the load\_unload without this data present. This option modifies the simulation of the load\_unload procedure to not apply constrained values on signals. Using this option validates if the constrained signals affect the behavior of the load\_unload procedure, as long as the constrained signals are sufficiently modeled in the design to reflect their operational effects.

---

## DRC Rule V34

### Message Text

(**Version 1**) Line *N* (*filename*), Invalid internal clock timing: There is no ClockTiming block named '*string*' in ClockStructures '*string*' (V34-1)

(Version 2) Line *N* (*filename*), Invalid internal clock timing: Invalid period for "*string*" (V34-1)

(Version 3) Line *N* (*filename*), Invalid internal clock timing: Missing period for "*string*" (V34-1)

(Version 4) Line *N* (*filename*), Invalid internal clock timing: Waveform events must be contained within one period for "*string*" (V34-1)

(Version 5) Line *N* (*filename*), Invalid internal clock timing: Waveform events must be monotonically increasing for "*string*" (V34-1)

(Version 6) Line *N* (*filename*), Invalid internal clock timing: Inconsistent off\_state specification for clock string (V34-1)

(Version 7) Line *N* (*filename*), Invalid internal clock timing: Waveform events must be contained within one period for "*string*" (V34-1)

(Version 8) Line *N* (*filename*), Invalid internal clock timing: Cannot slow down capture from '*string*' to '*string*' beyond 1 cycles. (V34-1)

(Version 9) Line *N* (*filename*), Invalid internal clock timing: Clock string cannot safely be synchronized in Synchronization Group string. (V34-2)

## Severity

Warning

## Description

This violation is reported when you make an incorrect specification in the ClockTiming block of the STIL procedure file. A clock timing definition with a V34 violation is discarded and the clock's timing relationships are not reported by the `report_clocks -capture_matrix` command. ATPG will treat the clock as asynchronous to the other clocks, which reduces the test coverage. The various versions of the V34 message are described as follows:

**Version 1** – The specification made by the `set_drc -internal_clock_timing` command did not match a named ClockTiming block in the STIL procedure file.

**Version 2** – The period for the "*string*" statement must include a positive integer followed (without a space) by ns or ps.

**Version 3** – The period for the "*string*" statement is missing or invalid. It must include a positive integer followed (without a space) by ns or ps.

**Version 4** – The argument for the Period statement is too short to contain both arguments for Waveform statement.

**Version 5** – The first argument for the Waveform statement must be at an earlier time than the second argument.

**Version 6** – The first argument of the Waveform statement must be equal to or greater than 0ns or 0ps.

**Version 7** – The difference between the arguments of the Waveform statement must be less than the Period argument.

**Version 8** – Because a MultiCyclePath is defined between the clocks, the pulses be moved farther apart. But the `set_drc -multiframe_paths` command was not set.

**Version 9** – This message accompanies one of the other versions of the V34 message. Correct one or more of the other message(s) first, and this version will no longer print.

### What Next

Fix the violated syntax in the STIL procedure file so ATPG can use the clock as an synchronous clock.

---

## DRC Rule V35

### Message Text

Vector Rule V35:

**(Version 1)** Line *N* (*filename*, Invalid clock constraint: *string* " is not a declared constrainable object" (V35-1)

**(Version 2)** Line *N* (*filename*, Invalid clock constraint: Non-clock entity "*string*" cannot be pulsed. (V35-1)

**(Version 3)** Line *N* (*filename*, Invalid clock constraint: Constraints assignment is truncated to the size of "*string*" (V35-1)

**(Version 4)** Line *N* (*filename*, Invalid clock constraint: Assignment to InstructionRegister "*string*" is incomplete; padding with dont\_cares (V35-1)

**(Version 5)** Line *N* (*filename*, Invalid clock constraint "*string*" is an unusable constraints assignment (V35-1)

**(Version 6)** Line *N* (*filename*, *string* is not a constrainable clock (V35-1)

### Severity

Warning



### Description

A V35 violation is reported when you make an incorrect specification in the ClockingProcedure block of the STL procedure file. A clocking procedure with a V35 violation is discarded and is not reported by the `report_clocks -constraints` command.

If other ClockingProcedure blocks exist that don't have V35 violations and are valid, then ATPG generates patterns. But if the violated clocking procedure represents real clocking behavior, the test coverage is lower than it should be. In this case, you need to fix the violated syntax so that the clocking procedure can be used by ATPG.

The various versions of the V35 message are described as follows:

*Version 1* – The clocking procedure specified a pin path name that is not specified in the ClockStructures block as either a clock or a clock instruction register element.

*Version 2* – The clocking procedure specified a P in the clock instruction register values.

*Version 3* – The clocking procedure specified too many bits for the clock instruction register.

*Version 4* – The clocking procedure did not specify enough bits for the clock instruction register.

*Version 5* – The clocking procedure specified invalid characters for a clock or in the clock instruction register.

*Version 6* – A reference clock is specified in the ClockingProcedure block.

### What Next

If the violated clocking procedure represents real clocking behavior, you need to fix the violated syntax so that it can be used by ATPG.

---

## DRC Rule V36

### Message Text

Vector Rule V36: Failed to map external chain *chain\_name* to an existing codec.

### Severity

Fatal Error

### Description

This message is generated for each externally visible chain that has a codec-association constraint statement defined (Compressor, LoadPipelineStages, or UnloadPipelineStages

statements) that does not match any defined codecs in the design. DRC is halted after reading the SPF on this error, and other errors are likely.

### What Next

Review the constraint definitions on this chain against the codecs defined for the design, and identify a proper matching codec to use.

---

## DRC Rule V37

### Message Text

Vector Rule V37: External chain *chain\_name* used default mapping to a codec; no pipeline or compressor data.

### Severity

Warning

### Description

This message is generated for each externally visible chain that does not have explicit codec-association constraints present (Compressor, LoadPipelineStages, or UnloadPipelineStages statements). This warning is only generated when one or more other externally visible scan chains are defined with codec-association statements.

Mixing explicit associations and default assignments creates a non-robust environment. This condition may change default assignments and the SPF. It also might cause the scan chains without associations to change to other compressors or map to an unexpected compressor that doesn't have the correct pipeline context. You should always use explicit association statements if any external chain has an explicit association statement.

### What Next

To eliminate this warning and provide consistent scan chain mapping, review the scan chain definitions and add explicit codec-association statements to the scan chain definitions identified in the message.

Chains with explicit association statements prevent codecs that match these constraints from being used as default codecs (for chains that are missing explicit association statements). A codec that has associated explicit external chains requires all associated chains to use explicit statements.

---

## DRC Rule V39

### Message Text

Vector Rule V39: Line L (filename), missing state in  
load\_unload(<additional info>).

### Severity

Warning

### Description

The indicated missing signal has a constraint value (0/1) but gets 'N' during the  
load\_unload procedure.

ATPG shows the following violation message:

```
Warning: Line L (filename), missing state in load_unload(Constraint pin  
"<missing signal>" is not set to its constraint value in "load_unload").  
(V39-1)
```

ATPG generates patterns that satisfy the defined list of PI constraints. But VCS might not get the constraint value during the load\_unload procedure if it is not specified in the STIL procedure file. This may lead to VCS failure. V39 check is added to avoid VCS failure.

### What Next

If this violation occurs during then the missing state is automatically added to the pre\_shift vector statement of the load\_unload procedure. You can write out the corrected DRC file using the command and compare it to the original. If the load\_unload procedure does not have a pre\_shift vector, then it does not add the missing signal.

# 11

## Category X - X-State Rules

---

### See Also

- [X1 - sensitizable feedback path](#)
- [X2 - X-source gates that propagate to a scancell](#)

---

## DRC Rule X1

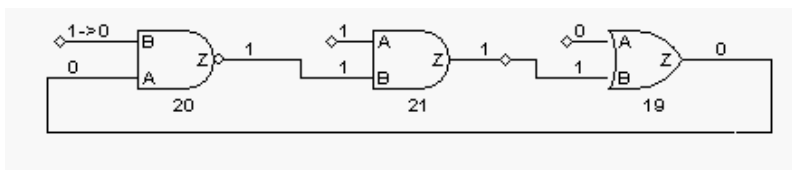
### Message Text

DRC Rule X1: Feedback path network N is sensitizable through source gate G.

### Severity

Warning

One example, many variations are possible:




---

## Example

### Description

Feedback paths must not be sensitizable.

A combinational feedback path was also found to be sensitizable. In other words, a feedback loop that completes the path back on itself and results in a potential oscillation or ring driver.

Violation of this rule is only an indication of a potential sensitizable feedback path. This rule considers the effects of PI constraints and ATPG constraints created with the `-drc`

option, but it does not consider the effects of nonscan cells with constant values, cell constraints, or the effects of PI equivalences. You need to investigate further to determine if there is truly a sensitizable feedback path.

Any patterns that would pose a risk of oscillation are identified and the expected values masked off so there is no danger of generating ATPG patterns that would miscompare. However, the opportunity for the design to oscillate during simulation of ATPG patterns is still present even though the expected data has been masked. This oscillation can degrade simulation performance.

In general, any feedback path (B23 warning) can cause the test generator to fail to justify all necessary test conditions, which can result in a lower test coverage. This failure to justify can happen for any feedback path but the risk is greater if the feedback path is sensitizable. The presence of B23 and X1 violations can produce degraded runtime performance.

In the violation message, *N* is the feedback path ID assigned by TestMAX ATPG when identifying combinational feedback loops. *G* is the gate ID of the ATPG primitive gate where there is the potential for oscillation to start.

### What Next

You can use the `report_feedback_paths` command to review the number of gates and sources in the feedback loop. You can also use it to get a detailed list of instance pathnames of gates in the loop. The "source" gates of the feedback loop are always listed first. The feedback path analysis uses test generation, whose parameters are controlled using the `set_atpg` command. X1 analysis is affected by the `set_atpg-abort_limit` switch, since it uses ATPG to figure out if the feedback path is sensitizable. The default setting is 10 aborts before the analysis can complete.

You can analyze an X1 violation using the graphical schematic viewer by selecting the **ANALYZE** button on the left, and then choosing the specific occurrence of the X1 violation from the menu. You can also perform an analysis by manually entering the `analyze_violation` command with the `-display` option. This sets the gate reporting to the pattern data that sets up the sensitization condition and displays the gates in the failing feedback path along with the net data that enables that path.

You can find the X1 violation is nothing to worry about due to some controls not considered during the analysis such as nonscan cells that are constant or PI equivalences. You can then safely ignore the violation.

If the potential for oscillation is real, you might wish to consider altering your design to eliminate the feedback loop or at least to prevent the X1 violation, at least in ATPG mode.

You might wish to consider manually breaking the loop for ATPG efforts by disconnecting selected gates along the way. The `add_net_connections` command along with the `TIEX` and `-disconnect` options can be used to disconnect all inputs from selected gates in the feedback path. However, the use of net connections can lead to performance issues

during circuit flattening, so you might want to modify the netlist (used only in the ATPG environment) to break the loop rather than use the `add_net_connections` command.

---

## DRC Rule X2

### Message Text

DRC Rule X2: X source T gate G1 is propagatable to N scan cells (G2).

### Severity

Ignored

### Description

Note the following definitions:

T is the gate type of the X-source gate

G1 is the `gate_id` of the X-source gate

N is the number of capturing scan cells of the X-source gate

G2 is the `gate_id` of the first capturing scan cell of the X-source gate

This rule checks for gates that are potential sources of X values that can propagate to a scan cell. The following possible X-source gates are considered:

Gates with a connection to the global TIEX gate

Gates (except BUS gates) with a connection to the global TIEZ gate

TIEX gates

TIEZ gates, except those that connect directly to a BUS gate

BUS gates identified as controllable or that can potentially attain a Z state

TSD or SW gates that do not connect directly to BUS gates

WIRE gates identified as controllable

DLAT gates identified as non-hot transparent latches

DLAT or DFF gates associated with scan cells constrained to X

DLAT or DFF gates identified as nontransparent nonscan cells

DLAT or DFF gates with user-selected capture masking

DLAT or DFF gates with capture masking due to a failing DRC rule

RAM gates

ROM gates with memory values that are not fully defined

Feedback path source gates of sensitizable feedback paths (those that fail [Rule X1](#))

PI or PIO gates constrained to an X value

Note that constrained values from pin constraints and constant value cells are considered when determining whether an X-source gate is propagatable to a scan cell. Any X-source gate considered propagatable to at least one scancell causes a violation of the X2 rule.

### What Next

No action is required to correct this problem: ATPG can tolerate captured Xs under normal conditions. If you want to avoid Xs during ATPG, you can use automated violation analysis to help debug this problem. To enable this analysis, specify the `set_rules X2 warning` command, then run DRC. This analysis displays the failing X-source gate, the capturing scan cell gate, and all the gates in the paths between them. The pin data is set to the `constrain_value` option.

# 12

## Category Y - PHDS Rules

---

### See Also

- Y1 - Cell is placed in DEF, but not defined in LEF
- Y2 - Cell has DEF-defined DIEAREA mismatch with LEF defined SIZE
- Y3 - Cell has DEF-defined DIEAREA offset mismatch with LEF-defined ORIGIN offset
- Y4 - Cell has multiple LEF definitions
- Y5 - Instance is placed in DEF, but is in the hierarchy of an undefined DESIGN Cell
- Y6 - Cell has dimensions less than or equal to zero
- Y7 - Cell cell\_name is defined in multiple DEF files
- Y8 - Instance not placed in DEF
- Y9 - Net exceeds maximum size and will be truncated in final PHDS
- Y10 - Net has name longer than supported by PHDS
- Y11 - Instance has name longer than supported by PHDS
- Y12 - Rule is used, but not defined
- Y13 - VIA is used, but not defined
- Y14 - LEF file in LEF directory is empty
- Y15 - DEF file in DEF directory is empty
- Y16 - DEF file in DEF directory was not used in PHDS creation
- Y17 - DEF file is missing the required DESIGN keyword
- Y18 - DEF file does not have corresponding LEF file
- Y19 - Cell missing corresponding LEF and DEF files
- Y20 - Cell missing corresponding DEF file
- Y21 - Less technology LEF files used than specified



- [Y22 - No layer data found in PHDS](#)
- [Y23 - No nets, VIAs, instances found in PHDS](#)
- [Y24 - More nets than net segments in the PHDS](#)
- [Y25 - Fewer nets than instances found in PHDS](#)
- [Y26 - No technology LEF file found/defined was violated d times](#)

---

## DRC Rule Y1

### Message Text

Cell *cell name* is placed in DEF, but not defined in LEF (Y1)

### Severity

Warning

### Description

A cell is included in a DEF file, but is missing from a LEF file.

### What Next

When creating the PHDS database, make sure the noted cell is defined in a LEF file.

---

## DRC Rule Y2

### Message Text

Cell *cell\_name* has DEF defined DIEAREA mismatch with LEF defined SIZE.  
(Y2)

### Severity

Warning

### Description

There is a size mismatch in the noted cell between the DIEAREA parameters included in the DEF file and the SIZE parameters defined in the LEF file. In this case, placing a mismatched set of net geometries and pins can disorient the cell, which leads to incorrect behavior when creating the PHDS database.

### What Next

Correct the DIEAREA parameters in the DEF file so they match the SIZE parameters in the LEF file. For example:

DEF File:

```
DIEAREA ( 0 0 ) ( 100 100 ) ;
```

LEF File:

```
SIZE 100.000 BY 100.000 ;
```

---

## DRC Rule Y3

### Message Text

```
Cell cell_name has DEF defined DIEAREA offset mismatch with LEF defined  
ORIGIN offset. (Y3-1)
```

### Severity

Warning

### Description

The noted cell has mismatched offsets between the lower left point in the DIEAREA statement of the DEF file and the ORIGIN parameters of the LEF file. This causes mismatch issues between the cell pins and the net geometries when creating the PHDS database.

### What Next

Correct the offset of the lower left point defined in DIEAREA statement of the DEF file so it matches with the ORIGIN parameters in the LEF file.

---

## DRC Rule Y4

### Message Text

```
Cell has multiple LEF definitions. (Y4-1)
```

### Severity

Warning

### Description

The noted cell is defined in more than one LEF file. If both definitions of the cell are identical, there may not be a problem with the PHDS database.

### What Next

Remove the extra LEF file and create another version of the PHDS database, if necessary.

---

## DRC Rule Y5

### Message Text

Instance *instance\_name* is placed in DEF, but is in the hierarchy of an undefined DESIGN Cell. (Y5-1)

### Severity

Warning

### Description

A cell is missing a DEF file that describes its nets and components.

### What Next

When creating the PHDS database, include a DEF file that describes the nets and components of the noted cell.

---

## DRC Rule Y6

### Message Text

Cell *cell\_name* has dimensions less than or equal to zero. (Y6-1)

### Severity

Warning

### Description

The dimensions for the noted cell are invalid.

### What Next

When creating the PHDS database, specify dimensions for the noted cell that are equal to or greater than one.

---

## DRC Rule Y7

### Message Text

Cell *cell\_name* is defined in multiple DEF files. (Y7-1)

### Severity

Warning

### Description

When creating the PHDS database, the noted cell is defined in more than one DEF file. If both definitions of the cell are identical, there may not be a problem with the PHDS database.

### What Next

Remove the extra DEF file and create the PHDS database again, if necessary.

---

## DRC Rule Y8

### Message Text

Instance *instance\_name* not placed in DEF. (Y8-1)

### Severity

Warning

### Description

A DEF file includes the noted instance, but does not specify its placement location.

### What Next

When creating the PHDS database, make sure to include the cell location of the noted instance in a corresponding LEF file.

---

## DRC Rule Y9

### Message Text

Net *net\_name* exceeds maximum size and will be truncated in final PHDS.  
(Y9-1)

### Severity

Warning

### Description

The noted net is too long and is truncated in the PHDS database.

### What Next

If you do not want the net truncated, reduce its size to less than 1024 characters and create the PHDS database again.

---

## DRC Rule Y10

### Message Text

Net *net\_name* has name longer than supported by PHDS. (Y10-1)

### Severity

Warning

### Description

The noted net name is too long for the PHDS database.

### What Next

Reduce the net name to less than 1023 characters and create the PHDS database again.

---

## DRC Rule Y11

### Message Text

Instance *instance\_name* has name longer than supported by PHDS. (Y11-1)

### Severity

Warning

### Description

The instance name is too long for the PHDS database.

### What Next

Reduce the instance name to less than 1023 characters and create the PHDS database again.

---

## DRC Rule Y12

### Message Text

Rule *rule\_name* is used, but not defined. (Y12-1)

### Severity

Warning

### Description

A non-default rule is used in the DEF file, but it is not defined in a LEF file.

### What Next

Check if the non-default rule is in a LEF technology file that was not included when creating the PHDS database. Create the PHDS database again, if necessary.

---

## DRC Rule Y13

### Message Text

VIA *via\_name* is used, but not defined. (Y13-1)

### Severity

Warning

### Description

A DEF or a LEF file is trying to use a VIA that is not defined by a LEF file.

### What Next

When creating the PHDS database, include the LEF file containing the VIA definition.

---

## DRC Rule Y14

### Message Text

LEF file *LEF\_file\_name* in LEF directory is empty. (Y14-1)

### Severity

Warning

### Description

During the creation of the PHDS database, an empty LEF file was discovered.

### What Next

Remove the empty LEF file from the LEF directory, and create the PHDS database again, if necessary.

---

## DRC Rule Y15

### Message Text

DEF file *DEF\_file\_name* in DEF directory is empty. (Y15-1)

### Severity

Warning

### Description

During the creation of the PHDS database, an empty DEF file was discovered.

### What Next

Remove the empty DEF file from the DEF directory and create the PHDS database again, if necessary.

---

## DRC Rule Y16

### Message Text

DEF file *DEF\_file\_name* in DEF directory was not used in PHDS creation.  
(Y16-1)

### Severity

Warning

### Description

During the creation of the PHDS database, a DEF file in the DEF directory was not used.

### What Next

Make sure the noted DEF file uses the correct design name and create the PHDS database again, if necessary.

---

## DRC Rule Y17

### Message Text

DEF file *DEF\_file\_name* is missing the required DESIGN keyword. (Y17-1)

### Severity

Warning

### Description

The DESIGN keyword is required in all DEF files.

### What Next

Include the missing DESIGN keyword in the DEF file and create the PHDS database again, if necessary.

---

## DRC Rule Y18

### Message Text

DEF file *DEF\_file\_name* does not have corresponding LEF file. (Y18-1)

### Severity

Warning

### Description

A cell included in a DEF file is not defined in a LEF file. This warning is important when a design has multiple DEF files. It is usually issued when DEF files are used for blocks and macros in a design.

### What Next

Make sure all cells in DEF files are defined in a LEF file when creating the PHDS database. It is safe to ignore this warning when the DEF file in the message is a top-level file since it does not have a corresponding LEF file.

---

## DRC Rule Y19

### Message Text

Cell *cell\_name* missing corresponding LEF and DEF file. (Y19-1)



### Severity

Warning

### Description

The noted cell is missing from LEF and DEF files.

### What Next

Include the noted cell in the LEF and DEF files when creating the PHDS database.

---

## DRC Rule Y20

### Message Text

Cell `cell_name` has corresponding LEF but is missing corresponding DEF file. (Y20-1)

### Severity

Warning

### Description

This message is issued when the `set_rules y20 -min_area` command is specified, and a cell with an area greater than the specified area has a corresponding LEF file but not a corresponding DEF file.

For example, the following command sets the minimum area to 0 to show all cells that have a LEF file but are missing a DEF file:

```
set_rules y20 -min_area 0
```

You can find the area of a cell in the `SIZE` parameter in the cell's LEF file:

```
MACROslnlb4  
CLASS CORE ;  
SOURCE USER ;  
ORIGIN 0 0 ;  
LEQ slnlb2 ;  
SIZE 16.81 BY 3.69 ;  
SYMMETRY X Y ;  
SITE unit ;
```

### What Next

Make sure the cell in the message is included in both LEF and DEF files when creating the PHDS database.

---

## DRC Rule Y21

### Message Text

Less technology LEF files used than specified was violated *d* times.  
(Y21-1)

Less technology LEF files used than specified (specified=*d*, used=*e*).  
(Y21-1)

### Severity

Error

### Description

This message is reported when the `set_physical_db -technology_lef_file` command specifies more LEF technology files than are used when creating the PHDS database. In most cases, this message indicates a LEF technology file is missing or unrecognized.

### What Next

Check the extensions of all LEF technology files in your LEF/DEF database. All LEF files, including technology files, must have a .lef extension.

---

## DRC Rule Y22

### Message Text

No layer data found in PHDS. (Y22-1)

### Severity

Error

### Description

This message is reported if only OVERLAP layers exist in the PHDS database with no other layers present, or if no layers exist in the PHDS database.

### What Next

The technology LEF file defines the layers of a design. Check this file for any missing layers. Also, check to see if any [Y21](#) violations (missing technology LEF files) were reported during PHDS generation.

---

## DRC Rule Y23

### Message Text

Error: No nets found in PHDS. (Y23-1)

Error: No VIAs found in PHDS. (Y23-2)

Error: No instances found in PHDS. (Y23-3)

### Severity

Error

### Description

These message variations indicate that incomplete or incorrect DEF files were read when creating the PHDS database.

### What Next

Check your DEF files and include the missing nets, VIAs, or instances.

---

## DRC Rule Y24

### Message Text

More nets than net segments found in PHDS. (nets=A, net segments=B)  
(Y24-1)

### Severity

Error

### Description

This message is reported when the DEF files are missing cell placement or routing information. If the DEF files have complete routing information, there should be more net segments than nets. If the DEF files are missing all net placement information, the number of net segments reported is 0.

### What Next

Identify the missing cell placement or routing information and include it in the DEF files when creating the PHDS database.

---

## DRC Rule Y25

### Message Text

`Fewer nets than instances found in PHDS. (Y25-1).`

### Severity

Ignore

### Description

This message indicates that filler or antenna cells might be included in the PHDS database.

### What Next

This is an optional check. The presence of filler or antenna cells are not an issue for the PHDS database. But these cells are never used by diagnosis and can be filtered out from the LEF/DEF files to reduce the PHDS size.

---

## DRC Rule Y26

### Message Text

`No technology LEF file found/defined was violated d times. (Y26-1)`

### Severity

Warning

### Description

This message is reported when the user does not specify the technology LEF file by command `set_physical_db -technology_lef_file`, and no LEF file name contains `*tech*` available under the LEF directory.

### What Next

The technology LEF file defines the layers of a design. Specify these files with the `set_physical_db -technology_lef_file` command or rename files to include `*tech*`. Also, check if any [Y22](#) violations (no layer data found in PHDS) were reported during PHDS generation.

---

## DRC Rule Y27

### Message Text

Instance has multiple definitions. (Instance\_name) (Y27-1)

### Severity

Error

### Description

When creating the PHDS database, the noted instances are redefined in DEF files. This can cause the PHDS to miss data of instances with the same name.

### What Next

Remove or rename instances with the same name and re-create the PHDS database.

---

## DRC Rule Y28

### Message Text

Subdevice device\_name generated fail. (Y28-1)

### Severity

Warning

### Description

This message is reported when there is a device list in the hierarchy file that is not generated during hierarchical PHDS generation.

### What Next

Remove the extra device in hierarchy file and create the PHDS database again, if necessary.

# 13

## Category Z - Tristate Rules

---

### See Also

- [Z1 - bus capable of contention](#)
- [Z2 - bus capable of holding Z state](#)
- [Z3 - wire capable of contention](#)
- [Z4 - bus contention in test procedure](#)
- [Z5 - BIDI pin not at input mode](#)
- [Z6 - ATPG constraint violation in test procedure](#)
- [Z7 - unable to prevent contention for circuit](#)
- [Z8 - unable to prevent contention for bus](#)
- [Z9 - bidi bus driver enable affected by scan cell](#)
- [Z10 - internal bus driver enable affected by scan cell](#)
- [Z11 - uncontrolled BIDI driver using BIDI control pin](#)
- [Z12 - BIDI control pin effects capture operation](#)

---

## DRC Rule Z1

### Message Text

Tristate Rule Z1: Bus gate N (G) failed contention ability check for drivers G1 and G2.

### Severity

Warning

### Description

A BUS primitive must not be capable of being in a contention state when PI constraints are set. A BUS primitive is automatically inserted into the design during flattening to resolve the driven value on multi driver nets. G is the gate ID of the BUS device and G1 and G2 are

two drivers which could contribute to a contention situation. More might be possible but the analysis needs only two to identify a Z1 violation.

You can think of this rule check as dividing the BUS gates into two groups: Those that require further monitoring (Z4, Z7, bus analysis), and those that do not.

This violation occurs when a pattern can simultaneously enable the drivers on two or more tristate drivers to the same net, or if the analysis is terminated before completion because the ATPG abort limit was reached. Unless you select the `-nomultiple_on` option of the `set_contention` command, it also is required that the data pins be at different values to be considered a violation. BUS inputs coming from weak drive strength gates are not considered when identifying this violation.

To achieve an accurate assessment of all potential contention BUS devices this rule check is performed independently of the effects of any DRC procedures such as `test_setup` or `load_unload`. Because the analysis is independent of all procedures, the effects of nonscan cells which might be a constant by the end of `test_setup`, or might be loaded to a constant by the end of the scan chain load, are not considered.

This check does not indicate that contention is occurring, only that it is possible for it to occur and so therefore must be monitored more closely during pattern generation.

Unless finding a pattern which avoids contention cannot be found, there should be no affect on test coverage. There is no danger that patterns created can fail simulation.

### What Next

If the violation was due to an aborted analysis, use the `analyze_buses` command in conjunction with the `-abort_limit` option of the `set_atpg` command to attempt to complete the analysis. After DRC, you should reduce the abort limit, otherwise your ATPG runtime is potentially much longer.

In general, BUS devices which fail Z1 and which also fail the Bus Analysis portion of DRC will require additional CPU effort during ATPG pattern generation to attempt to avoid contention. If there is an excessive number of bus contention failures as reported by the `analyze_buses` command, you might want to investigate.

The severity of this rule can be set to ignore, but be warned that this disables the BUS contention checking used by the Z4 and Z7 DRC checks and can cause those checks to fail. It is strongly recommended that you do not disable the Z1 checks.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "pattern data" and a pattern which causes contention is selected for display on the schematic. The GSV displays the gates in the backward trace cones of the enable lines of the contending drivers. Reviewing the gates displayed can give you some insight as to whether the problem can be corrected by changing a PI constraint or clock definition or whether the problem is inherent in the design.

## DRC Rule Z2

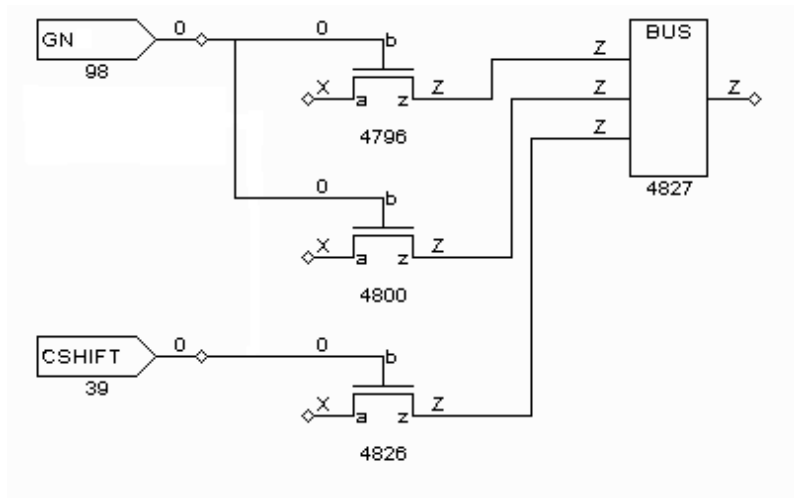
### Message Text

Tristate Rule Z2: Bus gate N (G) failed Z state ability check.

### Severity

Warning

One example, many variations are possible:



## Example

### Description

A BUS primitive must not be capable of being in a Z state when constraints are set. A BUS primitive is automatically inserted into the design during flattening to resolve the driven value on multi driver nets.

Failure to satisfy this rule will not affect test coverage or the ability to generate patterns that pass simulation. However, the presence of internal Z states become X's as they pass through other gates and can lead to an increase in tester mask requirements if the X's make their way into scan chains or outputs.

This violation occurs when analysis aborts before completing or a pattern that can simultaneously disable all of the drivers to the same net was found.

### What Next

If the violation was due to an aborted analysis, use the `analyze_buses` command to attempt to complete the analysis.



A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "pattern data" and a pattern which causes a Z state is selected for display on the schematic. The GSV displays the gates in the backward trace cones of the failing BUS primitive. Reviewing the gates displayed can give you some insight as to whether the problem can be corrected by changing a control in a test procedure or whether the problem is inherent in the design.

If this violation is acceptable, you can use the `set_rule` command to adjust its severity to ignore.

---

## DRC Rule Z3

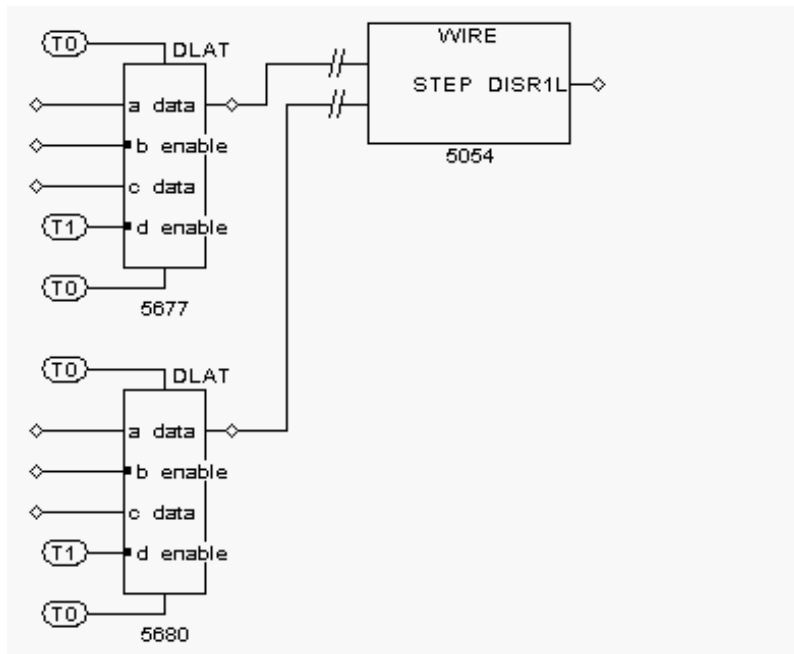
### Message Text

Tristate Rule Z3: Wire gate N (G) failed contention ability check for drivers G1 and G2.

### Severity

Error

One example, many variations are possible:



---

## Example

### Description

A WIRE primitive must not be capable of being at different state when constraints are set. A WIRE primitive is automatically inserted into the design during flattening to resolve the driven value on multi driver nets that do not use tristate drivers.

Failure to satisfy this rule results in an X state on the WIRE output, which can lead to a loss of test coverage. The contending condition can also cause harmful device stress if the non tristate drivers are allowed to be in contention.

A violation occurs when a pattern can be found which simultaneously drives a different value on the outputs of two gates connected in common to a WIRE primitive. A violation can also occur if analysis is aborted before completion (low `-abort_limit` of the `set_atpg` command).

### What Next

Review the `wire` option of the `set_contention` command.

If the violation was due to an aborted analysis, use the `analyze_buses` command to attempt to complete the analysis.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "pattern data" and a pattern which causes a contention on the WIRE primitive is selected for display on the schematic. The GSV displays the gates in the backward trace cones of the contending drivers. Reviewing the gates displayed can give you some insight as to whether the problem might be overcome by changing a contention check control or applying some sort of PI constraint or whether the problem is inherent in the design.

---

## DRC Rule Z4

### Message Text

Tristate Rule Z4: Bus contention on N (G) occurred at time T of X procedure.

### Severity

Warning

### Description

A BUS primitive must not be in contention during the simulation of any test procedure.

**N** is the instance pathname of the BUS primitive, **G** is its gate\_ID number, **X** is the name of the procedure, such as `shift`, `test_setup`, or `load_unload`, and **T** is the simulation time of that procedure.

Failure to satisfy this rule will not affect test coverage. However, the contention might be harmful to the device on the tester.

This violation occurs when simulation of the DRC procedures detects any BUS primitive with inputs in a contending state. A BUS primitive is automatically inserted into the design during flattening to resolve the driven value on multi driver nets.

### What Next

Common causes for this violation include:

- lack of control of internal tristate drivers from a top level port

- forgetting to disable bidi port drive during `load_unload` and `test_setup`

- forgetting to force Z on bidi ports during `load_unload` and `test_setup`

- disabling Z1 analysis by setting rule Z1 to ignore

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to one of the procedures: `shift`, `test_setup`, `load_unload`, and so forth., and displays the gates in the backward trace cone of the failing BUS primitive.

**Note** that when the violation occurs in the `test_setup` procedure, only a single simulation value will appear on the initial schematic display. This corresponds to the time reported in the violation being analyzed. For example "... occurred at time 200 of `test_setup`..." would indicate the single simulation value corresponds to event time 200. You might see all the simulated events of the `test_setup` procedure by clicking on the `REFRESH` button. To return to a single character display, you will need to repeat the `analyze_violation` command.

Certain periods of potential contention might be allowable in the `test_setup` or at the beginning of the `load_unload` procedure. However, it is not advisable to continue with potential contention during the `Shift` procedure as any patterns produced can lead to device damage when applied on real silicon.

If this violation is acceptable, you can use the `set_rules` command to adjust its severity to ignore.

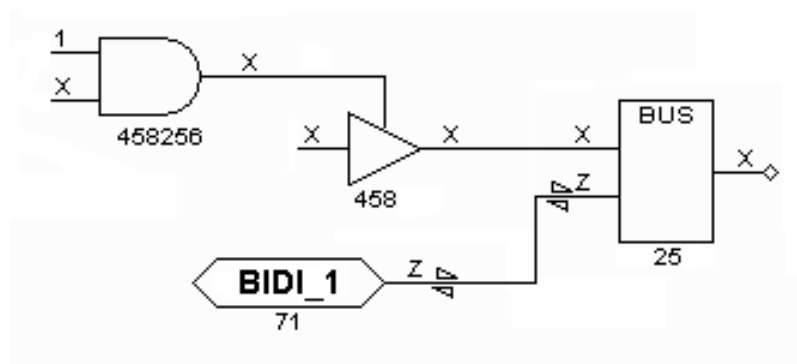
---

## DRC Rule Z5

### Message Text

```
Tristate Rule Z5: BIDI port P not set to input mode at time T of X
procedure.
```

One example, many variations are possible:



Certain test methodologies insist that bidirectional ports which are not scan outputs be in a driven input mode. This is primarily intended to reduce switching noise on the tester that can occur if the bidirectional ports are allowed to change direction or state during each shift clock. Your own specific test methodology cannot have this requirement, if so you might want to set this rule's severity to ignore using the `set rules` command.

---

## DRC Rule Z6

### Message Text

Tristate Rule Z6: ATPG constraint violation on N (G) occurred at time T of X procedure.

### Severity

Warning

### Description

Static ATPG constraints must be satisfied during the application of any test procedure.

### What Next

You can analyze a violation using the schematic analysis by selecting its violation ID number. This sets the gate reporting to the procedure pattern data at which the failure occurred and displays the gates in the backward trace cone of the failing ATPG constraint site.

Note that when the violation occurs in the test\_setup procedure, only a single simulation value will appear on the initial schematic display. This corresponds to the time reported in the violation being analyzed. For example "... occurred at time 200 of test\_setup..." would indicate the single simulation value corresponds to event time 200. You might see all the simulated events of the test\_setup procedure by clicking on the REFRESH button. To return to a single character display, you will need to repeat the analyze\_violation command.

---

## DRC Rule Z7

### Message Text

Tristate Rule Z7: Cannot set all buses to a noncontending state.

### Severity

Error

### Description

All BUS primitives must be capable of being simultaneously set to a noncontending state. A BUS primitive is automatically inserted into the design during flattening to resolve the driven value on multi driver nets.

Failure to satisfy this rule indicates that test generation cannot find a pattern that will avoid contention. Depending upon the contention checking conditions controlled by the set\_contention command, any or all patterns generated can end up being discarded

due to contention. This would then contribute to a lower test coverage and increased CPU time.

This check is performed by attempting to find a pattern which simultaneously avoids contention on all BUS devices in the design. A Z7 violation occurs if a pattern cannot be found.

Z7 checking is not performed if Z1 failures are not present. However, if there are Z1 failures, Z7 checking commences and could fail if ATPG constraints are forcing internal clocks-on in other than the first cycle.

### What Next

This violation has no automated schematic viewing analysis and none is possible.

If Z1 checking has been disabled by changing the severity of Z1 to ignore, then this can contribute to Z7 violations. Use the `report_rulesz1` command to determine if Z1 is set to ignore.

If the design has Z8 violations, then investigate these violations first. A Z8 violation is issued for a `single` BUS for which no pattern can be found to avoid contention. As long as a single BUS is failing, there is no point in troubleshooting the Z7 which is a check for all BUSES.

It is important to determine if the contention prevention effort which accompanies a Z1 analysis had completed or aborted. Follow these steps:

Issue the `report_buses-summary` command to check whether any of the bus analyses were aborted (`#abort` is nonzero for contention status). If the number of aborted analyses is zero, then contention prevention completed without aborting and you can skip steps 2-5 below.

If there were aborted bus contention analysis, then determine if there were also any Z1 failures using `report_rulesz -fail`.

If there were any Z1 failures, increase the ATPG effort using the `set_atpg-abort NNN` command and re-analyze for bus contention using the `analyze_buses-all -exclusive -update` command. This rechecks all current buses capable of contention and updates their contention status.

If step #3 still shows aborted analysis, continue to increase the ATPG abort limit and try again.

If the analyze bus command completes without aborting the analysis, then retry the contention prevention check by using the `analyze_buses-prevention -all` command.

Do a `report_violationsz7`. If your Z7 is of the form Z7-nn.A, then the "A" means analysis was aborted. Increase the ATPG abort limit and try again. If you have used `set_contentionfloat` try restoring this to its default of `set_contention`.

If you observe you have zero Z1 violations and the Z7 violations cannot be resolved by increasing the ATPG abort limit or by restoring `set_contentionnofloat`, then submit a problem report with a testcase.

If the failure to satisfy contention prevention was not due to an aborted analysis (in other words, the analysis completed but could not avoid the contention), then there is no way to avoid bus contention without modifying the design. Check for the presence of ATPG constraints using the `report_atpg_constraints` command. The presence of ATPG constraints can lead to a Z7 failure if the ATPG constraints cannot be satisfied. To continue you will either have to downgrade Z7 to a warning, or remove ATPG constraints, or disable contention checking for the design.

You can obtain a list of BUS devices contributing to the problem by using the `report_buses-contention fail` command.

If there are no Z8 violations, the inability to satisfy contention prevention is due to more than one BUS device and requires complex manual effort to find the cause. With a list of BUS devices which failed contention checks, you can use the `run_justification` command to assist in that effort.

Note: When you run the `report bus -contention fail` command, it is not uncommon to see a violation ID of the form: Z7-nn.A. the "A" at the end is an indication that analysis was aborted. You might benefit from increasing the ATPG abort limit using the `set_atpg` command.

---

## DRC Rule Z8

### Message Text

Tristate Rule Z8: Bus gate N (G) failed contention prevention ability check.

### Severity

Warning

### Description

A BUS primitive must be capable of being set to a non-contending state. A BUS primitive is automatically inserted into the design during flattening to resolve the driven value on multi driver nets.

Failure to satisfy this rule indicates that test generation cannot find a pattern that will avoid contention and still satisfy all other constraints and restrictions. Depending upon the contention checking controlled by the `set_contention` command, all patterns generated can end up being discarded due to contention that can contribute to a lower test coverage. If contention checking is disabled, patterns generated can fail to simulate.

This violation occurs when ATPG pattern generation can find no pattern which can simultaneously avoid contention on the BUS primitive while at the same time honoring all ATPG and other constraints.

#### What Next

Check the settings in use for the `set_contention` to ensure they are appropriate. This can be done using the `report_settings` command to display the current settings.

Check your define PI constraints and ATPG constraints (if any) to ensure they are appropriate.

A violation can be analyzed using the schematic viewer by selecting its violation ID number from the `ANALYZE` button. This changes the pin data reporting to "none" and displays the gates in the backward trace cone of the enable lines of the BUS drivers. Reviewing this displayed data can give insights in how to control the driver enables to avoid the problem or it can indicate that a design change is required.

---

## DRC Rule Z9

### Message Text

Tristate Rule Z9: Enable of bidi bus driver N (G) connected to scan cell gate (G) .

### Severity

Warning

### Description

The enable line of a bidirectional pin is controlled or connected to a scan chain cell. There is a potential for driver contention between the bidirectional pin and the device tester during both scan chain shifting and after the application of a capture clock when the contents of the scan cell will change.

This rule check is performed using design connectivity plus the affects of any PI constraints, or constant load cells.

### What Next

Use the `ANALYZE` button to select the violation by its violation ID and to have the violation drawn in the schematic viewer.

This rule only indicates that there are potential problems. If there are problems with scan chain shifting there will also be Z4 rule violations, which should be corrected to avoid contentions that might occur for every shift of every load.



To avoid contention due to a capture clock changing the contents of controlling scan cells, use the `-capture` option of the `set_contention` command. This causes offending patterns to be discarded if they are found to cause contention. A message is given indicating the number patterns that were rejected and there is likely some loss of test coverage due to the pattern rejection.

A general solution to avoid all problems associated with this rule is to modify the design to add circuitry which prevents the bus from attaining a contention condition in both normal operating mode as well as scan shift mode. The bus gate will now pass the Z1 rules check and will not be considered for this Z9 rule check.

---

## DRC Rule Z10

### Message Text

Tristate Rule Z10: Enable of internal bus driver N (G) connected to scan cell gate (G).

### Severity

Warning

### Description

The enable line of an internal three-state driver is controlled by or connected to a scan chain cell. There is a potential for driver contention between driver devices on the internal multidriver net both during scan chain shifting and after the application of a capture clock. This check is performed only for bus gates that fail the contention ability checking (rule Z1).

This rule check is performed using design connectivity plus the affects of any PI constraints, or constant load cells.

### What Next

Use the schematic viewer to analyze the violation ID.

This rule only indicates that there are potential problems. If there are actual problems with scan chain shifting, Z4 rule violations will occur. All Z4 violations should be corrected, especially those that occur during the shift procedure as these might be serious.

To avoid contention due to a capture clock changing the contents of controlling scan cells, use the `-capture` option of the `set_contention` command. This causes offending patterns to be discarded if they are found to cause contention or changed to a force Z if the contention occurs on a bidirectional port. A message is given indicating the number patterns that were rejected and there is likely some loss of test coverage due to the pattern rejection.

## DRC Rule Z11

## What Next

Use the **ANALYZE** button to select the violation by its violation ID and to have the violation drawn in the schematic viewer. This selects a pin data of `bidi_control_value` and displays the simulated values on the failing tristate driver gate along with its control logic cone and bidi control pin when the bidi control pin is at its off value.

If your design OR's the Scan Enable with the BCP control, you should try constraining the scan enable control to it's off state with an `add_pi_constraints` command.

Review the control logic of the tristate driver to ensure that an omission has not occurred in the defined STL procedure file procedure files. If there are no problems found in the procedure files you have a number of choices:

Change the design so that bidi control is achieved.

Look for an alternative bidi control pin.

Downgrade the severity to warning or ignore. However, it is likely that the patterns produced will have contention on the bidi pins not controlled.

Modify the procedures and command file to eliminate the use of the ReflectIO protocol. Instead, consider beginning each `capture_XXX` procedure with a cycle in which the PI's are forced, but the BIDI's are set to Z. Either later in the same cycle or in a separate cycle the BIDI's is forced to their intended value.

## DRC Rule Z12

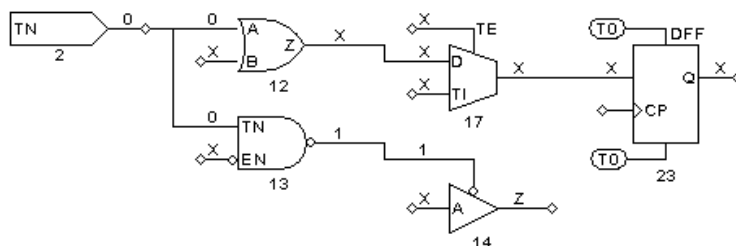
### Message Text

Tristate Rule Z12: BIDI control pin P (G1) effects capture operation of <type> (G2). (Z12-1)

### Severity

Error

One example, many variations are possible:



---

## Example

### Description

When a bidirectional control pin (BCP) has been specified by either the use of the `-bidi_control_pin` option of the `set_drc` command, or by the ReflectIO capture procedure, then it must not have a path to a DFF or DLAT state element or RAM.

`P` is the name of the primary input which has been identified as the BCP, and `G1` is its gate ID. The `<type>` is the type of device that can capture the BCP's value through a combinational path and is one of: DFF, DLAT, RAM. `G2` is the gate ID of this device.

The severity of this rule is set at error because when using the ReflectIO protocol, any pattern developed in which the BCP is asserted is changed by the ReflectIO protocol to force BCP off during the capture clock. The ATPG pattern will have been created assuming the BCP is asserted during clock capture, but ReflectIO protocol changes this port to be off during all captures. As a result, the expected value captured into the state element is wrong and will result in a simulation mismatch reported for every pattern in which the BCP is asserted.

### What Next

Use the `ANALYZE` button to select the violation by its violation ID and to have the associated gates drawn in the schematic viewer. This selects a pin data of `constrain_value` and displays the gates along the combinational gate path from the BCP to the state element which captures this value. The path drawn displays one unblocked combinational path from the BCP pin to the state element shown.

Look for a way to block the value from the BCP input point to the state element. This can be done using an `add_capture_masks` command or perhaps by locating a primary input that can be constrained with an `add_pi_constraints` command to block the path from the BCP to the state element.

Use of a PI Constraint to hold the BCP to an off state is possible, but it has a negative affect of inhibiting all bidirectional ports controlled by this BCP from ever being used in output mode. Test Coverage might be reduced by this approach.

It is highly suggested that the design be changed to eliminate the path from the BCP to the state element. If this is not possible, you might want to consider abandoning the use of the ReflectIO protocol in favor of an end-of-cycle protocol which begins each `capture_XXX` procedure with a cycle in which the PI's are forced, but the bidi's are set to Z. Either later in the same cycle or in the next cycle the bidi's is forced to their desired value (Bidi Dead-Cycle Protocol).

# 14

## Category Others - All Other Rules

---

### See Also

- [SEL\\_003 - Nothing implicitly matched](#)
- [SEL\\_004 - No objects matched](#)
- [SEL\\_005 - Nothing matched for %s](#)

---

### SEL-003

#### Message Text

Nothing implicitly matched '%s'

#### Severity

Warning

#### Description

The pattern which you specified did not match any objects of the classes acceptable for this command.

#### What Next

Check the pattern to see if it is what you expected.

---

### SEL-004

#### Message Text

No %s objects matched '%s'

#### Severity

Warning

### Description

The pattern which you specified did not match any objects of the class acceptable for this command.

### What Next

Check the pattern to see if it is what you expected.

---

## SEL-005

### Message Text

Nothing matched for %s

### Severity

Error

### Description

The patterns which you specified did not match any objects.

### What Next

Check the values which you entered.